

NANYANG TECHNOLOGICAL UNIVERSITY
SINGAPORE

SCHOOL OF MECHANICAL & AEROSPACE ENGINEERING

MA4012: Mechatronics Engineering Design

PROJECT REPORT

Team ROLLEX: Tennis Ball Retrieval Robotic Vehicle

GROUP 3 MEMBERS:

TAY KAI JIE, WORDEN	U2321648F
CHIA ANSEL	U2322571C
NG JIA MEI SHERMAINE	U2322150F
ENG BING XIAN	U2321594B
ARAVINDHAN S/O SUBBU PANDIAN	U2323632B
ASHER CHAN YONGRUI	U2322701L
LIM YUE FENG	U2322180L
NICKY CHUNG HAN JUN	U2322975K
OOI CHEE RAY	U2321939A

DATE: 17 Apr 2026

Table of Contents

Table of Contents 2
Abstract 6
Acknowledgements 7
List of Abbreviations 8
List of symbols 9
List of Figures 10
List of Tables 14
1 INTRODUCTION 15
1.1 Background 16
1.2 Motivation & Scope 16
1.3 Competition Constraints & Layout 16
2 DESIGN DEFINITION & SPECIFICATION 18
2.1 User Requirements 18
2.2 Relevant Authorities Consulted 20
2.3 Related Documents & References 21
2.4 Conditions of Material Parts 23
2.5 Design Characteristics 24
2.6 Performance 26
2.6.1 Ball Collection Performance 26
2.6.2 Ball Delivery Performance 27
2.6.3 Navigation Performance 27
2.6.4 Sensing Performance 27
2.7 Lifespan 27
2.8 Reliability 28
2.9 Control of Quality 29
2.10 Packaging and Protection 30
3 CONCEPTUAL DESIGN 31
3.1 Function Analysis 31
3.2 Ball Collection Concept Designs 34
3.3 Ball Delivery Concept Designs 37

3.4 Chassis Layout Concept Designs 39
3.5 Drive Base Layout Concept Designs 40
3.6 Navigation System Concept Design 42
3.7 Combined Morphological Chart 43
3.8 Concept Selection 45
Design Idea 1 (Blue Line) 45
Design Idea 2 (Green Line) 48
Design Idea 3 (Red Line) 51
3.9 Concept Evaluation 54
3.10 Rationale Behind Results 56
4 DETAILED MECHANICAL DESIGN 58
4.1 Ball Collector Mechanism 58
4.2 Storage & Delivery System 64
4.3 Drive Base & Chassis System 71
4.4 Vehicle Structural Frame 79
5 DESIGN CALCULATIONS 86
5.1 Theoretical RPM of Robot Drive System 87
5.2 Static Load Analysis 89
5.3 Torque and Traction Force Calculation 90
5.4 Estimated Power Consumption 91
5.5 Velocity of Ball Delivery 92
6 SENSOR SYSTEM & WIRING DIAGRAM 94
6.1 Electrical components 94
6.2 Power & Electrical Design 94
6.3 IR Line Tracking Modules 97
6.4 Sharp Distance Sensors 102
6.5 Limit Switches 109
6.6 Vex Micro-Controller & Battery Pack 112
6.7 Digital Compass 114
6.8 Wiring Pathway & Connections Design 115
7 PROGRAMMING SOFTWARE DESIGN 118

7.1 Overview of Software Strategy 118
7.2 Logic Flowchart 120
7.3 Helper Functions 121
7.4 Search Algorithm: First Pass 129
7.5 Search Algorithm: Second Pass 132
7.6 Home Return & Delivery Logic 136
7.7 Recovery & Failsafe Function 136
7.8 Edge Detection Logic 139
7.9 Opponent Detection Logic 140
7.10 Testing & Troubleshooting 141
7.10.1 Sensor Ghosting 141
7.10.2 Motion Control Functions: Swerving 143
7.10.3 Compass Inaccuracy in Cardinal Headings 144
7.10.4 Failsafe Logic for Sweep Inaccuracy 145
7.10.5 Same Side Boundary Bug 146
7.10.6 Opponent Detection Delay 146
7.10.7 Unreliable Home Delivery Logic 148
7.10.8 Lane Change Not Alternating 148
7.10.9 Other Minor Bugs & Improvements 149
8 FINAL ROBOT DESIGN OVERVIEW 150
9 COMPETITION 154
9.1 Competition Results 155
9.2 Observation 155
9.2.1 Our Robot's Performance 155
9.2.2 Observations of Competing Robots 156
9.2.3 Technical Issues or Unexpected Behaviours 157
10 LESSONS LEARNT: POST COMPETITION ANALYSIS 158
10.1 Correlation Between Design Choices and Performance 158
10.2 Observed Limitations 159
10.3 Recommendations for Improvement 160
CONCLUSION 162

GROUP REFLECTION 163
COLLATED PERSONAL REFLECTION 165
PEER EVALUATION 177
APPENDIX A: SOFTWARE CODE 186

Abstract

This project presents the design and development of an autonomous tennis ball retrieval robotic vehicle for the MA4012 Mechatronics Engineering Design competition at Nanyang Technological University. The objective of this project was to design a compact robot, within the prescribed size and hardware constraints, capable of autonomously searching for tennis balls in a bounded arena, collecting one ball at a time, returning to the home arena, and delivering the ball over the boundary wall without human intervention.

The final robot integrates mechanical, electrical, and software subsystems to achieve reliable navigation, ball retrieval, storage, and delivery. The mechanical design features a roller-based collection mechanism with passive guiding elements to improve ball intake under positional misalignment, together with a ramped storage and delivery system that secures the ball during traversal and enables controlled dispensing at the collection area. The electrical and control system uses onboard sensors for edge detection, object detection, and navigation, allowing the robot to avoid leaving the arena, locate balls, and execute return and delivery behaviours. The chassis and drive base were iteratively refined to improve rigidity, manoeuvrability, and resistance to collision-induced misalignment.

The design process followed a systematic engineering approach, beginning with requirement definition, function analysis, concept generation, concept evaluation, detailed design, prototyping, and testing. Key design considerations included compliance with competition rules, robustness against collisions, simplicity of fabrication, efficient use of limited power, and reliability of ball collection and delivery. Iterative testing was carried out to improve collection consistency, gear train performance, and structural stability.

Overall, the project demonstrates the application of mechatronics engineering principles in developing an autonomous robotic system under practical design constraints. The resulting robot provides a functional solution to the competition task and offers valuable insights into subsystem integration, design trade-offs, and iterative engineering improvement.

Acknowledgements

Our team would like to express our warmest thanks to Professor Heng Kok Hui, John Gerard, from School of Mechanical & Aerospace Engineering (MAE), for providing invaluable guidance, constructive feedback and support throughout the duration of course in MA4012.

Additionally, our team would also like to thank the Mechatronics Lab technicians, Ms. Lee, Mr. Ng and Mr. Burhan for providing the necessary resources and facilities for carrying out the project. Their continuous support and engagement truly created a welcoming and collaborative environment which made this project enjoyable.

Lastly, we would also like to thank everyone in the team for working so hard on the robot and report. This experience has been a significant milestone in our academic journey, and we are grateful for the opportunity provided to us.



List of Abbreviations

NTU	Nanyang Technological University
MAE	School of Mechanical and Aerospace Engineering
SI	International System of Units
NiMH	Nickel-Metal Hydride
BS	British Standard
mm	millimetre
cm	centimetre
m	metre
s	second
min	minute
V	volt
A	ampere
Ah	ampere-hour
mAh	milliampere-hour
N	newton
kg	kilogram
g	gram
rpm	revolutions per minute
°	degree

List of symbols

Symbol	Meaning	Unit
L	Overall Robot Length	mm
W	Overall Robot Width	mm
H	Overall Robot Height	mm
d	Diameter	mm
r	Radius	mm
h	Vertical Height	mm
θ	Angle/ Approach Angle / Ramp Angle	°
v	Linear Velocity	m/s
ω	Angular Velocity	rad/s or rpm
T	Torque	Nm
F	Force	N
m	Mass	kg
t	Time	s
P	Power	W
V	Voltage	V
I	Current	A
Ng	Number of Gear Teeth	-
G	Gear Ratio	-
μ	Coefficient of Friction	-

List of Figures

Figure 1: Final Build of Rollex 15

Figure 2: Arena layout of the Competition 17

Figure 3: (a) Tennis ball collector and (b)Past printed roller examples stored in NTU's recycled parts cabinet 21

Figure 4: Calculations of Sensor Mounting and Collector Height from Past Groups 22

Figure 5: Drive Base from Vex Competition Tutorial 23

Figure 6: Preliminary chart for autonomous robot 31

Figure 7: Simplified Function Chart for Autonomous Robot 32

Figure 8: Detailed Function Analysis Diagram for Autonomous Robot 33

Figure 9: Overall combination of conceptual design for design idea 1 45

Figure 10: Overall combination of conceptual design for design idea 2 48

Figure 11: Overall combination of conceptual design for design idea 3 51

Figure 12: Dimensions of Collector Roller Design and Mounting Height 58

Figure 13: Mechanical Collector Roller Self-Help Principle 59

Figure 14: Illustration of Squeeze Effect & Sudden Release 60

Figure 15: Allowance between roller and ramp decreases as the ball climbs from (a) to (b) 60

Figure 16: V-Guides Design & Reinforcement for Intake Assistance 61

Figure 17: Front view of robot with V guide installed 62

Figure 18: (a)Direct-drive configuration and (b)Gear-drive configuration 63

Figure 19: Overview of Storage Mechanism 64

Figure 20: Roller Dimensions for Dispenser 65

Figure 21: 3-Point Contact of Tennis Ball During Storage 66

Figure 22: Trigger Flap Mechanism for Ball Detection Inside Robot 67

Figure 23: Ramp limit switches to confirm ball collection 68

Figure 24: Overview of Ball Delivery 69

Figure 25: Ball Delivery to Overcome Gap of 9 cm 70

Figure 26: Empirical Testing Measured Result 70

Figure 27: Gear Train Ratio Configuration for High Speed 72

Figure 28: Drive Base Design Iteration 1 72
Figure 29: Pushback Test of Drive Base 73
Figure 30: Visible Flexing of Chassis System 74
Figure 31: Screw Joint of Wheels and Gears 74
Figure 32: Drive Design of Motor, Gears, Bearings, Load 75
Figure 33: 'Exploded' View of the Modular Drive System (Iteration 2) 76
Figure 34: Encoder Wiring Contact with Floor When Mirrored 76
Figure 35: Drive Base Design Iteration 2 77
Figure 36: Final Design Iteration of Drive Base (Bottom View) 78
Figure 37: Compartmentalization Sketch of Robot's Frame 79
Figure 38: Vertical Pillars & Side Plates 80
Figure 39: Top Mount & Close-up View of Mounting Pillars 81
Figure 40: Right Side View of Gear Drive for Dispense Motor 82
Figure 41: Left Side View of Gear Drive for Collector Motor 83
Figure 42: Front Vex Strip for Opponent Sensor & Roller Guard 83
Figure 43: Protective Cover for Gear Train 84
Figure 44: Handle by Anti-Slip Mat or Strings 85
Figure 45: Vex Motor Specification from Catalogue 86
Figure 46: Free Body Diagram of Robot Drive System 87
Figure 47: Layout of the competition area 88
Figure 48: Calculated delivery roller velocity 92
Figure 49: Calculated landing distance 93
Figure 50: Conceptual Sketch of Electrical System & Sensor Strategy 95
Figure 51: Circuit Diagram of Robot 96
Figure 52: Illustration of Illuminated Surface Values and Threshold 97
Figure 53: Illustration for Boundary Line 99
Figure 54: Measurement & Placement of Line Sensor 99
Figure 55: Mounting of Line Sensors Internally 100

Figure 56: Analog Values On-Site Testing & Calibration 101
Figure 57: Analog Values Compiled for Line Sensors 101
Figure 58: Illustration of Distance Sensors 102
Figure 59: Experimental Set-up for Calibration of Distance Sensors 103
Figure 60: Interpolation Results and Simulation 103
Figure 61: Calibrated Sensor Readings 103
Figure 62: Testing Sensor's FOV for Ball Detection 105
Figure 63: Distance Sensor Mounting Height Aligned with Ball's Centre 105
Figure 64: First Design Iteration of Distance Sensors on Robot (Front View) 106
Figure 65: First Design Iteration of Distance Sensors on Robot (Bottom View) 107
Figure 66: Final Design Iteration of Distance Sensors on Robot 108
Figure 67: Open/Close Ball Detection of Trigger Flap via Limit Switches 110
Figure 68: Back Limit Switch for Ball Delivery 111
Figure 69: Start Switch for Robot Handler upon Power ON 112
Figure 70: Battery holder & Vex Micro-Controller 112
Figure 71: VEX Micro-Controller I/O Layout 113
Figure 72: Digital Compass Mounting 114
Figure 73: Designed Wiring Pathways on Robot (Side and Back View) 116
Figure 74: Colour-Coded Wiring for Sensors 116
Figure 75: Wiring Layout from Top View 117
Figure 76: Arena Layout Dimensions & Ball Placement for Search Strategy 118
Figure 77: First Pass Ideation, Exploiting First ball collection 119
Figure 78: Flowchart Logic for Main Program 120
Figure 79: Arena's Grid Layout with Our Robot 129
Figure 80: Ball Detection During First Pass 130
Figure 81: Align to Ball when Detected at Sides 130
Figure 82: First Pass Scenarios 131
Figure 83: Second Pass Illustration 132

Figure 84: Ball Encountered During Forward 134
Figure 85: Ball Encountered During Sweep 135
Figure 86: Missed Ball During Second Pass..... 137
Figure 87: Failsafe Function to Recover Ball 138
Figure 88: Breakout Function 138
Figure 89: Schematic illustration of possible front and corner boundary-detection cases 139
Figure 90: Distance sensor placement 141
Figure 91: Left distance sensor placement..... 142
Figure 92: Visual Approximate Representation of Cardinal Ranges..... 144
Figure 93: Illustration of Opponent Sensor & Ball Sensor Offset Detection 147
Figure 94: Isometric View of ROLLEX 150
Figure 95: Right View of ROLLEX 151
Figure 96: Top View of ROLLEX..... 152
Figure 97: Front View of ROLLEX..... 153
Figure 98: Picture of designs of others group's robot on the arena. 154

List of Tables

Table 1: User requirements for our proposed robot 19
Table 2: Different functions of the robot 32
Table 3: Ball collection design concepts 37
Table 4: Ball delivery design concepts 39
Table 5: Chassis layout conceptual designs 40
Table 6: Drive base layout concept designs 41
Table 7: Navigation system concept design 42
Table 8: Combined morphological chart 44
Table 9: Analysis on overall conceptual design 1 47
Table 10: Analysis on overall conceptual design 2 50
Table 11: Analysis on overall conceptual design 3 53
Table 12: Weightage of evaluation criteria 54
Table 13: Total scores for the different idea drafted using criteria rating 55
Table 14: Electrical components used in the robot 94
Table 15: Design characteristics of ideal placement of sensors 98
Table 16: Wire layout characteristics and its respective reasons 115
Table 17: Helper functions used for our main task program 128
Table 18: Round Robin competition results against other teams in MA4012. 155
Table 19: Placement and the total amount of tennis balls each team deposited. 155

1 INTRODUCTION

This chapter introduces the overall context of the project, with the complete tennis ball retrieval robotic vehicle developed in this project shown below.

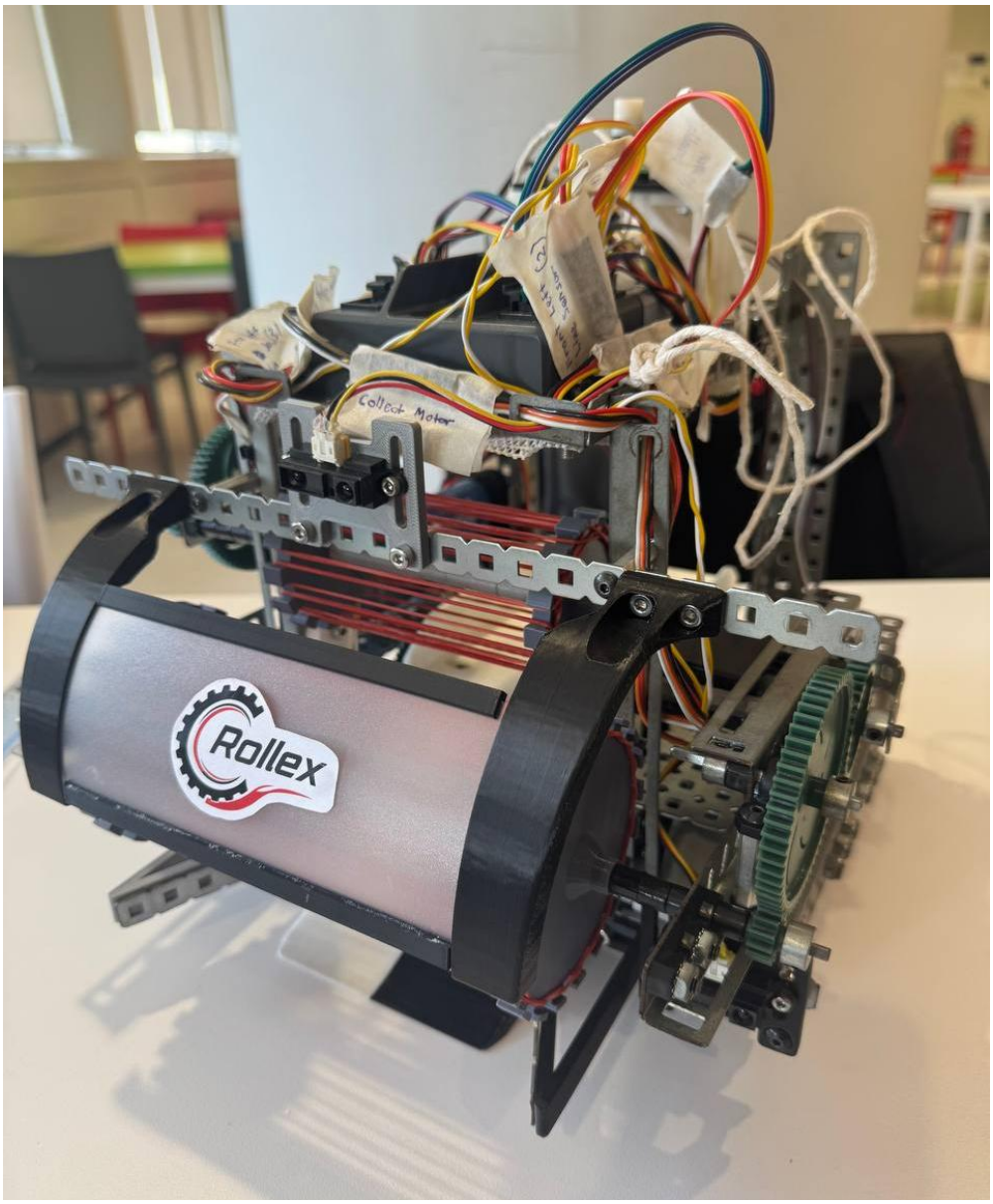


Figure 1: Final Build of Rollex

This final build of the robot represents the integration of the mechanical, electrical, and software subsystems described throughout this report. The design presented here is the result of iterative development, testing, and refinement. The subsequent chapters document the full progression of the project, detailing the conceptual design, engineering decisions, and improvements that led to this final implementation.

1.1 Background

The MA4012 course introduces engineering design processes, the role of designers and systematic approach in product definition, conceptual, embodiment and detailed design, applicable to practical project in designing autonomous mobile robots. Autonomous robots are common in industrial, service, and logistics applications to perform repetitive search, detection, and object retrieval tasks. Robotics systems rely on the integrating mechanical structures, electronic sensing, and software control to operate reliably in dynamic environments.

This project focuses on the design and development of a compact autonomous ball retrieval robotic vehicle. The robot must be capable of navigating within a bounded arena, detecting objects using onboard sensors, and performing automated retrieval and delivery tasks without human intervention. This simulates real-world engineering challenges such as limited power supply, restricted hardware resources, and time-critical operation.

1.2 Motivation & Scope

The motivation of the project is to design and build a compact ball retrieving robotic vehicle using the kit and equipment provided, testing engineering design competency and ability to design the best robotic vehicle within specified constraints.

The scope of the robotic vehicle must perform basic functions like edge detection, target tracking, searching and retrieval, while being pitted against another vehicle from other groups. The two vehicles will be placed inside an arena on the same side in the home boundary area, and three tennis balls will be placed around the track arena. The best designed vehicle can search and retrieve the most balls and return them in the delivery area behind the home boundary line will be the winner.

1.3 Competition Constraints & Layout

The following constraints of the design project for the robotic vehicle are listed below:

- The robotic vehicle should be **smaller than 30cm x 30cm x30cm** before the start of the event.
- The robotic vehicle is **not allowed to increase its footprint** after the start of the event.
- The robot vehicle should be made up by more than half coming from parts that are provided by the lab.
- Power supply to the vehicle is limited to a 6-cell NiMH 7.2V, 3000mAh rechargeable battery pack provided.
- Sensors are to be deployed at the edges of the vehicle to allow it to **detect the boundaries** of the arena.
- The robotic vehicle is to demonstrate **good manoeuvrability** (good wheel geometry, drive wheel or steering balance, etc.)
- A sweeping sharp sensor can be used to search for the balls.
- A grasping mechanism should be available to retrieve the ball and allow the robotic vehicle to return to the delivery area to deliver the ball.
- The robotic vehicle will then return to the arena to locate more balls.

- No additional sensors, servos, motors, batteries are allowed over that of what is provided.
- No restriction on the number of limit switches that you want to mount on your vehicle.

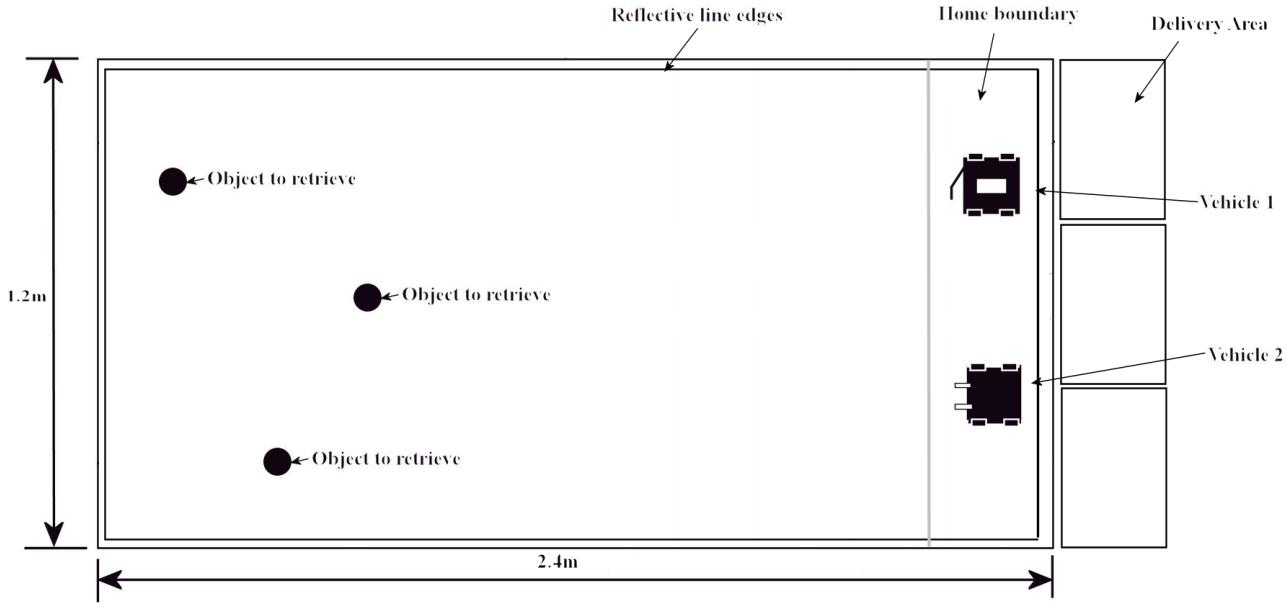


Figure 2: Arena layout of the Competition

- Each group will place their robotic vehicle on the same side of the arena. Handlers must **decide the location before the balls are placed** randomly in the arena
- 3 balls will be placed at random somewhere in the arena.
- A **time limit of 3 mins** will allow both vehicles to search and recover as many balls as possible.
- The arena will be **bounded by a yellow reflecting tape** to allow the robots to detect the edges. The edges of the arena will have a 30mm sloping high wall to prevent the ball from rolling out of the arena.
- Each robot can **only carry one ball at any one time**. (If robot collects 2 or more balls, it must release the extra balls immediately before moving)
- Upon the detection and collection of one ball, the robot is to carry it back to the collection area. (**No throwing or rolling from a distance** back to the collection area.) The collection area has a boundary wall in which the ball must be lifted over to be delivered.
- If both robots are ‘stuck’ together “each not giving way” or locked in a corner, the referee can order either an “untangle” or a “match reset”.
- The winner will be the group with the higher number of balls successfully delivered.

2 DESIGN DEFINITION & SPECIFICATION

The following specifications are in accordance with BS7373:1991 as a guideline, primarily written to communicate from a user and designer point of view.

2.1 User Requirements

Functional Requirements	The robot can self-navigate and autonomously search for balls within the competition arena. The robot must be able to retrieve tennis ball upon detection. The robot can transport and carry ball upon its return. The robot delivers the ball at the designated area and over boundary wall. The robot must withstand collision impact from opponent and not be affected during its navigation/search/transport of the ball. The robot will not move/be pushed out of bounds during navigation at any time.
Performance Requirements	The robot should operate reliably without any human intervention for every round of the competition. The robot should maintain stable movement and avoid getting stuck during operation (breakout function). The robot should be able to complete retrieval and delivery tasks before the opponent, using past seniors' competition timing as reference. The robot is efficient in power usage and smooth execution, allowing it to last for a long time.

Usability Requirements	The robot should be easy to start and reset between matches. The robot should allow easy serviceability and basic maintenance on critical mechanical and electrical parts. (E.g. Collars, Wires, Battery, etc.) The robot should allow easy access to key components for troubleshooting.
Safety Requirements	The robot should not damage the arena or competition equipment. The robot should operate safely without loose components or exposed wiring. The robot should prevent accidental release of the ball during transport. The robot remains rigid and not fall apart during possible collision impact by opponents.
Compliance & Constraint Requirements	The robot must comply with competition rules, layout and hardware constraints. The robot will only use the provided motors, sensors, controller, and battery pack. (Other constraints and compliances are listed previously in Chapter 1.3)

Table 1: User requirements for our proposed robot

2.2 Relevant Authorities Consulted

Technical consultation will be obtained from the course tutor, Dr. John Heng, for verification of design feasibility, clarification of competition requirements, and guidance on system integration of mechanical, electrical, and software components.

Laboratory technical staff are consulted for advice on safe equipment usage, component limitations, and fabrication support during prototype development.

Technical documentation from VEX Robotics is referenced for controller programming, motor specifications, and system configuration guidelines.

Industry-standard engineering design practices and laboratory safety guidelines are followed where applicable during design, fabrication, and testing phases.

2.3 Related Documents & References

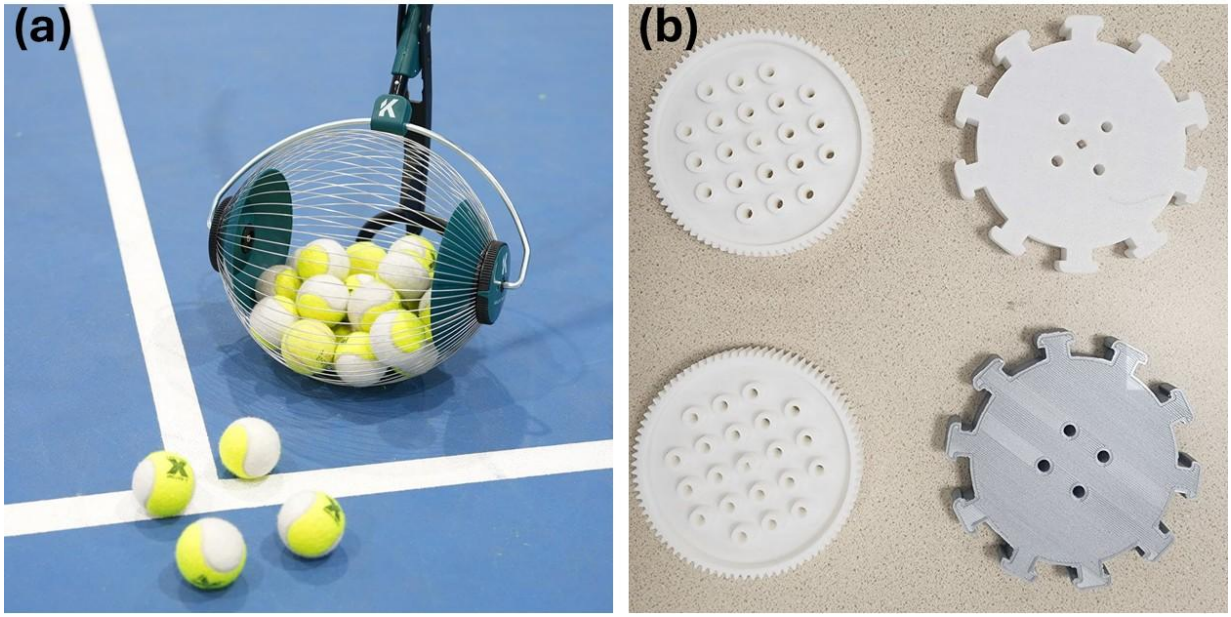


Figure 3: (a) Tennis ball collector and (b)Past printed roller examples stored in NTU's recycled parts cabinet

Our group's initial starting point of reference came from tennis ball collector machine, seen in **Figure 3a**. This design provided a practical basis for our collecting system, as it provided a practical example of how a tennis ball collecting system can be used to collect tennis ball effectively. Furthermore, **Figure 3** shows the past rollers used by previous groups. By referring to this design, our group was able to study how the roller mechanism interacted with the tennis ball during the collection process. These examples provided useful insights into how the roller position, ground clearance and degree of intake can influence the effectiveness of the intake system. As a result, our group was able to better understand how the roller establishes contact with the ball and guide it into the collection region, serving as an important reference in the development of our collection system.

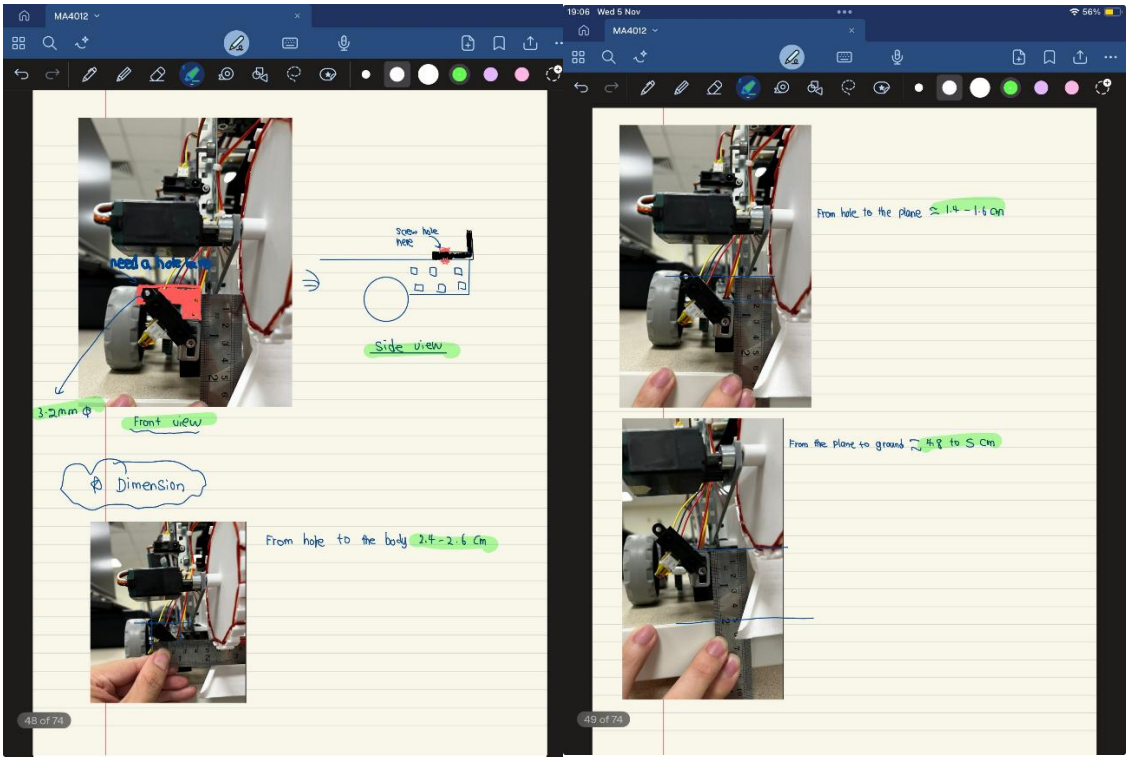


Figure 4: Calculations of Sensor Mounting and Collector Height from Past Groups

From past groups design, seen in **Figure 4**, the plane to ground distance was 4.8 to 5cm. Since the tennis ball is around 6.7 cm, this resulting difference of 1.7 cm is captured by the roller itself, allowing it to engage about 25% of the tennis ball. Through past video recordings of this roller system, this configuration was observed to be able to collect the tennis balls consistently and efficiently under competition conditions. The ball was observed to not bounce off from the rubber bands after contacting the roller and was able to roll into the collection area smoothly. This suggests that this configuration is reliable and serves as a strong reference point for our group, supporting our overall design for the collector mechanism. However, certain challenges have been faced by this configuration as well. It was noted that when the roller collides with other structures, it could get tangled, affecting the overall smoothness of the roller. To overcome this limitation, our group has decided to incorporate a roller guard.

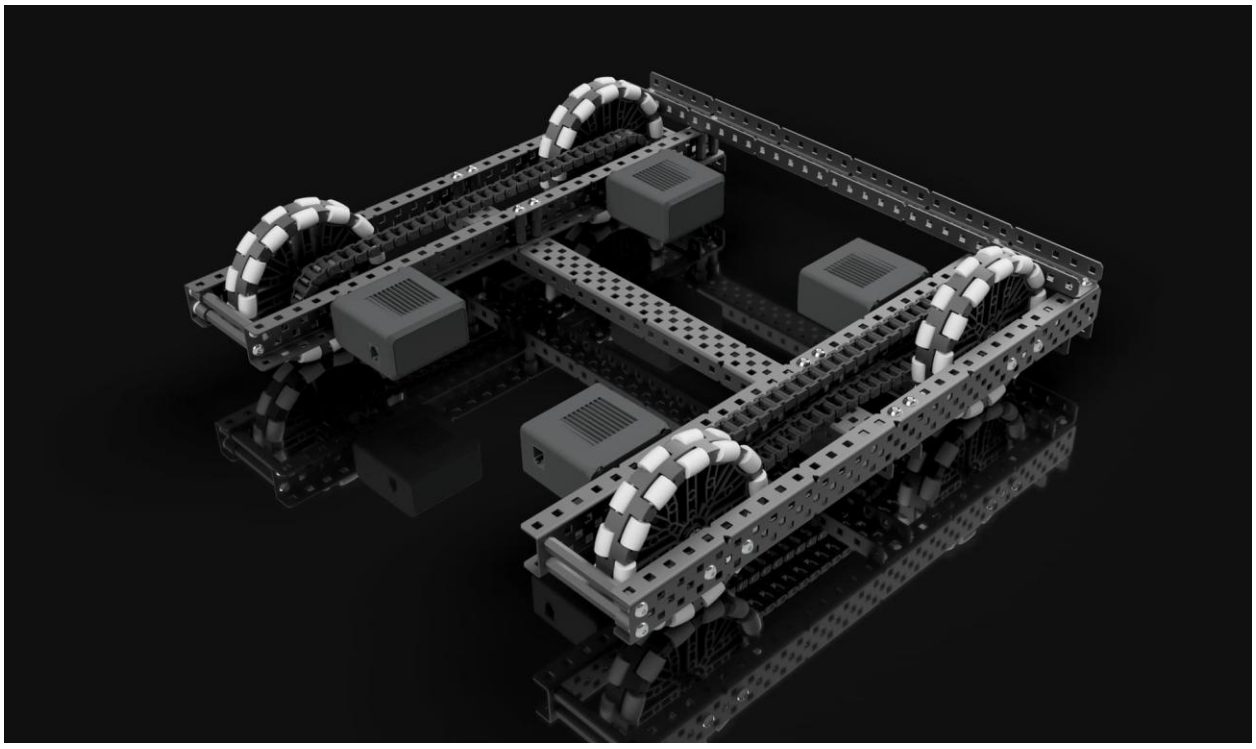


Figure 5: Drive Base from Vex Competition Tutorial

Our drive base was deeply inspired by vex robotics competitions overseas that follow a similar ball intake project, where competitors commonly use the tank drive design. This design was specifically rigid and allow the chassis and rotating elements to be well-protected. While there are some with chain drive, we use gear system for high-speed configuration for simplicity and existing components to fit our assembly. Through repeated testing of gear configurations, we settled on 60T:36T speed ratio for high speed and sufficient torque. Heavy loads will also further our pushing power due to the traction force by our wheels. The bearing system for each axle of rotating parts are also similar to these vex tutorials or forum discussions online.

2.4 Conditions of Material Parts

The robot is intended to operate in an indoor environment, but its material components must still be selected with consideration for durability under practical working conditions such as repeated loading, vibration, collision, dust exposure, and occasional moisture from handling or storage. Metallic parts should therefore possess adequate corrosion resistance or protective surface finishing to prevent rusting and long-term deterioration. Structural parts should maintain stiffness and dimensional accuracy under repeated use, while moving and contact components should resist excessive wear, frictional damage, and loosening. These requirements are important to ensure consistent mechanical performance, reduce maintenance frequency, and extend the service life of the robot.

2.5 Design Characteristics

Below table describes the various characteristics that we envision our robot to achieve, verified through standardized testing conditions listed in subsections below.

Collection Mechanism/Method:
• Wide capture zone to tolerate misalignment due to positional errors being off-centred (10 cm away, Min, 45-degree position error tolerance) • Self-Centering Behaviour due to passive guide into collector (15cm wide Min.) • Works from multiple approach angles (30-degree angle) • Low risk of ball bounce-out/getting stuck halfway (Pass Reliability Test) • Holds exactly one ball at a time • Doesn't require precise alignment for ball intake (Reduced risk of repeated attempts due to inaccuracy) • Ball intake is fast and smoothly executed, within a few seconds upon detection. • Survives impact without jamming/damaging/entangled if robot collides from front. (Pass Collision Test) • Made of minimal moving parts and compact design • Simple design to fabricate, enables easy maintenance and repair. (No complex range of motion that makes it unpredictable) • Conserves Power (No need to always actuate, only when ball nearby)
Storage & Delivery Mechanism:
• Lift ball by a vertical height of 8cm • Hold ball in place without risk of dropping • Reliable dispensing system without risks of jamming/stuck • Able to tolerate misalignment and overcome gap if off positioned during delivery (45-degree angle and 5cm minimum) • Release mechanism is simple and quick to deliver (No complex & unpredictable motion) • Made of minimal moving parts (Not more than 3 dynamic mechanisms)
Vehicle Structural Frame:
• The mechanical structure of the frame should be rigid and does not contain any non-constrained free moving components. • The mechanical structure of the frame should be well bolted and ensure all tightly secured without loosening during test runs. • Well compartmentalised with clear segregation of drive system, ball mechanism, and electrical system. • Able to withstand collision from opponent robot on all sides. • Low centre of gravity, whereby heaviest items (battery & motors) are below. • Does not exceed the limit size of 30cm x 30cm x 30cm.

Drive System/Chassis:
• The drive system should provide stable forward and reverse motion with adequate traction on the arena surface. • The chassis should maintain continuous contact of all intended wheels with the ground during straight motion and turning. • The robot shall be capable of turning in place or executing a turning radius of not more than 250mm. • The drive system should achieve a maximum straight-line speed of at least 0.3m/s, while remaining controllable during ball approach and edge avoidance. • The chassis should not experience gear-ground interference, and all gears shall maintain sufficient ground clearance of at least 5mm during operation. • The drive train should be designed to minimise backlash and lateral shaft movement using collars, spacers, and rigid mounting. • The drive system should remain operational after 10 collision trials without gear disengagement, wheel detachment, or severe loss of alignment. • The chassis layout should support balanced weight distribution to reduce wheel slip and improve manoeuvrability.
Navigation (Arena Behaviour/Opponent Response):
• The robot should autonomously search the arena for tennis balls without human intervention for the full match duration of 3 minutes. • The navigation system should detect the arena boundary and initiate corrective action before any wheel crosses out of bounds. • The robot should respond to edge detection within 0.5s and execute a retreat and turn manoeuvre to remain within the arena. • The robot should be capable of detecting a target ball within the sensor detection range and orienting toward it for collection. • The robot should include a search strategy that allows it to re-enter search mode if no ball is detected within 3s. • The robot should include a breakout or recovery behaviour if stalled, trapped or obstructed for more than 10s. • The robot should tolerate temporary blockage by the opponent robot and attempt re-positioning rather than remaining stationary. • The robot should return to the delivery area after successful collection of one ball before resuming search for the next ball. • The robot should avoid prolonged pushing contests with the opponent and prioritise recovery or re-routing where possible. • The navigation logic should remain functional under repeated transitions between search, collect, return, and deliver states.

Electrical Sensors/Wiring:
• All the sensors should be securely mounted to prevent positional shift during operation and collision. • Edge sensors should be positioned at the perimeter of the robot to maximise early boundary detection coverage. • Distance sensors should be mounted at heights and orientations suitable for detecting tennis balls and obstacles in the intended sensing region. • The wiring layout should be neat, secured, and routed away from moving parts such as wheels, gears, rollers, and shafts. • No exposed wiring should hang below the chassis or outside the robot envelope where it may snag on the arena or opponent. • Electrical connections should remain intact during repeated operation and after collision events. • Sensor readings should remain sufficiently stable to support reliable state transitions in the control program. • The electrical system should be powered solely by the provided **7.2V, 3000mAh NiMH battery pack** in compliance with competition rules. • The system should allow easy inspection and replacement of damaged wires or loose connections between matches. • The control wiring should be organised to minimise electrical noise, accidental unplugging, and maintenance difficulty.

2.6 Performance

The performance of the robot was assessed based on how well it could carry out the main competition tasks under practical operating conditions. These included ball detection, ball collection, ball delivery, stable navigation, and edge avoidance.

As the robot functions as an integrated mechatronic system, its performance was evaluated by observing how effectively the full system could complete these tasks in the competition arena rather than by using a single performance metric alone.

2.6.1 Ball Collection Performance

For ball collection performance, the robot was expected to detect and collect a tennis ball even when the ball was not perfectly aligned in front of the collector. The final design, together with V-guides and the roller mechanism, allowed the robot to collect the ball over a useful range of approach positions and angles. The robot was also able to approach the ball at different motion speeds, with slower motion used near the final collection stage to improve control and intake accuracy. Further details of the collector and storage mechanism are discussed in **Chapter 4** and the software search logic is discussed in **Chapter 7**.

2.6.2 Ball Delivery Performance

For ball delivery performance, the robot was expected to retain the collected ball, return to the home area, and release the ball into the collection box even when slight positional misalignment was present. The final dispensing system could overcome the delivery gap and benefited from the robot's reverse alignment against the collection box, which improved the final delivery position. Further details are discussed in **Section 4.2** and **Section 7.6**.

2.6.3 Navigation Performance

For navigation performance, the drive base and control system were required to provide sufficient speed for arena coverage while still maintaining controllability during turning, collection, and return-home movement. The robot was able to achieve high straight-line speed, but a more moderate operating range was preferred during actual use to improve stability, and sensing performance. The final navigation behaviour also allowed the robot to remain within the arena through boundary detection and corrective manoeuvres. Further details are discussed in **Section 4.3**, **Section 6.3**, and **Chapter 7**.

2.6.4 Sensing Performance

For sensing performance, the line sensors, distance sensors, and switch inputs were calibrated to operate within useful working ranges suited to the robot's search and collection strategy. The line sensors were able to distinguish the arena boundary from the playing surface, while the distance sensors provided a practical detection range for locating the tennis ball. Further details of the sensing system are discussed in **Chapter 6**.

Overall, the robot demonstrated adequate performance in the required areas of collection, delivery, navigation, and sensing, allowing it to meet the intended functional demands of the competition tasks.

2.7 Lifespan

The intended lifespan of the robot was defined based on the duration of the MA4012 project, from the early prototyping stage in approximately Week 4 up to the final competition in Week 13 of Semester 2, AY25/26. In this context, lifespan refers to the ability of the robot's main structural, mechanical, electrical, and printed components to remain functional throughout the development, testing, and competition period without major failure. This includes the chassis, drive system, sensor mounts, wiring, collector roller, delivery mechanism, and other repeatedly used components.

To achieve this intended lifespan, the robot was designed with durability, maintainability, and repeated testing in mind. Components that were expected to experience repeated motion, vibrations, or impact were checked and adjusted regularly during the testing phase. In addition, reasonable care was taken during assembly, handling, and troubleshooting to avoid unnecessary damage to the robot or premature wear of its parts. Further discussion of the durability-related design is provided in **Chapter 4**, and **Chapter 6**.

2.8 Reliability

To test the reliability of the robot, our team have devised a structured reliability testing plan to test the reliability of the robot.

Domain	**Test devised**	**Results**
Ball collection	Repeated trials of ball collection to see if the collector stalls	No stalling was observed, the ball travelled through the collector region consistently at high speed.
	Robot collecting ball at different speed	The robot can collect the ball consistently at different varying speed.
Ball storage/retention	Test whether the robot can keep the ball inside storage area during: • Turning • Reversing • Sweep motions • Return-home motion	Ball was consistently secure throughout all tests.
Ball delivery	Repeated delivery trials at slight misalignment	Robot could deliver the ball consistently even under slight positional misalignment.
	Checks for jamming or incomplete release	No significant jamming or incomplete release was observed.
Collision resistance	Robot subjected to light frontal and side collisions with walls or obstacles	The chassis and bumper remained structurally intact after light collisions, and the robot was still able to continue operating without major misalignment or component detachment.
Environmental Detection	Tracking to see if the robot will cross the boundary lines	The robot will perform its programmed task to return into the arena.
Sensor Reliability	Repeated trials run at off angles from the ball	The robot is consistently able to differentiate the ball from the opponent and engage it over 25cm ~ 40cm reliably. Line sensors for Edge Detection were reliable and consistent, able to course-correct itself in any orientation or traversal functions.

Home return/recovery	Repeated return-home and failed collection recovery trials	Homing and recovery routines were effective under typical test conditions.

2.9 Control of Quality

Method of checking compliance	- Trial runs under controlled conditions to confirm that the robot performs within the required standards. - Programming functions are separately tested to be fully functional, before integrating. Combined codes are fully tested again to update the final code. - Troubleshooting through analysing video recording of robot's behaviour, group discussion participation and consultation. - Sensor inspection by lab technicians and group members, for any damage to wiring port or PCB, and calibration and debugging to analog/digital values. - 3D-printed parts are wholly intact, no damage or severe cracks.
Equipment Inspection	- Ensure received parts are in good working condition. - Verify that the robot fits within the space limits in the competition
Surface Inspection	- Tension of screws, nuts, and bolts must be checked to ensure that the parts do not come loose or detach during robot operation. - Cable management is ensured to prevent no loose wires that could get trapped between moving components in the robot. - Surfaces should be inspected for burrs.

2.10 Packaging and Protection

The robot should be designed to incorporate handles for ease of pickup by key stakeholders such as the robot handler or lab technicians. Such designs must provide clarity and no complex mechanisms, whereby people can easily see and understand how to hold the robot with ease. Furthermore, the robot handles must be designed for ergonomic usage, preventing any awkward handling positions which could cause fatigue, injuries, stress, or accidental complications to physical body. For example, the handles must be sized to fit any and all various types of hand sizes without bias, and must consider the carrying positions of the arm/hands by the handler. Handles should be clear of any sharp surfaces or exposed wirings, as well as fragile parts during rough handling. Design must also account for biological conditions, such as sweaty palms, that could cause slippage or accidental drop of the robot.

Robot must also have clear identification of its name, allowing the announcer to know and call for the robot during the event. For ease of identification, the names must be eligibly read, through correct colours and contrast, font size, placement and no wear and tear that could remove visibility.

For maintenance, key members should know what orientation to fix the robot, especially the undercarriage. For example, knowing how to prop up the robot and elevate it under a stable and heavy object, to prevent from accidental running during programming or troubleshooting. Another example is placing the robot on its side or other orientation, where it is safely designed for level and rigid surface of the robot, without any sensitive components like sensors in the way. Installing a start switch when power on could be a safe way to prevent accidental running during programming.

Storage of robot should be dry indoor environment and enclosed, to prevent dust from collecting, reduced corrosion, or accidental damage when unattended. Robot should not be running on power when not in use, ensure sufficient space for storage, clear of any obstruction of foreign object or water. Regular inspections should be done to check the condition of the robot, ensuring it is fully functional and not damaged in any way during carrying or storage.

3 CONCEPTUAL DESIGN

3.1 Function Analysis

After identifying the requirements of the design, function analysis helps to identify the essential functions and consider constraints of the project. It is an important technique for problem decomposition, dividing complex problems into smaller simpler steps. This makes the problems more manageable and simplifies our understanding of how the robot works.

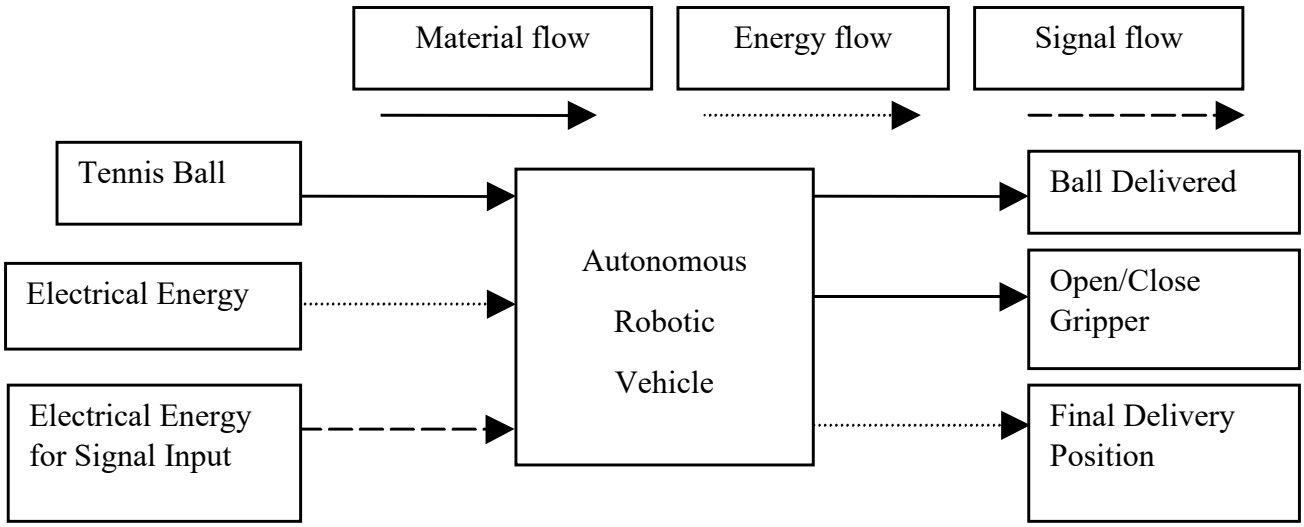


Figure 6: Preliminary chart for autonomous robot

Main Functions	Sub Functions
Navigation	Ball Search & Return Home Pathing
Object Detection	Detect Ball and Detect Opponent
Ball Collection	Grabbing & Guiding Ball Inwards, Aligning to Ball's Position
Ball Storage	Sense Ball in Possession, Lift & Hold Ball inside during traversal
Ball Delivery	Actuate Dispensing System, Overcome gaps and height upon delivery
Anti-Collision	Avoid Opponent, Breakout, Re-position, Prevent Fall/Stall/Pushed

Edge Detection	Re-Position, Prevent Out-of-bound, Detect when delivery position

Table 2: Different functions of the robot

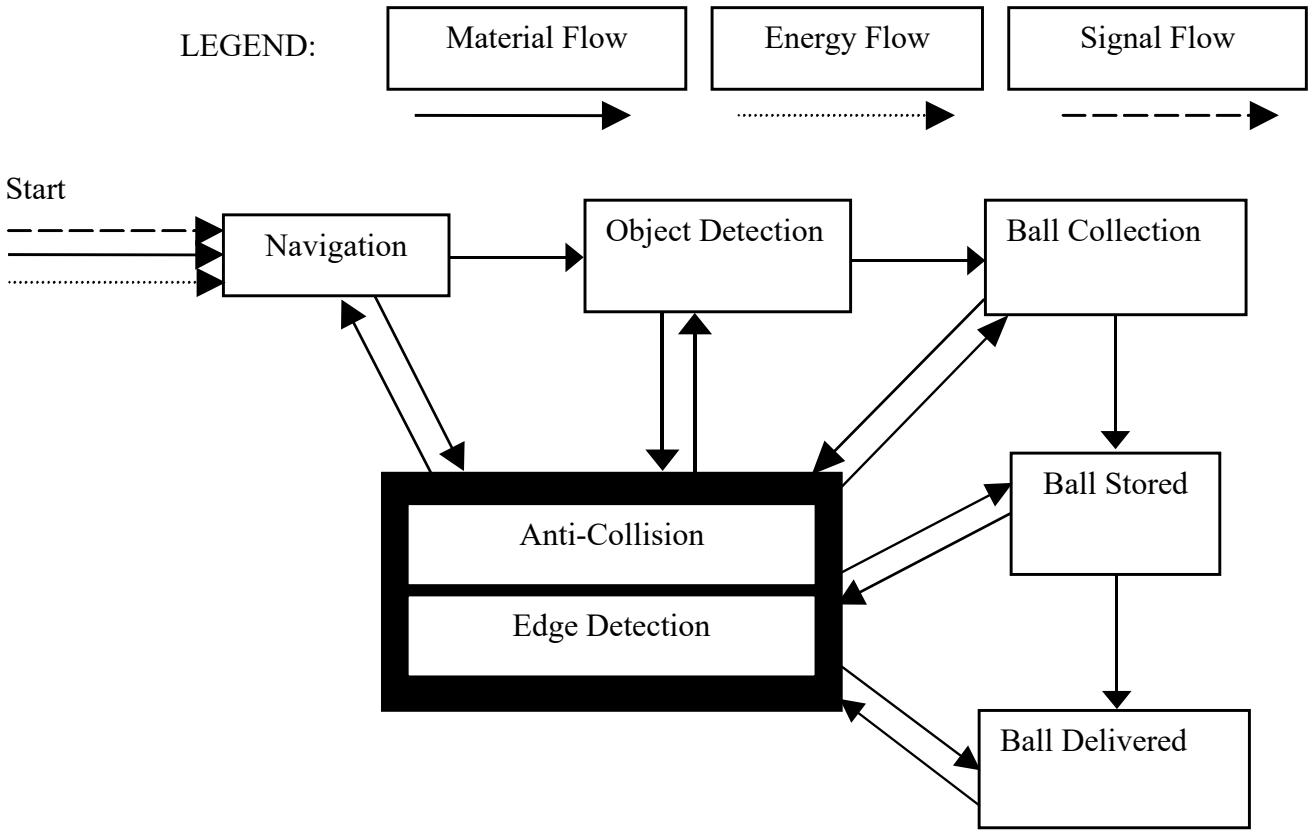


***Figure 7**: Simplified Function Chart for Autonomous Robot*

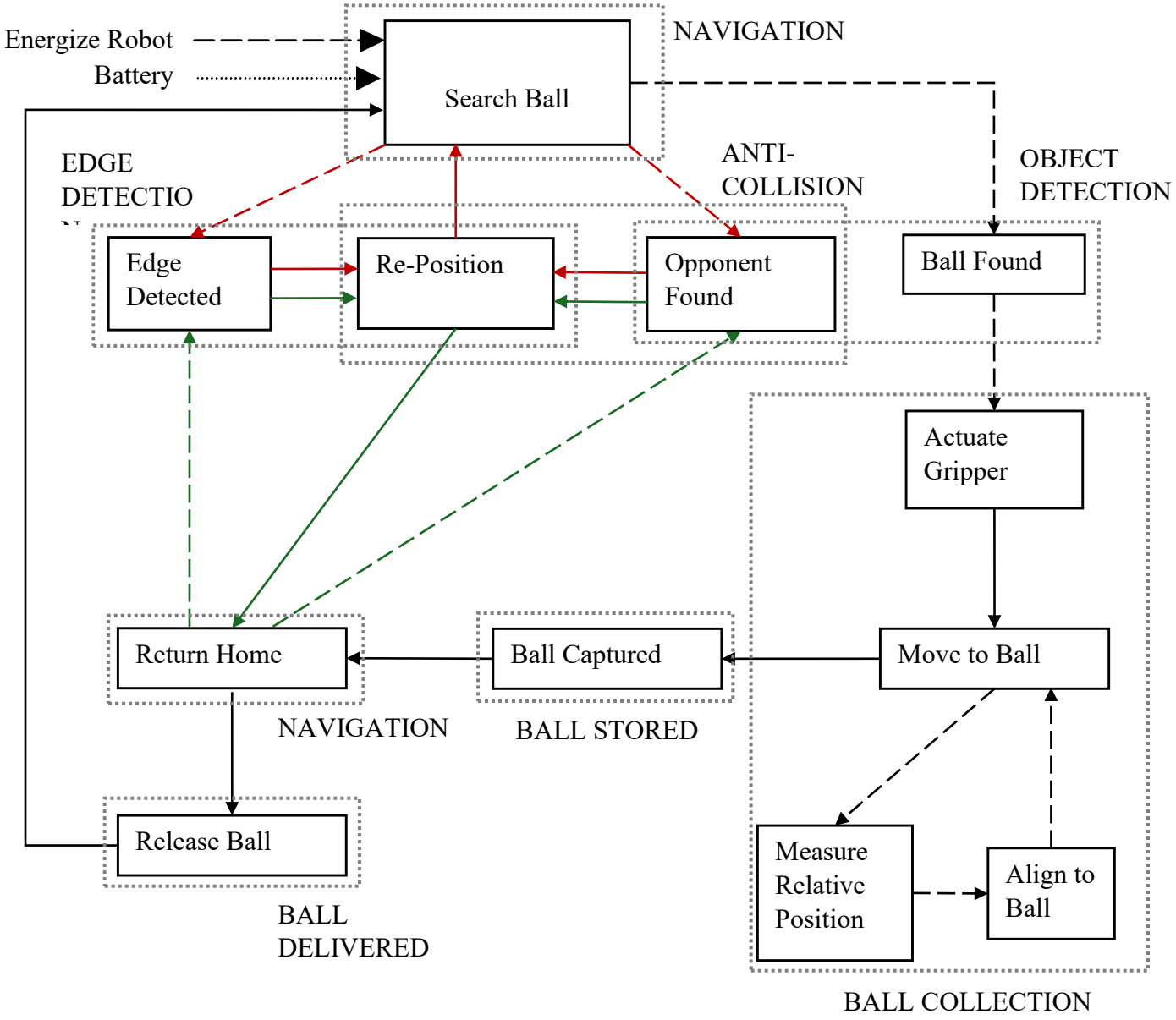
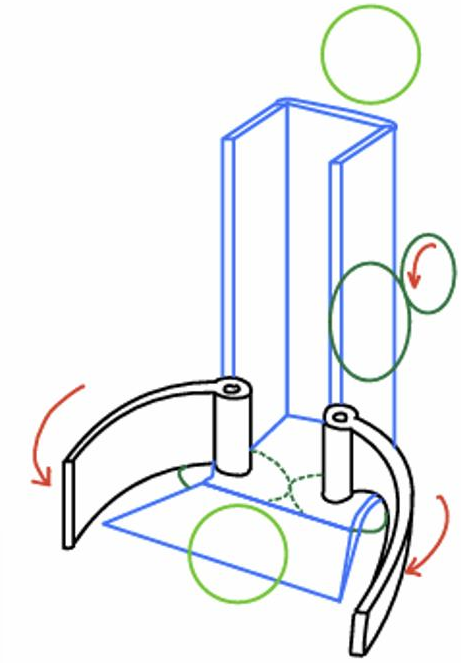
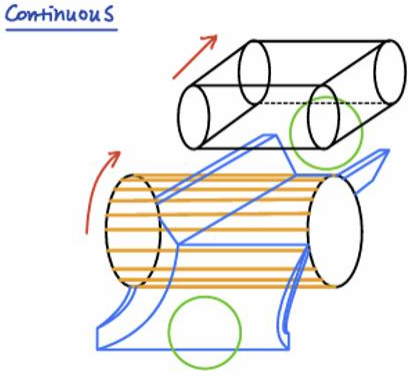
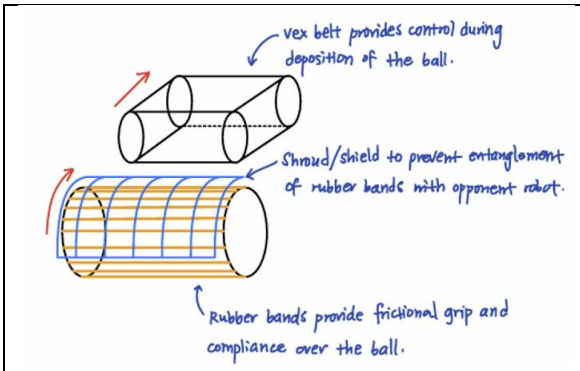
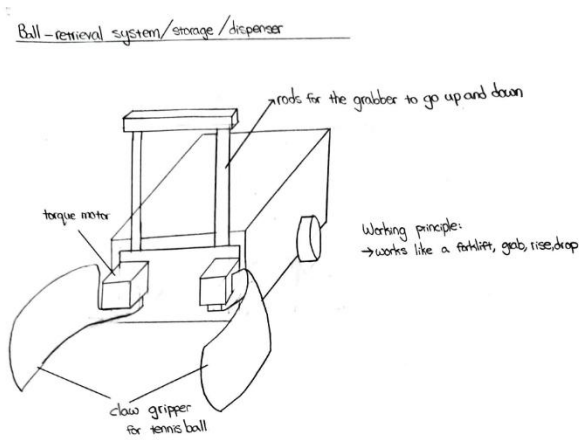
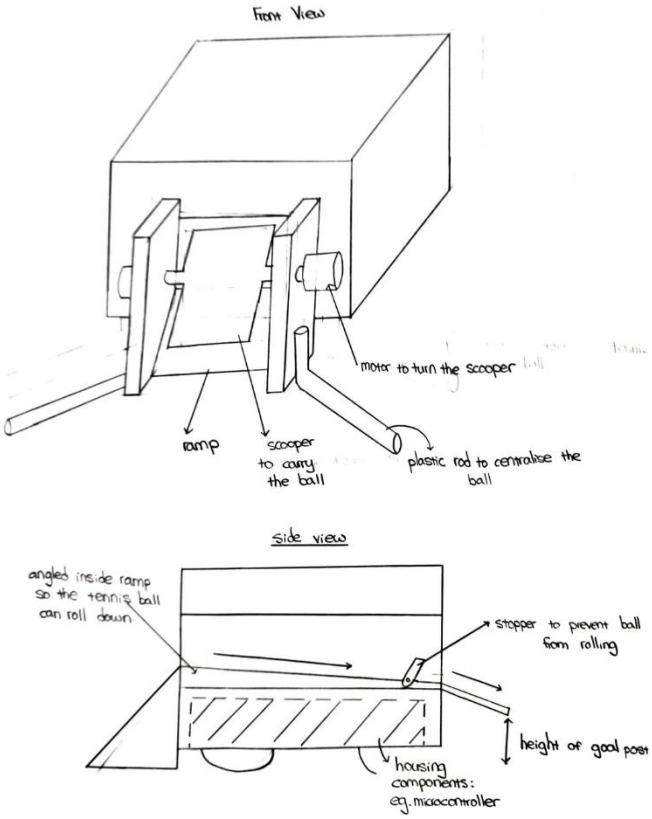


Figure 8: Detailed Function Analysis Diagram for Autonomous Robot

3.2 Ball Collection Concept Designs

Design	Description
	**Wide Ball Gripper mechanism 1** - Upon reaching deposit area, collection module is rotated in a circular motion, tipping the collected ball through the tunnel and expelling out the rear of the robot. - Non-continuous operation saves battery power usage **Strength:** - Wide-end-effector provides a large intake and a tolerance for misalignment during collection process, increasing the chances of securing the ball - Tripping motion reduces robot base movement as the depositing procedure on require reversing, with the ball guided by the tunnel. **Weaknesses:** - Rotational motion of the grip takes up a big portion of the robot's real estate, which may cause issues structurally for the base constructions.
	**Sweeper Mechanism 1 (Rubber-Band)** **Strengths:** - Wide funnel with passive guidance intakes ensures the ball will be captured, allowing more tolerance for sensor detection to operate efficiently - Straight forwards, high control of ball collection and deposition, easy to maintain **Weakness:** - Exposed rubber bands can create entanglement issues upon contact with opponents' robot. Therefore, a shield is needed to cover the exposed top section of the roller.

	
	**Gripper Mechanism 2** Works like a forklift; grab, rise and drop **Strengths:** - Wider grips allow more reach therefore providing more tolerance for sensor detection **Weaknesses:** - Claw gripper can create space constraints, especially when there is a max allowance due to competition rules - Requires more motors



Sweeper Mechanism 2

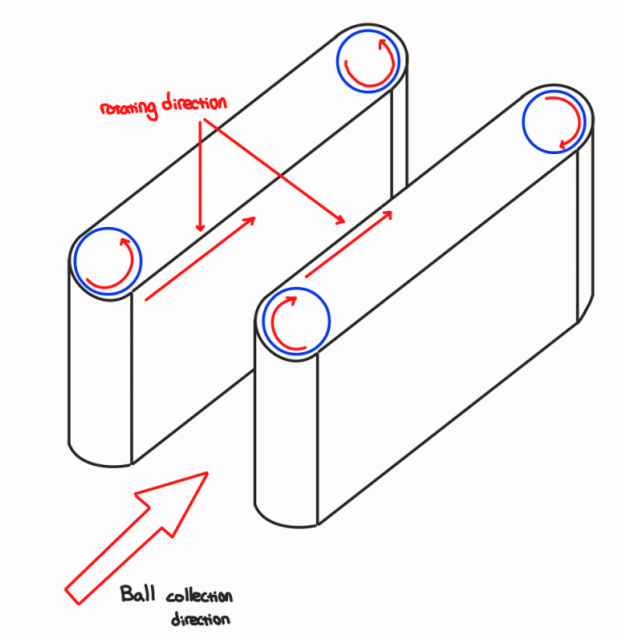
Using a scooper, to sweep the ball up a ramp, after which there is a ramp with a gate, for the ball to roll downwards after reaching the goalpost.

Strengths:

- Provides a larger intake area, making it more reliable.
- Simple to design and fabricate, as long as it is flat

Weakness:

- May be less effective if the ball approaches from an unfavourable angle.
- The scooper must be adjusted when rotating such that it would not push the ball away once guided by the V-guides.



Sweeper Mechanism (Belt) 3

Using a rotating belt to actively draw the ball into the robot during collection. The distance between the 2 belts can be calibrated such that the tennis ball can be grip firmly.

Strengths:

- Simple to design

Weakness:

- Possible occurrence of slippage, if one of the belts moves at a slower speed, the ball may be shot outwards instead.
- Though simple to design, it does require a lot of function moving components which may cause issues during operations.

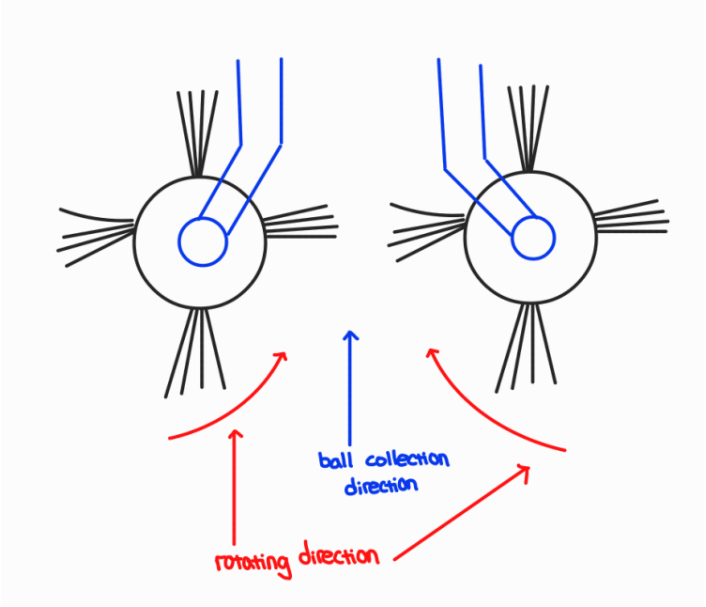
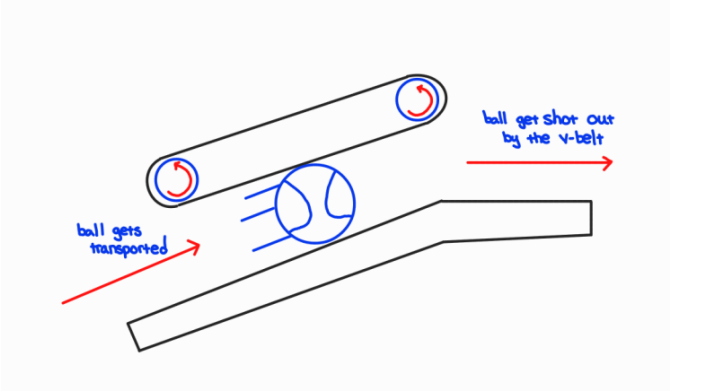
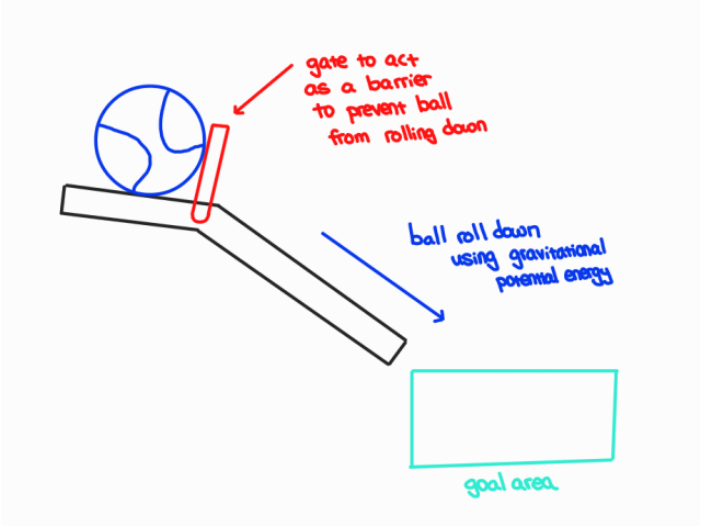
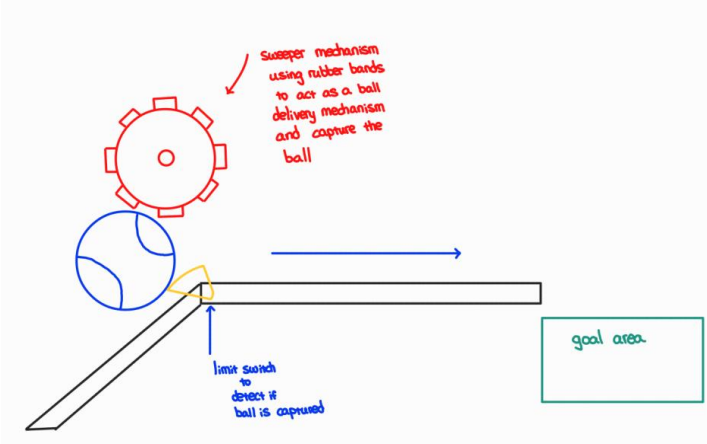
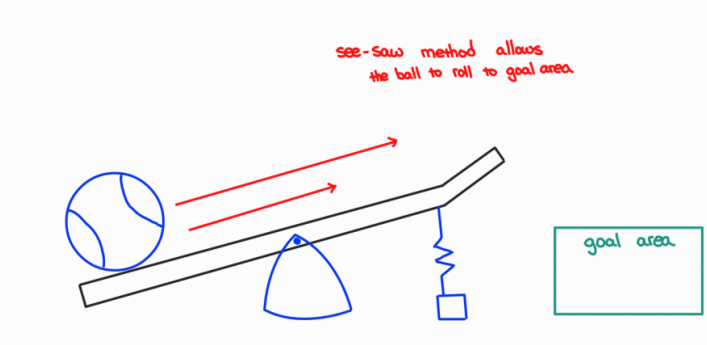
	**Sweeper Mechanism 4** Using brushes to sweep the ball into the collection area during operation. **Strengths:** - Wider reach to capture the ball, which helps with intake efficiency **Weakness:** - If the brush speed and arrangement is not aligned properly, ball may not be guided in effectively. - The length of the brush can create multiple constrains; if it is too long, the ball could be pushed away and affect the max size limit given by the competition's rules.

Table 3: Ball collection design concepts

3.3 Ball Delivery Concept Designs

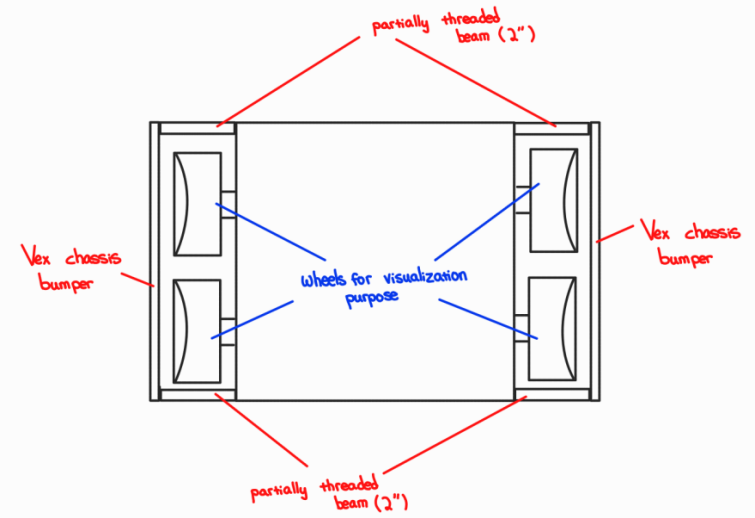
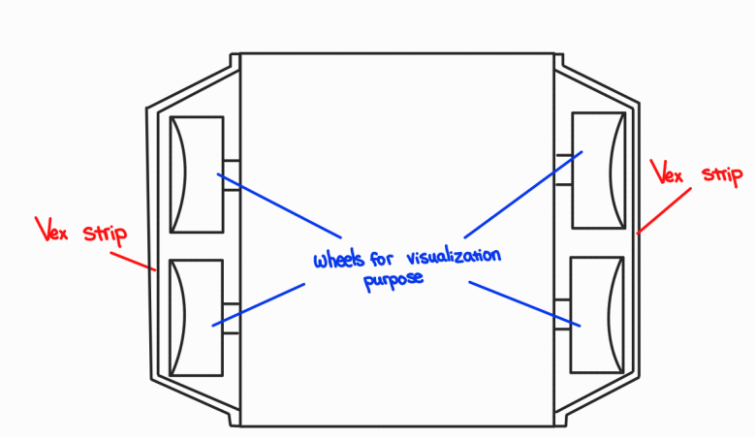
Design	Description
	**Sloped Conveyer** The slope conveyer system transports the ball at an angle and deposits them into the goal zone. **Strengths:** - Inclined arrangement can allow better usage of the space below the ramp for components. **Weakness:** - Mechanism could impose on height limits set by the rules.

	**Slope ramp with gate** Using gravitational potential energy, where the ball rolls down the ramp after the gate releases. **Strengths:** - Simple and easy to design **Weakness:** - Mechanism could impose on height limits set by the rules. - Ramp would also be needed to build higher to accommodate the higher ramp height
	**Sweeper mechanism** Similar to our ball collection concept, by replicating the mechanism, it allows us to control our overall ball handling system better. **Strengths:** - Simpler to design - Allows for much better flexibility in tension due to the properties of rubber band **Weakness:** - Rubber bands used for the sweeping mechanism might get caught with loose materials surrounding it.
	**See-saw slope mechanism** Functions like a seesaw, pully will tile the see-saw to allow the ball to roll downwards **Strengths:** - Requires fewer moving parts, just a small pully and a pivoting point **Weakness:**

	- Requires too much space both vertically and horizontally - Angle of tilt must be precise and fast as well, since the ball would be moving along the see-saw as it yaws, hence too slow of a yaw would not give the ball enough momentum to cross into the goal area.

Table 4: Ball delivery design concepts

3.4 Chassis Layout Concept Designs

Design	Description
	**Protected Vex chassis bumper layout** The Vex chassis bumper provides the higher protection and acts as a mounting for the wheel assembly. **Strengths:** - Does not bend easily - High protection to protect critical components. **Weaknesses:** - Could possibly take too much space in the horizontal direction hence, not meeting the constraints given in the competition rules
	**Protected Vex-strip layout** Similar to the Vex-chassis bumper layout, however, the vex strip only provided little protection during collisions. **Strengths:** - Helps to reduce impact during collision. **Weaknesses:** - Bends easily when flexed or collision occurs.

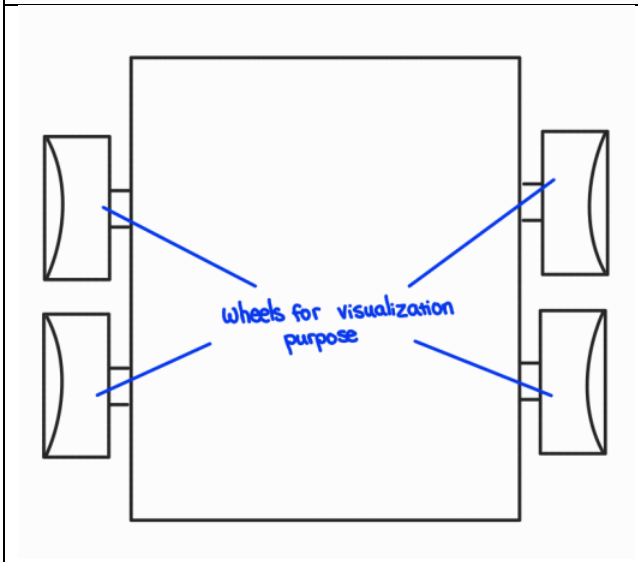
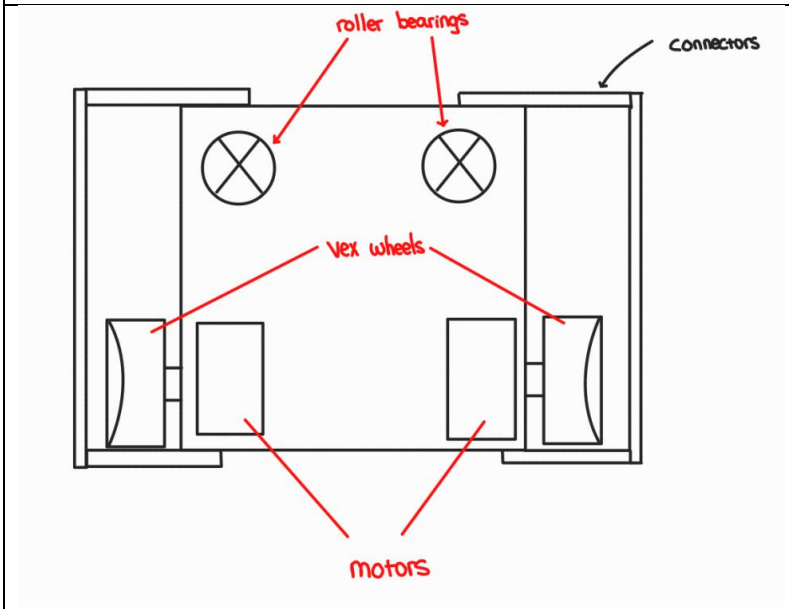
	**No protection layout** The wheels are mounted directly onto the chassis without additional protection. **Strengths:** - Simple, little components used. **Weaknesses:** - Wheels are exposed to impact or damaged, especially in a competition scenario, when collision is common. - High risk of misalignment after stuck

Table 5: Chassis layout conceptual designs

3.5 Drive Base Layout Concept Designs

Design	Description
	**Rear wheel drive with 2 roller bearing in front** **Strengths:** - It is simple and easy to implement. Reduces number of functioning components. **Weaknesses:** - Since the roller bearings do not provide steering and driving forces, this can create wobbling and skiing during operation.

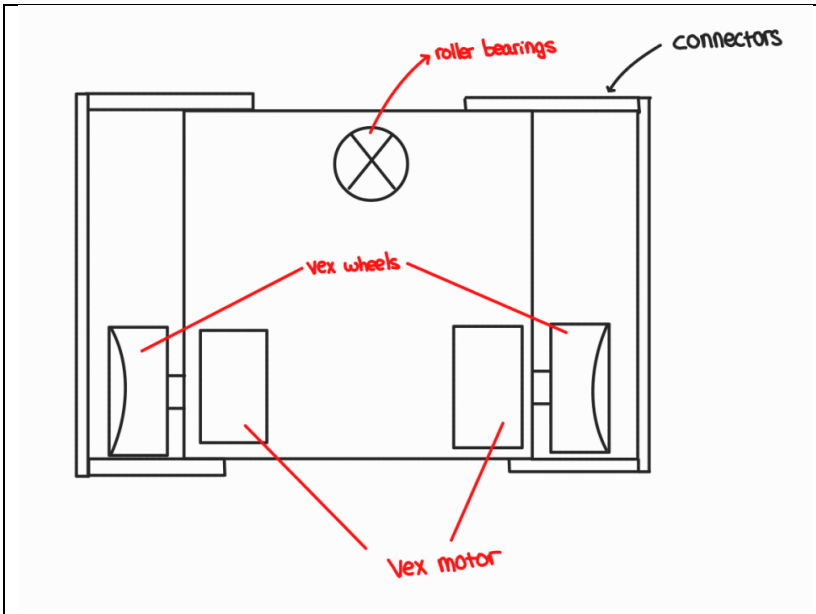
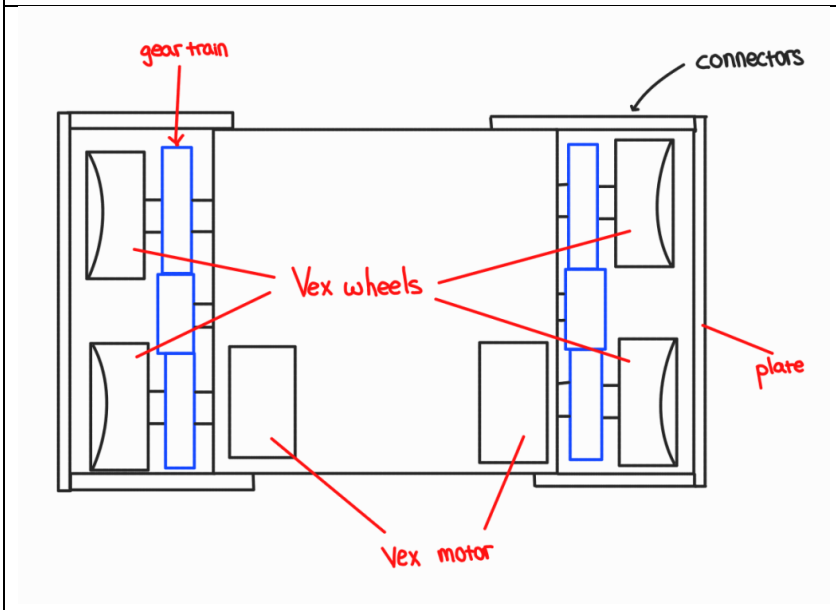
	**Rear wheel drive with 1 roller bearing in front** **Strengths:** - Like the configuration with 2 roller bearings, reduces number of components used, simplifying overall the mechanical design **Weaknesses:** - Very prone to wobbling and instability. Additionally, with the added weight from the ball collection mechanism, further unbalance weight distribution will be introduced which can affect the overall centre of gravity which is not desired.
	**Rear wheel drive layout** **Strengths:** - Offers better weight distribution and balance compared to other designs. Offers good traction as well from the rubber tires. **Weaknesses:** - It is more mechanically complex, as it requires a gear transmission arrangement.

Table 6: Drive base layout concept designs

3.6 Navigation System Concept Design

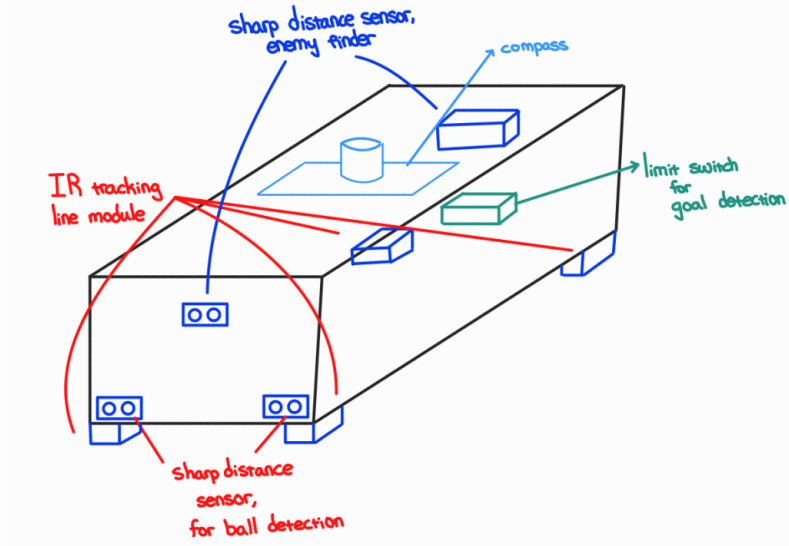
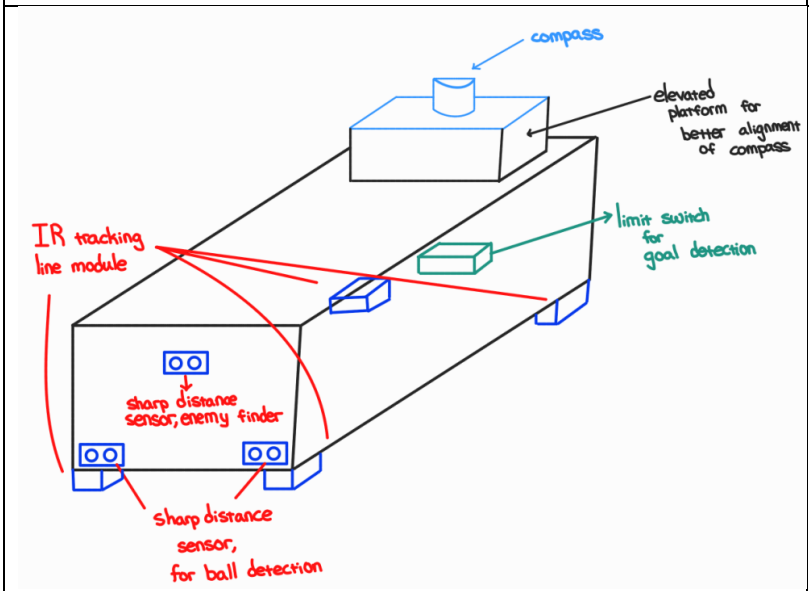
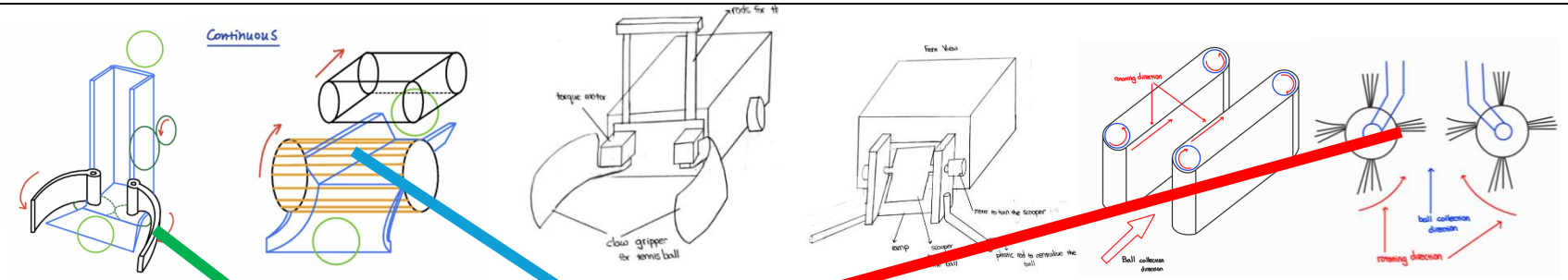
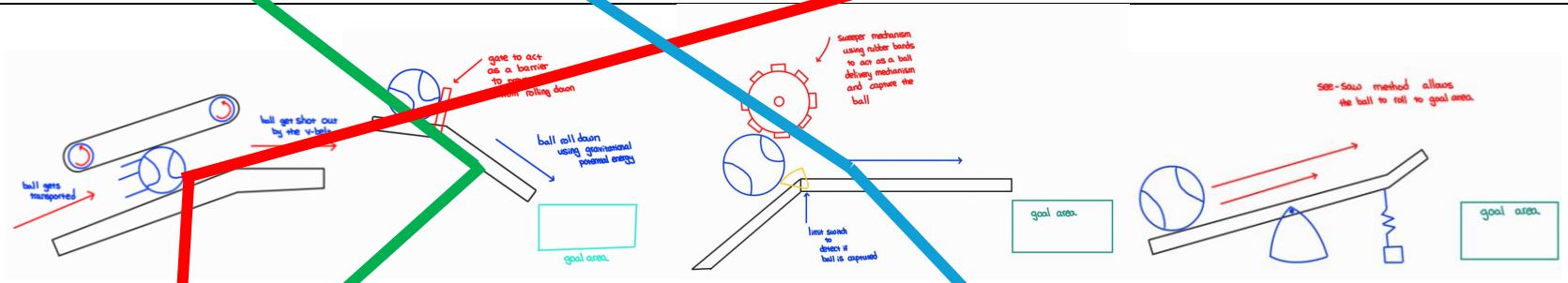
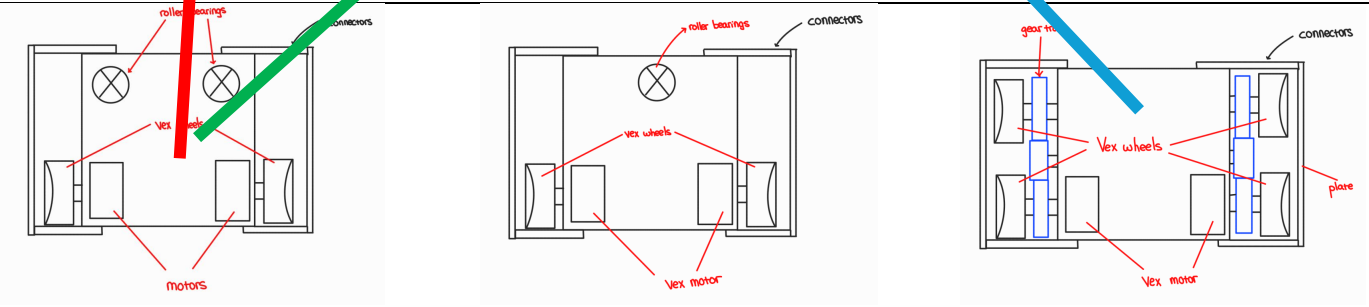
Design	Description
	**Navigation Layout 1** **Strengths:** - Has back sensor for enemy detection - All round navigation **Weaknesses:** - Compass get receive interference from other external electronic.
	**Navigation Layout 2** **Strengths:** - Has better compass navigation abilities, since it is on a elevated platform; away from potential interferences eg. Vex microcontroller. - Save on resources from one less distance sensor **Weaknesses:** - No back distance sensor for enemy detection.

Table 7: Navigation system concept design

3.7 Combined Morphological Chart

Components	Concept illustrations
Ball collection	
Ball delivery	
Drive base layout	

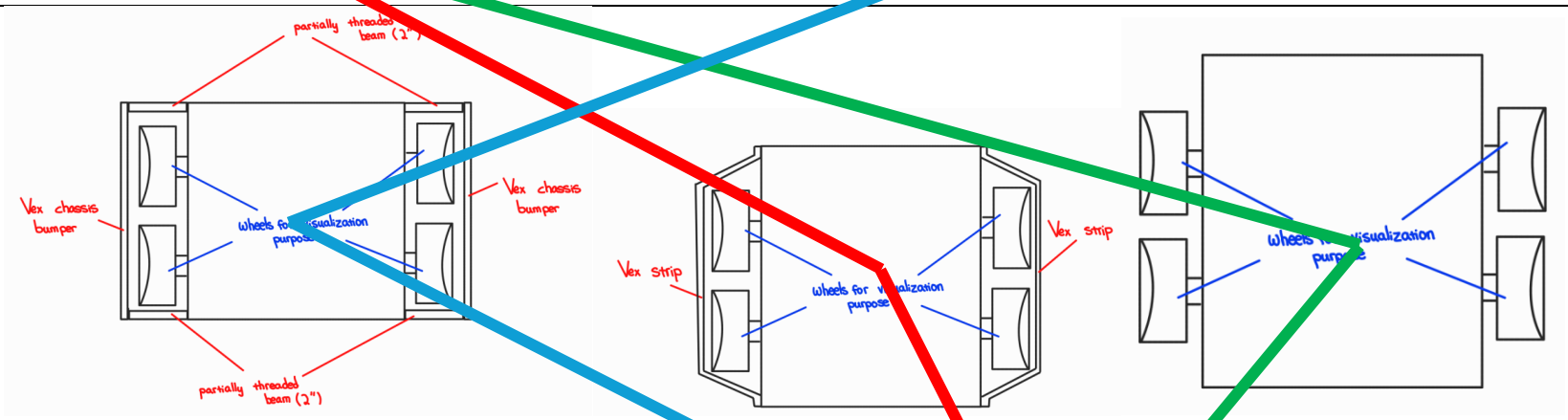
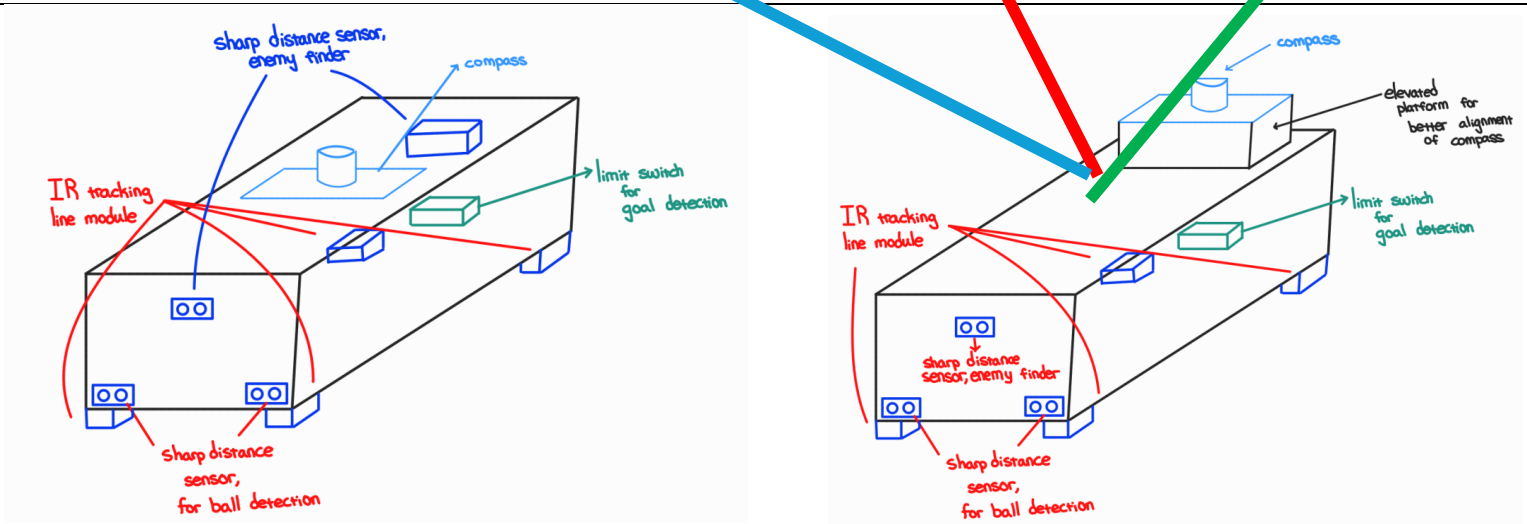
Chassis Layout	
Navigation System	

Table 8: Combined morphological chart

3.8 Concept Selection

Design Idea 1 (Blue Line)

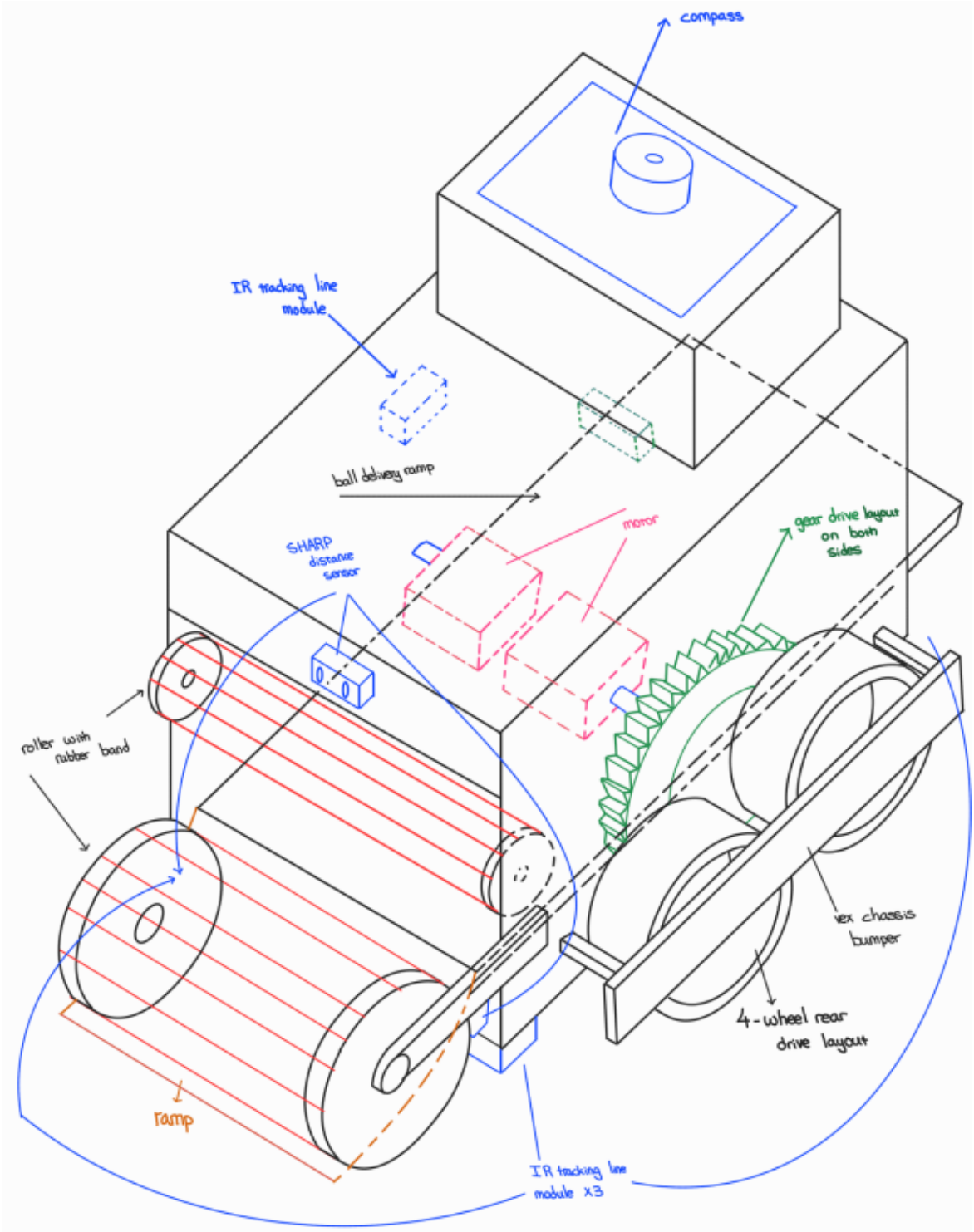


Figure 9: Overall combination of conceptual design for design idea 1

Component	Chosen Selection	Description
Ball Collector	Using rollers with rubber band	This mechanism can smoothly grab the grab since it uses tension from the rubber bands. As the rollers rotate, it will help guide the ball inwards while maintaining sufficient contact during collection. Additionally, due to the flexible nature of the rubber band, it allows the mechanism to better adapt to slight variations in ball position, making the collection process more forgiving and reliable.
Ball Delivery	Using rollers with rubber band, with a limit switch embedded in the ramp	In a similar working functionality to the ball collection system, the roller used for ball delivery can catch the ball once collected without slipping. Additionally, this configuration also enables the input of a limit switch where it can sense when the ball is collected, serving as more reliable option.
Drive System	4-wheel layout with motor driving the gear train	No slippage occurs when operating, although it requires a larger turning area to turn, it can be mitigated using software coding. Additionally, when supporting heavier front loads such as the ball collection mechanism, this configuration can help to distribute the weight more evenly, which improves balance during operation. Furthermore, by having a gear train, it allows power to be evenly distributed across the wheels, ensuring a stronger pushing force against other robots during competition.
Chassis layout	Using Vex L bumper plates with partially threaded beams	This provided the best protection among the others, although it can help shield the wheels and gear train during collisions. Additionally, this can also increase the robot's overall weight as well, making it a more robust structure.

IR Sensor	4 IR Line sensor placed at the outmost exterior of the robot	This placement of the sensors allows the tracking module to be able to track the edge at the earliest moment, preventing the robot from going out of the arena.
Sharp Sensor	3 Sensors placed in front of the robot	The top sharp sensors are used to detect the opponent while the left and rights sensor is used for tracking the tennis ball's location. As the robots continues to sweep across the arena, the ball is detected when it goes sensing range.
Compass	Using an elevated platform to place the compass.	An elevated platform is used to mount the digital compass so that it is positioned further away from the motors and other electric components that may generate magnetic interference. This helps to improve the accuracy and stability of the compass during the robot operation, overall improving the robot's orientation for navigation and directional control.

Table 9: Analysis on overall conceptual design 1

Design Idea 2 (Green Line)

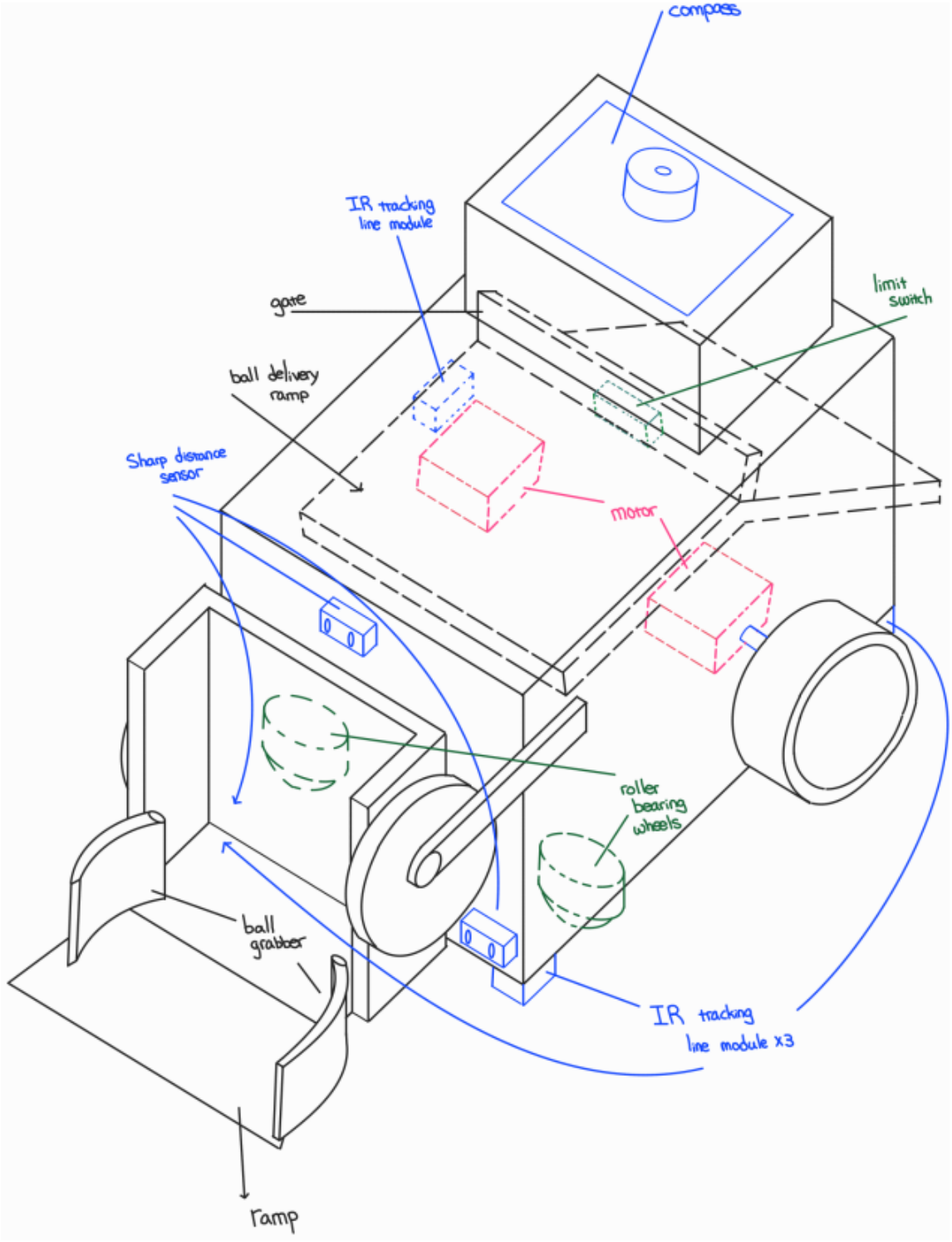


Figure 10: Overall combination of conceptual design for design idea 2

Component	Chosen Selection	Description
Ball Collector	Ball grabbing configuration with rotating base	Works similarly to a loading machine. As the robots moves towards the ball, the ramp and the grabbers will work in tandem to push the ball into the collection area. After which, the rotating base will direct the ball into the delivery area.
Ball Delivery	High ramp with gate installed.	This configuration uses a high ramp to guide the ball towards the delivery area through gravitational potential energy. The gates also help prevent premature release, ensuring that the ball is only delivered only when at the delivery point. This facilitates easier installation and control method.
Drive System	Direct rear drive system with two roller bearing wheels in front	The roller bearings help to improve manoeuvrability, as they can rotate freely with a much lesser turning radius to accommodate changes in direction. Therefore, it allows the grabber to be able to grab much more easily.
Chassis system	No protection	It is to facilitate a quicker movement of the robot by having as less weight as possible, although the robot will be more vulnerable to impact during operation.
IR Sensor	4 IR Line sensor placed at the outmost exterior of the robot	This placement of the sensors allows the tracking module to be able to identify the boundary set and

		respond timely. It helps to support a more reliable navigation method during the competition.
Sharp Sensor	3 Sensors placed in front of the robot	The top sharp sensors are used to detect the opponent while the left and rights sensor is used for tracking the tennis ball's location. As the robots continues to sweep across the arena, the ball is detected when it goes sensing range.
Compass	Using an elevated platform to place the compass.	An elevated platform is used to mount the digital compass so that it is positioned further away from the motors and other electric components that may generate magnetic interference. This helps to improve the accuracy and stability of the compass during the robot operation, overall improving the robot's orientation for navigation and directional control.

Table 10: Analysis on overall conceptual design 2

Design Idea 3 (Red Line)

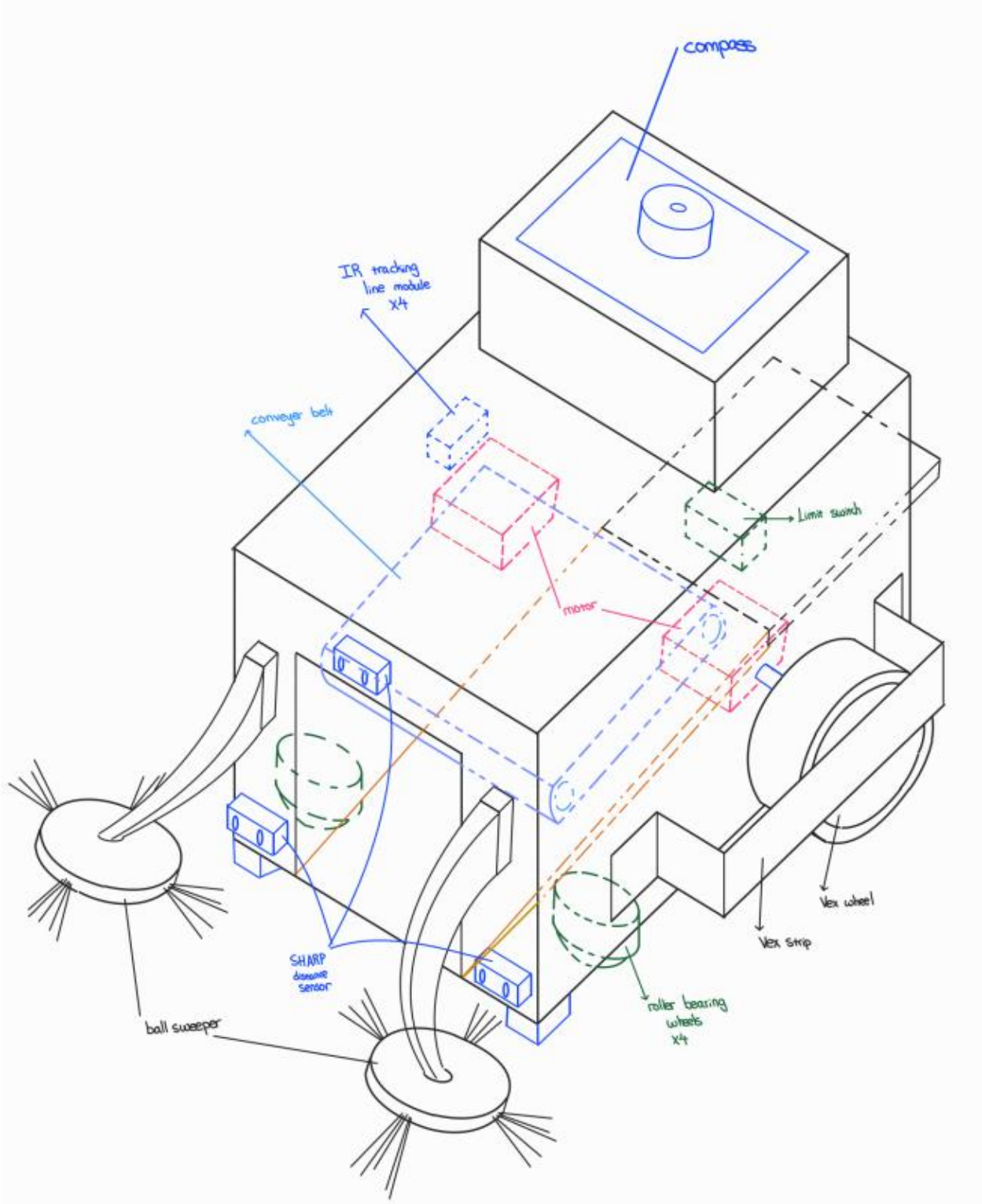


Figure 11: Overall combination of conceptual design for design idea 3

Component	Chosen Selection	Description
Ball Collector	Sweeping mechanism	This mechanism work very similarly to the road cleaning machines, where the rotating sweeper helps to direct the ball into the intake. Furthermore, it can cover a wider collection region, reducing the need for highly precise alignment during approach.
Ball Delivery	Using a conveyer belt with an angled ramp	This mechanism delivers the ball using a conveyer belt with tiny hooks to be able to grab onto the hairs of the tennis balls. With a continuous mechanism such as a conveyer belt, it can also prevent ball rollback and improve overall reliability.
Drive System	Direct rear drive system with two roller bearing wheels in front	The roller bearings help to improve manoeuvrability, as they can rotate freely with a much lesser turning radius to accommodate changes in direction. Additionally, by using a direct drive system, it has a more reliable power transfer to the wheels during operation, as such having better momentum to push the ball into the sweeping mechanism easier.
Chassis layout	Using a vex strip	The VEX strip provides a simple and lightweight structure that can be easily integrated into the chassis while still offering basic support for mounted components. However, it

		offers a much lesser protection and rigidity compared to reinforced chassis layout.
IR Sensor	4 IR Line sensor placed at the outmost exterior of the robot	This placement of the sensors allows the tracking module to be able to identify the boundary set and respond timely. It helps to support a more reliable navigation method during the competition.
Sharp Sensor	3 Sensors placed in front of the robot	The top sharp sensors are used to detect the opponent while the left and rights sensor is used for tracking the tennis ball's location. As the robots continues to sweep across the arena, the ball is detected when it goes sensing range.
Compass	Using an elevated platform to place the compass.	An elevated platform is used to mount the digital compass so that it is positioned further away from the motors and other electric components that may generate magnetic interference. This helps to improve the accuracy and stability of the compass during the robot operation, overall improving the robot's orientation for navigation and directional control.

Table 11: Analysis on overall conceptual design 3

3.9 Concept Evaluation

Evaluation Criteria	Details	Weightage (%)
Ball collection capability	Ability to pick up the tennis balls in the arena	25
Ball delivery capability	Ability to pass the ball from dispersion mechanism to the goal area	20
Search reliability	Ability to detect and move towards the ball	20
Environment Detection	Ability to detect the arena and other robots	15
Pushing Ability/Resistance	Ability to push or resist against other robots	10
Robot Agility	Ability to get towards desired location in the shortest possible time	10

Table 12: Weightage of evaluation criteria

Evaluation Criteria	**Weightage (%)**	**Criteria Rating**	**Idea 1**		**Idea 2**		**Idea 3**	
			Value	**Weighted Value**	**Value**	**Weighted Value**	**Value**	**Weighted Value**
Ball collection capability	25	Scale: 1-5 1 - Worst 5 - Best	5	125	3	75	3	75
Ball delivery capability	20		5	100	4	80	3	60
Search reliability	20		4	80	4	80	4	80
Environment Detection	15		3	45	3	45	3	45
Pushing Ability/ Resistance	10		5	50	1	10	2	20
Robot Agility	10		5	50	4	40	3	30
Total				**450**		**330**		**310**

Table 13: Total scores for the different idea drafted using criteria rating

3.10 Rationale Behind Results

Ball collection capability:

Our group goals in selection of collection strategies were to find a configuration that could easily catch the tennis balls regardless of intake alignment or travelling speed of the robot. Among the proposed concepts, the roller intake demonstrates better collection capability as compared to the other configurations, due to the inherent flexibility of rubber bands. Idea 2 had a gripper that would require the robot to align directly in front of the tennis ball while idea 3, could accidently push the ball away if not align directly, due to its high rotation speed and bristles.

Ball delivery capability:

Our group chose Idea 1, which uses a second roller with a limit switch to support a more controlled delivery process. The delivery roller can deliver the ball at a high speed, which means it can overcome gaps, if any, between the robot and the collection area. Furthermore, the limit switch enables a more precise detection of the ball's position, which tells robot if the ball is collected. Idea 2 is not as suitable as a delivery system that relies on gravity could affect the maximum height given by the competition. If the ramp is too low, the ball would not gain sufficient speed to be propelled effectively during delivery. Furthermore, in Idea 3, although the usage of a conveyer belt can be feasible, our group preferred to avoid incorporating too many mechanisms introduced by the additional complexity of the conveyer belt. Our group also tried to test several Velcro materials with the tennis ball given, but they could not effectively grip onto the ball's surface hairs, making the concept less reliable than considered.

Search reliability, Environment Detection:

The three different concepts all share the same search reliability score, as they share the same SHARP sensor, compass, and limit switch placement and arrangement. Since the sensor configuration remained unchanged across the concepts, no significant difference in search performance was expected. However, it is important to also note that this is due to the elevated positioning of the compass, where it is less affected by external interferences, making this configuration preferred within the group.

Pushing Ability/Resistance:

Push resistance of the robot is contributed largely by the drive system but also the collection and delivery systems. Typically, a 4-wheel layout will have a much better traction compared to the 2-ball bearing wheel with rear-wheel-drive setup. Furthermore, given that Idea 1 has the most robust chassis compared to the other concepts, it is expected to provide better structural durability and protection during operation, particularly when subjected to collisions or external impact in the competition arena.

Robot Agility:

The robot agility is also a good contribution to the design as it can give us the edge over our competitors during tie-breaks or higher chance of first ball guarantee before any opponent can intercept ours or get their own first ball. This cascading effect would then give us higher edge of detecting the second ball first. Although the wheel base from Idea 1 is not typically the fastest design, the gearing ratio can help increase the speed beyond the motor's nominal rpm speed, hence allowing for wider range of increased speed to operate with. Furthermore, a higher top speed can provide greater momentum or offensive strategy during collisions, hence increasing the pushing ability further in the competition arena.

Conceptual Finalization:

Therefore, seen from **Table 13**, Idea 1 accrues the highest number of points and is thus selected as the design for this project. Discussions on optimizations and challenges faced will be discussed in following chapters.

4 DETAILED MECHANICAL DESIGN

4.1 Ball Collector Mechanism

Based on the selected conceptual design, our selected ball collector design is a roller which was designed with a gear shaped profile, with rubber bands stretched across both ends overlapping two teeth-by-two teeth manner. These teeth are designed as 'hooks' to hold the rubber band tension in place, without any means of loosening rubber bands unintentionally. The roller is 8.5 cm in diameter to ensure that the robot does not exceed the 30 cm length limit, and it is positioned at 9 cm height. ***It is important to understand that these dimensions were developed based on relevant embodiment design principles with respect to the lecture notes.***

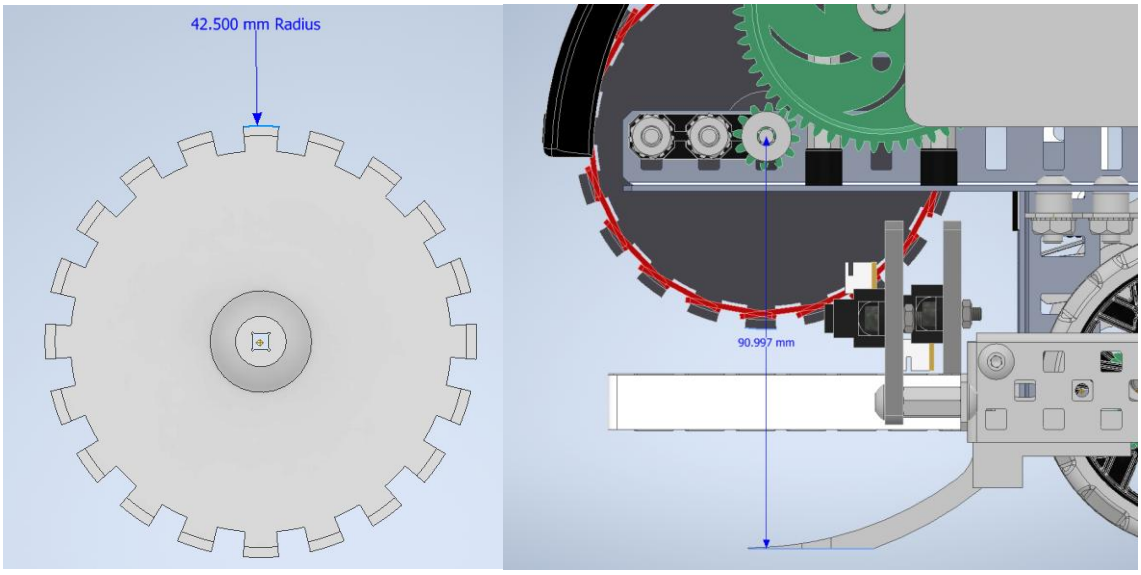


Figure 12: Dimensions of Collector Roller Design and Mounting Height

The roller also consists of custom-printed 10cm spacers, which allowed for adjustable width of rollers and applies the principle of self-help through force balance. As the rubber band tension pushes the gears profile inwards, these spacers can counteract the forces by preventing axial movements through reaction forces outwards. This prevents any accidental snapping from the gears coming together, supporting safety design for robot handling and maintenance and prevent catastrophic failure of collector.

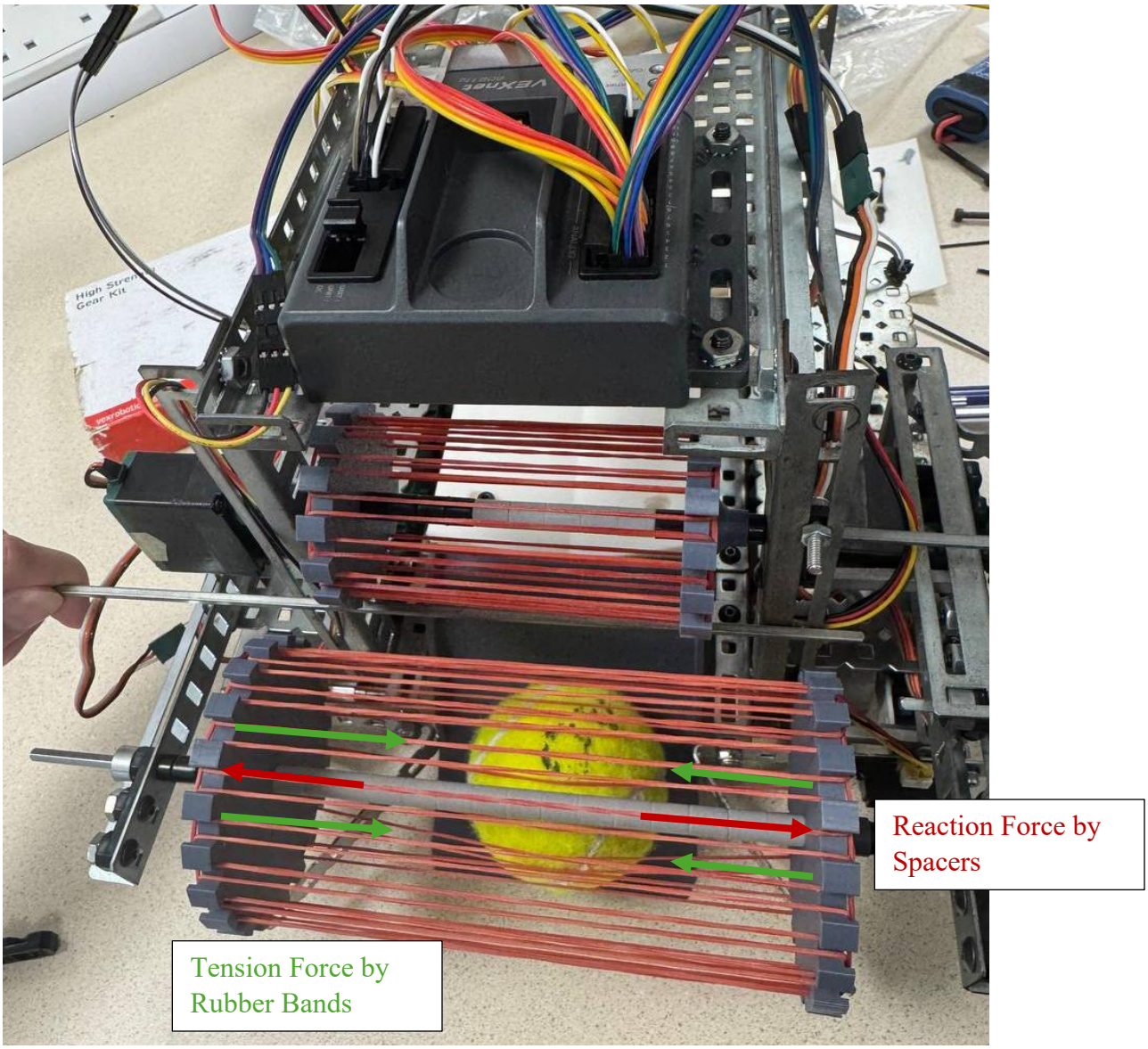


Figure 13: Mechanical Collector Roller Self-Help Principle

In terms of force transmission, the mechanism relies on the motor and gear train to transfer rotational motion effectively to main roller and subsequently to the ball. Furthermore, the ramp is designed such that is at the same elevation with the goal height, this is especially critical, as rotational force transferred into the ball must be able to climb till the minimum goal height at 8 cm.

Secondly, the principle of self-help can be seen between the ramp and the roller. The ramp is designed such that as the ball climbs, the distance between the roller and the ramp shrinks, seen from **Figure 15**. This allows the ball to become more secured during the transfer, while maintaining continuous motion towards the delivery roller. This shrinking allowance between the roller and the ramp also creates an additional squeezing effect on the ball, giving it momentum and preventing it from getting stuck as it clears the collecting region, ensuring a

reliable transfer. The tensions from the rubber bands are also weak enough such that it would not push the ball away during collection but strong enough to support the ball for it to climb the ramp as the gradient of the ramp increase with height.

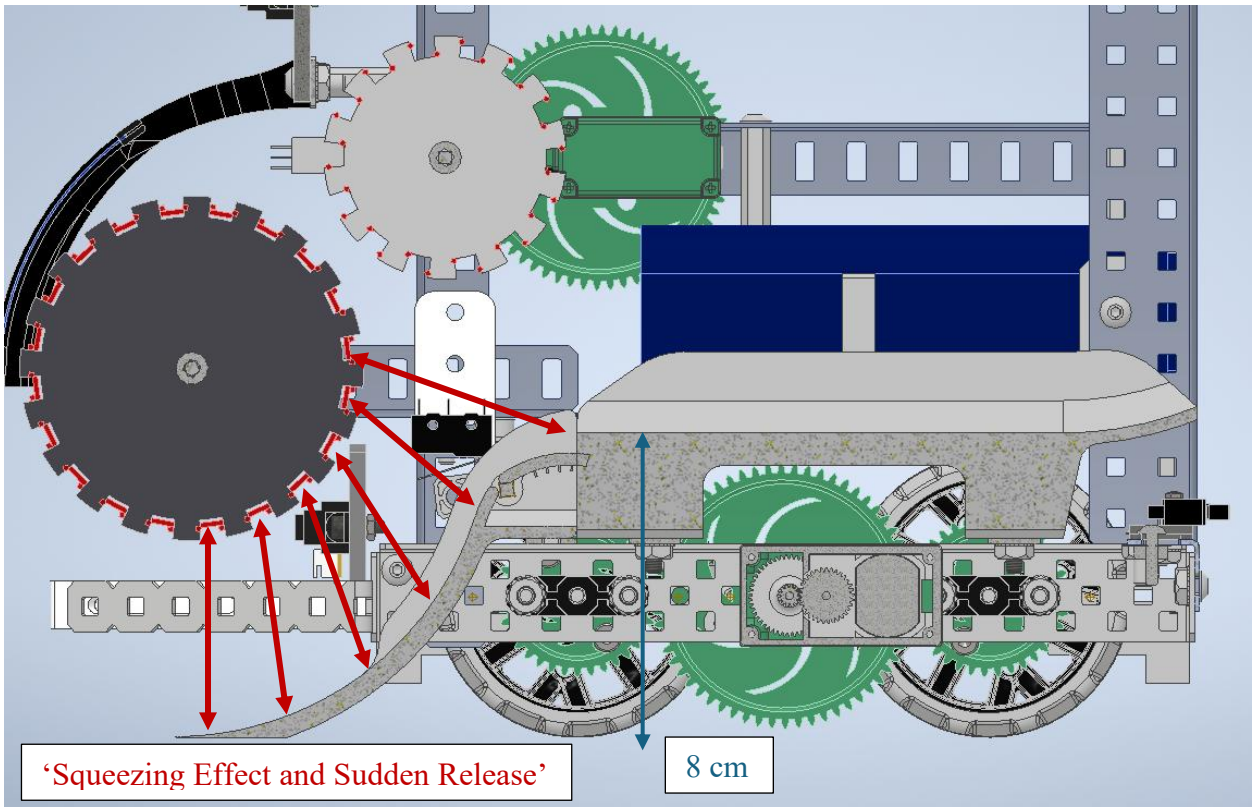


Figure 14: Illustration of Squeeze Effect & Sudden Release

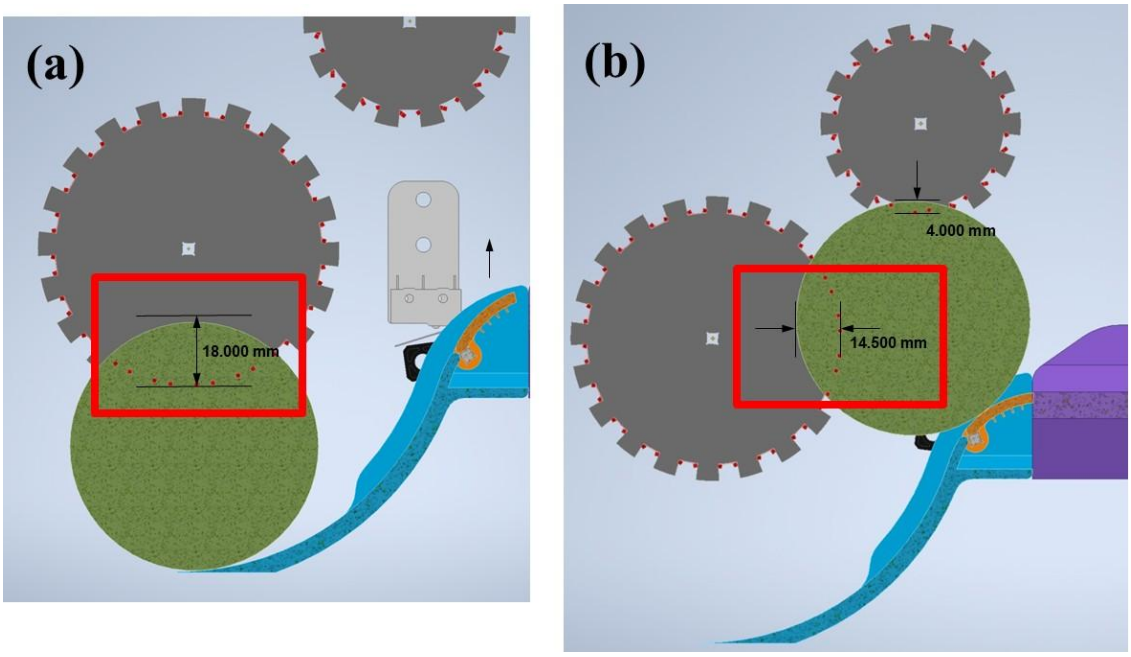


Figure 15: Allowance between roller and ramp decreases as the ball climbs from (a) to (b)

Stability was another important consideration. The ball needed to remain sufficiently controlled within the intake and transfer path to prevent loss of contact, while at the same time avoiding over compression that could either stall the roller or push the ball away due to the lack of intake space.

Overall, our continuous roller enables our robot to achieve fast ball intake upon slight contact with the rubber bands, while also lifting the ball up to the ramp's height. However, these features alone are not enough, as there can be a tendency to knock the ball away if misaligned. This is especially troublesome if the gear profile were to contact the ball instead of the rubber bands. Hence, having some form of guidance mechanism to guide the ball inwards and centre it is necessary.

In our design, V shaped guides were added to allow the ball to be guided into the collector more effectively, providing better positional control and improving stability during ball intake. This helps to aid the robot as a helping hand to successfully capture ball in the event the robot is unable to centre the ball during collection. This is further validated through testing, where the team rolled the ball from various off-centre angles to assess its ability to still engage and capture it effectively. It should be noted that the V-guide angles cannot be too steep, as that would increase longer intake duration. At the same time, exceeding the gear profile by too much will cause the ball to get jammed in between during intake. Hence, it is crucial for the vex strips to stay at a specific angle for optimal performance.

Since the angles can change drastically from impact collision by opponent or foreign objects during operation, 3D-printed supports are used to reinforce and counteract any opposing forces acting towards the V-guides, preventing further deformation that might change the desired angles.

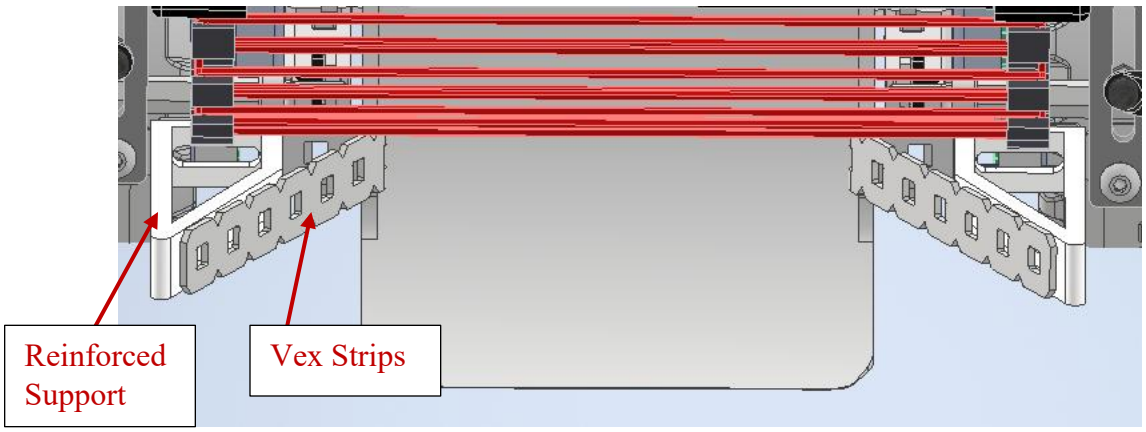


Figure 16: V-Guides Design & Reinforcement for Intake Assistance

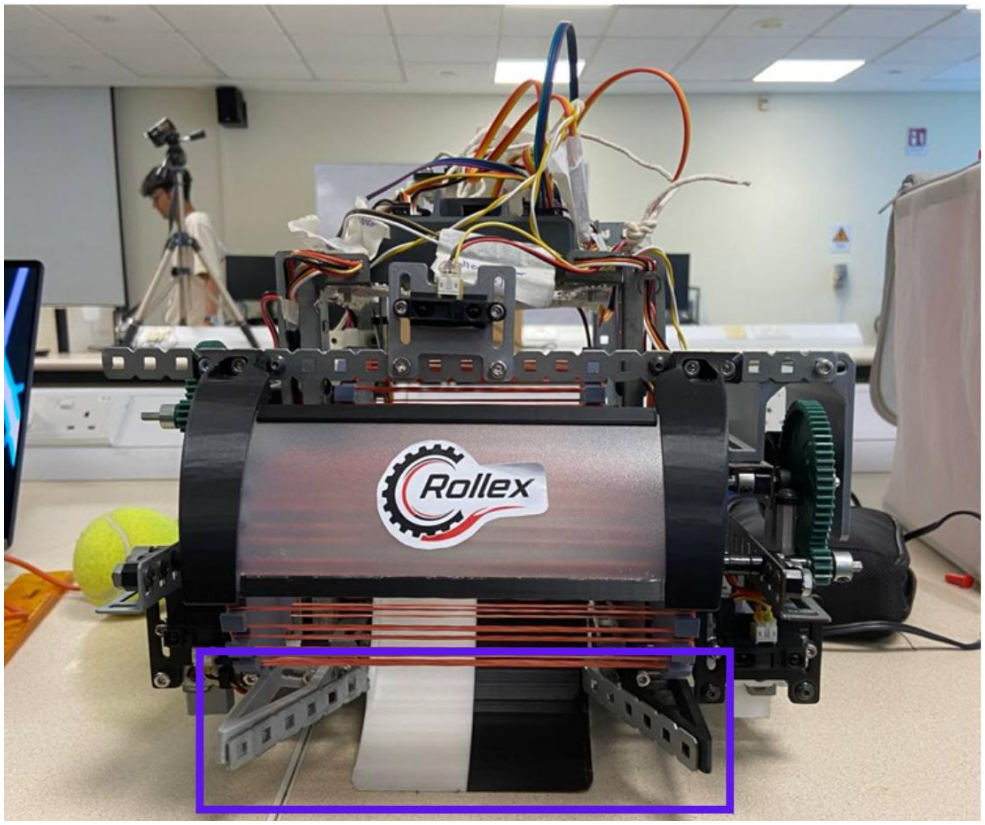


Figure 17: Front view of robot with V guide installed

This roller configuration was not developed entirely from the beginning. References were taken from past examples by past seniors and similar gear rollers stored inside of mechatronics' lab recycled parts cabinet. These examples show that gear shaped rollers are a feasible concept. However, although these rollers may have for worked well for them, their performances still must be refined and re-evaluated to fit our robot. As such, several challenges were encountered.

In our first challenge, given that the roller was directly driven by the motor, the ball did not have enough rotational momentum to overcome the ramp. Once the roller passes through the collector region, it was observed to roll on top of the main roller. Initially, we tested to roll the ball at a high speed into the roller, simulating the robot traveling; however, the same problem occurred. Since the roller was already spinning at 100% rpm based on the code, this indicated that the problem was not necessarily caused by the motor but rather due to the mechanical limitations of direct-drive setup. Therefore, our group used a gear train assembly from the motor to the roller at a gear ratio of 3:1 to improve the rotational speed. This improved RPM provided enough momentum for the ball to overcome the steep ramp and enhanced overall collection reliability.

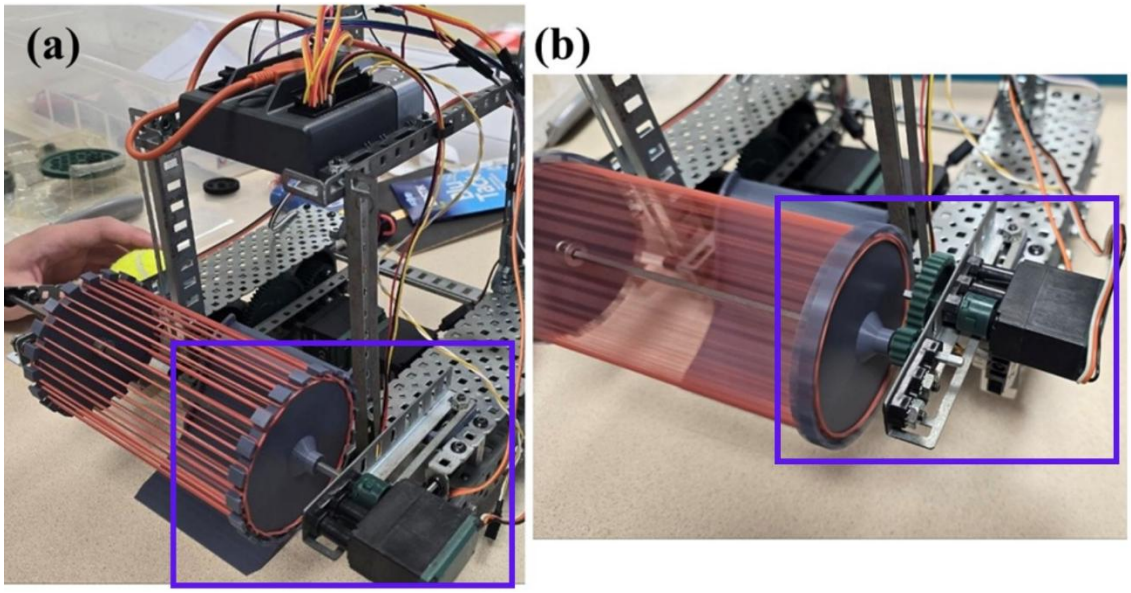


Figure 18: (a)Direct-drive configuration and (b)Gear-drive configuration

For our second challenge, we found that as the rubber bands overlapped after one another, the roller gears maybe offset at an angle or even snap together at times due to the tension. Increasing tension for our rollers are needed to provide better effort and stored energy to collect and deliver the ball quickly over a distance. Initially, we tried using collars to tighten on the square rods to prevent the roller gears from moving due to tension. However, the collar screws were insufficient due to slipping and required a different strategy. Hence, our solution was to print spacers to maintain proper separation and alignment between the two ends.

By overcoming these challenges through repeated testing, the resulting roller was able to provide consistent reliable ball collection which aligns with our group's ideal design.

4.2 Storage & Delivery System

The ball storage system is the second stage after ball is collected, where the robot will return Home for ball delivery. The core purpose of the ball storage system is to securely retain the ball once it has been successfully collected, so that it is not lost during subsequent robot motion, turning, sweeping, or return-to-home maneuvers. After the ball is detected as being inside, through the limit-switch feedback, the storage system acts as a holding mechanism that prevents the ball from rolling back out of the collector area. This is important for the robot to preserve possession of the ball while continuing to move through the arena and eventually return for delivery. In this sense, the storage system functions as a transition stage between collection and delivery, ensuring that the collected ball remains contained, stable, and ready for controlled release at the home zone.

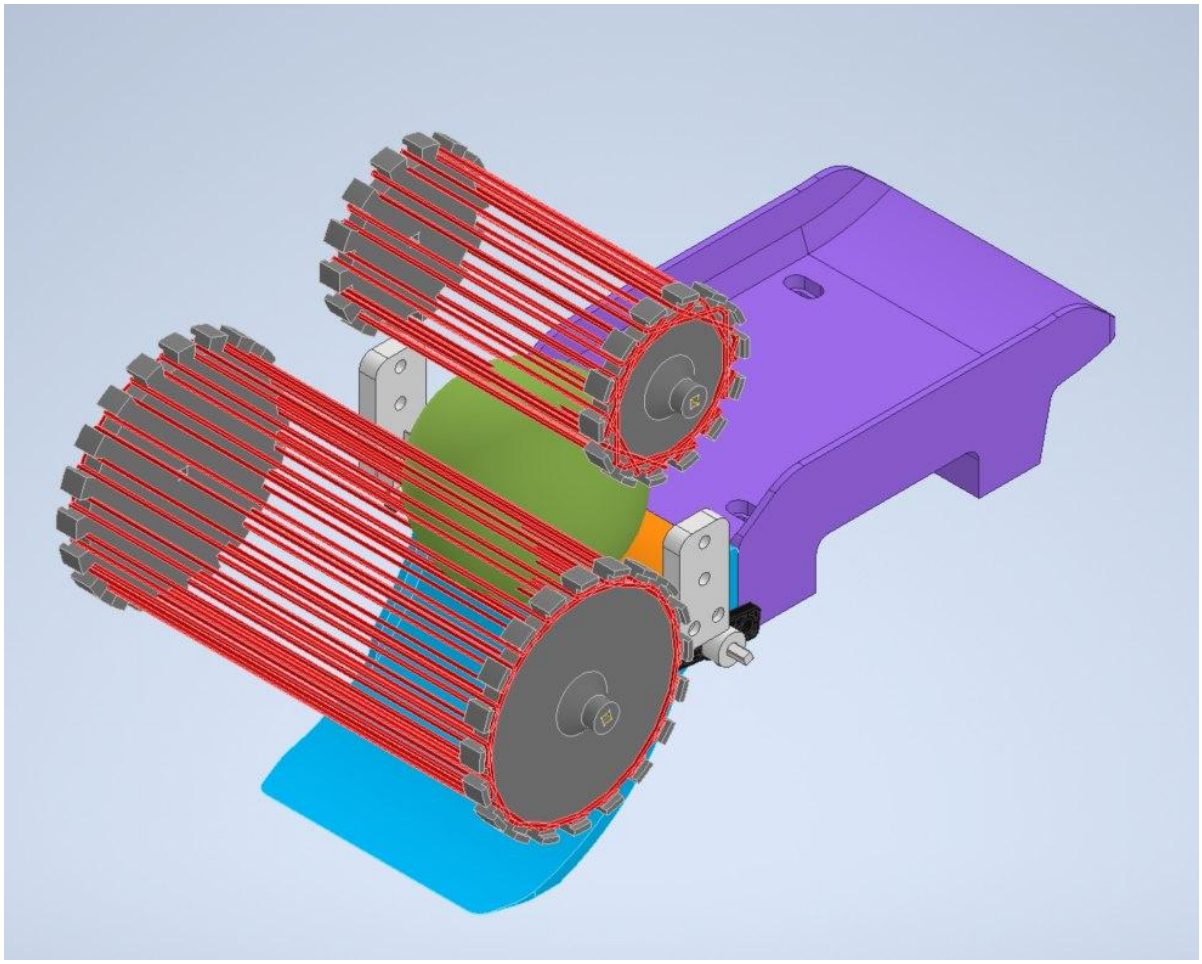


Figure 19: Overview of Storage Mechanism

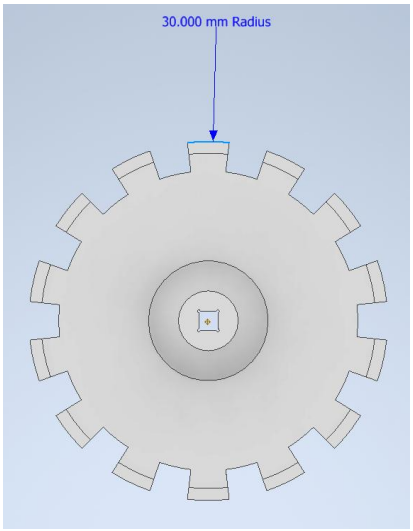


Figure 20: Roller Dimensions for Dispenser

Another smaller roller of diameter 30mm with rubber bands stretched across 3 gear-teeth in a horizontal overlapping pattern is used to both stop the ball and deliver it. Like the collector roller, there are custom-made spacers used to prevent axial movement of the gear rollers due to high tension forces of rubber bands. At the top of the ramp, there is a hinged flap that will be depressed when the ball has been collected and has sufficient tension to be ready for delivery. When the flap is depressed, it will activate a limit switch to stop the collector roller from spinning. When the delivery conditions are met, the delivery roller will push the ball down the straight ramp into the box, at the same time the limit switch is released when the ball leaves the ramp.

The figure below illustrates how the tennis ball is held in place through 3-point contact: from both the collector and dispenser rollers together with the trigger flap on the ramp. However, the depression depth of the rubber bands on the collector roller is deeper than that of the dispenser roller. This means that there are uneven tension forces on the tennis ball, and the lack of tension force on dispenser roller means that ball delivery may not be as effective as ball collection. Due to a smaller diameter size, it is expected for the dispense system to have less power than the collector, as seen by the difference of depression depth of rubber bands. Hence, our team decided to increase the tension by stretching the rubber bands over 3 teeth instead, as well as increasing the gear drive speed ratio (60:12) to much higher RPM than the collector, roughly 5 times faster than the motor's default speed. This ultimately help to compensate and provide similar power to both collection and delivery mechanism, allowing the programmers to use the same speed values for both motors.

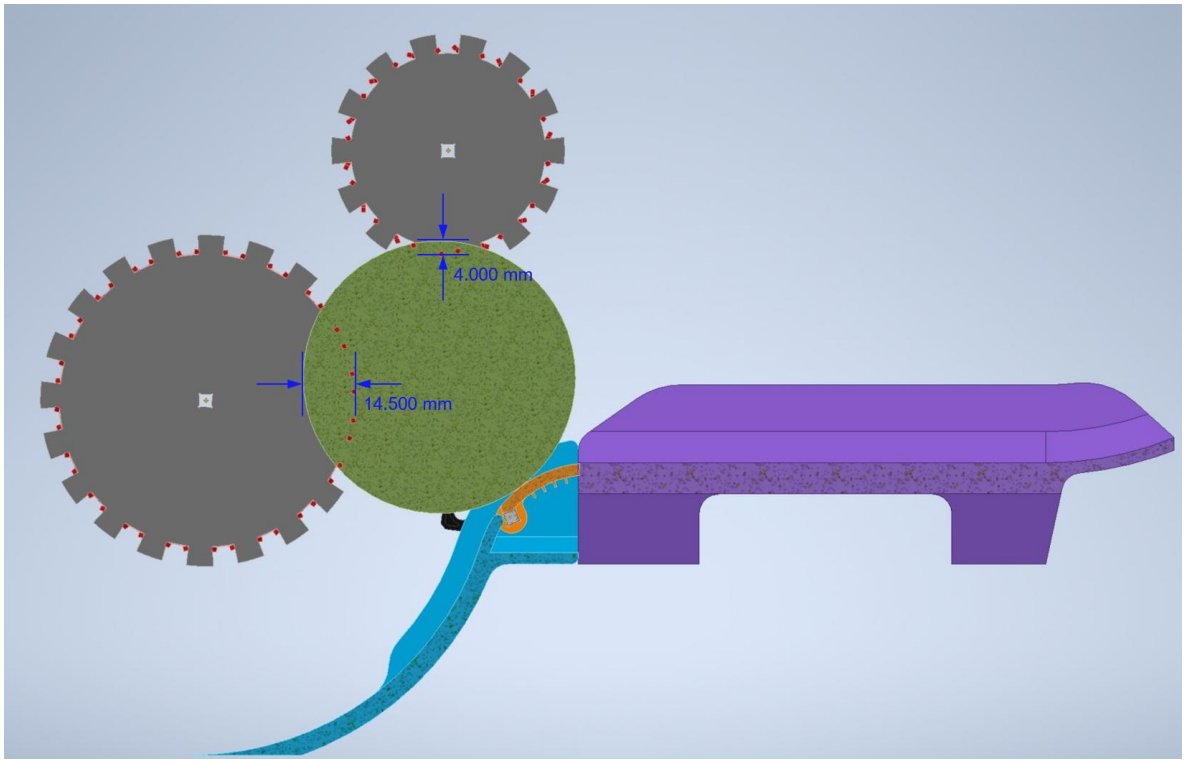


Figure 21: 3-Point Contact of Tennis Ball During Storage

The trigger flap mechanism functions like a lever, where the weight of the ball will press down onto the flap, rotating the square rod together with the bearing which presses against the inverted limit switch to actuate ball detection and stop the collection motor. This is important to prevent the collection roller from overturning continuously while in contact with the tennis ball. When that happens, there is a significant risk the ball could start moving towards between both rollers due to the high frictional torque from both rollers. However, it is also risky for the collector motor to stop immediately when detected, as the ball may not fully pressed down on the flap yet. Hence, it is important for the programmer to tune and ensure that the ball rests directly and fully closed the trigger flap without being squeezed out from overturning. In our program, the ideal time delay after detection is around 300ms to 500ms.

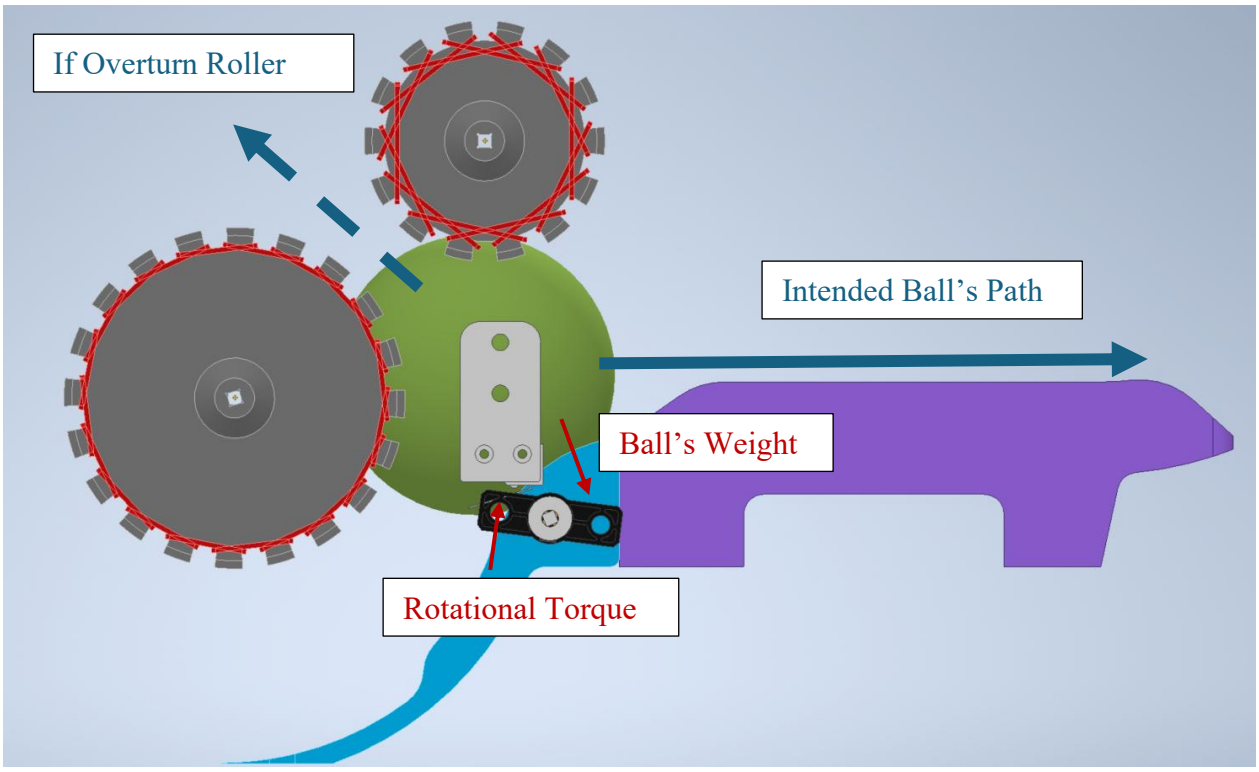


Figure 22: Trigger Flap Mechanism for Ball Detection Inside Robot

More than one digital sensor is used to support state confirmation and directional control, showing the left and right ramp limit switches that were used to confirm whether a ball has successfully travelled up the collection ramp and is secured inside the robot. This allows the software to distinguish between a ball that has merely been detected and one that has been collected. Besides redundancy in case of failure, this design helps to double the spring back force when the ball leaves the trigger flap, applying the embodiment principle of equal tasks to lift the trigger flap back up when empty more easily.

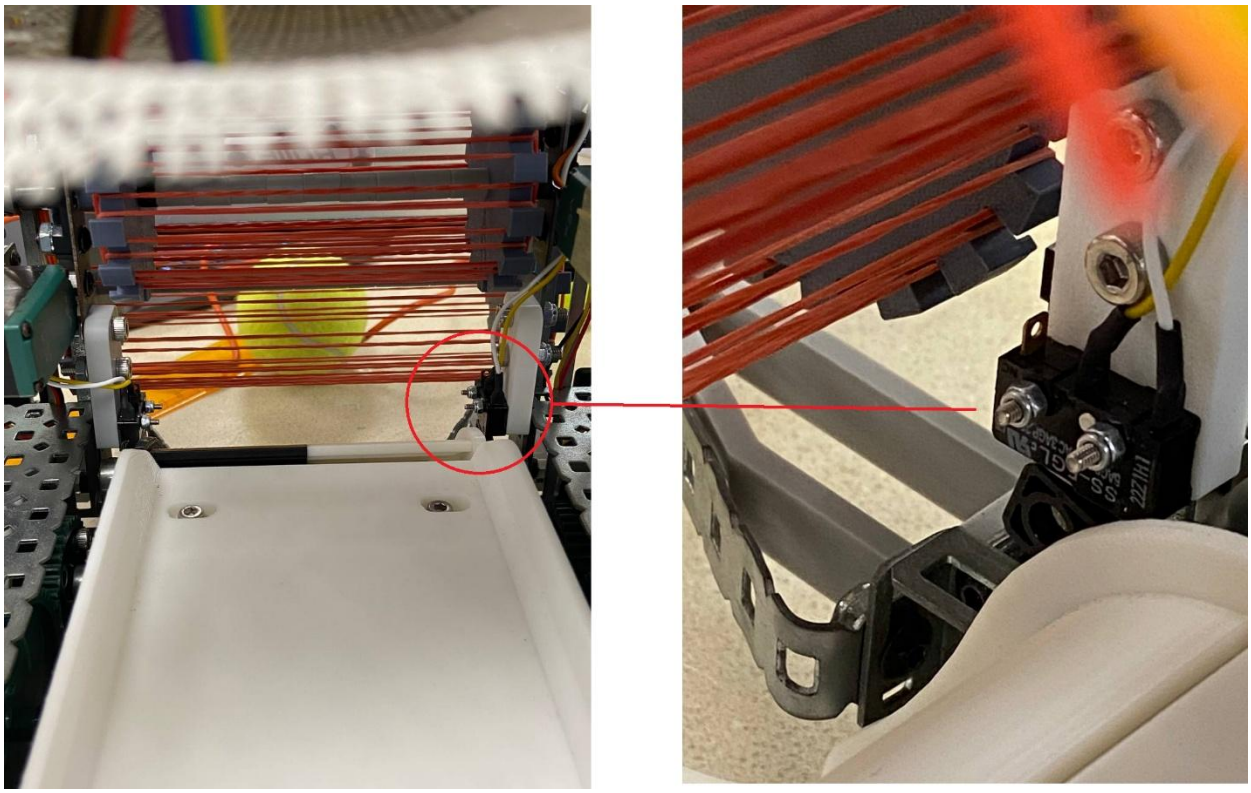


Figure 23: Ramp limit switches to confirm ball collection

The dispensing system was designed to release the stored ball in a controlled and reliable manner once the robot had successfully returned to the home zone. Its core purpose is to transfer the ball from the storage region to the collection area without requiring complex repositioning or manual intervention. A good dispensing system must therefore be simple, consistent, and strong enough to overcome any friction or obstruction that may prevent the ball from exiting smoothly.

The dispensing system also supports the software control strategy by acting as the final output stage after home detection has been confirmed. Upon reaching the delivery area, the controller activates the dispense motor for a fixed duration, allowing the ball to be pushed or guided out of the robot in a predictable manner. This controlled actuation is important because it reduces the risk of incomplete release, accidental jamming, or premature ball loss during earlier stages of motion. In other words, the dispensing system ensures that the ball is only released when the correct delivery condition has been met. This improves the reliability of the overall robot operation, since successful collection only becomes meaningful when the ball can also be deposited effectively at the intended location.

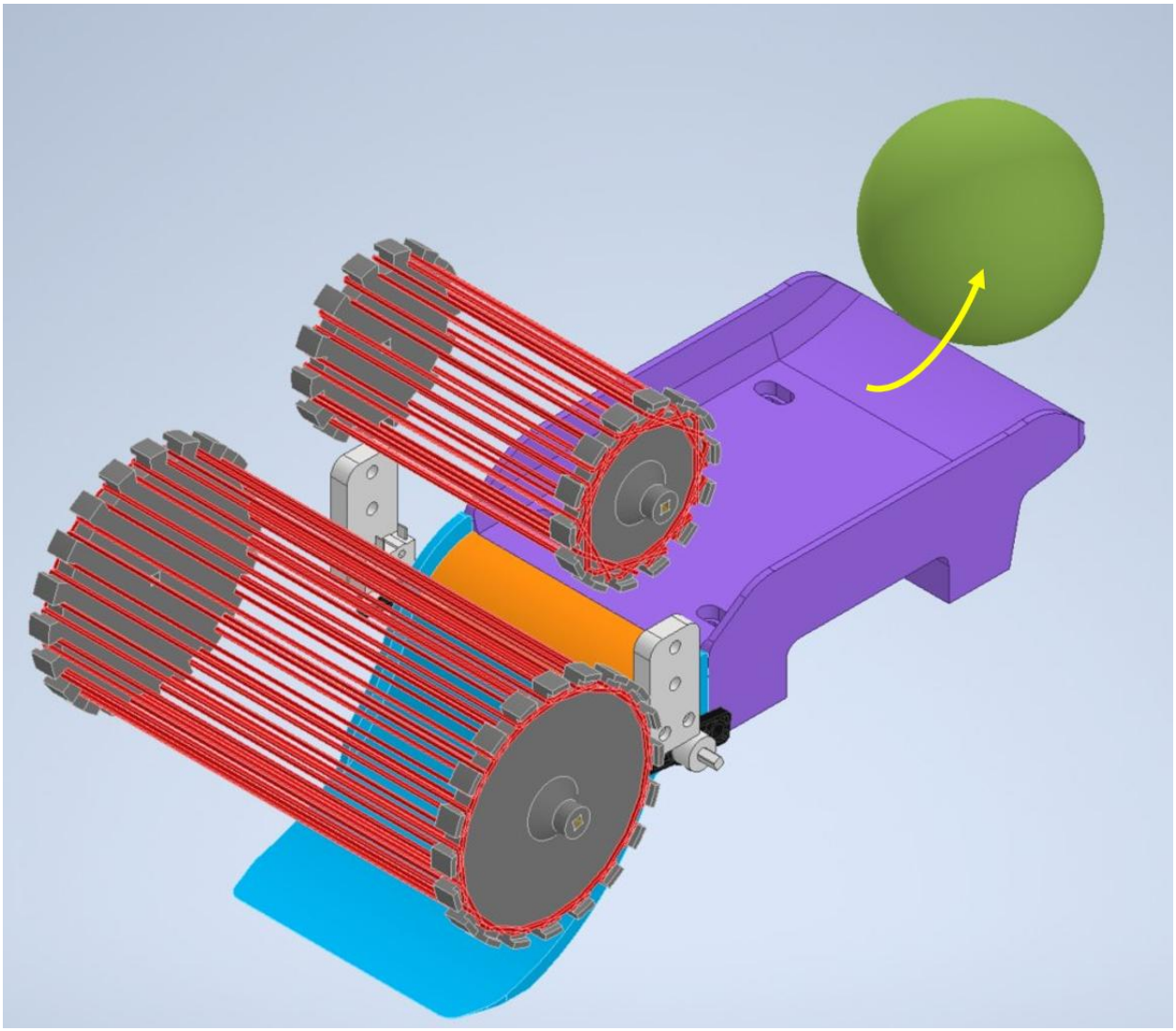


Figure 24: Overview of Ball Delivery

The curvature of the back ramp is designed to be flat and then curved upwards towards the end. Although it seems contradictory, the dispenser motor is able to accelerate from rest immediately at high speeds, together with frictional torque from rubber bands, gaining enough momentum to push the ball out at high speeds. This upwards curvature allows the ball to fall from a greater height, and overcome a significant distance of 9 to 12 cm away from the robot, giving the system an edge at delivering. This also means that the robot is able to deliver the ball by a larger range of positional error tolerance, even at angles beyond 45 degrees.

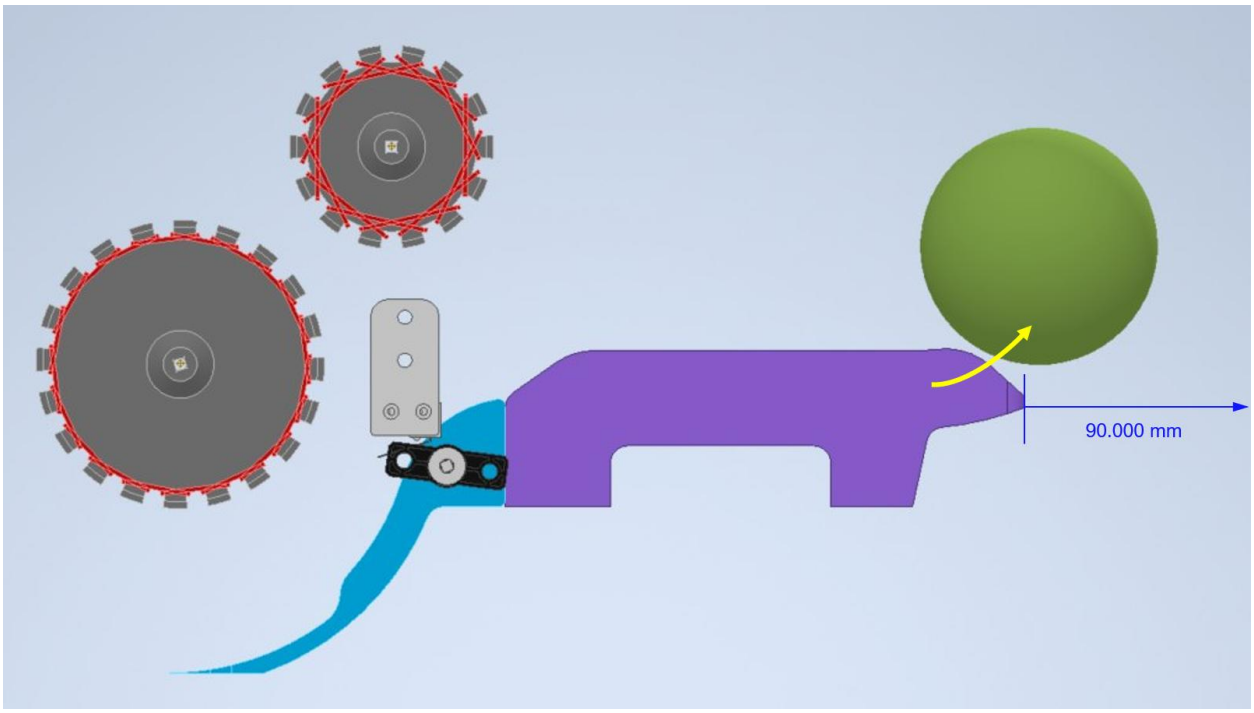


Figure 25: Ball Delivery to Overcome Gap of 9 cm

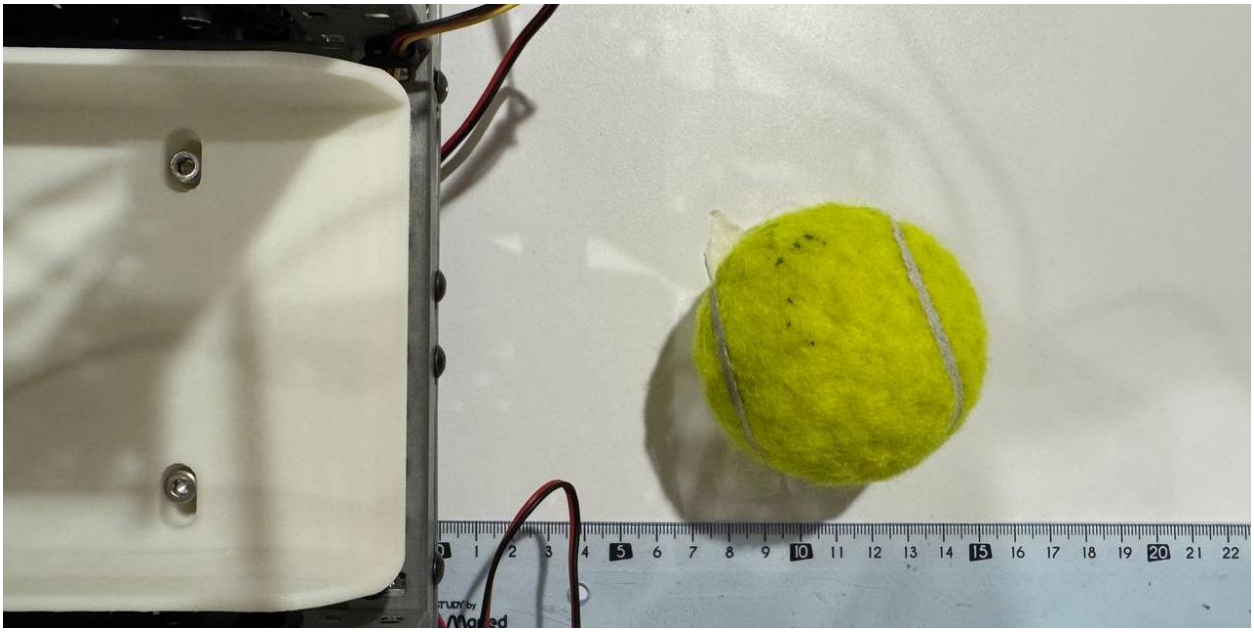


Figure 26: Empirical Testing Measured Result

4.3 Drive Base & Chassis System

The drive base was designed with the aim of achieving stable motion, reliable traction, and sufficient speed for efficient performance in the arena. We chose to use a four-wheel drive base rather than a two-wheel drive configuration because it provides better overall balance, weight distribution, and contact with the ground. With four points of contact, the robot is less likely to tilt, wobble, or lose stability during rapid acceleration, turning, or contact with the arena boundary. In contrast, a two-wheel drive base would have been simpler, but it means greater dependence on fewer contact points for traction and stability, making the robot more vulnerable to slipping, uneven turning, and reduced control when the centre of mass shifts. We foresee that having two wheels would make the robot tendency to swerve when moving straight much higher, where as four wheels provides more consistent and desired drive direction.

Additionally, the four-wheel drive base also helped improve traction and power transfer to the ground. With more wheels are engaged along the surface, the available driving force can be distributed more effectively, reducing the chance of wheel slip and allowing the robot to move more efficiently and much harder to be pushed against upon opponent collision. This was beneficial not only during forward search and ball collection, but also during reverse motion when returning home for delivery.

However, one limitation is that the robot has added static friction, which makes it vulnerable to jerking or erratic vibration during point turns, which creates reading spikes from sensors and higher wear and tear, and higher risks of screws and nuts loosening. This means that the drive base assembly must be properly secured with lock nuts to prevent this risk, while also ensuring that all 4 wheels are fully in contact with the ground. Our team spent many hours mitigating these limitations to ensure that all the drive system are assembled as closely similar as possible, and eliminate any ‘rocking’ motion from uneven wheel contact with the floor.

Another important design consideration was manoeuvrability. Although four-wheel drive can sometimes increase turning resistance compared to a simpler two-wheel system, the chosen base was still able to achieve a sufficiently tight turning behaviour through differential drive control, where the left and right sides could rotate at different speeds or opposite directions. This allowed the robot to pivot, sweep for balls, and make directional corrections with reasonable accuracy. The wider footprint of the four-wheel base also helped stabilise the robot during turning, especially when the collection mechanism was active or when the robot was carrying a ball. As a result, the design was not only fast in straight-line motion, but also stable and controllable during repeated turning and search maneuvers.

Overall, the drive base was designed as a balance between speed, stability, traction, and control. The use of a four-wheel drive configuration provided a more stable and reliable mechanical foundation than a simpler two-wheel alternative, while the use of a speed-increasing gear ratio enabled faster navigation and more efficient arena coverage. Together, these design decisions supported the robot’s intended strategy of rapid search, controlled collection, and dependable return to the home zone.

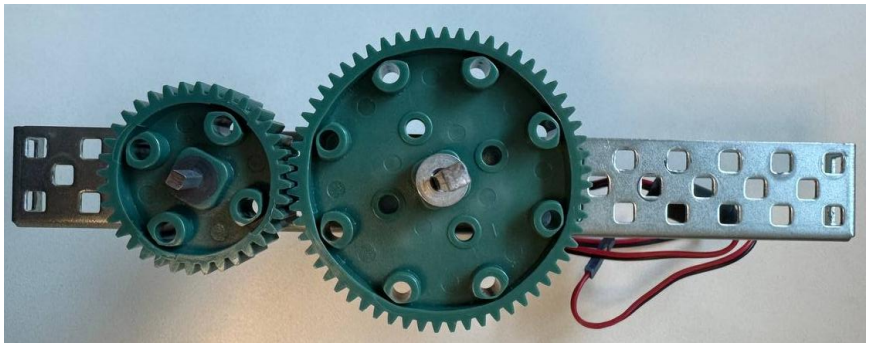


Figure 27: Gear Train Ratio Configuration for High Speed

To improve speed performance, a geared transmission was used to increase the wheel rotational speed relative to the motor output. The decision to use a gear ratio for higher speed was driven by the need for the robot to cover search lanes quickly and reduce wasted time travelling between ball-detection zones and the delivery area. A faster drive base allows the robot to scan the arena more efficiently, reach target areas sooner, and complete more actions within a limited run time. By selecting a speed-increasing gear ratio, the motor's output speed at the wheels was amplified, allowing the robot to travel faster than it would under direct drive or a torque-prioritised ratio. This was suitable because the robot's task required quick navigation rather than pushing heavy loads or overcoming extreme resistance.

However, increasing speed through gearing required careful consideration of the trade-off between speed and torque. A higher-speed ratio reduces the torque available at the wheels, which can make the robot less powerful during acceleration, climbing over resistance, or recovery from collisions. For this reason, the drive base design had to strike a practical balance to be fast enough to search and reposition efficiently, but not so over-geared that it became unstable, underpowered, or difficult to control. The four-wheel drive arrangement helped compensate for this trade-off to some extent, because the improved traction allowed the robot to make better use of the available torque at the wheels. Hence, although the gear ratio prioritised speed, the four-wheel base helped preserve usable motion quality and reduce losses from slipping.

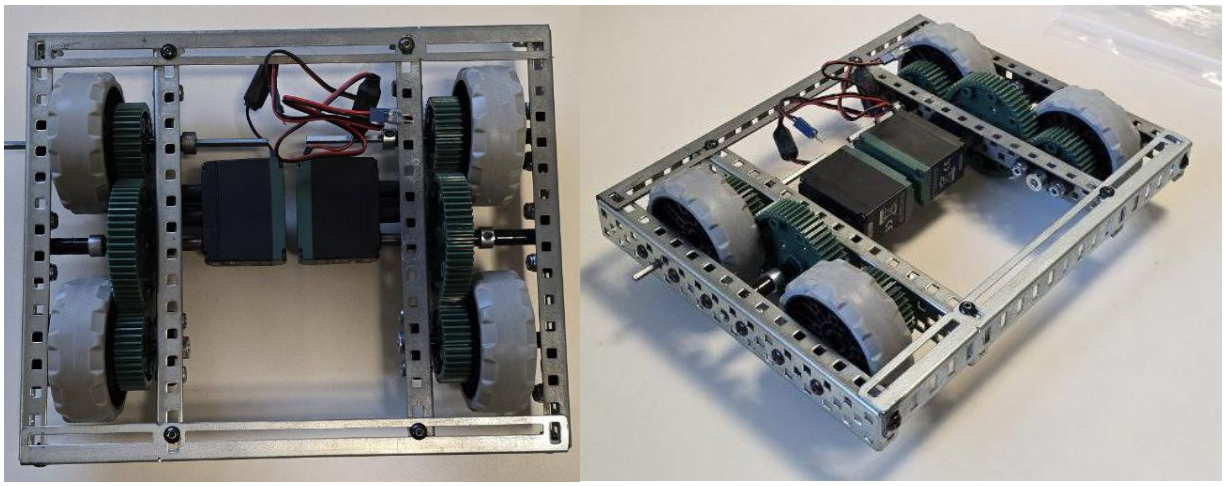


Figure 28: Drive Base Design Iteration 1

Our first design iteration consisted of 4-wheel drive using high strength gear ratio of 60T to 36T on each side. It is designed as a square based, wider in width than length for smoother point turning, and ensures all rotating elements are well-protected against external collisions. This drive design is inspired in reference to our literature review from Vex competitions overseas, where their drive base design often incorporated tank-drive design through protected wheels and gearing systems.

At full speed, the robot is measured to reach 90cm/s, while at half speed, the robot can reach 40cm/s. However, running the robot at top speed is not recommended as there is large inertia force that could cause the robot to hop and jerk, especially when sudden change of motion during turning, traversal or collision. By tuning the speed of the robot, the comfortable range of speed is roughly 35cm/s to 50cm/s on average. Despite its high speed and low torque configuration, the robot was still able to generate large amount of pushback under a short period of time. Through our testing, the robot was able to push back the storage box of heavy components (estimated weight of typical opponent robot) by 15 cm under 2 seconds at max speed. Although the speed is reduced, the eventual full build of the robot would add significant weights to the wheels to generate higher traction force that could aid in its pushing force further.

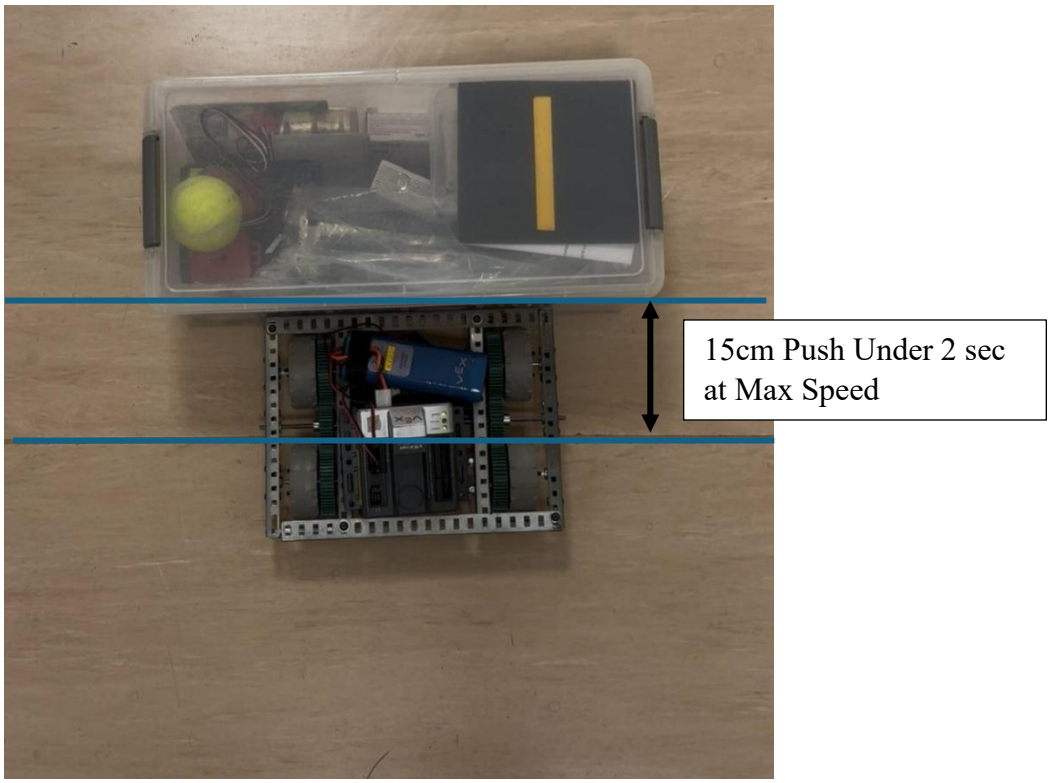


Figure 29: Pushback Test of Drive Base

However, this initial design raised many problems during our testing and physical assembly. Firstly, the fixture is prone to flexing and deformation, making the drive base highly unstable. The square frame tightened on corners means that it is prone to bending about diagonally, flexing the wheels upwards and hence unable to ensure full contact of the wheels. This means that the drive system is prone to swerving, unpredictable behaviour, and reduced manoeuvrability. To overcome

this limitation, it is important to further increase the rigidity via more constraints and fixtures on the frame. For example, using a cross-brace to reinforce the chassis frame and minimize flexing.

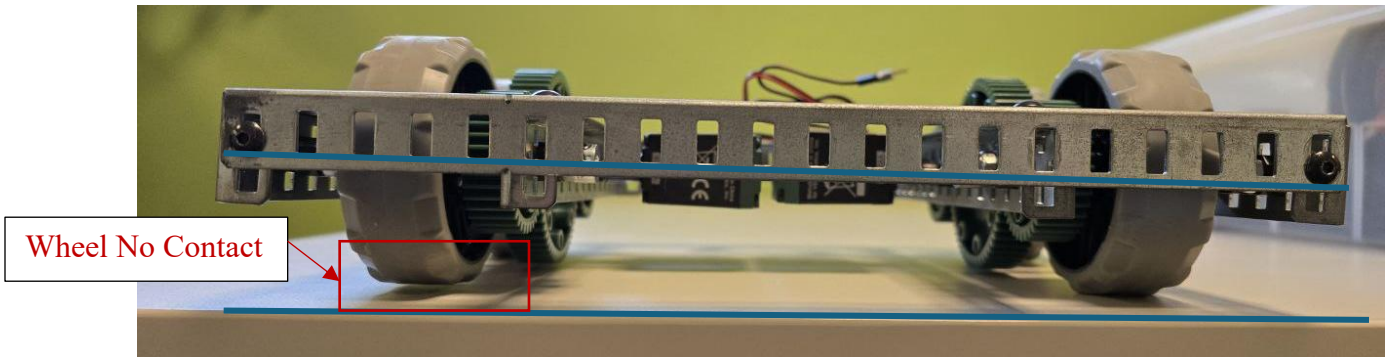


Figure 30: Visible Flexing of Chassis System

Another problem we encountered was the accumulative backlash between the rotating elements of wheels, gears, and bushings. This backlash effect can make the robot wheel turn 45-degree angle freely without actuating the motors. Although it may seem like a minor issue, it can have a detrimental effect on our drive system, where both sides of the wheels start at differently. This could mean that the robot would swerve during its forward traversal. To overcome this limitation, our team decided to use screw joints to directly fix the wheels to the gears, ensuring that both elements rotate with each other. This feature helped to eliminate any backlash or misalignment between the wheels and gears. However, there is still minimal backlash from the internal gearbox of the motors. This was later mitigated by manual handling of the robot, moving the wheels backward until the motors are actuated to eliminate free movement and allow a straighter motion during drive.



Figure 31: Screw Joint of Wheels and Gears

For our second design iteration, we decided to re-design the drive base into modular units instead of an entire piece. For this design, the bearing system is similar to the previous iteration, except with improved bearing mounting positions and collar placements. To prevent any free lateral movement, each axle requires tightened packed spacers & collar along the square rod. Each axle for both the driver and driven gear consists of two bearings, two collars and a short spacer. The

bearings help to stabilise and reduce friction of the rotating axle. The short spacer prevents any contact between the frame and rotating gears which could induce noise or friction, while also ensuring that all gears' surfaces are flushed together for proper meshing. Two collars are tightened whereby one of them prevents the gear and wheel from moving axially to left or right, together with the spacer, holding the wheel in place. The other collar helps to prevent the axial movement of the axle (square rod) itself from moving left or right, together with the spacer to hold it in place.

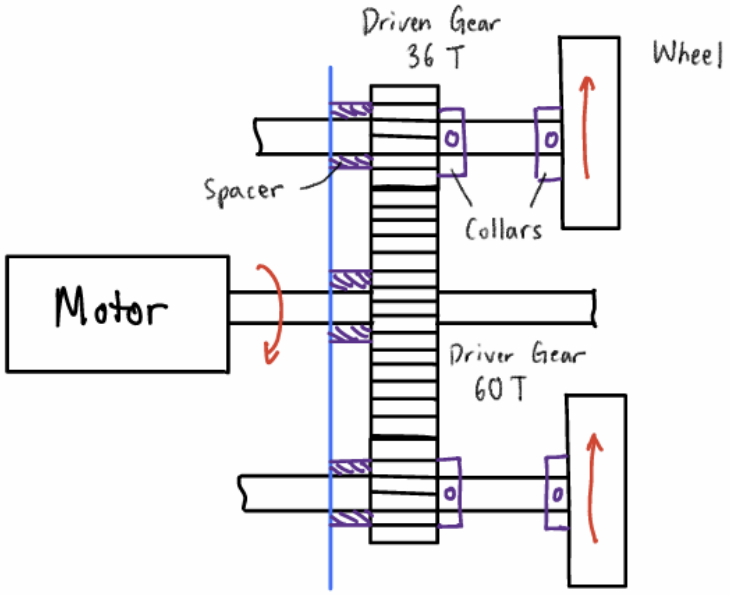


Figure 32: Drive Design of Motor, Gears, Bearings, Load

Overall, this chassis system design also ensures ease of assembly and disassembly, maintenance and troubleshooting in a short period of time and minimal effort. The user only needs to loosen the collars and can remove the axle rod from one end of the frame, accessing the components inside the drive system. Once done, the user can put the axle back through one side of the frame, and stuff the components back onto the axle rod, and finally tightening the collars.

The frame that makes up the chassis is also redesigned to have two vex metal held apart by spacers to house the bearings, wheels, gears, the entire drive system module. This is because the spacers can be made of different lengths, hence offering versatility and adjustable width during assembly, allowing us to test and try various sizes for the ideal width. By using both aluminium and short plastic spacers, the module's width can be adjusted to the desired dimensions to house the drive system and line sensors within the frame at a minimal distance to conserve space. Moreover, the spacers embody the principle of direct force transmission, whereby the frame can be tightly secured together, further enhancing the rigidity of the chassis and reducing deformation or flexing.

This design also includes the encoder, which the team initially wanted to have for extra capability of measuring the displacement of the wheel for accurate distance.

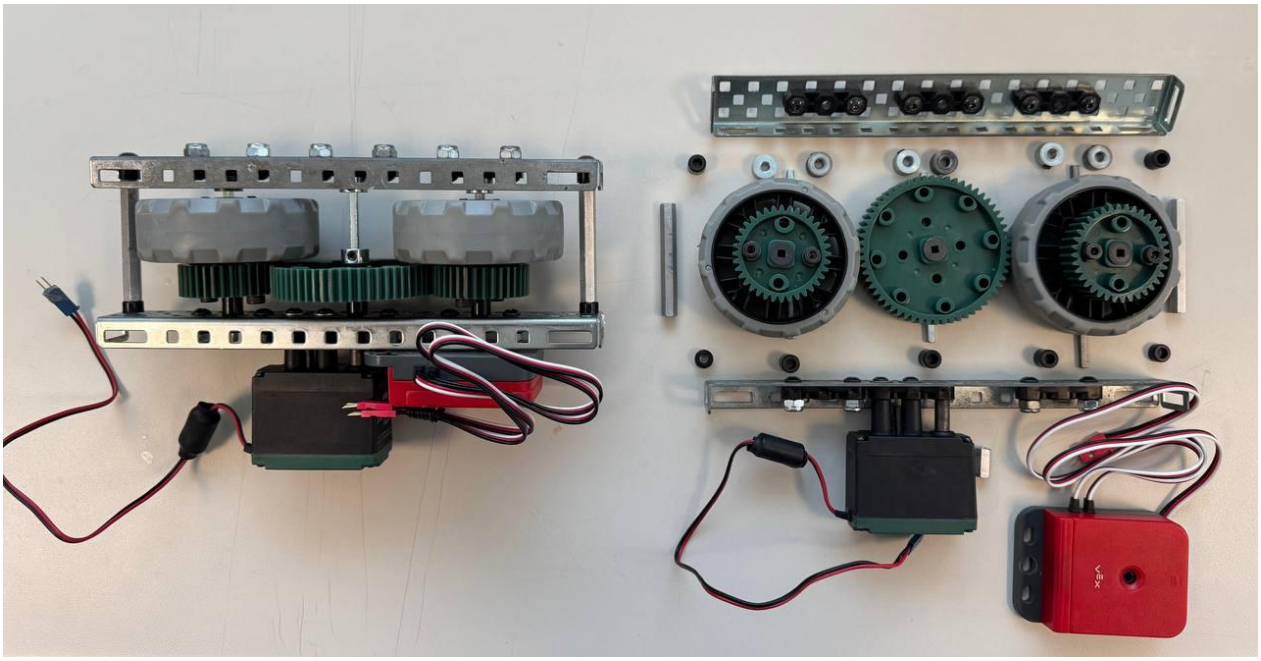


Figure 33: 'Exploded' View of the Modular Drive System (Iteration 2)

Since the encoder is not mirrored, the encoder needs to be flipped and be asymmetric. Otherwise, the rubber casing of wires will rub against the floor and damage the component.

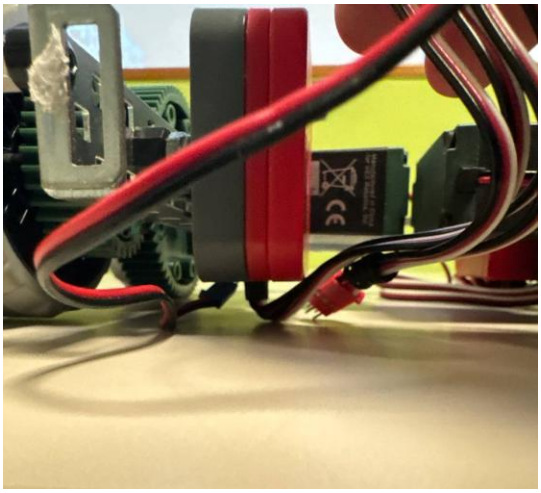


Figure 34: Encoder Wiring Contact with Floor When Mirrored

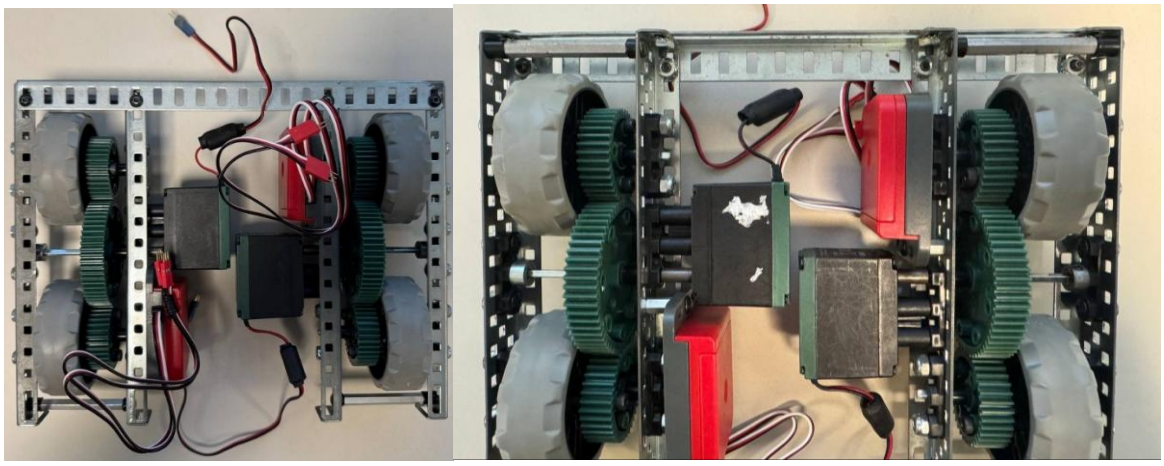


Figure 35: Drive Base Design Iteration 2

Structurally, the drive base was designed to be compact yet strong enough to support the motors, wheels, sensors, and controller without visible flexing. A low and balanced layout was preferred so that the centre of gravity remained close to the ground, improving stability during high-speed motion and sudden turns. The positioning of the wheels and motors was also considered in relation to ease of assembly, servicing, and access to internal components. This is important because a good drive base should not only perform well in theory, but also be practical to build, maintain, and troubleshoot. A more serviceable structure reduces the time needed to replace components, adjust wiring, or correct mechanical issues during testing.

However, while this design iteration is functional on its own, it still raises issues for mounting sensors and mechanical ramp for ball collection. Because of the encoders, there is insufficient space at the centre to house mechanical ramp needed to collect the ball, which means that the ramp's slope gradient drastically increases with minimal space given. Eventually, our team decided to remove the encoders and align the motors to be symmetrical, giving more space for the ramp to have a gentle slope. This is because the encoders were not as useful as we thought since the wheels can be prone to slip and cause the displacement values to be inaccurate and useless. Furthermore, there wasn't a need for precise location or distance for the robot to travel for its search algorithm that we thought could be beneficial, and many other external factors like backlash or collision could cause the encoders to misfunction. Hence, it was better to revert to a simpler design of motors only, while conserving space. Moreover, we decided to use thinner driver gears and shorter spacers to make room for line sensors within the chassis. The driven gears remain as thick high strength ones, to provide extra width to connect with the wheels by screw joints. In this way, our team was able to make modular changes with little effort, without needing to change the entire frame or disassemble entirely components.

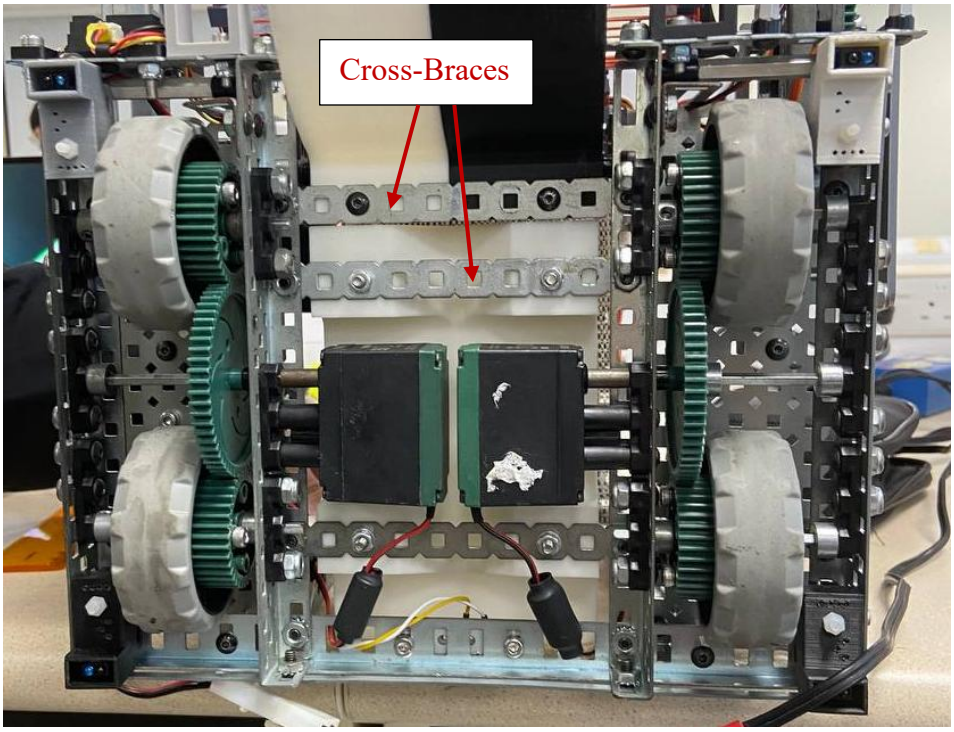


Figure 36: Final Design Iteration of Drive Base (Bottom View)

Our final design of the drive base retained all the strengths of the previous designs, while improving on their limitations. This design allows the chassis and drive system to be modular, conserve space for additional mechanical parts, well-protected frame to house line sensors and rotating elements, screw joints to minimise backlash, ease of assembly and maintenance by collars. Although the modular drive is already highly rigid and secured, we decided to further increase the durability and rigidity of the overall drive base by adding cross-braces, which also help secured the 3D-printed ramps for ball collection and delivery. This is because the drive base is a C-shape, which means it is prone to flex vertically upwards and downwards on both sides. Hence, adding these braces help to minimise this deformation and increase the overall strength of the drive base's rigidity.

4.4 Vehicle Structural Frame

For our vehicle overall structure, we have designed the frame to compartmentalise five various sections over three sections or 'layers'. The first layer at the lowest is the drive base and chassis system as discussed before, housing the drive system, sensors and motors. The second layer on top is split into three sections, the left and right side where base plates are mounted over the wheels, and the middle for the ball's trajectory for collection and delivery. This means the middle section is where the mechanical rollers and ramp will be installed for managing the ball. The base plates on the side are meant to carry the heaviest components such as the battery, additional motors, gears, and other structural members, allowing extra weight to be distributed directly to the wheels for more traction force to provide greater pushing power by friction. Finally, the topmost layer is to house any components that require easy accessibility by robot handler, such as the Vex Controller, Compass, Handles, Wiring, LEDs, etc.

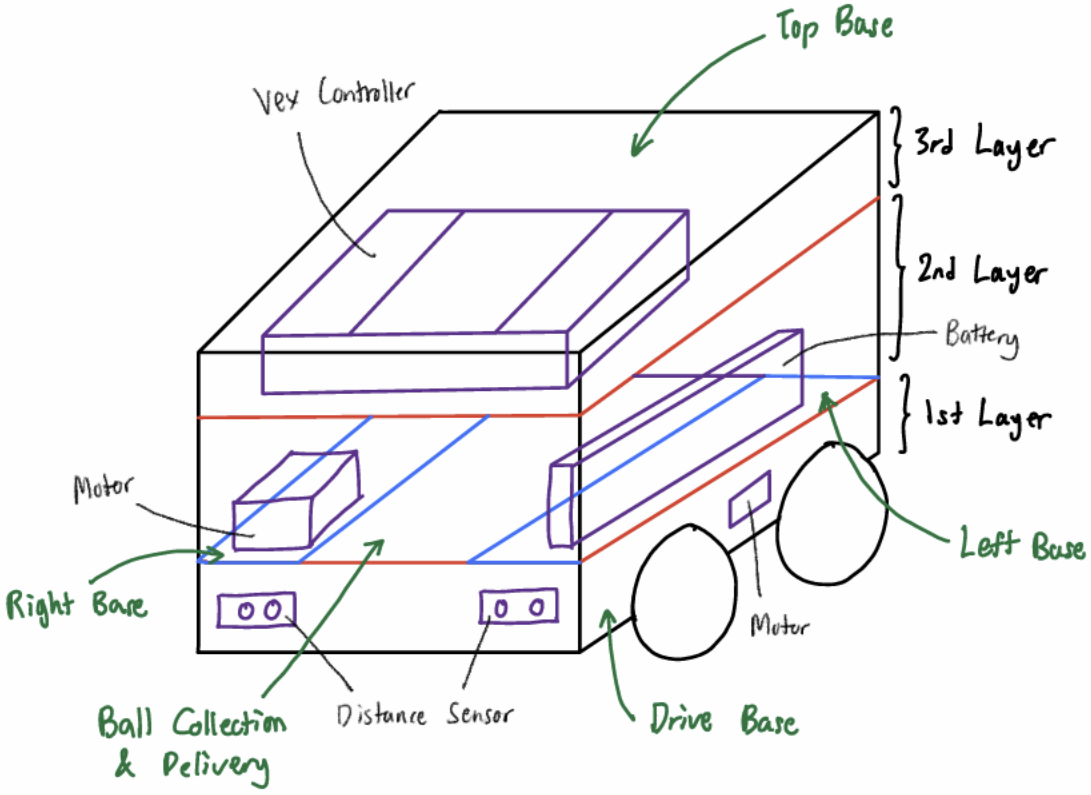


Figure 37: Compartmentalization Sketch of Robot's Frame

The vertical pillars are then mounted on all 4 corners in between base plates and the drive base frames. They apply the principle of division of tasks, where the same members can provide multiple functionalities to complement and value-add to each other. Firstly, the vertical is securely tightened by screws and 'wedged' between the base plates and drive base frame, supporting the base plates by the corner on one side without the need for attaching screws. At the same time, this creates a self-balancing force on each other, which further reinforces the rigidity of the base plates and the pillars, applying the principle of self-help. Furthermore, the pillars act as connection points to the topmost layer, which can act as pathways for wires to run through externally without interfering with any rotating elements or the ball collection/delivery. At the same time, it makes assembly and troubleshooting on wiring more visible and easier given external access. Mechanical clips are used to firmly hold wires onto these pillars, protecting and prevent accidental damages from collision or entanglement. Lastly, these pillars can be tightly connected to each other by topmost layer, further strengthening the overall robot frame, and reducing the flexing tendency of C-shape drive base.

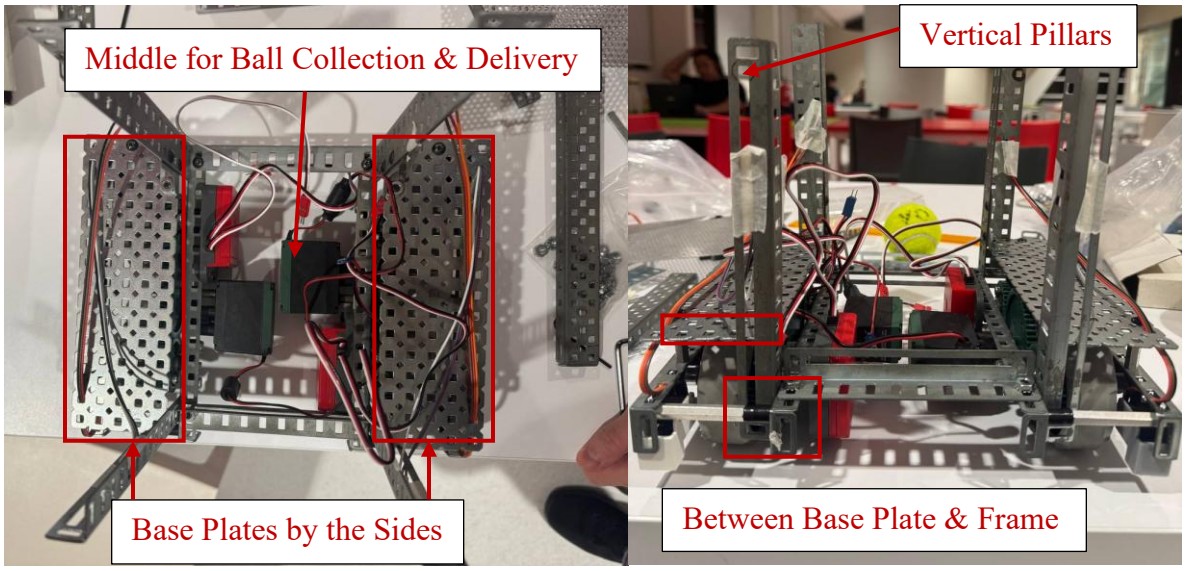


Figure 38: Vertical Pillars & Side Plates

As seen in the figures below, the top most layer which houses the Vex Controller enables complete accessibility to any wires running through the four vertical pillars, while providing ease of access and clarity to the operator. The back vertical pillars are mounted with L-plates as shown since there was insufficient space to wedge between the drive base frame, but still provided great reinforcement to the overall structure.

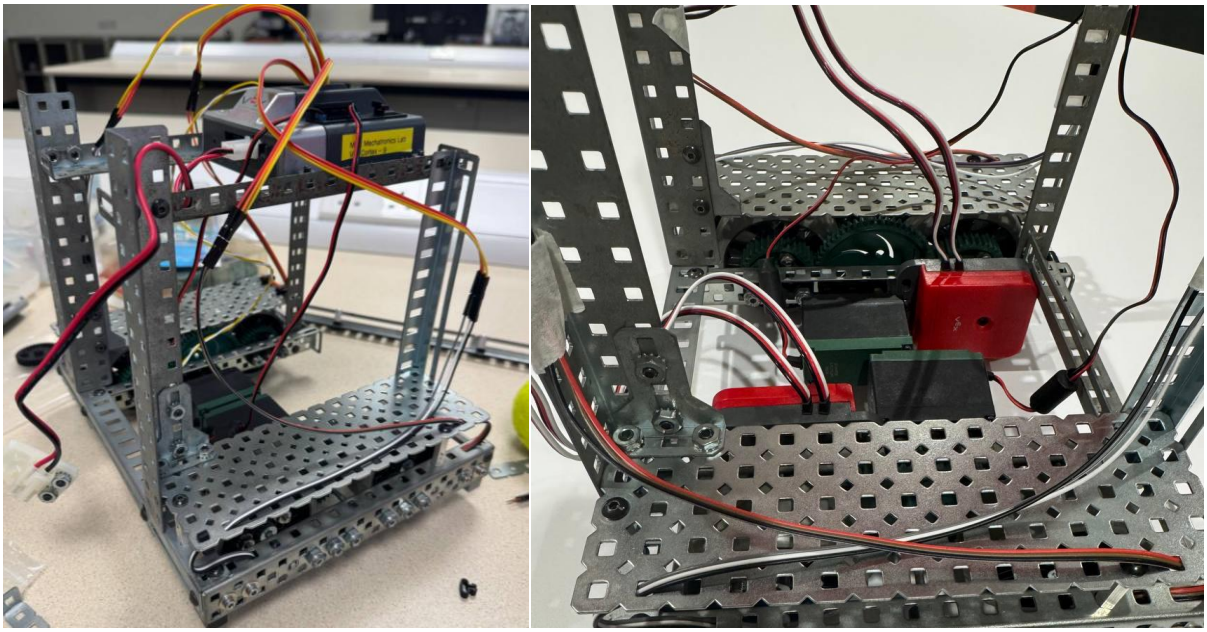


Figure 39: Top Mount & Close-up View of Mounting Pillars

The mounting of the dispenser and collector motor is through an elevated structural arm, to house both the gear train and the motor in place. Given that the dispenser roller is smaller in size than the collector roller, it was necessary to increase the rubber band tension and the rotational speed to achieve the same power as the collector roller. The arm is elevated at a height to accommodate the axles for motor, gears and roller, allowing the dispenser to be above the ball on the ramp upon contact. Collars are then used to hold the gears in position and prevent any axial free movement. The short Vex plate is installed to the structural arm and the driver gear, so that the motor's axle rod does not move out freely and secured in place, enabling the gears to mesh properly in place during any vibration or collision.

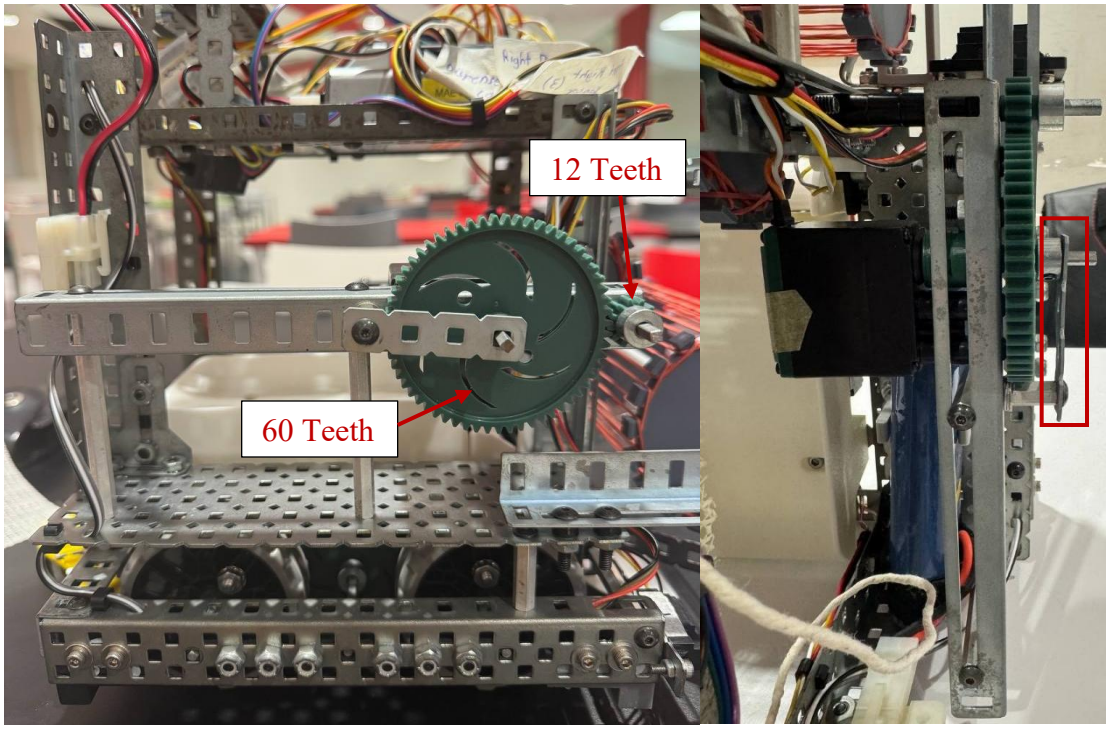


Figure 40: Right Side View of Gear Drive for Dispense Motor

For the collector motor, the gear drive required is lower than the dispenser motor due to a larger roller diameter which increased the tension, hence sufficient for a 36T to 12T speed ratio to increase the RPM by 3 times the motor speed. However, a large idler gear is needed to ensure the motor orientation is within the base plate and mounted directly above the wheels, applying the principle of direct force transmission to the drive base. While there are other means to mount the motor, they are not ideal simply because they stick out too much and risk collision damage or have poor load distribution to the wheel drive itself. Hence, the team decided to stick with this gear drive design and helps to be symmetrical similar to the dispenser motor for better aesthetics.

However, because of the awkward height of the collector roller and the motor's position, the centre distance of the meshing gears are placed at an angle of 45 degrees between the driven gear and idler gear. Using a larger idler gear helps create distance needed to ensure the ideal motor mounting position with the driver gear. The structural frame members of the driver and the driven gear are offset and separately mounted, which can cause some inaccuracies in meshing. To overcome this, we tighten the two members together through spacers and bearings to help connect both pieces into one entire rigid structure, preventing any possible misalignment during meshing. Furthermore, collars are used to hold the gears and spacers securely, while the motor's axle rod is also held securely by the bent vex strip, creating a 'spring' effect to resist any free movement by the axle.

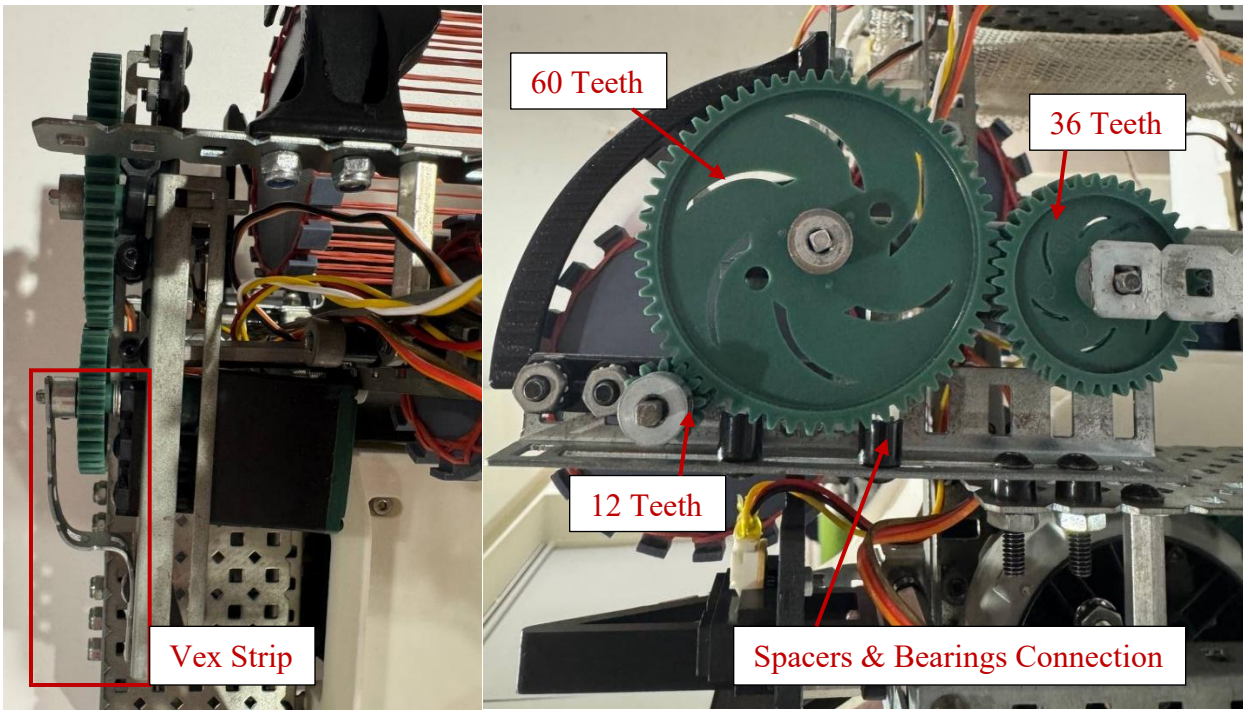


Figure 41: Left Side View of Gear Drive for Collector Motor

Over the weeks, the vex strip is mounted on the front vertical pillars to assist as mounting positions to house the roller guard, oppoenent sensor and the protective cover for the gear train. One of the main concern was how easily the strip could be deformed when collided by the front against the roller guard. To overcome this, we use the protective cover of the gear train to connect with other structural members, assisting to reinforce rigidity and resist against any opposing forces.

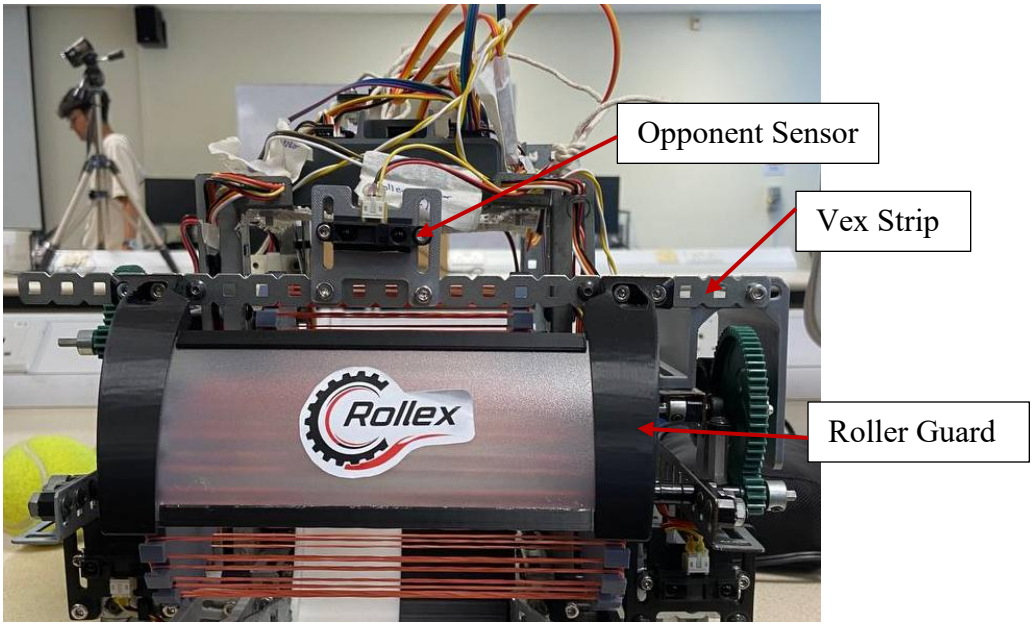


Figure 42: Front Vex Strip for Opponent Sensor & Roller Guard

While the protective cover acts as a shield from any collision damage to the gears that could cause misalignment of jamming during meshing, the mounting of screws is designed to hol the vex strip in place together with surrounding structural members to withstand any opposing force from the front that could cause it to bend and deform.

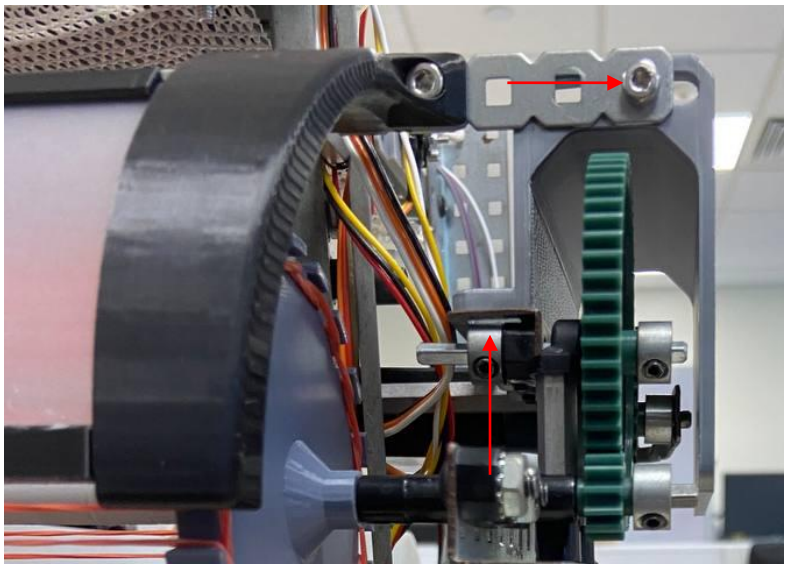


Figure 43: Protective Cover for Gear Train

To provide a proper handling for the operator, it is important to design handles on the robot to be as ergonomic as possible. This means that anyone regardless of their stature or orientation is able to handle it easily, clearly obvious and simple, and provide safety for them.

The team has incorporated an anti-slip mat underneath the wiring and Vex Controller, where the operator can safely hold the robot from underneath at any angle. This is ideal as the wide gap from the sides and back means that any operator is able to put their hand through to lift up the robot regardless of the orientation. Moreover, the anti-slip mat acts as a covering, preventing the wirings from spilling out below, while also protecting the user's hands from getting entangled or scratch by electronics above. The additoonal friction of the mat prevents slippage, allowing users with sweaty palms to be able to hold onto the robot and minimise risk of dropping the robot on accident.

However, holding on to the robot with our palms can be tiring over prolonged periods, which is why strings are tied in figure-8 knots that self-tightens and allow users to carry robot from above like a bag. The precuation is to ensure that the strings are neatly packed back with the wires to prevent them from spilling out and interfere with any sensors or rotating elements around.

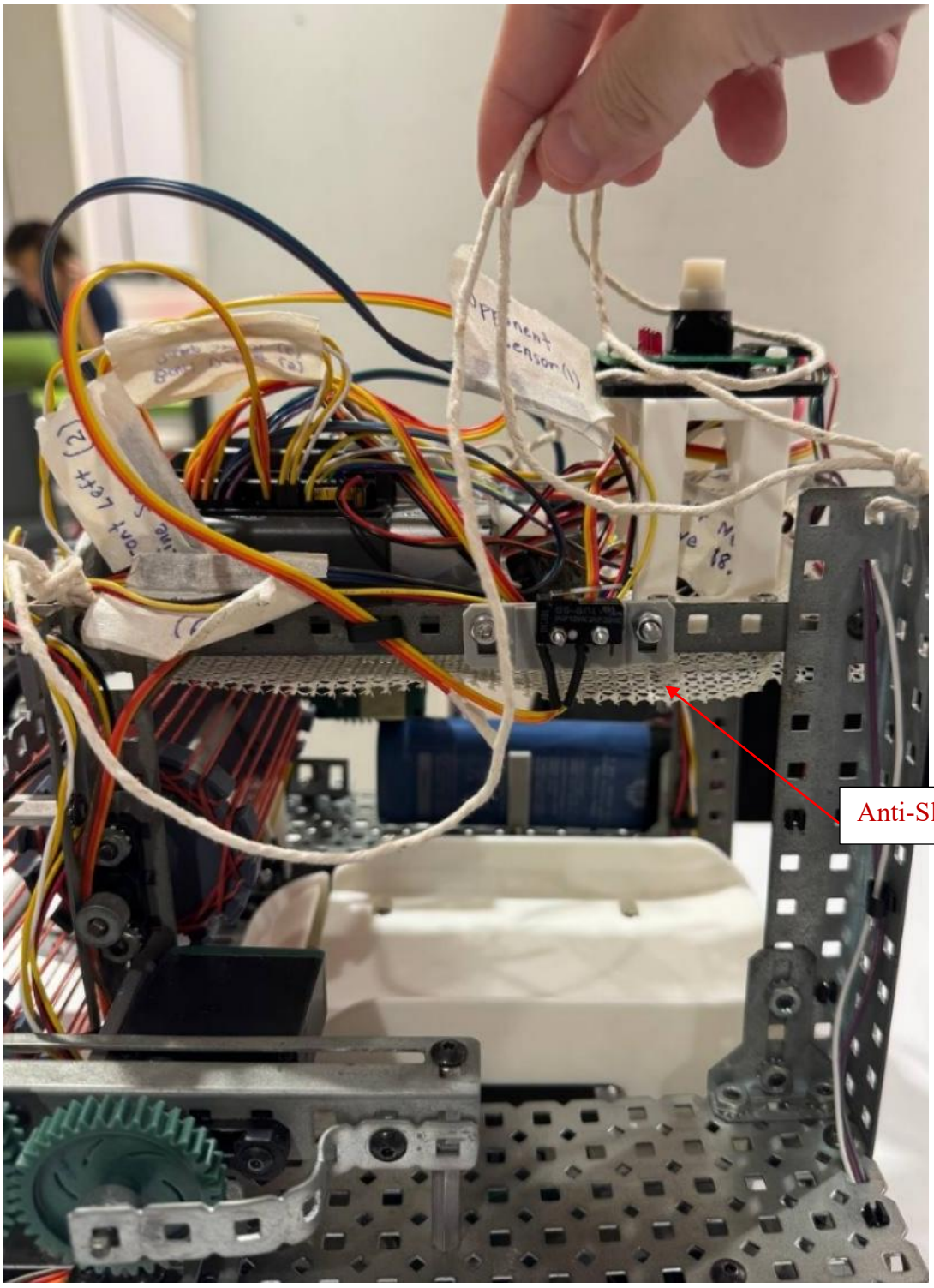
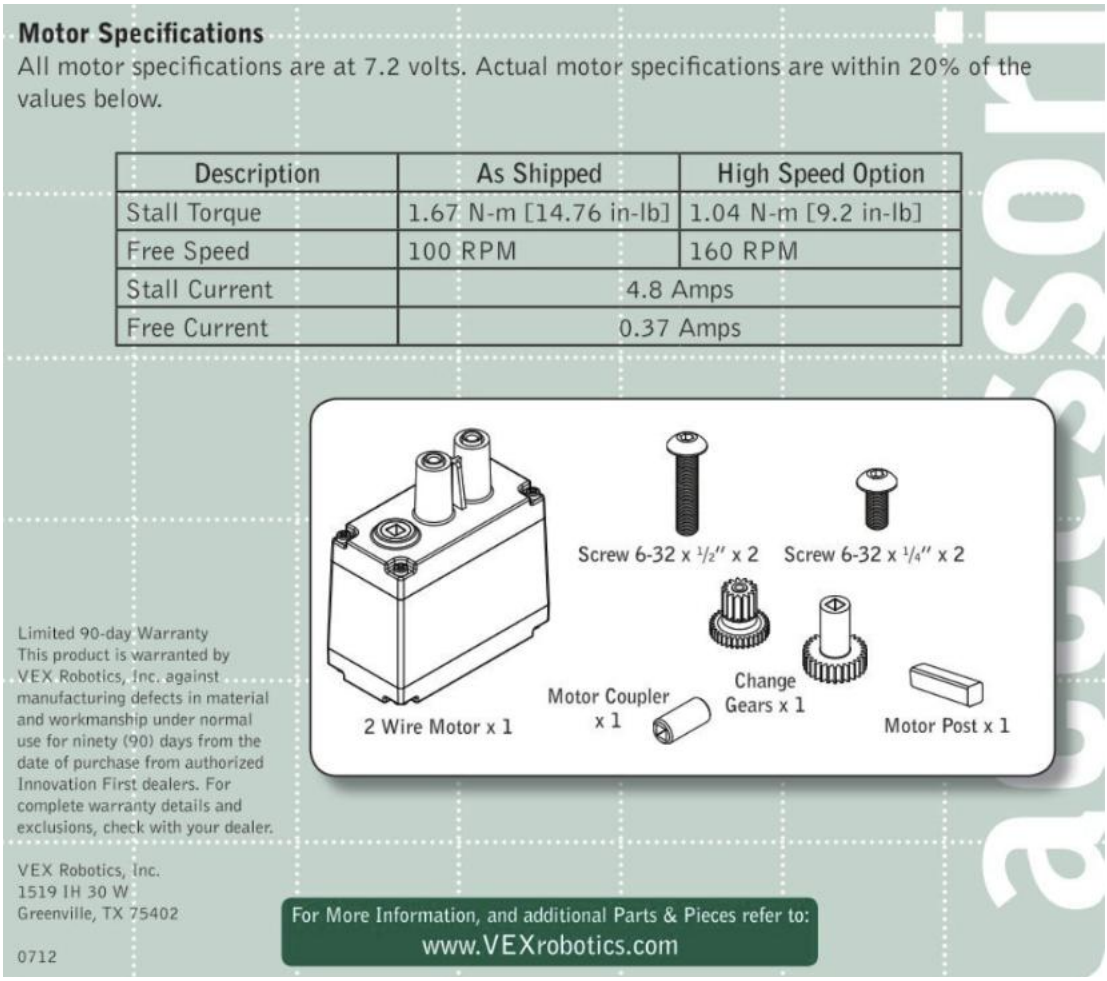


Figure 44: Handle by Anti-Slip Mat or Strings

5 DESIGN CALCULATIONS

This section presents the key calculations used to guide the design of the robot. These calculations were carried out to determine suitable dimensions and performance requirements based on the competition requirements.



Motor Specifications

All motor specifications are at 7.2 volts. Actual motor specifications are within 20% of the values below.

Description	As Shipped	High Speed Option
Stall Torque	1.67 N-m [14.76 in-lb]	1.04 N-m [9.2 in-lb]
Free Speed	100 RPM	160 RPM
Stall Current	4.8 Amps	
Free Current	0.37 Amps	

Figure 45: Vex Motor Specification from Catalogue

From the above motor specifications, we can calculate the robot's drive base rpm speed and forces, the collector and delivery mechanism, power requirements, as well as the load speed/torque required for motor to drive. This allows us to do sizing and calculation of gear ratio for the effective speed and forces as theoretical values. These can then be further proved by empirical testing to validate the results and pass the performance test of the robot.

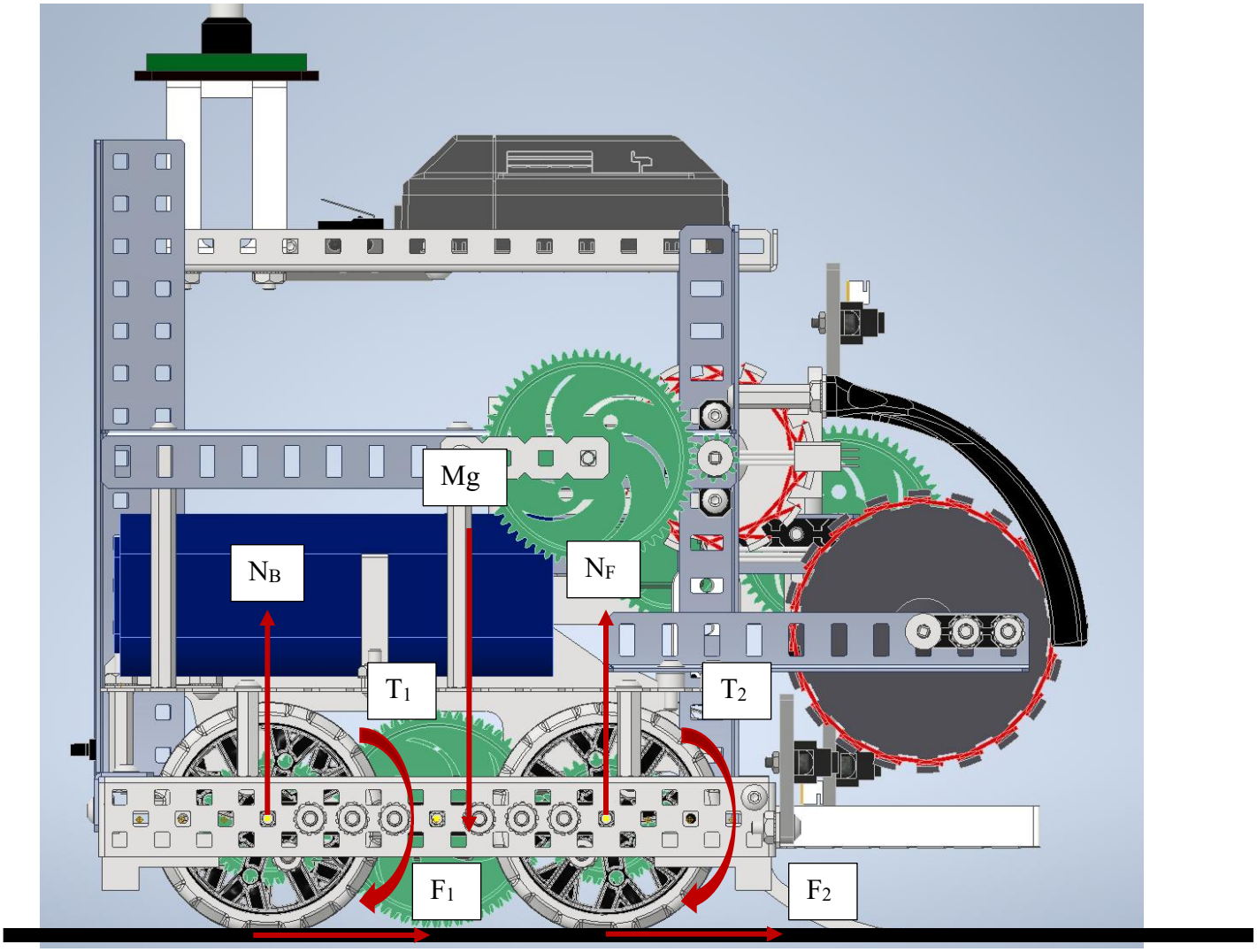


Figure 46: Free Body Diagram of Robot Drive System

5.1 Theoretical RPM of Robot Drive System

The following design calculations is to find out the theoretical speed of the robot's drivetrain at half speed and max speed, given the motors and gearing speed ratio. Note that the output RPM speed is unaffected by gearbox efficiency, but only through the gear speed ratio.

$$RPM\ of\ Wheel = RPM\ of\ Motor\ \times Speed\ Ratio$$

$$Linear\ Velocity = RPM\ of\ Wheel\ \times \frac{2\pi}{60} \times Radii\ of\ Wheel$$

Speed Ratio: $\frac{RPM\ of\ Wheel}{RPM\ of\ Motor} = \frac{Input\ No.of\ teeth}{Output\ No.of\ teeth}$, Diameter of Wheel = 70mm

At maximum speed:

$$V_{Max} = \frac{\pi D(RPM_{wheel})}{60} = \frac{\pi(0.07)}{60} \times 160 \times \frac{60}{36} = 0.977m/s$$

At half speed:

$$V_{Max} = \frac{\pi D(RPM_{wheel})}{60} = \frac{\pi(0.07)}{60} \times 80 \times \frac{60}{36} = 0.488m/s$$

Assumptions:

- Taking RPM motor speed to be high-speed configuration, because the actual values align more closely with it. In reality, it could be due to overload speed for initial speed over a period for traversal, or battery voltage at fully charged at 8.6V or actual high-speed configuration in motor's gearbox.
- Running at no-load speed, where the robot with loading will run slower than the theoretical values calculated.
- No slip condition (v=rw) and wheel diameter is constant,
- Ideal Gear Transmission whereby friction in practice could reduce the rpm speed.
- Stable Battery Voltage at 7.2V but speed can change drastically based on voltage level.

The theoretical results of the robot's drive speed support the recorded results (when fully charged at 8.6V), whereby its full speed is roughly 90cm/s and half speed is around 45cm/s. The robot's actual speed used for the competition is 50% of the motor's rated value, and recorded to be around 45cm/s.

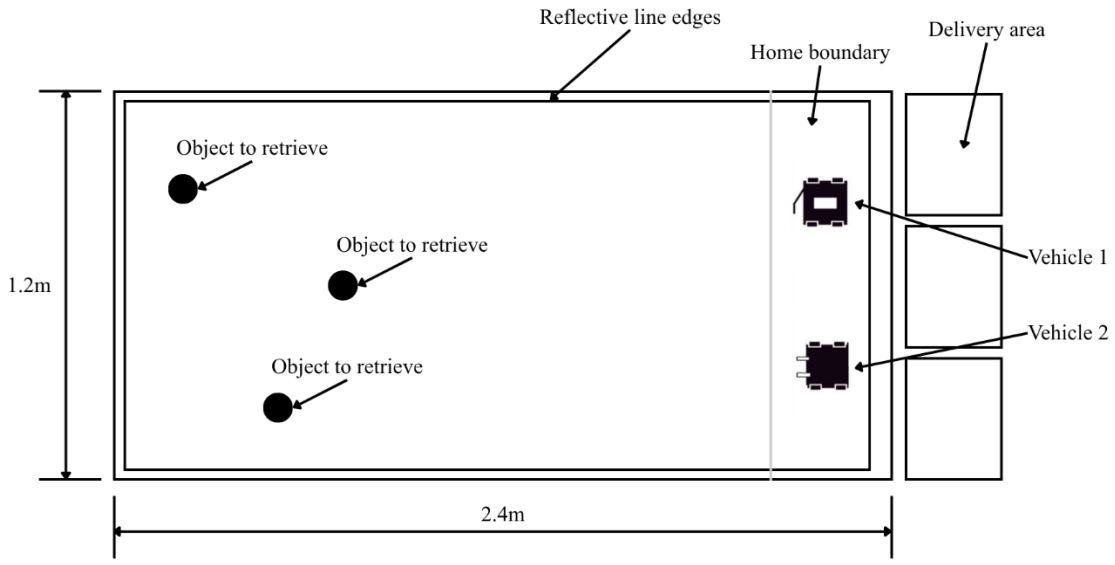


Figure 47: Layout of the competition area

Firstly, for a total of 3 minutes in each round and 3 tennis balls placed in the arena, seen in **Figure 47**, we would need to find out the minimum distance needed for the robot to travel and collect all the balls.

$$Min.\ distance\ to\ collect\ all\ 3\ balls\ =\ (2.4-0.4)\ \times\ 6\ =\ 12m$$

Additionally, we would also need to consider for collision between robots and accidental breakaways of the balls, therefore, an additional 5 m distance is added to the total minimum distance travelled by the robot to 17 m.

Theoretically, the total time taken for the robot to finish the match with all 3 balls is $\frac{17}{0.45} = 37.78\ s$. However, this value is not wholesome because of potential misses and unpredictable events during the competition. Hence, the only suitable and applicable calculation is the time taken for the very first ball, where the variables are undisturbed and opponent collision is minimal, with a straight path ahead.

The total time taken for 'first pass' and deliver the first ball is $\frac{4}{0.45} = 8.89\ s$. This means our robot when driving straight can guarantee the first ball within 7~9 sec depending on the ball placement, unless blocked or mishaps during the delivery Home. Nevertheless, compared to previous semesters robots and our research, it typically took the former champions more than 10 seconds to grab the first ball smoothly. This gives us confidence that our robot speed has an advantage over others in a straightforward fashion, provided the mechanical intake and programming functions are executed smoothly.

5.2 Static Load Analysis

To find the force exerted by the robot on the ground, which can influence its stability and traction:

- Mass of the robot (m): 5 kg
- Acceleration due to gravity (g): 9.81 m/s²

$$Weight = mg = 5\ \times 9.81 = 49.05N$$

Therefore, the force exerted by the robot due to gravitational potential energy is 49.05N.

Assumptions:

- Left and Right wheels are geometrically similar, as well as the front and rear wheels.
- Weight of robot is equally distributed between each of the wheel, based on the symmetrical and identical component assembly of each axle.

Reaction force for front and back wheels: $N_F = N_B = \frac{mg}{4} = 12.26N$

5.3 Torque and Traction Force Calculation

Assumptions:

- Take friction coefficient to be 0.7. For VEX-style wheels on various competition arena surfaces online, a coefficient in the range of about 0.6 to 0.8 is commonly used as an engineering estimate. Choosing 0.7 is a sensible mid-range assumption: not too optimistic, not too conservative.
- Assume gearing efficiency to be at typical value of 90%. Some energy losses due to bearing friction, shaft misalignment, meshing, and backlash.
- Assume equal weight distribution to both front and rear wheels due to mechanically identical and carrying similar loads for simplification. This is reasonable because the chassis, wheels, and drivetrain are designed symmetrically.
- Assume flat ground with no incline, since the arena is designed to be as level as possible.
- Assume front and rear normal reactions are identical and approximated to be similar due to centered CG based on symmetrical assembly and weight transfer from acceleration is small. It is acceptable to estimate the available traction and pushing force, rather than its dynamic suspension behavior.

Solving for **F_1 and F_2** due to **frictional** forces:

$$F_{f1} = \mu N_B = 0.7(12.26) = 8.582N$$

$$F_{f2} = \mu N_F = 0.7(12.26) = 8.582N$$

$$Total\ Frictional\ Forces\ F_{f,Total} = F_{f1} + F_{f2} = 17.164N$$

Solving for **F_1 and F_2** due to **torque**:

$$Speed\ Ratio: \frac{T_{out}}{T_{in}} = \frac{Output\ No.of\ Teeth}{Input\ No.of\ Teeth} \times 0.9 = \frac{36}{60} \times 0.9$$

Given that input torque is rated at 1.04Nm, the output torque is 0.5616 Nm.

Since $T_1 = T_2 = \frac{T_{out}}{2} = 0.2808Nm$:

$$F_{T1} = \frac{T_1}{r} = \frac{0.2808}{0.035} = 8.023N$$

$$F_{T2} = \frac{T_2}{r} = \frac{0.2808}{0.035} = 8.023N$$

$$Total\ Torque\ F_{T,Total} = F_{T1} + F_{T2} = 16.046N$$

Hence, since $F_{Torque} < F_{Friction}$, this means that the robot is motor-limited and could have been tuned to apply more torque. The friction limit is 17.164 N, but the drivetrain can only produce 16.046 N, so the robot is motor-limited.

Traction Force = Pushing Force = 16.046N (theoretically)

It should be noted, that although the calculations indicate motor-limited design, but these values are very speculative and could be traction-limited instead. This is because the actual input torque could be higher at 1.67Nm instead from catalogue for standard design, whereas the robot might be lighter than 5kg, hence $F_{Torque} > F_{Friction}$. It could also be possible to have a higher traction force as calculated above.

Nevertheless, we can conclude that based on the selected assumptions, the calculated torque-limited and traction-limited forces are of similar magnitude. This means the robot does not clearly fall into an extreme motor-limited or traction-limited case. Instead, its maximum usable drive force is likely governed by both factors together, with the dominant limit depending on the exact motor cartridge, supply voltage, robot mass, and floor condition during operation. In practical terms, this suggests that the drivetrain is reasonably matched to the available traction, which is desirable because neither subsystem is excessively overdesigned relative to the other. Hence, the drive system design is very well-executed by balancing both high speed and torque in its performance.

To account for sudden stops and collisions with obstacles or other robots, the robot is considered to decelerate from its travelling speed to rest over a short duration, conservatively estimated at 0.1 s.

5.4 Estimated Power Consumption

Assuming an average power consumption of 10 Watts and a battery capacity of 21.6 Wh:

$$Operating\ time\ = \frac{Energy}{Power\ Consumption} = \frac{21.6\ Wh}{10W} = 2.16\ hours$$

Therefore, this duration comfortably exceeds the requirement of 3 minutes per round, confirming the adequacy of the battery capacity for competition demands. At maximum charge, the battery is operated at 8.6V, while the minimum charge before the robot stops functioning is at around 7V.

5.5 Velocity of Ball Delivery

Assuming 100% rotational force transfer from delivery roller to ball, from Datasheet, a standard 393 Motor has 100 rpm with a gear ratio of 5:1 from motor to roller:

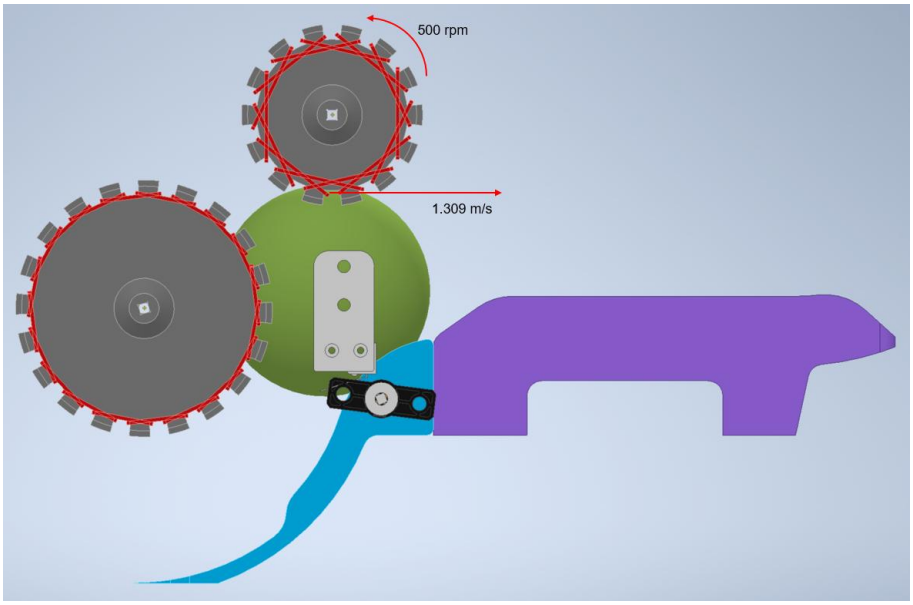


Figure 48: Calculated delivery roller velocity

$$RPM\ of\ Roller = 5 * 100 = 500\ rpm$$

$$V_{delivery\ roller\ at\ 70\%} = V_{ball} = \frac{2r\pi(RPM)}{60} = \frac{(0.05)(\pi)(500)}{60}\ \text{x}\ 70\% = 0.916\ m/s$$

$$Time\ taken\ to\ land = \sqrt{\frac{2h}{g}} = \sqrt{\frac{2(0.08)}{9.81}} = 0.1277\ secs$$

$$Landing\ distance = 0.916\ \text{x}\ 0.1277 = 0.117\ m$$

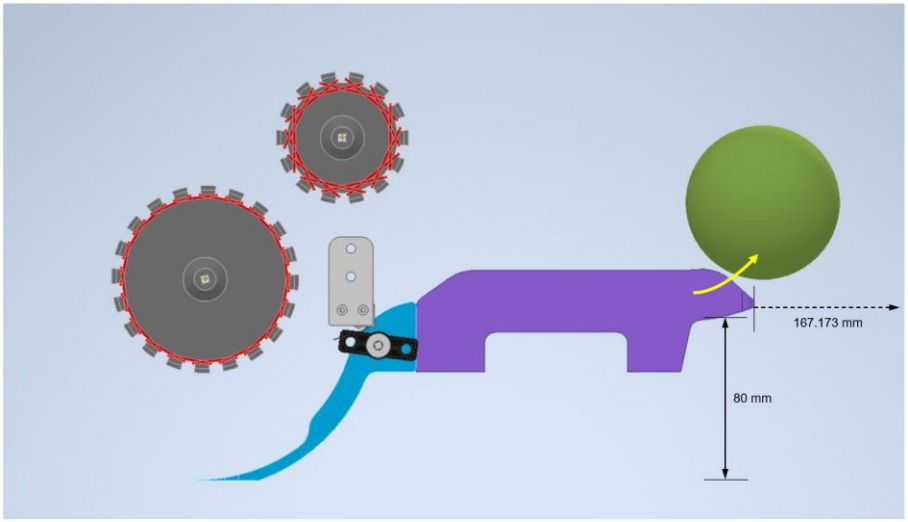


Figure 49: Calculated landing distance

Therefore, our approximate landing distance is assumed to be 11.7 cm which is the same as the landing distance seen from **Figure 25.**

6 SENSOR SYSTEM & WIRING DIAGRAM

This section presents the control system of the VEX robot, including the key electronic components used for robot operation and the wiring connections between them. The control system is responsible for coordinating power distribution, motor actuation, and sensor input, allowing the robot to perform its intended tasks reliably during testing and competition.

6.1 Electrical components

Description of Item	Qty
Vex ARM Cortex-based Microcontroller	1
NiMH Battery Pack 7.2V, 300mAH	1
Battery Extension Cable	1
2-Wire Motor 393 c/w screws x2, coupling x1, shaft x1	2
Sharp Distance Sensor (10 to 80cm)	3
Sharp Distance Sensor (4 to 30cm)	1
IR Line Tracking Module	4
Limit Switch	4
Digital Compass 1490	1
40 ways M-F wire	1
Line Tracking module cover	4

Table 14: Electrical components used in the robot

6.2 Power & Electrical Design

DESIGN REQUIREMENTS / SPECIFICATIONS:

- Provide reliable power to the drive motors, collector motor, sensors, and controller for a full match cycle.
- Maintain clean, organized wiring so sensors do not disconnect or shift during impacts.
- Keep the electrical layout compact, serviceable, and easy to diagnose.
- Place sensors so they support the control strategy without mutual interference.

ROLE & FUNCTIONALITY OF POWER SUPPLY:

- Supply stable voltage and current to all active subsystems.
- Support high transient demand from drive motors during turning, reverse breakout, and collector startup.

- Distribute power safely so motor loads do not physically stress sensor wiring or controller ports.
- Enable prolonged power for robot to function fully and optimally for test runs and competitions. Should be able to handle at least 8 to 9 hours of continuous testing and power on without charging
- Quick pre-match checks from ammeter or charging via easy accessibility

ROLE & FUNCTIONALITY OF ELECTRICAL SYSTEM:

- Read boundary line sensors, ball-detection distance sensors, opponent sensor, ramp switches, rear switch, and compass inputs (8 different directions).
- Drive the left and right traction motors, collector motor, and delivery motor with predictable port mapping.
- Keep analogue sensors separated and clearly labelled to reduce debugging time.
- Sensor placement should match the software assumptions: front line sensors at corners, side distance sensors near front half, center opponent sensor in front, and rear switch centrally mounted.

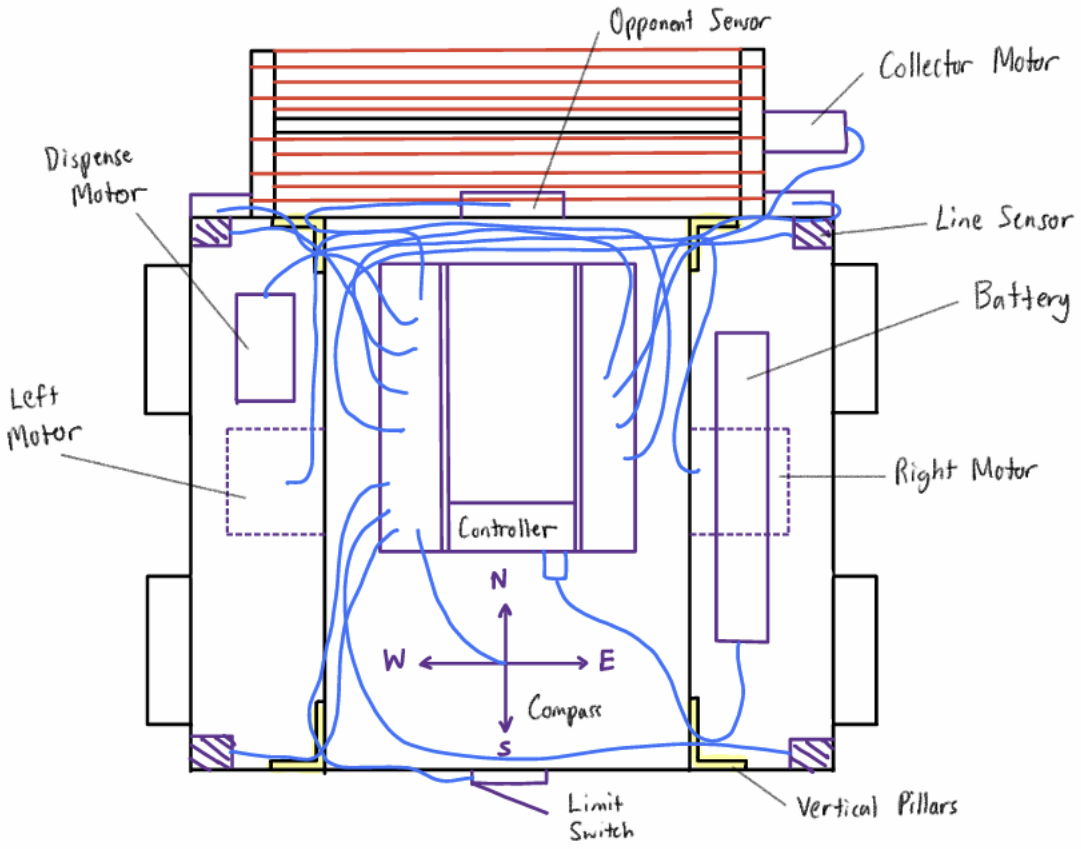


Figure 50: Conceptual Sketch of Electrical System & Sensor Strategy

The above sketch illustrates our team's initial idea for our electronics design of the robot. With all four line sensors installed at corners for effective edge detection, limit switch at back for Home delivery (extra limit switches for detecting ball inside upon collection), 2 drive motors and 2 motors for collection and dispensing respectively, ball detection sensors on left and right, opponent sensor detection at elevated height and centred, battery, microcontroller and compass. The battery and motors are designed to be placed at the sides of the robot to directly apply load on the wheels for stability and traction, whereas the controller is mounted high up for ease of accessibility, together with compass to create distance to minimize ferro-magnetic materials from distorting its magnetic field. The electrical layout is designed to run through the vertical pillars (highlighted in yellow), which act as pathways to connect between the Vex Controller and the electronic components. This ensures that the pillars act as 'housing' to hold wires in place and protect them from free movement that could entangle rotating elements or external collisions, while also creating indirect routes for more free slack and indirect force transmission, making wire connections harder to disconnect when force is applied during rough handling.

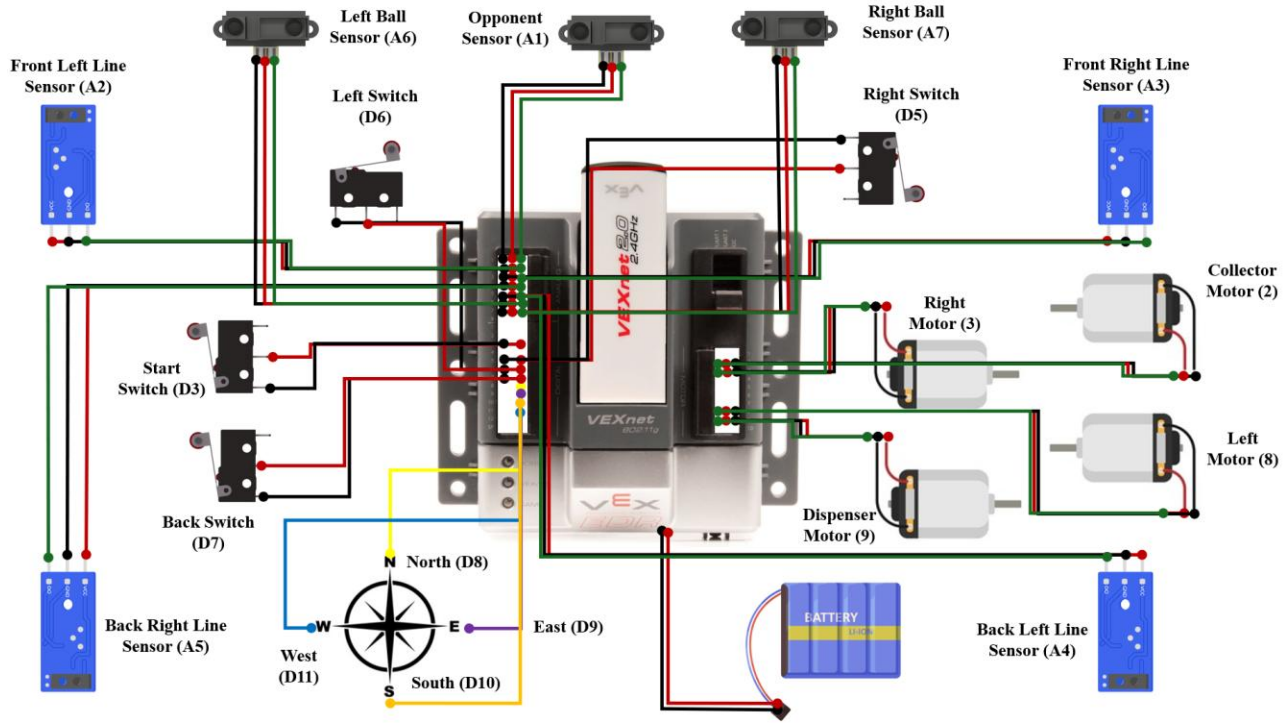


Figure 51: Circuit Diagram of Robot

The above figure is an illustrated diagram of how the circuit of the robot would be rigged up respectively, for each of the I/O from sensors, sensors and motors to the vex micro-controller. The port number is also included accordingly. The physical circuit will have unique colour coding to differentiate VCC, GND, SIGNAL, and the nomenclature will be labelled with description and port numbers. This will embody clarity design to prevent shorting from wiring mistakes that reverse polarity, and provide programmers ease of troubleshoot checks when studying the circuit.

6.3 IR Line Tracking Modules

Infrared Line Tracking Modules, or more commonly referred to as 'Line Sensors' are useful for differentiating surface reflectivity based on different values as an analog sensor. These readings can vary as low as 20~100 when reading bright surfaces, and very high values of ~2000 when reading dark surfaces. These readings are also affected by the height and placement of sensors on the robot due to the ambient light conditions. Overall, the group used 4 of these sensors for Edge Detection capabilities, ensuring that the robot can self-correct upon 'seeing' yellow boundary lines to prevent out-of-bounds, and also use the lines as means for other purposes, such as back line detection for reaching 'Home' to deliver the ball.

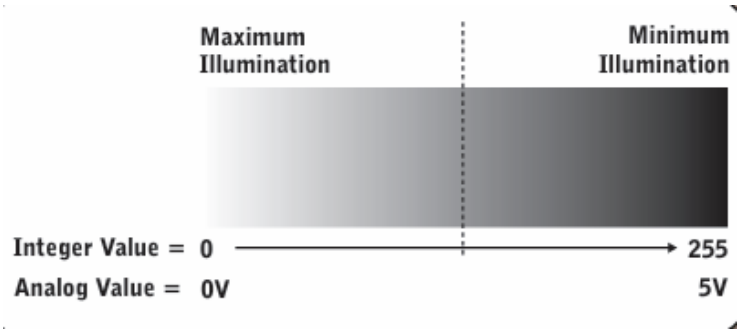


Figure 52: Illustration of Illuminated Surface Values and Threshold

According to our calibration testing of the given line sensors, not all line sensors hold the same range of values. Three of the line sensors tend to read higher analog values as compared to one of the line sensors (located on the front left of the robot). This is because that particular sensor was replaced and has different calibration settings, which required us to manually test and program value threshold against different surfaces to ensure they are proportionally equal despite different values. Since the values can spike and jump about, this also means that the analog values should be recorded over a period of time, roughly one minute, and then comparing their averaged values. This makes testing more reliable, reduced measurement noise, and no biased from occasional spikes due to ambient light conditions. Moreover, the set-up of our testing and calibration must ensure that the positional mounting orientation stays the same for reliable results, not affected by distance between sensors and the surfaces.

It is important to note that although the line sensors are used for edge detection, it is insufficient to only test the line sensors by the black arena and yellow lines. This is because the robot might move out of the arena, detecting the aluminium ramp by the sides, as well as the floor outside.

Overall, our line sensors have been iteratively designed for optimal sensing distance with engineering embodiment principles, while calibrating to optimise the programming

threshold, as well as eliminating potential issues that we foresee to minimise troubleshooting.

Our team has extensively discussed the following characteristics in order to decide the ideal placement of line sensors:

Design Characteristics:	***Reasons:***
Line Sensor 1cm Gap to Surface	Optimal distance for reliable readings as calibrated, well-supported by evidence from testing values, literature review from past seniors and lab technicians.
Well-Protected	Prevents damage to sensors upon collision and placement position will remain unchanged regardless of force applied.
Outermost Corners	Placed beyond the robot's drive base so that the sensors can detect the line before the wheels, especially if moving at high speeds which must account for deceleration.
Entry Hole Accessibility	Account for wiring port and pathway from sensor to microcontroller above, without getting entangled by rotating components (wheels, gears, etc)
Under Robot & Not Exposed	Ambient Light can act as measurement noise to spike the reading values, causing erratic movement or inaccurate behaviour. It is important for the housing to narrow and focus its view, while having the robot's shadow to block out ambient light.

Table 15: Design characteristics of ideal placement of sensors

Figure 53 below illustrates the yellow ruler as the 'boundary' line, showcasing why outermost corners are critical for sensor placement. As illustrated, the wheels would tend to exceed the boundary before the line sensors can detect anything, increasing the likelihood for out-of-bound. This concern suggests that they must be placed on the very corners of the drive base, along the vex frame. However, it is difficult to install sensors in horizontal orientation due to the assembly build of the drive base, where the spacers are obstructing their placements and no mounting holes available. Hence, it made more sense to mount them vertically, where they can be placed below the spacers and mounted freely at different heights on the vex frame mounting holes on sides.

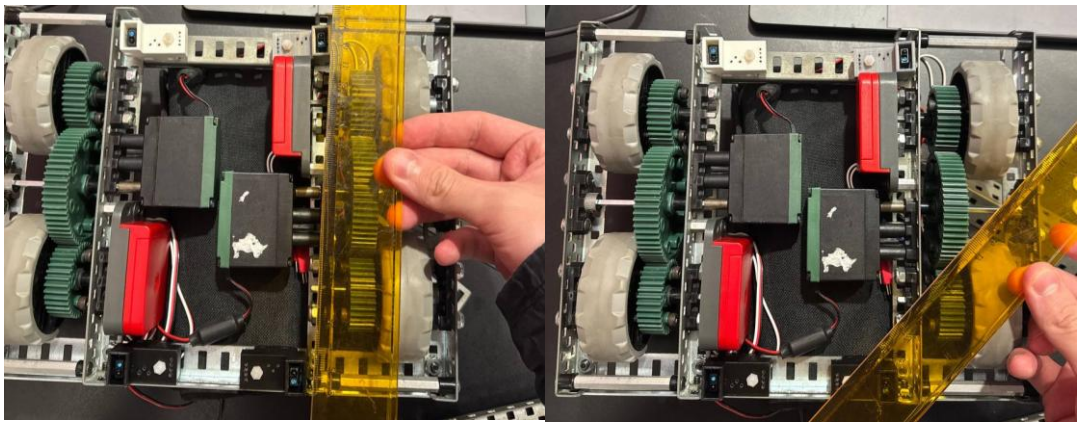


Figure 53: Illustration for Boundary Line

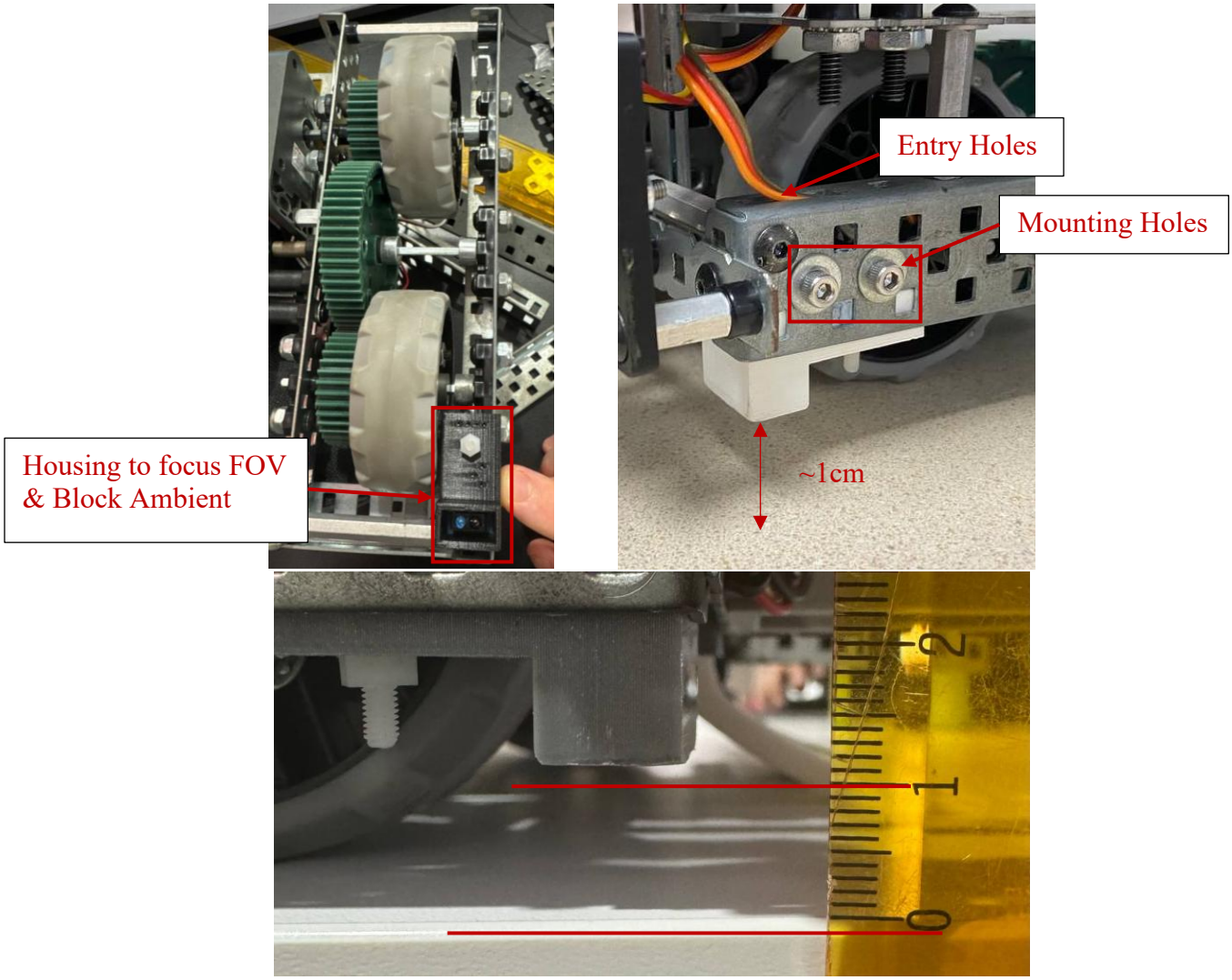


Figure 54: Measurement & Placement of Line Sensor

As seen from **Figure 54**, all the desired characteristics are met. Each of the 4 sensors are well protected within the drive base of the vehicle against any collision, preventing potential damages. By placing them within the robot and 3D-printed housing, the sensors FOV are narrowed to focus on the floor and minimised any ambient light to interfere as noise that create inaccuracies or erratic behaviour. With the vertical orientation instead of horizontal, the sensors can be nicely aligned at the corners with ideal mounting positions with the vex frame, placed below the spacers to not obstruct with the current assembly, and have accessible entry holes for wires to run through directly without getting caught by wheels or gears. These entry holes help hold the wires in place during vibration or movement, without fear of entanglement or loosened wires.

However, ideal sensor placement is insufficient when it comes to practical hands-on build, where mounting, tightening, and assembly techniques matter. It is highly recommended that the sensors must not be freely loose during vibrations or collision. Hence, the figure below illustrates our innovative approach to incorporate embodiment principles for mounting line sensors.

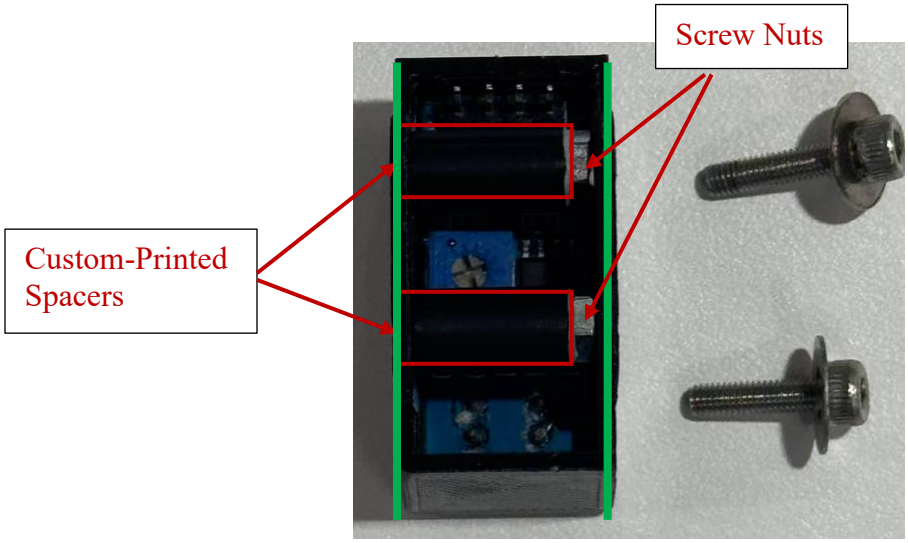


Figure 55: Mounting of Line Sensors Internally

If we attempt to tighten the screws and nuts through both walls of the housing, the nuts would be outside the housing and possibly interfere with the rotating wheels. This also make any assembly and disassembly complicated and inaccessible since the nuts cannot be reached with wheels or gears blocking them. Hence, through the principle of self-help, the screws and nuts can be tightened using one side of the housing wall, allowing the housing on other side to be flushed without touching the wheels.

Firstly, the screws and nuts have a direct force transmission on only one side of the wall, which means that no matter how tightly it tightens, the housing is harder to break or less deflection which affects the sensor's position as compared to tighten both walls. The custom spacer applies 'self-reinforcing' feature by applying reaction forces outwards to

both walls internally when wedged in between. This helps to strengthen the durability of the housing, while also holding down the nuts when tightening the screws. Hence, this self-reinforcement means only the screws need to be tightened without accessing the nuts, enhancing the ease of assembly. Additionally, this design can help to stabilise and protect the housing as well, where both thin walls will not deflect or break during tightening, allowing for a highly rigid assembly. In the event of vibration where the screws may loosen, the custom-spacers also prevent the nuts from moving axially and dropping out, holding it in place. It also means that loosened screws can only protrude outwards which makes maintenance more accessible and obvious to fix, helping us in the long run overall. Hence, this self-help principle enables the sensors to stay intact and rigid, well-protected and ease of operation.

Once the sensors are fully installed, the sensors are calibrated accordingly against the arena's black surface and boundary lines, to further verify that the sensor thresholds can perform under actual conditions when fully assembled, and not just during initial calibration phase.

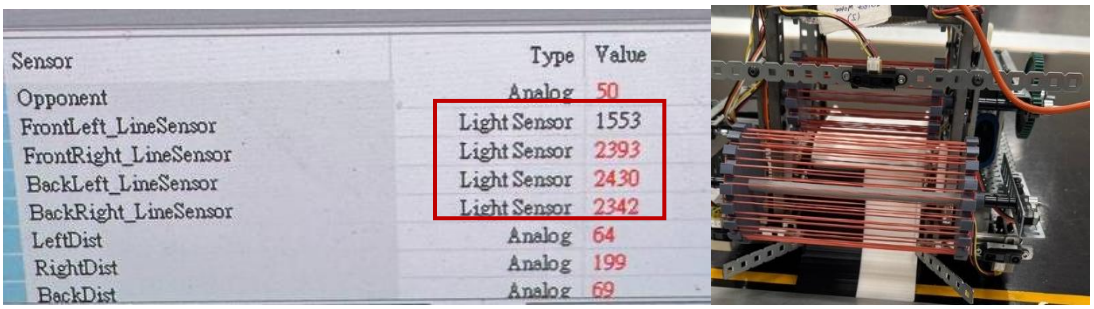


Figure 56: Analog Values On-Site Testing & Calibration

	FrontLeft	FrontRight	BackLeft	BackRIght
Yellow	180	170	140	200
Black	1000-1560	2400	2500	2300

Figure 57: Analog Values Compiled for Line Sensors

From **Figure 56** and **Figure 57**, shows the analog values logged when tested on the arena surface for each of the sensors respectively. As forementioned, the other sensors have higher readings than the 'FrontLeft', but this did not pose an issue because the values range vary drastically for black surface and yellow lines. We then decided to test the values against the arena's ramp and our results showed that the sensors read the values similar to the yellow lines. Hence, it is conclusive to put the threshold around 600 to differentiate the boundary for edge detection.

6.4 Sharp Distance Sensors

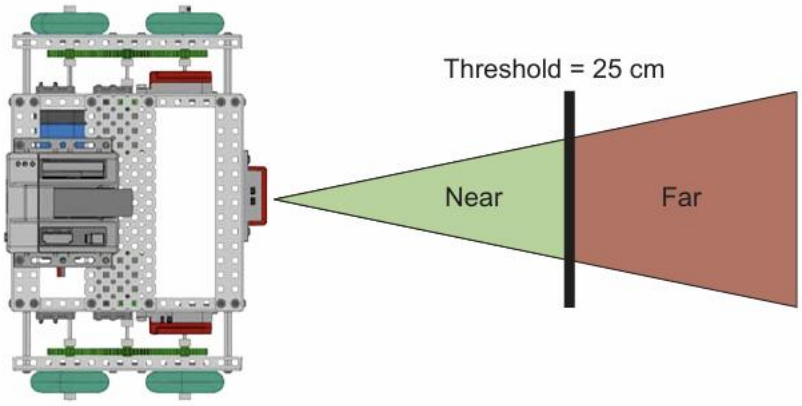


Figure 58: Illustration of Distance Sensors

The sharp distance sensors are used for ball detection through ultrasonic rangefinders. The ultrasonic sensor determines the distance to a reflective surface by emitting high-frequency sound waves and measuring the time it takes for the echo to be picked up by the detector. The ultrasonic sensor consists of two parts: an emitter, which produces a 40kHz sound wave; and a detector, which detects 40kHz sound waves and sends an electrical signal back to the microcontroller. Through the manual setting of a threshold value, the robot can determine what constitutes as object detected, and what distance is considered undetected. Hence, it is necessary to do our own experimental calibration of each distance sensor and find out the optimal and most reliable distance for each sensor.

During the early phase of assembly, our group conducted iterative tests to experiment with the analog values and distance measured. As part of the sensor calibration process, an interpolation test was conducted on the distance sensors to better understand the relationship between measured distance and raw analog output. Four distance sensors, comprising of three long-range sensors, and one short-range sensor were tested. To reduce any biased results or measurement noise, all sensors are conducted under the same environment, under the same mounting height and orientation, including a control set-up. For each sensor, the analog reading was recorded at 10cm intervals up to its usable sensing range. These measured data points were then tabulated in Microsoft Excel, where an interpolation table was generated to estimate the corresponding distance for intermediate analog values. The intention of this exercise was to convert raw sensor readings into approximate physical distances in centimetres, so that ball-detection thresholds could be set in a more intuitive and visually meaningful manner. The interpolation results were later prepared for implementation in RobotC.

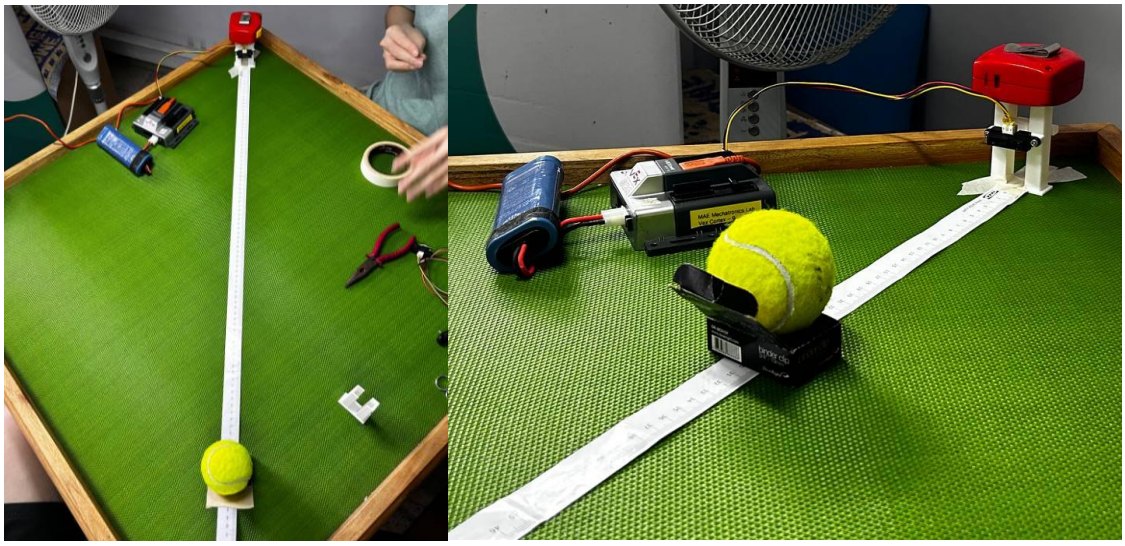


Figure 59: Experimental Set-up for Calibration of Distance Sensors

Actual Distance	RobotC Value
0	0
5	2550
10	2070
15	1400
20	1060
25	840
30	710
35	600
40	530
45	480
50	450
55	440
60	440
65	430
70	390
75	390
80	350

Parameters

Sensor height 6th teeth from the top

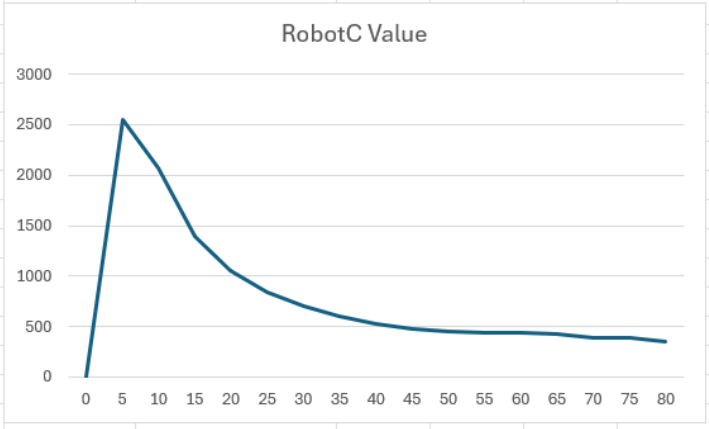


Figure 60: Interpolation Results and Simulation

	Left Distance Sensor	Right Distance Sensor
10	2400	2440
20	1120	1150
30	800	700
40	560	590
50	270	470

Figure 61: Calibrated Sensor Readings

This test served as a useful robustness and calibration check, because it allowed the sensing behaviour of each individual sensor to be examined more closely. In particular, it provided a clearer understanding of how the output varied across the sensing range and whether different sensors behaved consistently under the same measurement conditions. The above results showed that the best threshold is around a reliable and optimal distance of between 10cm to 40cm, regardless of the sensors' mounting orientation and higher height. Any distance beyond that would be subjected to external noise which makes the reading unreliable. Nevertheless, it is unadvisable to use the distance sensors at a further range because it makes the sensors too sensitive to inputs, such as being too close to the ground, jerking during operation and noisy environment during testing. The threshold value cannot be too near as that would make the search algorithm inefficient and redundant. Hence, the ideal distance is roughly 25cm to 30cm, which is comparable to the width of the robot. However, during troubleshoot and testing, we observed and continuously changed this threshold value accordingly to suit our robot's search algorithm and mechanical design.

Although the interpolation approach improved interpretability, it was not used in the final implementation. The main reason was that the robot's decision-making did not depend on precise distance estimation, but rather on whether the detected object had entered a suitable sensing zone for alignment and collection. Direct analog threshold values were therefore sufficient and avoided introducing additional conversion steps into the control loop. Furthermore, real test conditions showed that sensor response could vary with target angle, surface reflection, local mounting geometry, and slight differences between sensors. Under such conditions, converting analog output into centimetres could give a false impression of precision without significantly improving control performance. For this reason, the final software retained analog thresholding, while the interpolation test remained valuable as a calibration and robustness exercise.

Next, our group tested the sensor's field of view and measured the positional angles and extreme limits for ball detection. As seen in the figure below, the range of ball detection can detect at an angle 8 cm away on one side of the sensor. We also tested different mounting orientation such as vertical, horizontal and angled positions, but the horizontal orientation produced the most reliable and desirable results. This is because the horizontal orientation offered this wider FOV which is useful and advantageous to our robot behaviour later, when moving forward in a straight line. This specific feature is what enables our robot to cover more than the entire width of the grid layout of the tennis ball placements. To put the FOV into perspective, the extra angular sensing distance of 8 cm is similar to the tennis ball's width, which means it can detect one square away from the centre of the sensor.

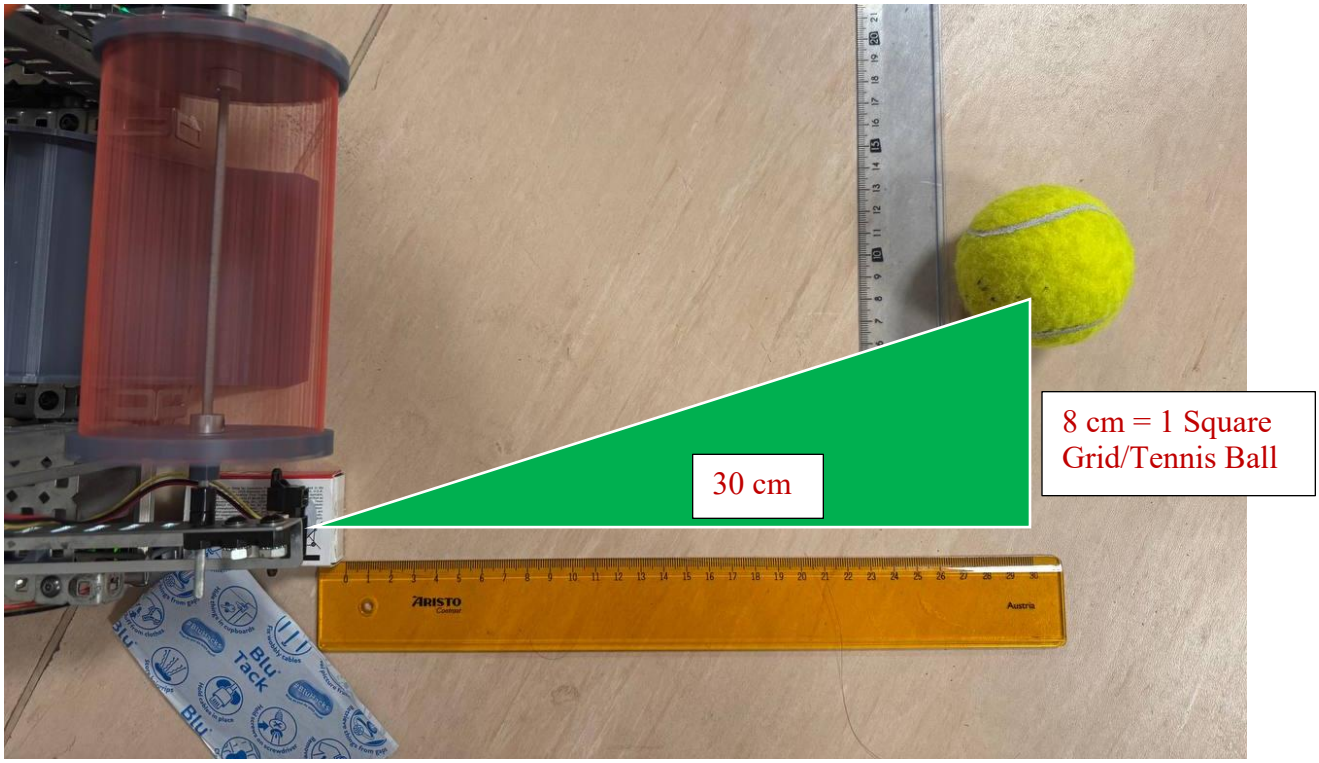


Figure 62: Testing Sensor's FOV for Ball Detection

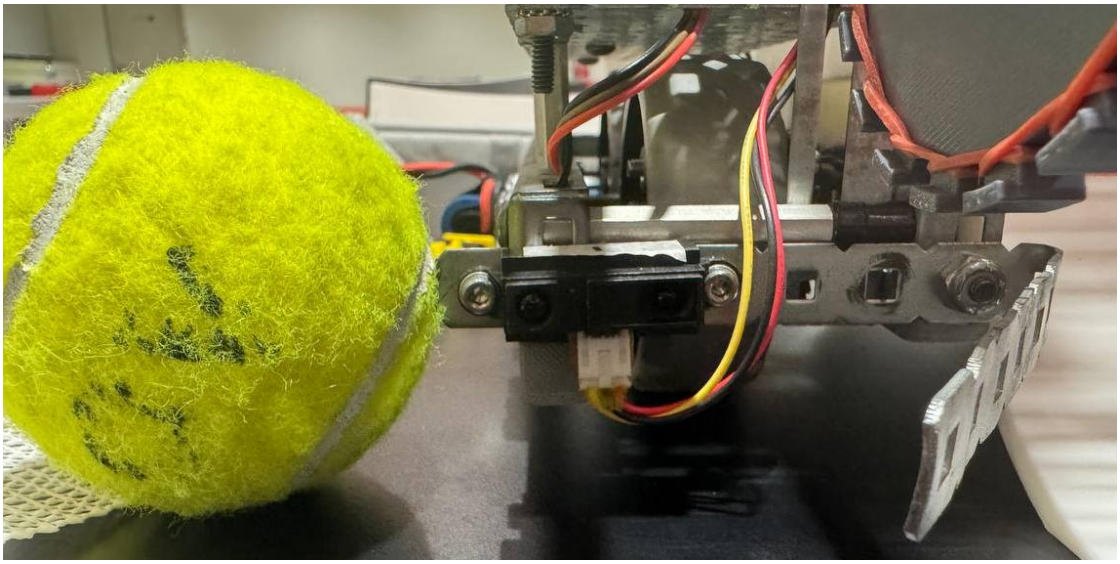


Figure 63: Distance Sensor Mounting Height Aligned with Ball's Centre

We initially mounted the sensor at the same height as the ball's centre for reliability and ease of assembly to the robot's frame.

Our first design iteration of the distance sensors are placed as seen above (**Figure 62** and **Figure 63**). The robot has one left and one right distance sensor to detect the tennis ball on both sides of the robot, with a centralised short sensor for opponent detection on top at an elevated height in

order to differentiate between the tennis ball and the opponent robot. Another distance sensor is also installed at the back of the robot, to assist detection of any obstructions when returning Home for ball delivery. The team also initially thought that this sensor could be useful for searching algorithms, where turning can search for the ball in front and at the back, making our search more efficient. However, this idea was later scrapped, because it would be difficult to prioritize which tennis ball to go for upon detection, and the mounting of the back sensor is too low which often leads to accidental detection of the arena's ramp or even collision when robot moves out of bound, damaging the sensor.

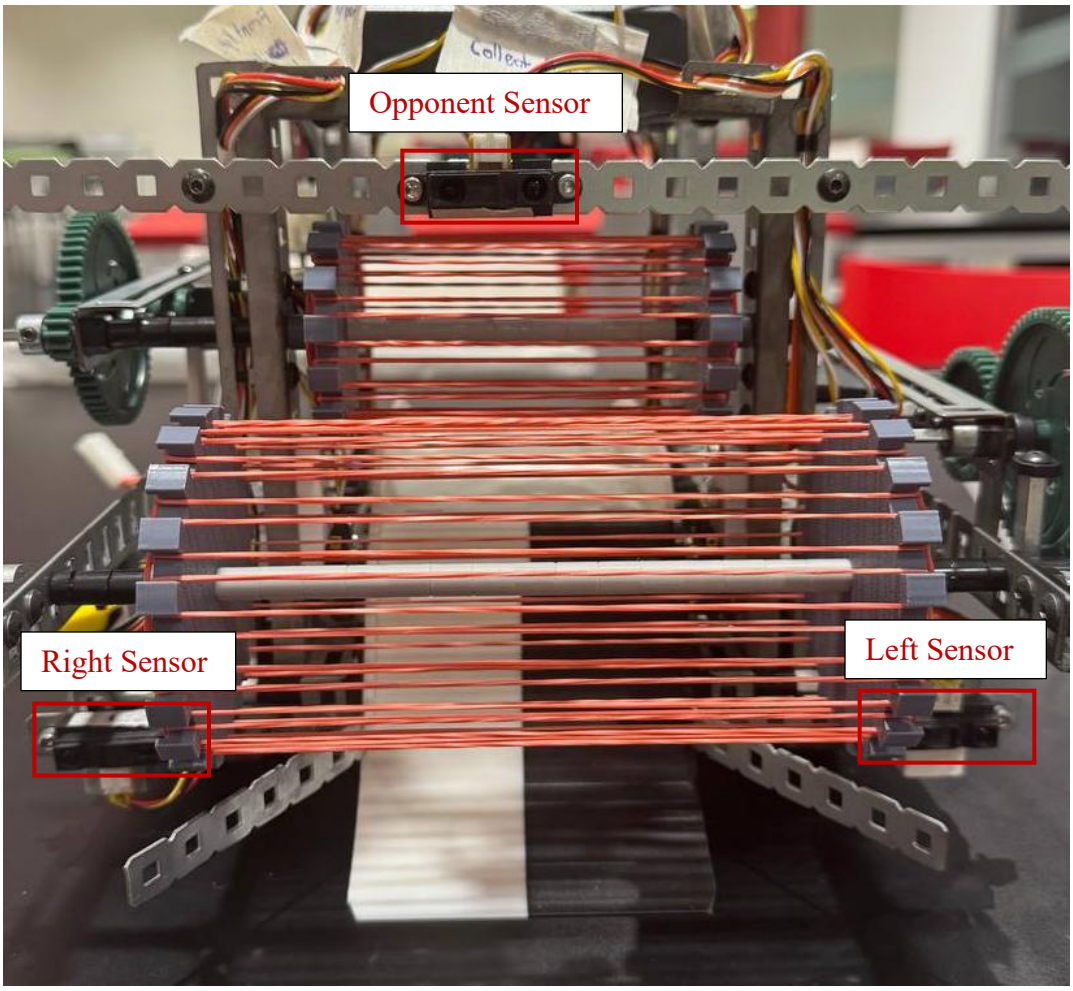


Figure 64: First Design Iteration of Distance Sensors on Robot (Front View)

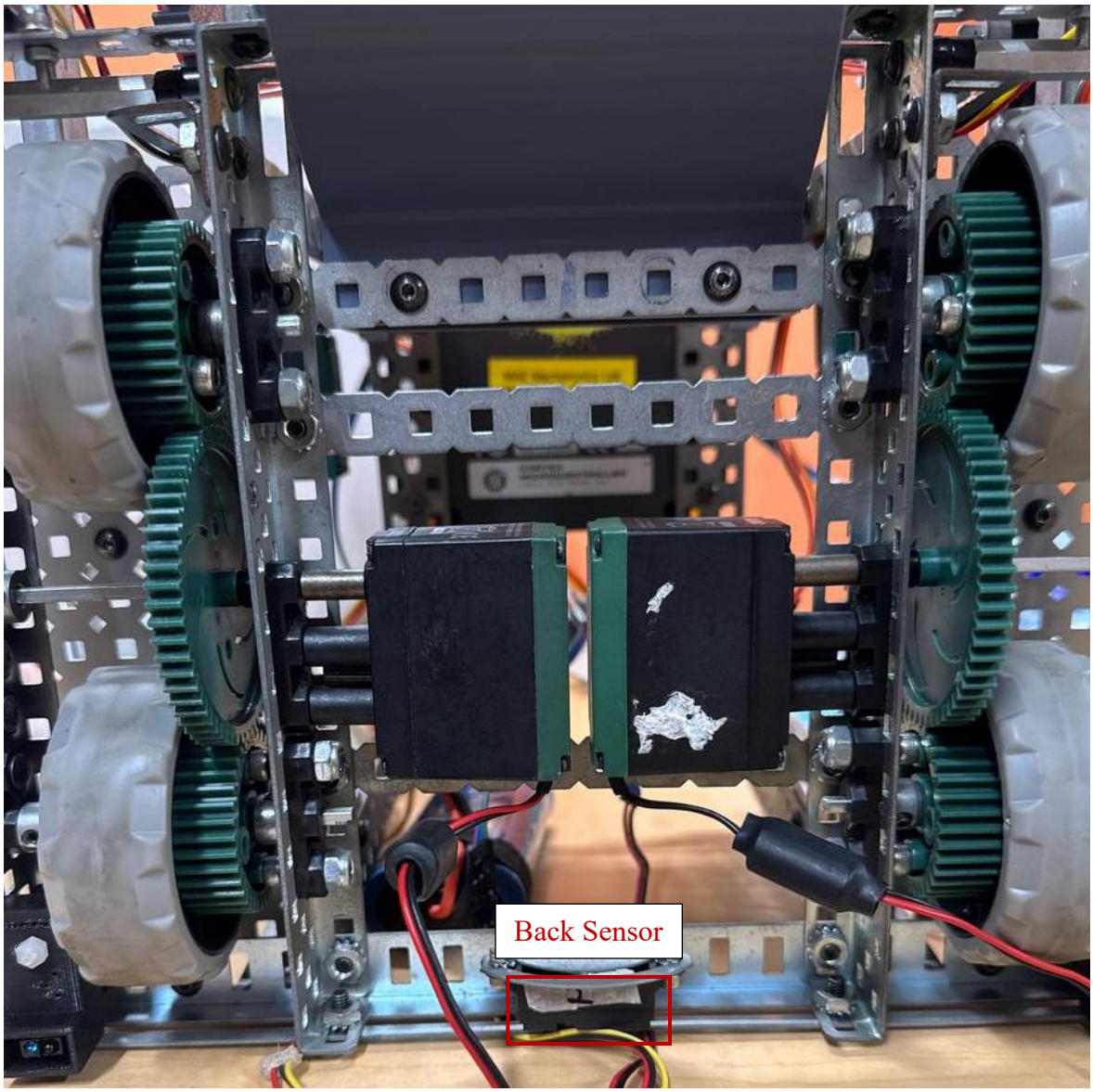


***Figure 65*: First Design Iteration of Distance Sensors on Robot (Bottom View)**

However, there were many problems encountered for this design layout of sensors. During testing, the left sensor often ‘ghost’, meaning it detects something when nothing is there at all. This could be due to various reasons which will be discussed under Error! Reference source not found.. One o f the reasons was because the sensor was mounted too low near the ground, which ended up reducing its effective range of detection and cause the analog values to spike when jerking or vibration during movement, especially at high speeds with high inertia. On the other hand, the right sensor is mirrored so the wiring port is below the sensor instead, which makes it vulnerable to damage from collision against the arena’s ramp when out of bound. This was a serious issue as the PCB broke off and rendered our sensor useless for the entire day, which means the current design was poor and must be further refined by changing the mounting orientation. Moreover, because the sensors were too low, this means that the sensors would tend to detect the ‘ramp’ as the tennis ball and chase after it. We tried to solve this from software side, but it became more

complicated with lots of software bugs. Fortunately, the damaged sensor made us replace it and re-design our design layout of the sensing system, allowing us to modify the mounting height to negate this issue, becoming a blessing in disguise. Hence, for our final design iteration, we designed sensor holders for adjustable heights for the side sensors, and removing the back distance sensor to replace the short sensor for opponent detection. This design change alone eliminated the risk of arena ramp detection, further opponent detection, reduced chances of ghosting and protected our sensors from collision with the arena ramp.

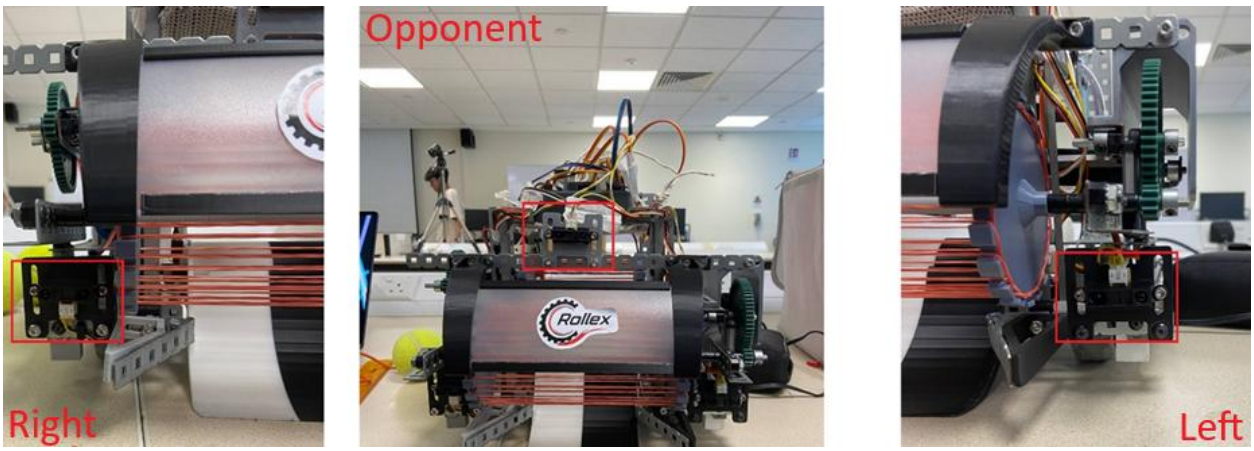


Figure 66: Final Design Iteration of Distance Sensors on Robot

6.5 Limit Switches

Our limit switch layout serves three different purposes, one for ball detection to detect that ball is collected inside the robot, another is to assist detection when delivering the ball when returned to Home, and lastly for start switch when power ON.

The trigger flap is actutaed by two inverted limit switches, closing when the flap is pushed down by the weight of the ball. As mentioned before, the square rod attaches with the flap and the bearing flat will rotate together when the ball is pushed the flap down. This rotational motion will push the limit switch upwards and detects that the ball is inside and ‘collected’.

When the back limit switch is pushed against the collection boxes and activated (together with back line sensors for yellow line), the robot will actuate the motor to deliver the ball. The absence of the ball will push the flap back up due to the spring force of the limit switch. However, to ensure that there is sufficient force to push the flap downwards after delivery, two limit switches are used to increase the spring force.

This design embodies the principle of similar tasks to share the function equally, splitting the force required and improves the safety measure via greater redundancy, in the event where one fails to actutate, the other can support as failsafe. Both the bearing flat (trigger arm) and the switch holder are positioned relative to each other, such that the arm can trigger directly on the switch with minimal effort, effectively using the principle of direct and short force transmission. Lastly, by inverting the limit switch upside down, the assistance of gravity helps reduce the effort needed to push the flap back up after delivery, embodying the principle of self-help.

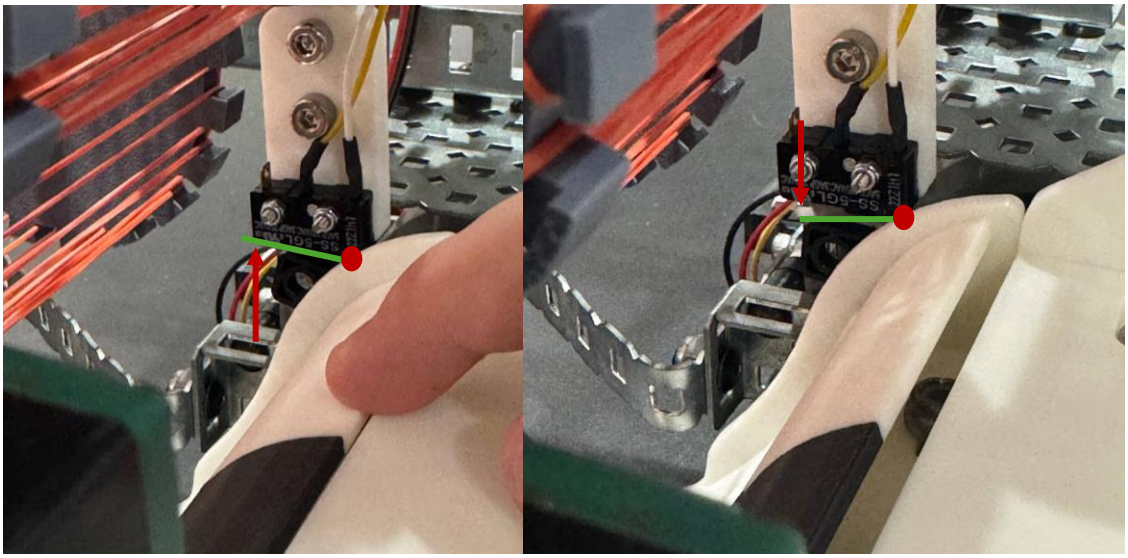


Figure 67: Open/Close Ball Detection of Trigger Flap via Limit Switches

Initially, our team wanted two limit switches at the corners of the robot to ensure that the robot must be straight upon contact for delivery. However, this would further constraint how our robot must be oriented in order to deliver the ball. Since the robot will always self align when bumping into collection boxes as it returns Home, it is much simpler and more effective to design a centered limit switch to activate. However, this design's limitation can be dire if opponent robots collide and accidentally actuate the switch, causing the robot to deliver the ball into the opponent. Fortunately, this design works since our opponents do not have a rigid structure at the sides of their robot at that height and have large empty spaces upon studying their design, as proven by collision during the competition. Nevertheless, this limitation can be overcome and improved upon through software code.

It is important to note that limit switch or line detection alone is insufficient to guarantee 'Home' detection for ball delivery. The robot must rely on both for guaranteed chance of success. The wall of the boxes can flex, which made switch alone to not trigger at times, while if moving too fast, the line sensors might misread the yellow line sometimes. To further increase our chances, the team shifted the limit switch mounting slightly outwards more to ensure full contact despite the flexed wall upon impact.

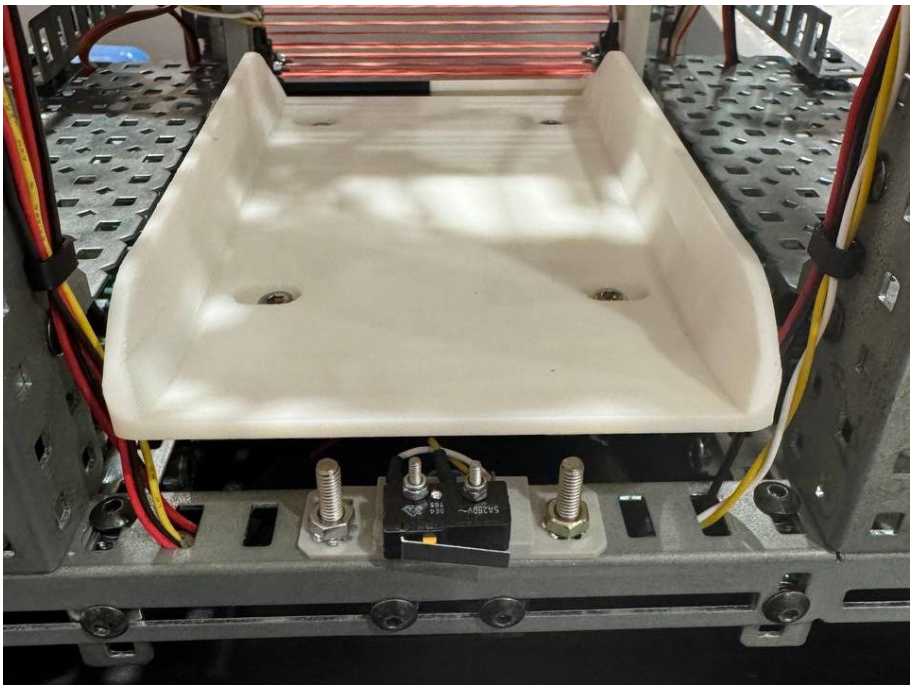


Figure 68: Back Limit Switch for Ball Delivery

The start switch is designed to be mounted directly beside the Vex Microcontroller for easy access and visibility for robot handler to start the robot. This mounting orientation ensures that the switch is tightly held in place and secured to prevent any accidental movement to trigger without intending to. Moreover, it has to be mounted at a position where its inaccessible by possible collision or rough handling, hence above the ‘handle’ by the anti-slip mat.

Initially, we thought that the start switch was redundant and did not include it within our design. However, trying to power on and run the robot immediately can lead to unpredictable volatge spikes. The start switch was implemented as a controlled start mechanism to allow the sensors and electronic components to stabilise before the robot begins executing its program. Rather than using the switch as a direct power-on trigger for the microcontroller, the microcontroller and main electronics are powered first so that the system can initialise under stable conditions. This is important because powering the robot and immediately forcing program execution can expose the controller and sensors to unpredictable voltage spikes, transient fluctuations, or start-up noise, which may lead to unreliable sensor readings or unstable behaviour at the beginning of operation. By using the start switch only to begin the robot’s motion and decision-making sequence after the electronics have already warmed up and settled, the design improves reliability, reduces the risk of false sensor triggers, and ensures that the robot starts from a more stable and repeatable operating condition. Hence, this was one of the many improvements to further reduce the chances of our distance sensors from ‘ghosting’, detecting objects that are not there.

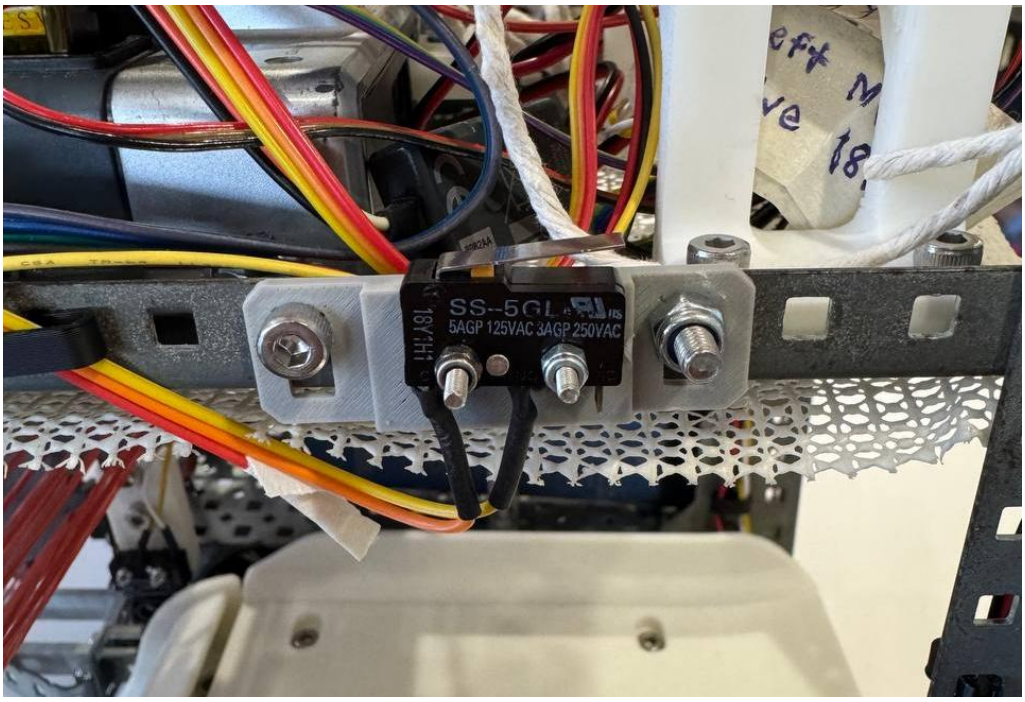


Figure 69: Start Switch for Robot Handler upon Power ON

6.6 Vex Micro-Controller & Battery Pack

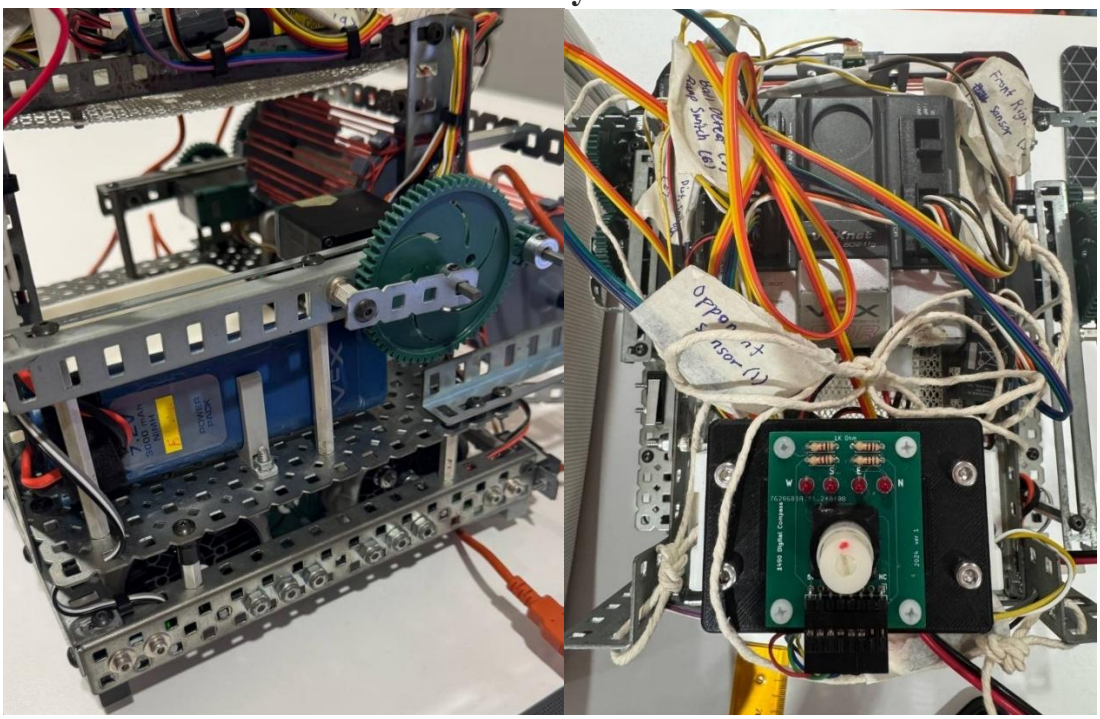


Figure 70: Battery holder & Vex Micro-Controller

While simple at hindsight, both the battery and microcontroller placements have significant design considerations. The battery itself holds a significant amount of weight, which means that it must be placed at a low position to lower the robot's centre of gravity for high stability as embodiment principle. As seen from the robot's structural frame design, there are two base plates, one on each side to house heaviest weights such as the battery, motors, gears, vex metals, and the roller mechanism itself. This is helpful by distributing the load to the wheels directly, allowing for more balanced movement and traction to accelerate itself by friction through reaction force. The shifting of weights can also help to control any tendency to swerve motion when trying to drive straight. The battery holder is designed to hold it in place to not move freely during any violent collisions or vibration movements, preventing any possible obstruction that could spill into the ramp and block ball's delivery. Moreover, the battery itself must be easily accessible by design. However, given the cramped space of the layout, an alternative is to access the battery port from the micro-controller itself for charging, hence value-adding to placing the controller above.

The Vex controller is designed to be easily accessible and maintained since there are many wiring ports, power button, USB cable port and battery port for user interface and testing. With dedicated wiring routes throughout the robot's frame, the microcontroller can be placed on top without any concern for constraints. Identification for troubleshooting wiring or controller would be obvious and require least effort for user intervention.

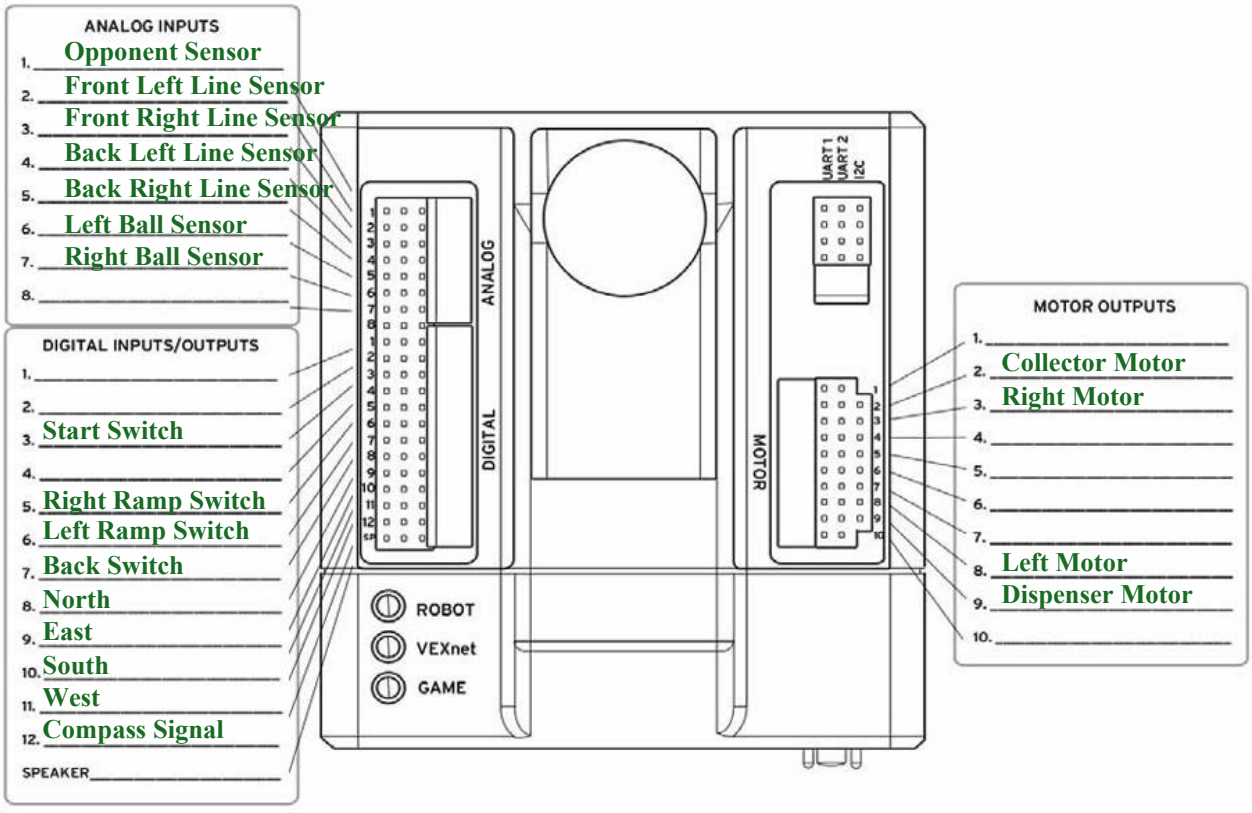


Figure 71: VEX Micro-Controller I/O Layout

6.7 Digital Compass

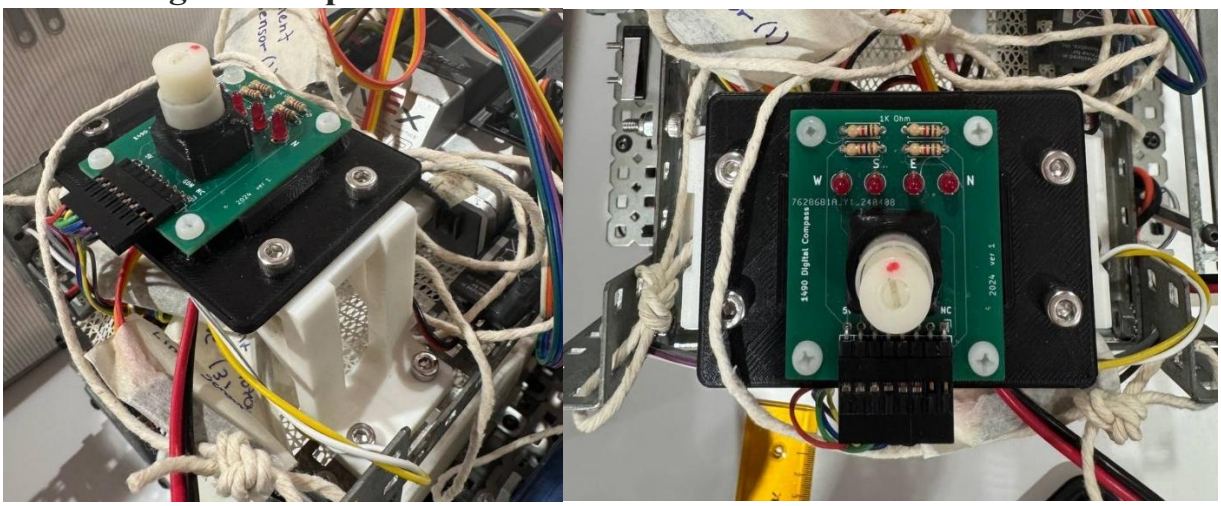


Figure 72: Digital Compass Mounting

The digital compass is the final sensor installed when the robot has been fully assembled. It has a simple functionality of orientating the robot to the Earth's magnetic field, consisting of 8 different directions: North, North-East, East, South-East, South, South-West, West, North-West. It outputs digital '1' as OFF and '0' as ON, turning on respective LEDs to indicate the direction of robot.

The compass is designed to be mounted as high as possible to maximise the distance between the compass and any ferro-magnetic parts that could distort the magnetic field, such as motors or aluminium frames. However, the compass has its own limitations, where it can be prone to inaccuracies which lead to the robot not adjusting to the direction accurately, hence can only be used for a general direction rather than a specific direction. This is because the compass only knows 8 directions and each direction has its own angle of tolerance.

Moreover, the compass heading may point in between 2 directions at same time, causing the LED to flicker, and causing the robot to behave erratically between both directions. This is noteworthy for programming as it is highly possible that the robot can behave differently and is indecisive due to inaccuracies.

Our group managed to overcome this limitation through tuning time delay to centre the compass heading, as well as tuning the turning speed or using this limitation to our advantage. For example, instead of forcing the robot to move straight, it could be better to allow inaccurate heading so that the robot can 'drift' to left and right during searching to cover more ground. Upon returning Home, the robot will self-adjust to true North and align straight upon bumping into collection boxes behind, given its wide base.

6.8 Wiring Pathway & Connections Design

Design Characteristics:	***Reasons:***
Stationary & Firmly Secured	Wiring must be held in place to prevent any free movement that risks entanglement or hooked by foreign objects. Must be secured together and prevent loose connections and standalone strands.
Avoid Rotating Elements	Prevent getting stuck or damaged by gears or wheels or rollers.
Clear Segregation	Wiring is clearly separated by each I/O sensors and motors for ease of maintenance and checks.
Loose Slack	Wiring should not be taut so that any sudden pull or yanking will not loosen the connection ports
Colour-Coded	Clear Wiring colours to differentiate for VCC, GND, Signal to prevent shorting when handling
Nomenclature	Clear Labelling for ease of identification of wiring port numbers and respective I/O
Well-Protected	Not exposed outside of robot's frame that risk collision damage, as well as well-insulated and no signs of copper wire exposure

Table 16: Wire layout characteristics and its respective reasons

The intended layout of wiring is to use the vertical pillars of robot's frame as pathways to travel from sensors to the microcontroller on top. Access points are open by the side of the robots or from above, whereas wires go through mounting holes of the frame as 'entry points' to hold the wires together. Mechanical clips are also used to bind these wires together on the frame to minimise any free movements or gaps that could cause entanglement. These wires are always within the structural frame and never exposed outside so that opponent collision will not damage these wiring connections. Although wiring pathways can be straight for ease of clarity, it is important for them to be very slack and curved a lot near the wiring ports at sensors and microcontroller. This embodies the principle of indirect force transmission, by giving flexibility and deformation of wiring to reduce the force transmission needed to pull out the wires. This can be observed by the arrangements of wires in the figure below.

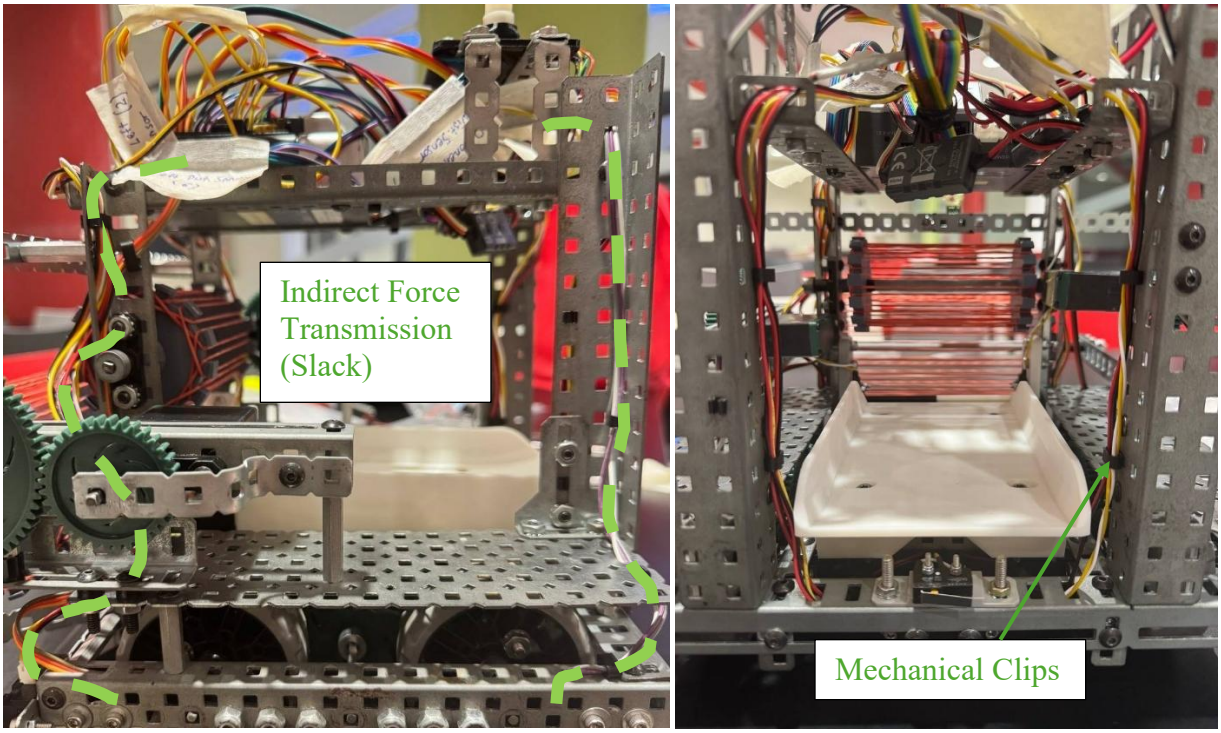


Figure 73: Designed Wiring Pathways on Robot (Side and Back View)

To ensure consistent colour coding for each sensor, the dedicated colours for line sensors are Red, Orange and Yellow, which translates to VCC, Gnd, Signal respectively. For the distance sensors, the colours are Green, Blue, Purple for Signal, VCC, Gnd respectively.

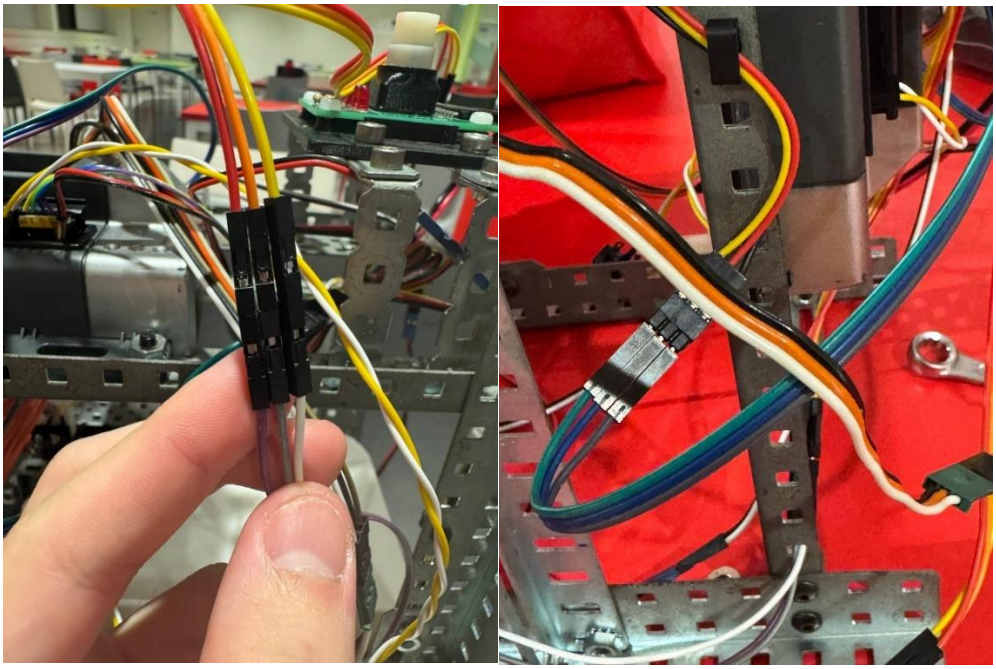


Figure 74: Colour-Coded Wiring for Sensors

When finalised, the intermediate wiring connections are taped together to ensure connections are tightly connected and no loose ends that could accidentally come off during testing when some force is applied such as vibration, jerking or rough handling. These tapes are also used as nomenclature for clear identification to reference the port number and sensor names for programmers to check and maintenance on the correct sensor system correspondingly. Overall, the electrical design layout optimizes both user interface for handling and maintenance, and robot's functionality during operational runs.

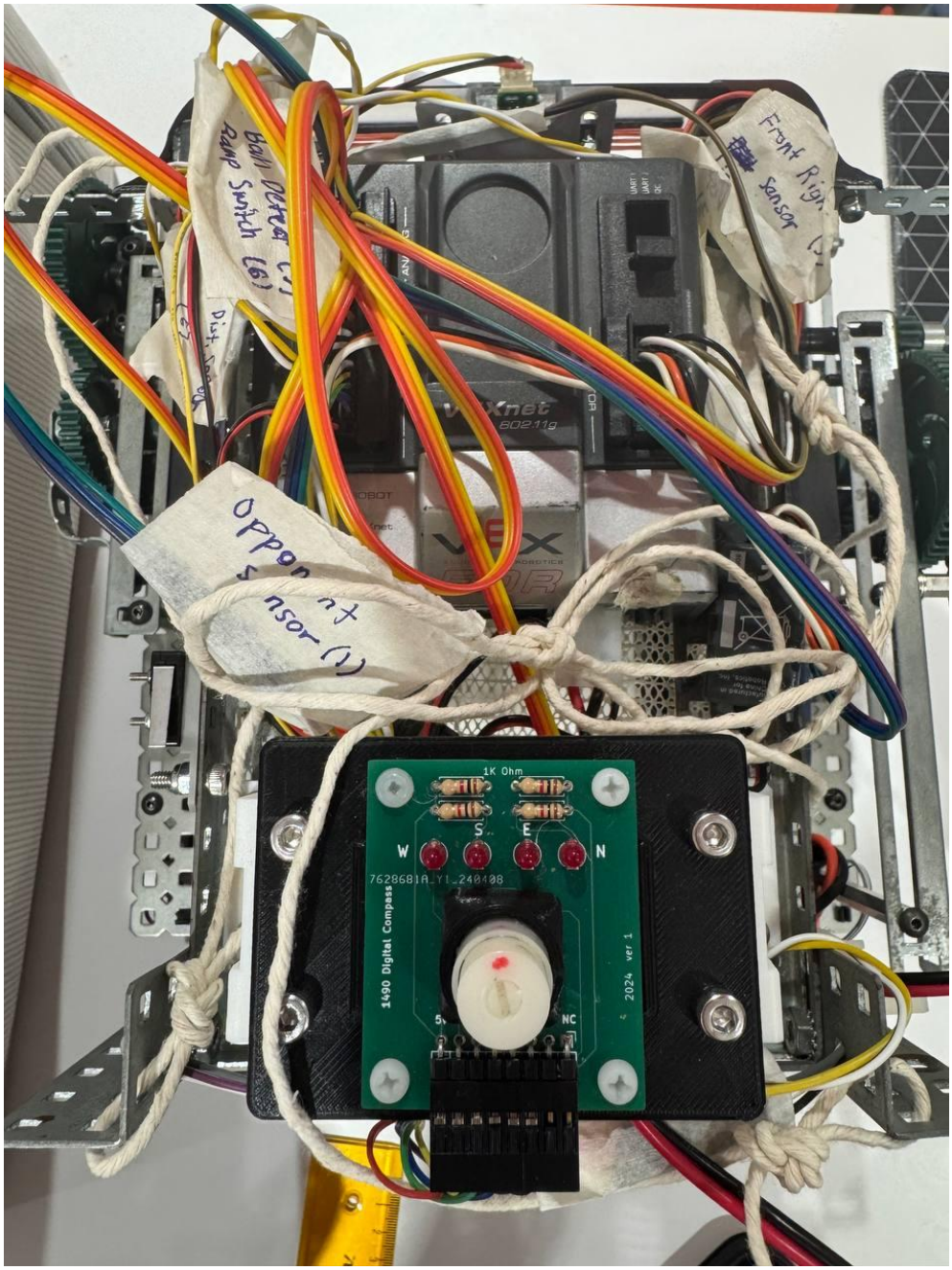


Figure 75: Wiring Layout from Top View

7 PROGRAMMING SOFTWARE DESIGN

7.1 Overview of Software Strategy

The software of the robot was developed as a rule-based autonomous control system to coordinate searching, ball collection, boundary avoidance, homing, and delivery during the competition. Its main purpose is to allow the robot to operate independently after the start switch is triggered, using real-time sensor feedback to decide its next action throughout the match. The control logic was designed to be simple, modular, and robust, so that the robot could respond reliably.

At a high level, the software operates as a closed-loop autonomous control system. The robot continuously reads sensor inputs, interprets the detected conditions, decides the next action based on predefined logic, and then executes the required motor commands. This feedback-driven cycle allows the robot to react to ball detections, boundary conditions, heading changes, and collection outcomes in real time.

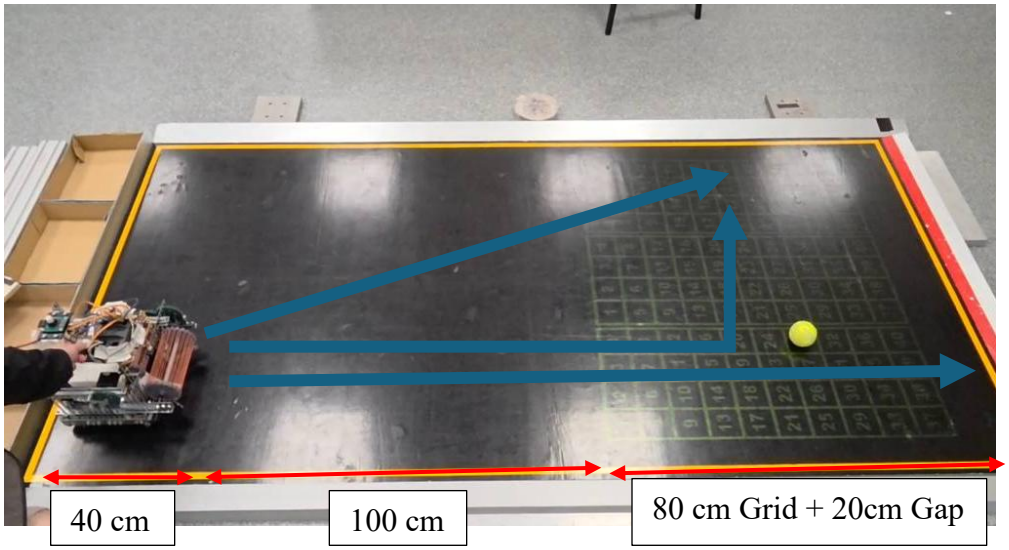


Figure 76: Arena Layout Dimensions & Ball Placement for Search Strategy

During the competition, a project grid layout is shown the arena whereby each set has 40 grids numbered from 1 to 40. There will be three sets corresponding to one tennis ball each, random number generated. To guarantee the first ball, our team believes the most straightforward method is to drive straight for the shortest path to the first ball. Next, we could follow-up a second pass with L-Pass or Diagonal Pass to intercept the opponent as quickly as possible. However, it could prove tedious to ensure the robot aligns to the correct direction when the compass is often inaccurate with positional errors in between. Moreover, there were other challenging considerations, such as if the opponent were to try and diagonally intercept and cut us off when moving straight, or how would the robot know which direction to change lane to. Nevertheless, each consideration will be tackled and discussed in the following sections.

Our software program logic is as follows:

After the robot is started, it first performs an initial forward pass while running the collection mechanism, or what our team calls as “first-pass”. During this stage, the robot attempts to secure

a nearby ball quickly by moving straight ahead and monitoring its front distance sensors and collection sensors. If a ball is detected, the robot turns towards it, aligns itself, and moves forward to attempt collection. If the ball is successfully secured, the robot transitions to the home-return stage. If the first collection attempt fails, the software includes a recovery scan to try to reacquire the ball before abandoning the attempt. This approach allows the robot to exploit early collection opportunities on its starting lane while avoiding excessive search time at the beginning of the run.

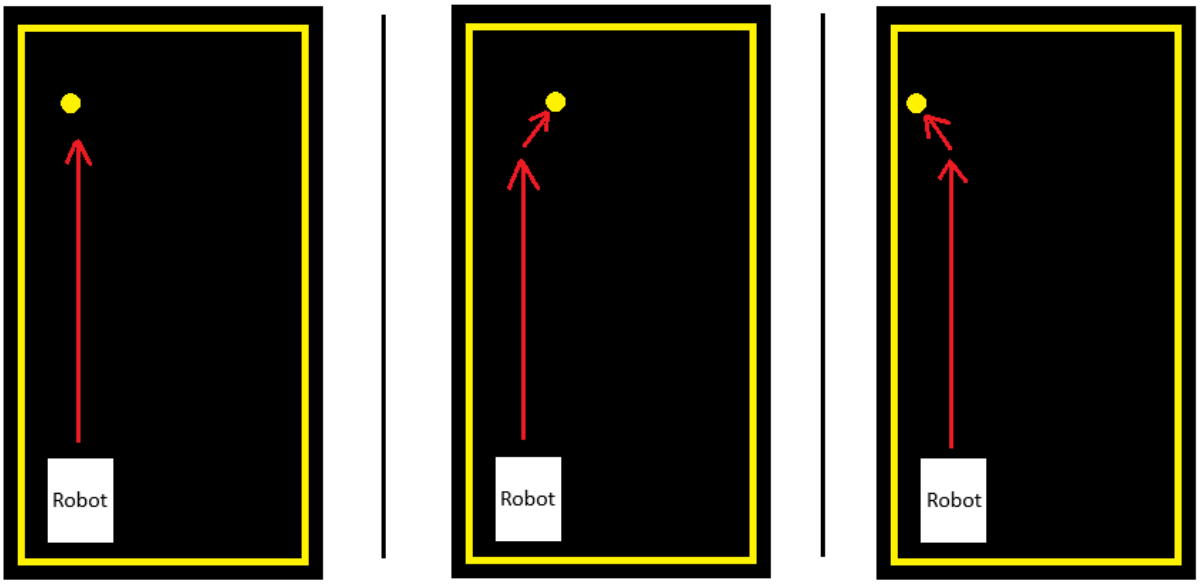


Figure 77: First Pass Ideation, Exploiting First ball collection

After delivering the first ball, the robot will no longer search for that same set of grids and 'change lane', cutting across diagonally at a 45-degree angle and points straight ahead. It will then run a loop of repeated cycles of sweep motion, by moving forward, sweep right, sweep left, and repeat until a ball is found. Otherwise, it will return Home, change to next lane, and repeat the whole sweep function again. This describes the second pass, third pass and so on.

The software was organized into modular functions so that each major behaviour could be implemented and tested independently. Low-level functions were used for motor actuation and sensor interpretation, while higher-level functions combined these basic actions into complete robot behaviours such as searching, centring on a ball, avoiding the boundary, returning home, and delivering the ball. This modular structure improved code readability and debugging efficiency, and it also made it easier to tune individual parts of the robot's behaviour during physical testing. Overall, the software strategy was designed to prioritise reliable autonomous operation, simple decision-making, and repeatable match performance within the constraints of the VEX controller and the competition environment.

7.2 Logic Flowchart

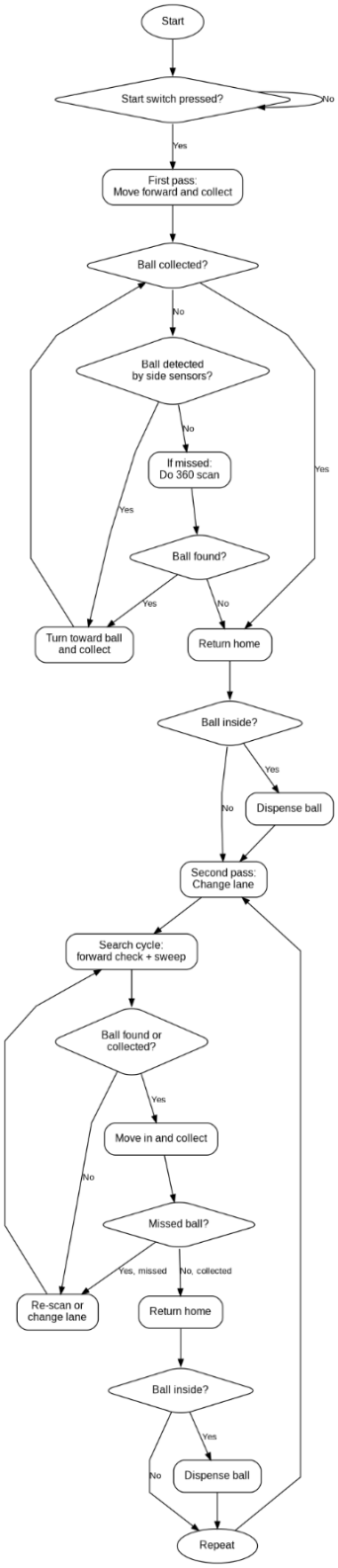


Figure 78: Flowchart Logic for Main Program

7.3 Helper Functions

Helper functions are useful for integrating the same commands and functions over the program repeatedly, which helps programmers reduce careless mistakes and tedious work when required to make a change.

The table below describes the helper functions used for our main task program:

Function Name	Description
void forward()	Drives both motors forward at speed 70 (left) and 75 (right). The slight difference in values compensates for motor imbalance to keep the robot moving straight.
void reverse()	Drives both motors in reverse at speed -70 (left) and -75 (right), again with offset compensation.
void turnLeft()	Sets the left motor backward at -65 and the right motor forward at 65, causing the robot to turn left.
void turnRight()	Sets the left motor forward at 65 and the right motor backward at -65, causing the robot to turn right.
void slowdown()	Drives both motors in reverse at a reduced speed of -60. This is used when approaching a detected ball to improve collection accuracy.
void reverseSlow()	Drives both motors in reverse at a reduced speed of -60. This is used after ball collection to back away gently.
void stopDrive()	Sets both drive motors to 0, bringing the robot to a complete stop.
void collect()	Runs the collection motor at speed 90 to drive the roller mechanism and pull the ball up the ramp.
void stopCollect()	Stops the collection motor completely.
void dispense()	Runs the dispense motor at speed 90 to release the ball into the collection point.
void stopDispense()	Stops the dispense motor.

bool ballCollected()	Return true if ramp limit switches are triggered. This indicates the ball has physically travelled up the ramp and is secured inside the robot.
bool opponentDetected()	Returns true if the top-mounted distance sensor reads above 320, indicating the opponent robot is nearby. Used to suppress false ball detections when the opponent is in the sensor's field of view.
bool leftDistDetected()	Returns true if the left distance sensor reads above 850 AND the opponent is not detected. Used to identify a ball on the left side of the robot.
bool rightDistDetected()	Returns true if the right distance sensor reads above 800 AND the opponent is not detected. Used to identify a ball on the right side of the robot.
bool leftDistNotDetected()	Returns true when the left distance sensor drops below 850. This is used as the exit condition when turning to centre the robot onto a detected ball. The robot will pause its turning if the opponent is in the way.
bool rightDistNotDetected()	Returns true when the right distance sensor drops below 800. This is used as the exit condition when turning to centre the robot onto a detected ball. The robot will pause its turning if the opponent is in the way.
int frontHitType()	Reads both front line sensors and returns an integer code describing the type of boundary contact: • 0 = no hit • 1 = left sensor only • 2 = right sensor only • 3 = both sensors When only one sensor triggers, it waits up to 100ms to check if the second sensor also

	triggers before confirming a corner hit, preventing false positives from slight angled approaches.
int backHitType()	Same logic as frontHitType() but for the two back line sensors. Returns 0, 1, 2, or 3 for the same respective conditions.
bool isFacingNorth()	Returns true when the compass reads north pin active, and all others inactive. This is to confirm that the robot is facing directly north.
bool isFacingNorthWest()	Returns true when both the north and west compass pins are active, used as an intermediate reference point during alignNorth().
bool isFacingSouthEast()	Returns true when both the south and east pins are active, used as the target direction for the right sweep in sweepMotion().
bool isFacingSouth()	Returns true when only the south pin is active, used as the target direction for the left sweep in sweepMotion().
void pointNorth()	A compass-based alignment function. Continuously checks the compass and turns the robot right if the west pin is active, and stops when then the robot is facing north. Used for quick heading corrections throughout the program.
void alignNorth()	A two-staged, more precise alignment function. Stage 1 turns the robot to face north-west as a consistent intermediate reference point. Stage 2 then turns right from north-west until the compass reads exactly north. This approach ensures north is always approached from the same direction, reducing overshoot.
bool frontLineDetected()	Returns true when frontHitType() returns 3, meaning both front sensors have hit the yellow boundary simultaneously, indicating the robot has reached the front wall of the arena.

bool backLineDetected()	Returns true when backHitType() returns 3, meaning both back sensors have hit the yellow boundary, indicating the robot has reached the home delivery zone.
bool backLeftHit()	Returns true when backHitType() returns 1, meaning only the back left sensor has detected the yellow boundary, indicating that the robot is drifting or angled to the left when reversing towards the home delivery zone.
bool backRightHit()	Returns true when backHitType() returns 2, meaning only the back right sensor has detected the yellow boundary, indicating that the robot is drifting or angled to the right when reversing towards the home delivery zone.
bool frontleftCornerHit()	Returns true when only the front-left sensor detects the boundary, indicating the robot is approaching the edge at an angle from the left.
bool frontrightCornerHit()	Returns true when only the front-right sensor detects the boundary, indicating the robot is approaching the edge at an angle from the right.
bool backleftCornerHit()	Returns true when only the back-left sensor detects the boundary, indicating the robot is drifting left while reversing home.
bool backrightCornerHit()	Returns true when only the back-right sensor detects the boundary, indicating the robot is drifting right while reversing home.
void boundaryFrontCheck()	Called when a front corner sensor detects the boundary. Reverses the robot away from the line for 600ms, then applies a brief corrective turn in the opposite direction of the hit sensor to straighten the robot back into the arena. Loops until no corner sensors are active.
void boundaryBackCheck()	Called when a rear corner sensor detects the boundary during home delivery. Moves the robot forward for 600ms to clear the line, then

	applies a brief corrective turn to realign. Loops until no rear corner sensors are active.
bool rightUntilNorth()	Turns the robot right while scanning with the left distance sensor until the compass reads north. If a ball is detected during the turn, it stops and "fine-tunes" itself by continuing to turn left until the robot centres on the ball. Returns true if ball is found, and false if north is reached without detection.
bool leftUntilSouthWest()	Turns the robot left while scanning with the right distance sensor until the compass reads southwest. Same ball centring logic applies. Returns true if a ball is found.
bool rightUntilSouthEast()	Turns the robot right while scanning with the left distance sensor until the compass reads southeast. Returns true if a ball is found during the turn.
bool check360()	Rotates the robot left for up to 5 seconds while scanning with the right distance sensor for a ball. If detected, the robot turns right until the sensor drops below threshold. This is used as a recovery mechanism after a failed collection attempt. Returns true if a ball is acquired, false if the full rotation completes without detection.
bool checkForward()	Moves the robot forward for up to 1000ms in 20ms increments while simultaneously scanning both left and right distance sensors. If a ball is detected on either side, it turns to face it and returns true. If the yellow boundary line is detected, it stops and returns false.

	If the ball is collected during forward movement without sensor detection, it also returns false to signal and unseen collection. Used at the start of each sweep cycle to check for balls directly ahead.
bool sweepMotion()	Executes a full 180-degree rotational sweep by calling rightUntilSouthEast(), then leftUntilSouthWest(), then rightUntilNorth() in sequence. If any of these detects a ball, it returns true immediately. Returns false if the full sweep completes with no detection. Yellow boundary line detections during the sweep cause the respective functions to abort early and return false.
void changeLaneRight()	Turns the robot to face north-east at 45 degrees, then drives forward for 10 counts to move into the right lane. If the front line sensors detect the yellow boundary during this movement, the robot re-aligns north, reverses home, and sets state = 2 to indicate the right boundary was reached. Otherwise it completes the move and calls pointNorth().
void changeLaneLeft()	Turns the robot to face north-west at 45 degrees, then drives forward for 10 counts to move the robot into the left lane. Used when state = 2, indicating that the robot is already at the right boundary.
void goingHomeToDeliver()	Calls pointNorth() to first ensure that the robot is facing north, then reverses towards the home delivery zone. Monitors the compass for drift and applies mild motor corrections if the robot is facing east or west.

	Also calls boundaryBackCheck() if back corner sensors detect the yellow boundary during reversal. Stops when both rear line sensors detect the yellow boundary and the compass confirms north. If the robot has not reached home after 15 seconds, it nudges forward briefly then resumes reversing, preventing the robot from getting permanently stuck.
void ballDelivery()	Stops the drive motors and activates the dispense motor for 1 second to release the ball into the collection point, then stops the dispense motor.
task main()	Main program flow The program waits in a loop until the start limit switch is pressed. Once pressed, the robot executes the following sequence: First pass - The robot moves forward while running the collector motor, scanning for balls with both front left and front right distance sensors. If the sensors detect a ball, the robot turns to align with the ball and drives slowly forward with the collector running until the limit switch confirms capture of it the front boundary line is reached. If the front boundary is reached without collecting a ball, it executes a 360-degree spin in attempt to acquire the ball. After the first pass, regardless of capture success, the robot reverses home and delivers the ball if one was collected. Second pass - The robot changes lane (to the right by default, and to the left if the right boundary was already reached), and then

	repeatedly executes a forward scan followed by a 180-degree sweep, up to 4 times. If a ball is detected, it drives forward to collect it. If collection fails, it attempts a 360-degree reacquisition spin before giving up and returning home. After delivery, state is reset to 1 and the second pass loop repeats, this allows the robot to continue searching and collecting.

Table 17: Helper functions used for our main task program

7.4 Search Algorithm: First Pass

The robot's first pass is described to be as simple as moving forward straight ahead and collecting the first ball, which is the most ideal case scenario. This is because we have designed the robot to encompass the entire width of the grid set, spanning a total of 4 grid squares which measures to be 35 cm estimated. Although the robot cannot exceed 30cm, we recalled that the sensor's FOV is able to sense 8cm more towards one side. This means that our robot is more than sufficient to detect the first ball when driving straight, regardless of where the ball might be. Furthermore, when we tested our robot against the grid layout, we realised that the V-guides span as wide as 2 grids, while the left and right ball sensors are at the centre of the outermost grid square. This confidently tells us that our first pass in the most ideal case will work.



Figure 79: Arena's Grid Layout with Our Robot

In the ideal case scenario, our first pass should fall under three scenarios. One is the ball is at the middle (2 grids) between the sensors, which means the robot will just continue moving forward until it detects the ball is inside without seeing it. Second is it detects a ball on the left, stops, and adjusts itself at an angle to centre the robot, before moving in to collect. Third is the same for the right side of the robot.

To centre the robot, the robot must first detect the ball by whichever side the sensor is on. It then turns in that direction until the sensor no longer detects it. Finally, the programmer will tune the time delay, calibrating accordingly to ensure that the ball is always centred after turning.

Given the robot's size, it can cover up to 4 squares, and its sensors also cover a range of up to 4 squares. This means that ideally, the robot can detect the first ball just by moving in halfway through, which means that the first pass will be short, quick and simple when executed correctly, especially given our high-speed configuration.

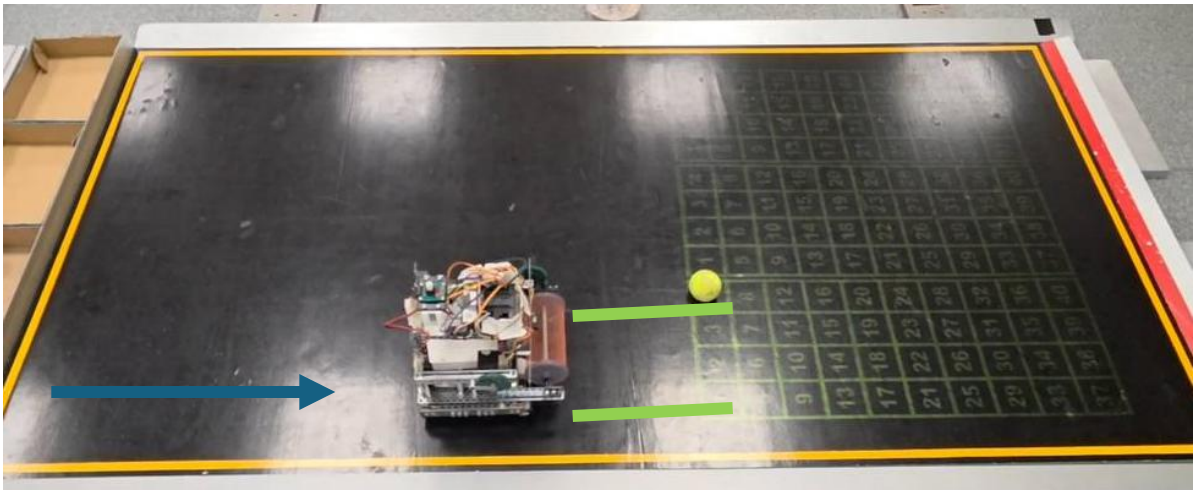


Figure 80: Ball Detection During First Pass

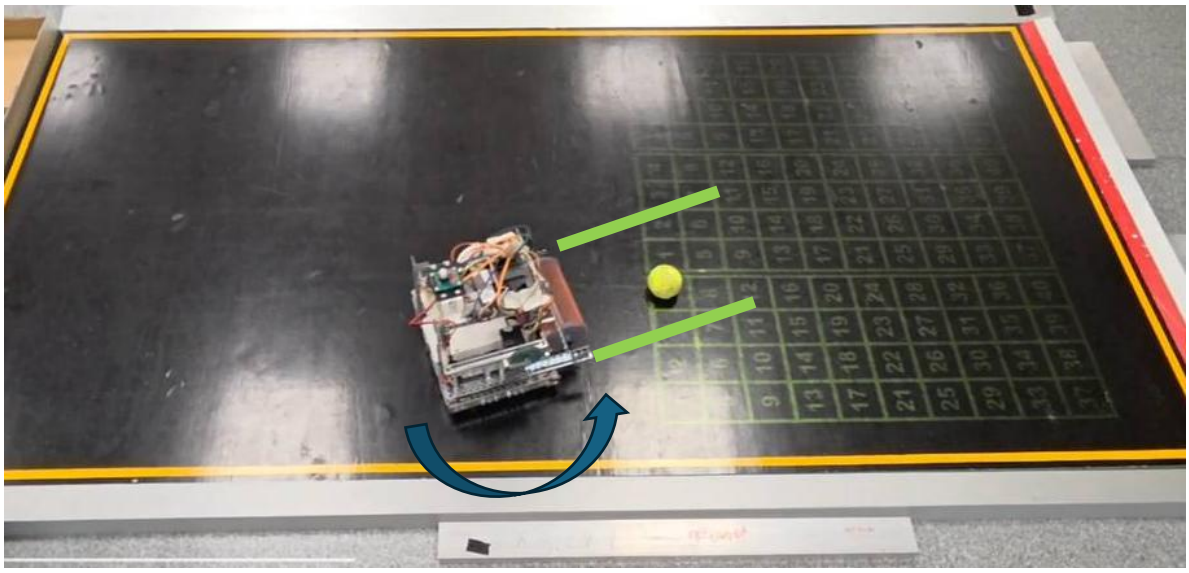


Figure 81: Align to Ball when Detected at Sides

However, our robot can be prone to swerving left or right due to the difference in traction between the wheels and arena, the changes in assembly of drive base, the weight distribution and centre of gravity, as well as the battery voltage. This means that if the robot does not drive straight, the chances of success for first pass will lower drastically and can often miss or go out-of-bound. This leads us to the other worse-case scenarios, such as when it misses the ball or if it detects boundary line.

To mitigate and overcome these limitations, we re-assembled the wheels and gears screw joints to ensure that the wheels are fully in contact and not misaligned. We realised that the spacers used between for screw joints are uneven due to tightening one side more than the other, hence we changed spacers to bearing lock plate which helps even out both sides during tightening, as compared to individual spacers. This change not only significantly reduced the swerving tendency of our robot, but also made our point turns much smoother with less 'jittering' vibration from slight uneven wheels.

We also coded failsafe function into the first pass, where a miss will cause the robot to reverse and scan 360 degrees around it to search for the missed ball for a second attempt.

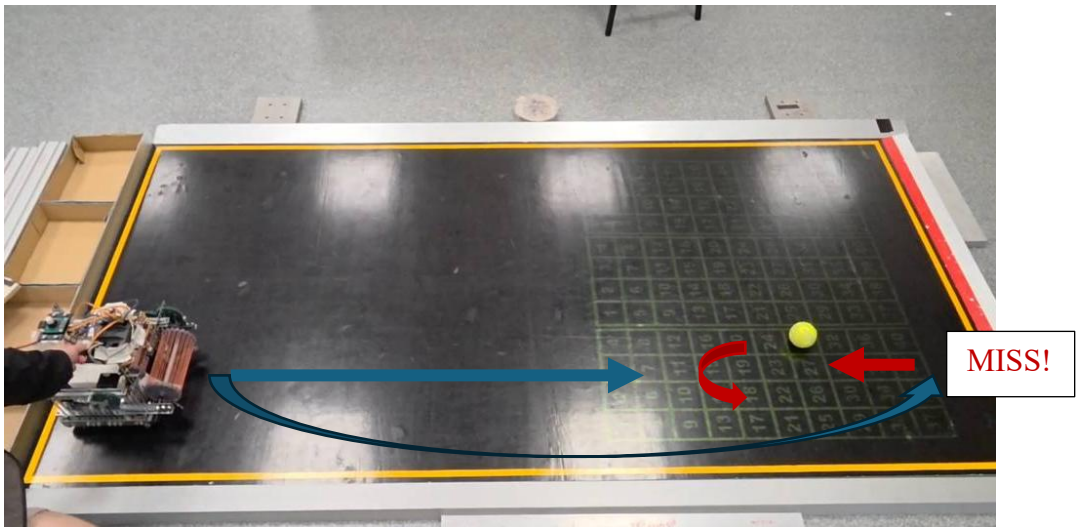


Figure 82: First Pass Scenarios

Furthermore, we tested numerous runs to fine-tune the speed accordingly at different voltage levels. We realised that at max charge, the robot's left motor turns faster which causes the robot to swerve left, at medium charge it would go relatively straight, and at low charge the right motor spins faster, swerving the robot to the right. To improve our chances of success, we tracked our voltage levels and fine-tune accordingly for max and medium charge, especially because the projected grid layout is slanted and it benefits us to swerve slightly left. Moreover, our robot handler practices run weekly to make himself well-prepared to manually position the robot to make it move straight. For example, it is important to place the robot and push the robot forward, hearing the motor gears turn, eliminating the risk of backlash that could cause differential speeds in turning and worsen the swerving tendency if one starts later than the other wheel.

7.5 Search Algorithm: Second Pass

The second pass logic was designed to provide a more systematic and structured search strategy after the initial straight-line collection attempt had been completed. Unlike the first pass, which prioritised speed and direct acquisition, the second pass focused on improving arena coverage and increasing the chance of detecting any remaining balls that were not located along the initial search path. To achieve this, the robot combined lane changing, short forward scans, angular sweep motions, and repeated collection checks in a cyclical manner. This allowed the robot to move beyond a single corridor of search and instead inspect multiple adjacent regions of the arena in a controlled sequence.

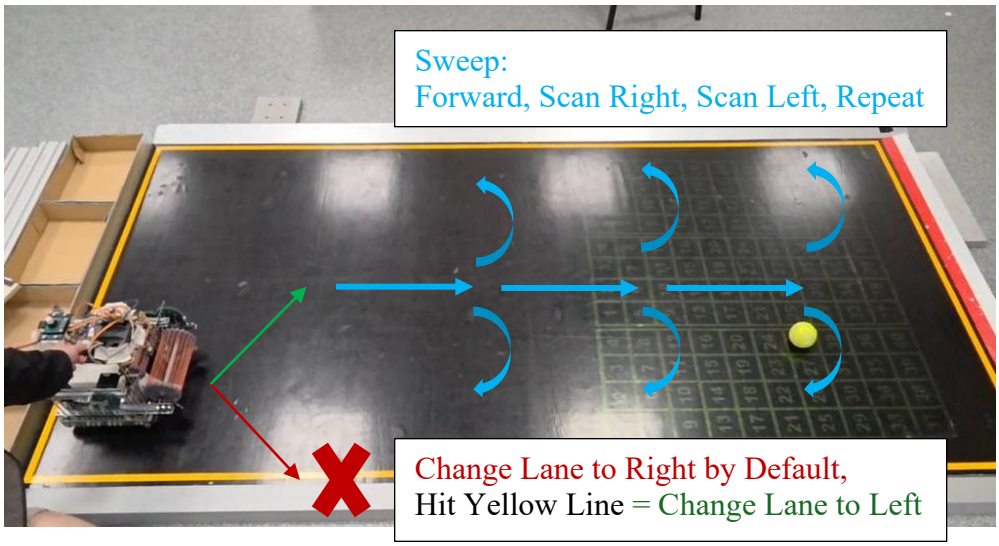


Figure 83: Second Pass Illustration

At the start of each second-pass cycle, the robot performed a lane-change manoeuvre in a given direction (always turn right), which determined whether it should shift toward the left search lane or stay right search lane. This state-based lane change was important because it prevented the robot from repeatedly scanning the same path and instead encouraged broader arena coverage over time. After shifting into the next lane, the robot reoriented itself to face north so that each search cycle began from a known reference direction. This reduced navigational drift and helped maintain consistency in the sweep pattern, making the robot's behaviour more repeatable and easier to tune. It is important to note that the robot is clueless which direction it should change lane, which is why we used the yellow line detection as an indicator. If the robot encounters yellow line under a certain distance, it will attempt to change lane, otherwise it has chosen the correct direction for lane change.

Once aligned, the robot entered a short forward-check phase, where it moved forward while monitoring the ball sensors, line sensors, corner detection, and opponent sensor. The purpose of this stage was to exploit any nearby ball that could be collected quickly without requiring a full angular sweep. If a ball was detected during this forward motion, the robot immediately corrected its heading toward the target and transitioned into collection behaviour. If a ball was collected without being explicitly detected, such as rolling into the collector unexpectedly, the software still allowed the robot to proceed correctly by checking the storage confirmation condition. This made

the logic more robust, as the robot was not entirely dependent on one sensing route for successful collection.

If the forward-check phase did not produce a valid target, the robot then executed a sweep motion. This consisted of turning through defined angular ranges to scan neighbouring sectors of the arena using the side distance sensors. The sweep allowed the robot to search laterally without drifting too far from its intended lane. By combining multiple sweeps with intermittent forward advancement, the robot could inspect a larger area than would be possible through straight motion alone. This was especially useful for locating balls positioned slightly off-axis from the robot's current heading. The software repeated this sweep-and-check cycle for several iterations, allowing multiple opportunities to detect and approach a target before giving up on the current lane.

Once a ball was detected during the second pass, the robot entered a collection phase in which it moved forward slowly with the collector active. During this stage, the software continuously monitored whether the ball had been successfully stored, whether the front line had been reached, or whether a front corner hit had occurred. This ensured that the robot could safely pursue the target while still respecting arena constraints. If the collection attempt failed, the robot reversed slightly and executed a 360-degree reacquisition routine to attempt recovery. This gave the robot a second chance to regain the target rather than immediately abandoning the effort. If no ball was found after this recovery step, the robot realigned itself and prepared to move on, avoiding unnecessary time loss.

After each second-pass search cycle, whether successful or unsuccessful, the robot executed its return-home routine. If a ball had been successfully collected, it was delivered at the home zone before the next lane-change cycle began. This repeated process of lane change, forward check, sweep search, collection, and return home formed the core logic of the second-pass software design. Overall, the second pass played a crucial role in improving search coverage, enabling repeated structured exploration of the arena, and increasing the robot's effectiveness beyond the simpler first-pass strategy.

Here are some examples to illustrate the possible scenarios for ball capture during sweep function:

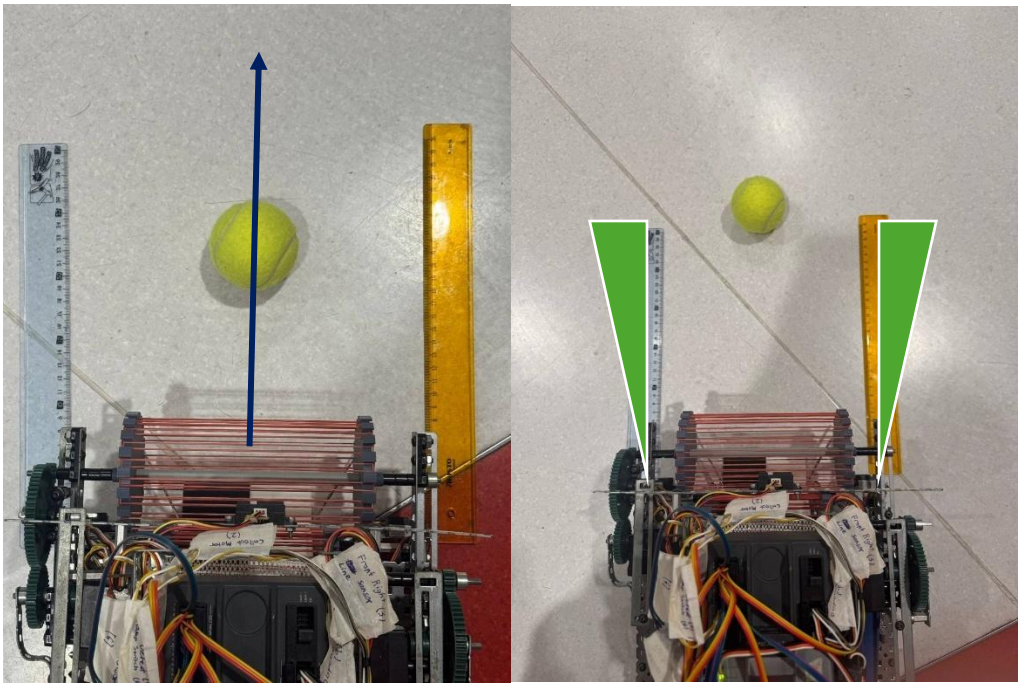


Figure 84: Ball Encountered During Forward

Suppose the tennis ball is within 30 cm after lane change or between each sweep, this means the robot should be able to pick up the ball during its forward motion of 1 sec, which is about 45cm forward. During this period, the robot behaves similar to its first pass function, over its 3 possible scenarios. Either the robot takes the ball blindly without sensor detection (in front), or it detects ball on either side and turns to centre itself before collecting it. Either way, the robot will guarantee the ball in between sweep function. It should be noted that there is a chance the robot could be exactly 45cm, which means the robot stops before ball is fully collected and just touched. Hence, it is crucial to keep actuating the collector for half a second more after stopping to ensure the ball can be somewhat collected inside the robot. If the ball is beyond 45cm, then the robot will finish scanning sweep and get it in its next forward iteration.

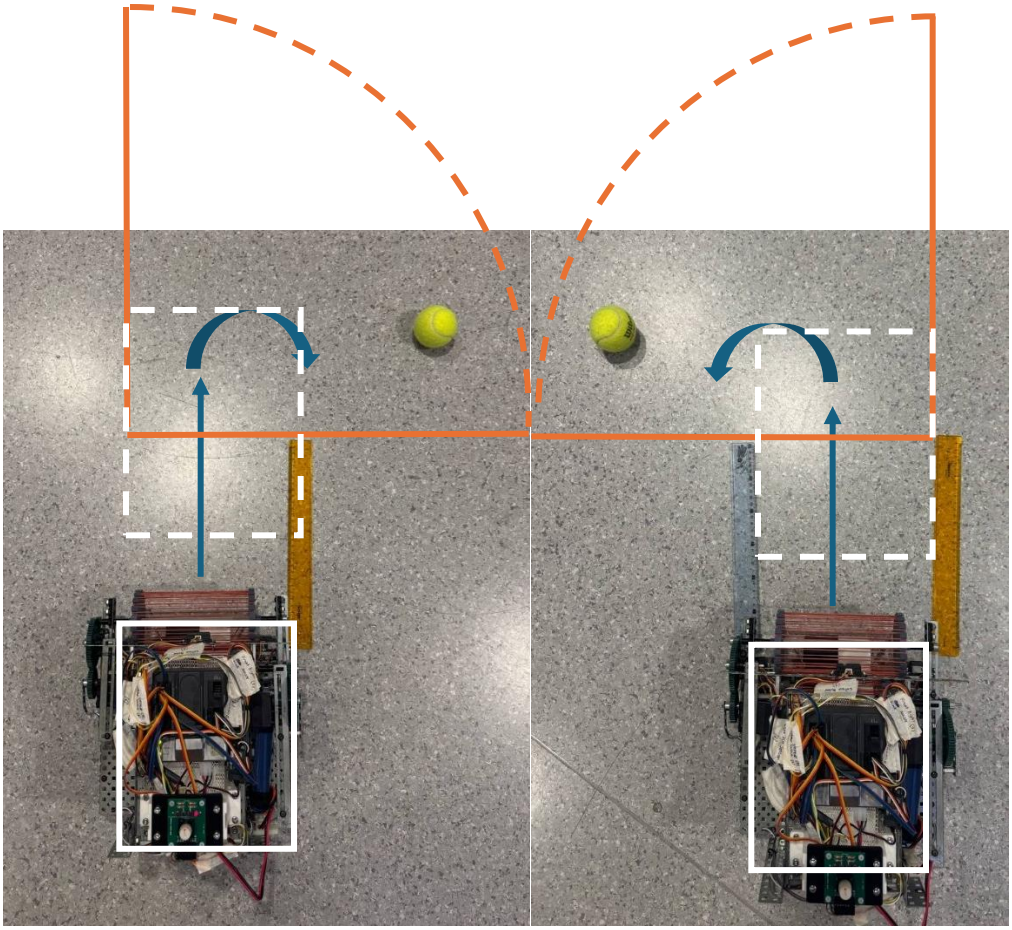


Figure 85: Ball Encountered During Sweep

If the ball happens to be placed or knocked away to another lane close by, the robot will be able to scan right and then left, giving itself a 180-degree view of its vicinity and find the ball successfully. During the scan, the robot will only use one of its sensors for detection. For example, when it turns right, it triggers the left sensor and will compensate by turning right until the ball is 'gone' to centre the ball aligned with its collector, drive straight for collection. Likewise, during left turn, it will trigger the right sensor and compensate by turning left until the sensor no longer detects it, centring the robot to the ball.

Overall, the second pass is to iteratively search the arena through sweep and scanning motion, chasing after the first signal detected as a ball and commit to it. If undetected, the robot will change lane and try again.

7.6 Home Return & Delivery Logic

After a ball is successfully collected, the robot executes a home-return routine to bring the ball back to the delivery area. Before reversing, the robot first uses the compass to align itself to face north. This ensures that the return movement starts from a known heading and improves consistency when travelling back to the home zone. Once aligned, the robot reverses while monitoring the rear line sensors, rear switch, and compass signals.

During the return motion, small heading corrections are applied if the robot begins to drift left or right. If the rear corner sensors detect the boundary at an angle, the robot performs a correction routine before continuing to reverse. This helps prevent poor alignment when approaching the home area. The robot stops when both rear line sensors detect the boundary and the heading condition is satisfied, indicating that it has reached the intended delivery position.

Once the robot reaches home, the drive motors are stopped, and the dispensing motor is activated for a fixed duration to release the ball into the collection area. This separates the homing and delivery stages clearly, so that the ball is only released after the robot has fully reached the correct position. Overall, this homing and delivery logic allows the robot to return and score in a controlled and repeatable manner after each successful collection.

7.7 Recovery & Failsafe Function

Several recovery and robustness features were implemented to improve the robot's performance under real testing conditions, where ball detection, collection, and navigation did not always succeed perfectly on the first attempt. These features were included to reduce the chance of the robot getting stuck, repeatedly making the same mistake, or losing too much time after a failed action.

One important recovery feature was the use of a 360-degree scan after an unsuccessful collection attempt. If the robot detected a ball but failed to capture it, it could rotate and attempt to reacquire the ball before giving up and returning home. This improved the chance of recovery when the ball shifted position during contact or when the first intake attempt was slightly misaligned. What makes this function impressive is that the robot can endlessly commit to the ball forever, until either the ball is knocked too far away, or the ball is stolen away, but the chances are low given how difficult it is to push our tank drive robot aside. Of course, to prevent the endless chase from going out of bound, it is important to try and reverse back to the same spot each miss, to increase the chance of search within the same vicinity area.

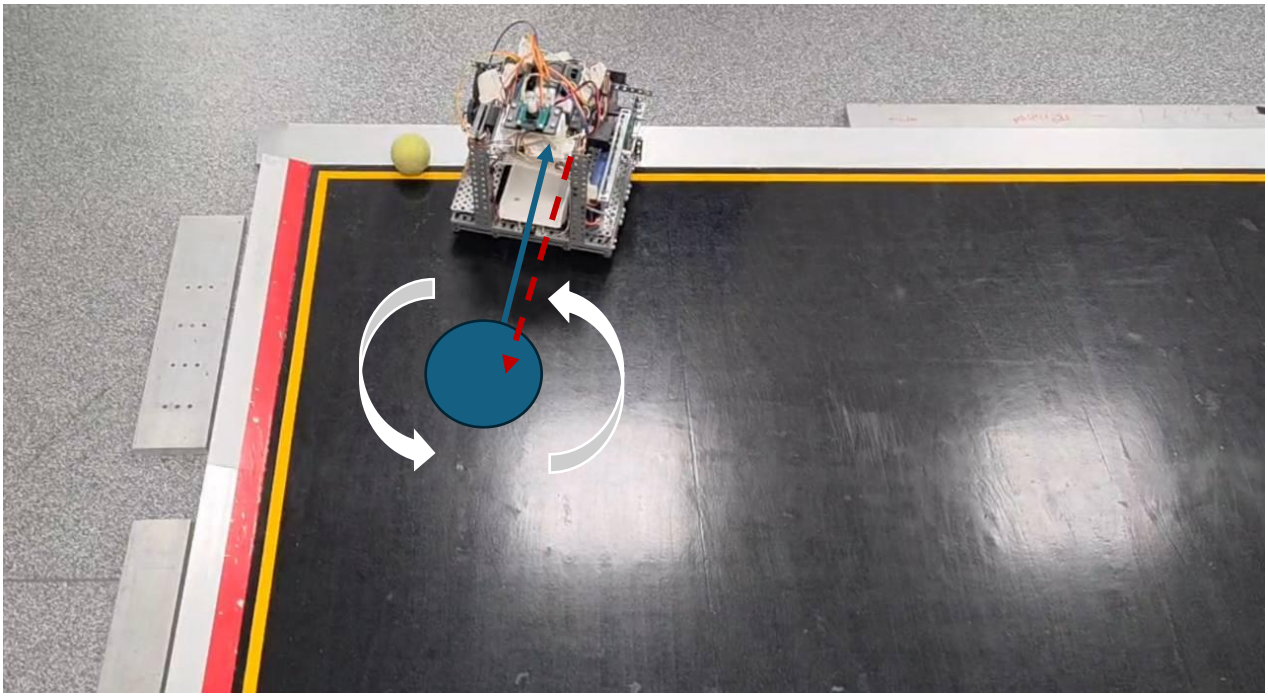


Figure 86: Missed Ball During Second Pass

For example in this scenario, the robot misses the ball during its sweep function, activating its fail safe function. It reverses back to its last known location, turn left to scan 360 degrees until it detects the ball and attempts to collect again. Note that only right sensor is used to trigger detection, and when detected it will compensate by turning right until undetected, centering the ball to the robot for collection.

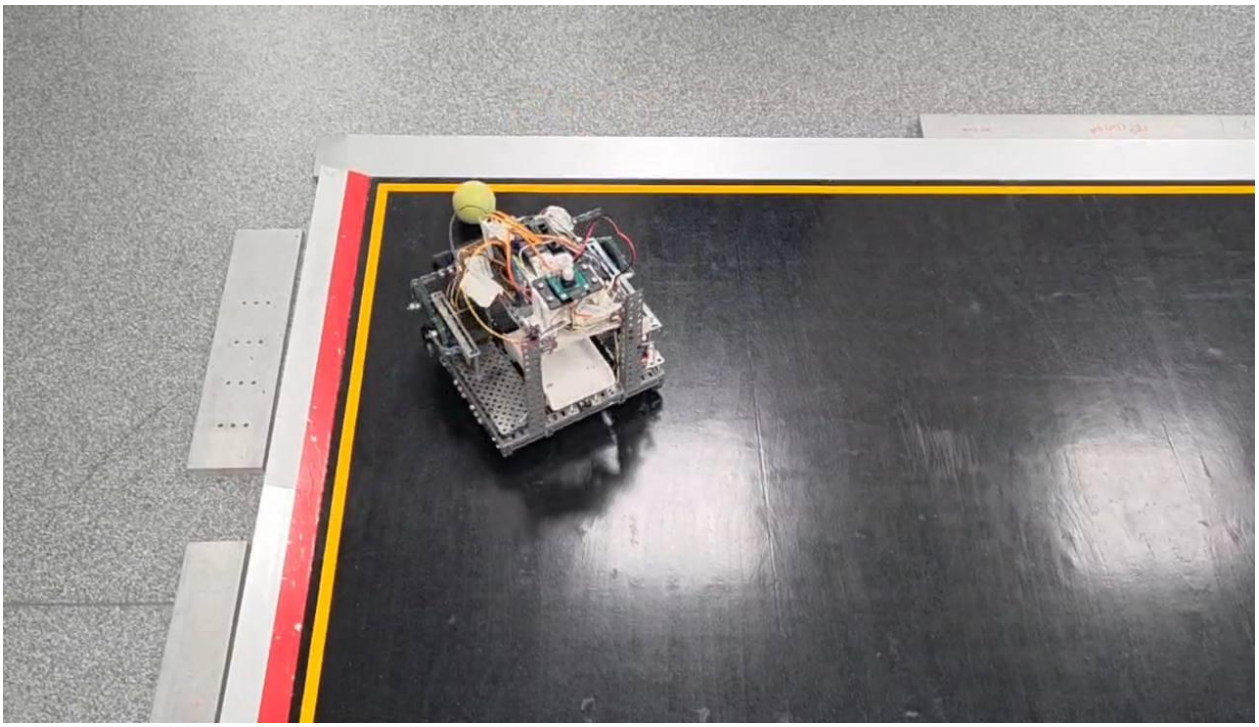


Figure 87: Failsafe Function to Recover Ball

Robustness was also improved through boundary correction and homing recovery behaviour. When the robot approached the arena boundary at an angle, corner-detection logic allowed it to reverse or move forward slightly and correct its heading before continuing. During homing, a breakout timer was included so that if the robot failed to reach the home area within the expected time, it would briefly move forward and then retry the reverse motion instead of remaining stuck. This helped prevent the robot from wasting time in an unproductive state.

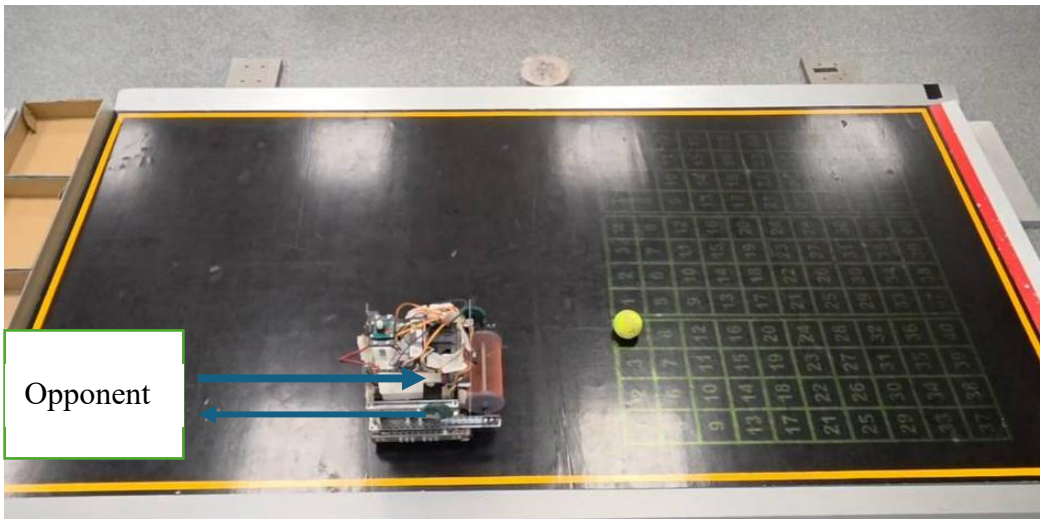


Figure 88: Breakout Function

7.8 Edge Detection Logic

Four line sensors were used to detect the arena boundary. Two are placed at the front of the robot and two at the rear. Their purpose is to determine whether the robot has approached the yellow boundary either head-on or at an angle.

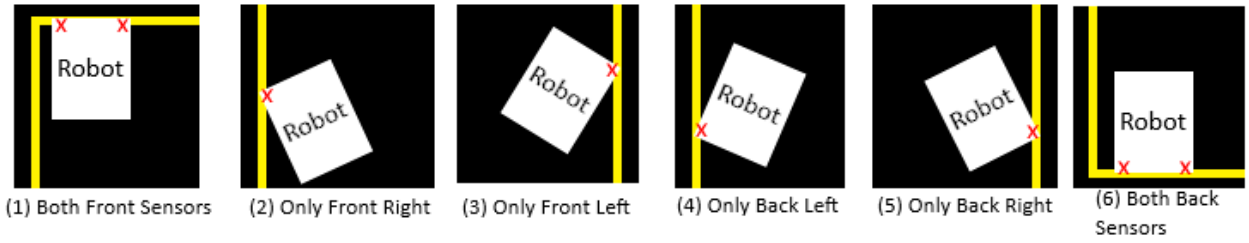


Figure 89: Schematic illustration of possible front and corner boundary-detection cases

In the software, these sensors are not only used as simple on-off boundary detectors but are also interpreted in combinations to identify the type of contact. For example, if both front sensors are triggered together, the robot interprets this as reaching the front limit of the arena. If only one side is triggered, the software interprets this as a corner or angled approach and applies a corrective response. The same logic is used at the rear during the home-return stage. This improves robustness because the robot can react differently depending on how it reaches the boundary rather than treating every line detection in the same way.

However, this logic has an underlying problem:

Suppose the robot is reversing Home to deliver the ball, and the back right sensor touches the yellow line (in scenario 5). How is it able to differentiate between Scenario 6 at Home delivery and Scenario 5 to course-correct and move straight? To the robot, unless the robot is straight completely and activate both back sensors together, it will always treat it as course correction, including at Home to deliver the ball, where it will keep course correcting.

Hence, the team's solution is to differentiate through the use of a delay timer. After the corner sensor activates, the robot will count a 100ms delay checking what is the next sensor that triggers. If the other back sensor triggers, it means it has reached the back line for Home to deliver the ball. Otherwise, if a front sensor activates next, then it has hit the side line and must course-correct to point North. This way, the robot can now uniquely identify each scenario and apply unique strategies to each. For example, any side lines will course correct accordingly, back line means to deliver, front line hit means miss and activate failsafe.

7.9 Opponent Detection Logic

The opponent detection logic was introduced to prevent the robot from wrongly interpreting the opposing robot as a ball target during search and collection. In a dynamic arena, the opponent may enter the field of view of the distance sensors and produce readings similar to those of a ball, especially during turning or forward scanning. Without a dedicated filtering strategy, this could cause the robot to chase the wrong object, waste time, or lose positional control. For this reason, the opponent sensor was used as a higher-priority interference signal that temporarily suppresses ball-targeting behaviour whenever the presence of the opposing robot is detected. This improves the reliability of the search algorithm by ensuring that the robot distinguishes between a valid collectible target and a moving obstacle.

In the software design, opponent detection plays a blocking and stabilising role rather than an aggressive combat role. During forward checking, if the opponent is detected, the robot stops its drive motion while keeping the collector active, allowing the system to pause safely without losing the chance to collect a nearby ball if it rolls inward. The program then waits until the opponent sensor clears and, in addition, requires the side distance sensors to remain clear for a short duration before resuming normal search behaviour. This extra waiting period is important because the opponent may leave the central sensor's view while still being visible to the side sensors, which could otherwise cause false ball locking immediately after the obstacle appears to disappear. By introducing a brief "all-clear" confirmation, the software reduces the chance of misclassification and makes the robot's response more stable.

Opponent detection is also integrated into the sweep logic using a timer-based ignore window. During angular scanning routines such as `rightUntilSouthEast()`, `leftUntilSouthWest()`, `rightUntilNorth()`, and `check360()`, detection of the opponent resets a timer that temporarily blocks ball detection for a short period. This was necessary because a centrally placed opponent sensor may only detect the opposing robot for a brief angular window during turning, whereas the side distance sensors may continue to see the same object immediately afterwards. Without this timed suppression, the robot could momentarily lose opponent detection and then instantly relabel the same obstacle as a ball. The timer-based logic therefore acts as a memory effect, allowing the robot to continue its sweep without prematurely committing to a false target. This makes the scanning behaviour more robust under moving-opponent conditions.

Overall, the opponent detection logic serves as a protective layer within the robot's decision-making system. Its purpose is not to engage with the opponent directly, but to preserve search accuracy, reduce false detections, and prevent unstable reactions caused by dynamic obstacles. By using opponent detection as a temporary suppression signal during both forward search and sweep scanning, the software improves target discrimination and ensures that the robot's collection logic remains focused on actual balls rather than on the presence of the opposing vehicle. This contributes to more reliable autonomous behaviour and supports the overall strategy of safe searching, controlled collection, and efficient delivery.

7.10 Testing & Troubleshooting

While we have already discussed all the errors and problems faced previously in each section, they will be reiterated here again, and the more major problems will be discussed in full details.

7.10.1 Sensor Ghosting

Sensor Ghosting is one of the most troublesome issues to troubleshoot for our team, and it was also one of the many problems encountered by other teams who could not overcome it but us. The problem is the left sensor is seeing a 'ball' that does not exist, and when detected it tried to turn towards it, and centre itself by turning until not detected. But since it keeps ghosting, it just keeps spinning continuously until it faces the collection boxes at the back and treats it as the ball again. There are a few ways to reduce and minimise ghosting, but it is inevitable that it happens at times. One way is to optimise the sensor placement, by customising a sensor holder for adjustable height and angular mounting of sensor. Our left distance sensor tend to have readings that spike high during test runs, as if it reads an object in front except there is nothing, hence the term 'ghosting'. Initially, we tried lifting the sensor at a higher height, as we thought the sensor was reading the ground constantly, being too low. However, even after adjusting to higher height, the ghosting still happened.

Next, we decided to replace the sensor with a new one, assuming that the sensor is faulty in some way. However, the issue persisted and we concluded it was not the sensor's fault but some interference.

The next thing we did was to push the sensor outwards by a set distance using a spacer, as shown in **Figure 91**. We thought that the sensor might be having some interference with the mechanical frame on top, and pushing it out further would help minimise FOV intersecting with the frame. While the ghosting is reduced, it still happens and persisted.

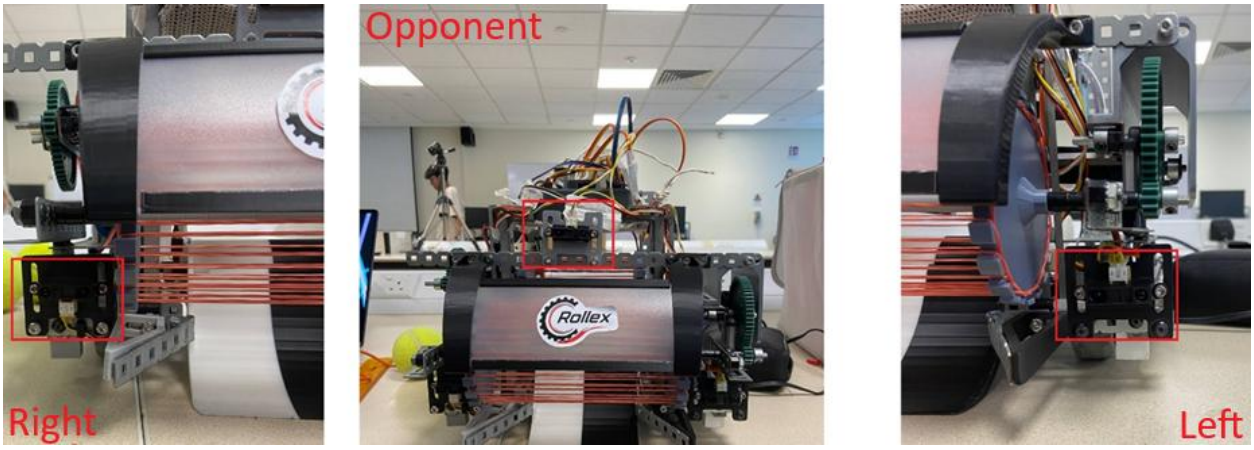


Figure 90: Distance sensor placement

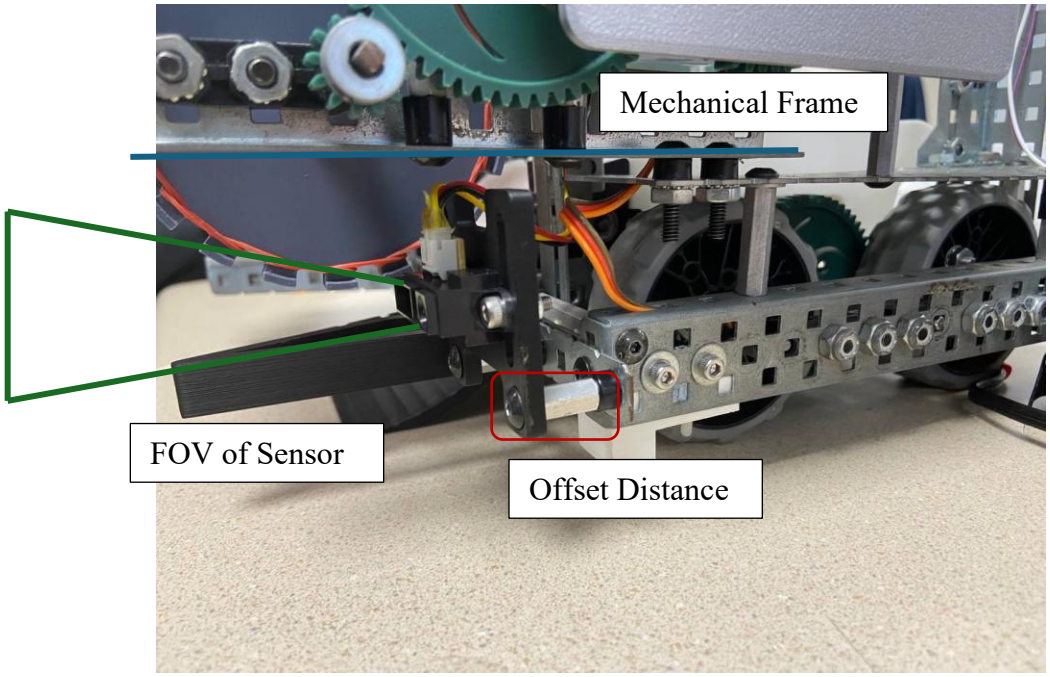


Figure 91: Left distance sensor placement

Following that, we assumed it could be the arena interfering due to uneven ground or ambient noise, because the problem only happens at the start of the program before the first pass. Hence, we hard code to force the robot to travel down further by 2 seconds, more than halfway down the arena before the first pass. However, the ghosting came back right after the 2 seconds finish. Somehow, the robot left sensor is detecting a ball that is not there and keeps turning.

Next, we added a start switch to the robot. An additional digital input, namely the start limit switch, was introduced to improve the reliability of the robot at the beginning of each run. During testing, it was observed that some sensor readings were not immediately stable when the VEX system was first powered on. If the autonomous routine was allowed to start instantly at power-up, the robot could react to these unstable initial readings and begin moving incorrectly. By requiring the robot to wait for a manual start-switch trigger, the controller and sensors were given time to stabilise before the software began executing the main search routine. This acted as a simple but effective sensor initialization measure and improved repeatability between runs.

The idea is that when the robot is ON, there is an initial voltage spike that could cause ‘ghosting’ on the sensors. This means that the sensors need some time to warm up before running the robot. Hence, we always ON the robot, and then pressed the switch 15 sec later to start the run. While this addition decreased the ghosting tendency, the problem eventually came back to haunt us once again.

Finally, we tuned the threshold of the sensor to be less sensitive. This code edits somehow made the ghosting disappeared almost completely, allowing us to overcome this persistent bug over the weeks of testing and troubleshooting.

7.10.2 Motion Control Functions: Swerving

The robot's movement is controlled using a set of basic motion functions for forward movement, reverse movement, left turning, right turning, slow approach, slow retreat, and stopping. These low-level functions form the basis of all higher-level behaviours such as ball searching, alignment, boundary correction, homing, and delivery. By organizing the code into reusable motion functions, the software becomes easier to tune, debug, and modify during testing.

The robot uses a differential drive system, where the motion of the left and right wheels is controlled independently. Straight-line movement is achieved by driving both wheels in the same direction, while turning is achieved by driving the wheels in opposite directions. During testing, it was observed that one drive motor was slightly faster than the other even when given the same command value. To compensate for this, slightly different motor speeds were assigned in the forward() and reverse() functions so that the robot could travel more consistently in a straight line. This helped improve navigation accuracy during searching and homing.

However, tuning these speeds are insufficient, because these motor speeds can vary according to the battery voltage level. This is because the more charge there is, the faster the motor RPM will be, and vice versa. This difference in speed can cause significant change in motion, especially if the physical assembly has different friction and losses already and accumulate. For example, at maximum battery charge, the robot would tend to swerve to the right, whereby the left motor is faster. At medium battery charge, the robot will drive relatively straight at same speeds. At low battery charge, the robot will swerve to the left, whereby the right motor is faster. Hence, we decided to always charge and test our robot around 70%~100% charge and tune the speeds within that range. It is also important for the robot handler to understand and practice positioning to maximise the chances of first pass to be successful.

Another important solution is to keep troubleshooting the mechanical assembly of the drive base. Minor tweaks such as stabilising the screw joints ensure that our robot wheels are completely flat with no 'rocking' motion when laid. Initially, we used individual spacers between the gear and wheel for each screw joints. However, this means uneven tightening will cause the gear and wheels to be slanted. Hence, by replacing the spacers with bearing lock plate to connect both, it ensures flat and even tightening, making the wheels and gears completely flushed, improving the drive motion and reducing the swerving significantly.

7.10.3 Compass Inaccuracy in Cardinal Headings

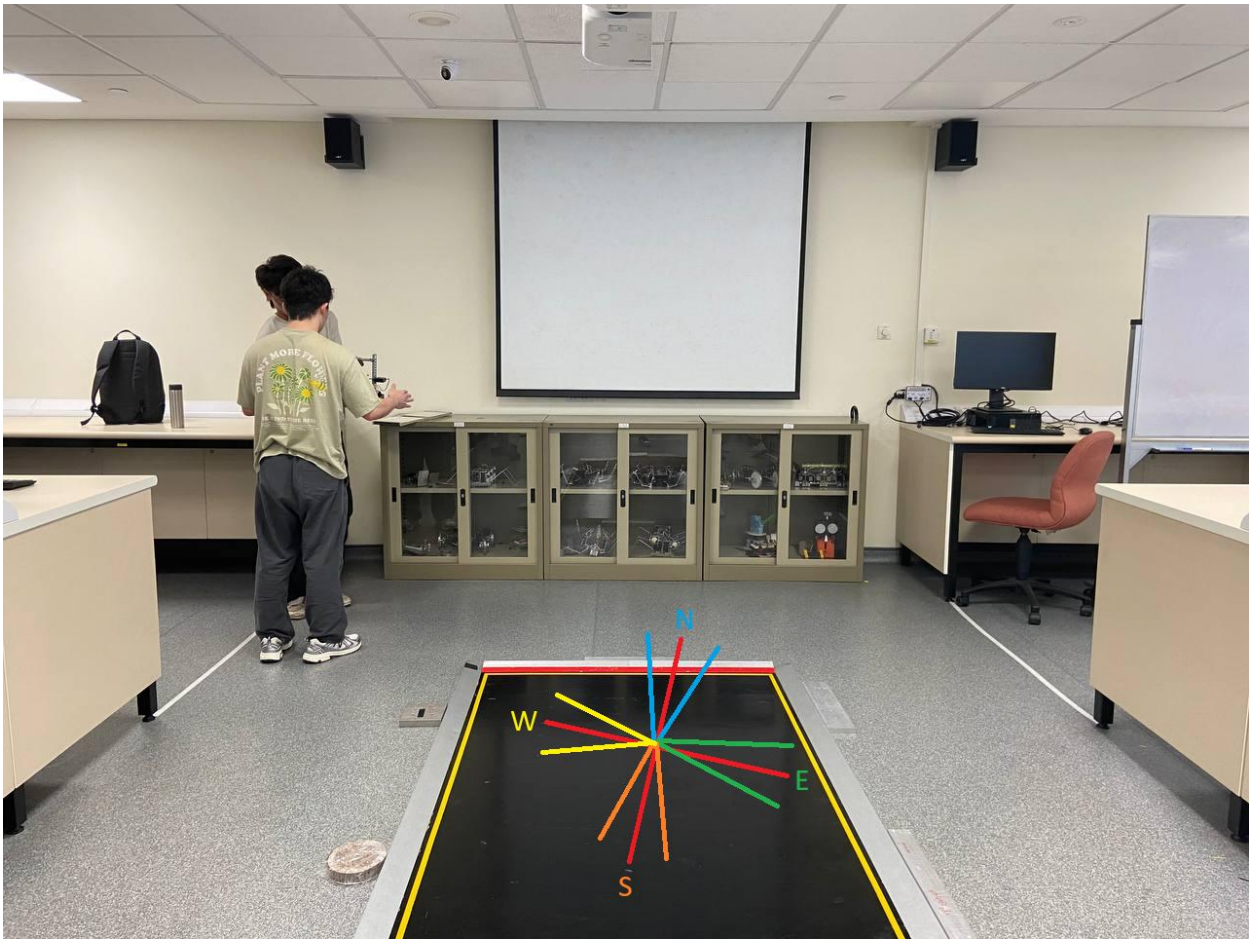


Figure 92: Visual Approximate Representation of Cardinal Ranges

During our initial testing of the sweep movement, we had coded the robot to do a sweep from North, to East, back to North, to West, and lastly back to North. However, it was observed that the compass-based cardinal directions behaved more like small angular ranges rather than exact single headings, as illustrated in Figure 93. As a result, approaching North from different directions did not always produce the same final physical orientation. In particular, returning to North from the East side tended to leave the robot slightly biased towards the right. To improve consistency, the sweep sequence was then changed such that the robot would end by approaching North from the West side. This allowed the robot to stop at the first detected point of North, which was found to be more accurately aligned and more repeatable for the next forward search motion.

However, this was difficult to achieve as well. If we tried to hard code to force the robot to turn North-West first before turning to North to point true North, there is a tendency for the compass LED to flicker between North-West and North, forcing the robot to constantly loop and turn between the two headings and get stuck. Nevertheless, our team decided to scrap forcing the robot

to point true North and let the robot misalign in the general direction to North. To us, we feel that this gives us an advantage within our search function for second pass.

The compass North was initially only important for second pass sweep function and the Home delivery function. However, going Home to deliver the ball is no longer an issue, because the robot can use the yellow lines by the side to course-correct and turn manually by time delay, until it is eventually parallel. Even if it reaches back line at an angle, the Edge Detection function is able to differentiate and its wide mechanical base means it will definitely align straight when forcefully bumping against the collection boxes. This makes the aligning North redundant. On the other hand, if we allow the robot to misalign on purpose during its sweep function, the robot is able to cover more ground, as long as it is generally heading in the North direction no matter how inaccurate. If we force it to be truly North, then the robot is moving very linearly and cannot cover a wider area. Forcing unpredictable sweep motion will provide us higher chance of second ball possession, opponent obstruction, and give us more various starting positions after second pass, third pass and so on, allowing us to constantly search more broader ground. In the end, this judgement call helped us to outperform other robots as well.

Nevertheless, the programming team also needed to adjust the cardinal headings, whereby pointing South allows the robot to turn to its left, and pointing South-East allows the robot to turn to its right. This is because the robot's left and right turn are different speeds and affect how and when the compass reads these values. This means it is really necessary to tune each compass heading for both left turn and right turn of the robot.

Besides software programming, another way to overcome this limitation is to change the physical mounting orientation of the compass. Through this, the robot is able to point true North by adjusting the compass physically.

7.10.4 Failsafe Logic for Sweep Inaccuracy

If the robot fails to collect a detected ball, it can perform a 360-degree scan to attempt to reacquire the target before giving up and returning home. This recovery behaviour improves robustness, especially when the ball shifts position during contact or when the initial intake attempt is slightly misaligned.

This failsafe was initially developed because the second sweep function is not 100% accurate. Unlike the first pass where the robot will always predict the ball within 4 grid squares in front, the sweep function when ball detected may not always be able to centre itself to the ball. This is because the slower it turns, the longer time it needs to re-compensate for centring, which means it misses ball nearer to the robot. On the other hand, faster turns will need shorter compensation time for centring, making balls further away easy to miss. We tried tuning for both cases accordingly, but the battery voltage makes a significant difference and would still miss rarely. Hence, the failsafe was developed to commit to the ball and redeem itself.

Overall, the search and collection logic combines direct forward searching, sensor-based alignment, controlled sweep scanning, confirmed ball capture, and recovery scanning to provide a simple but effective autonomous ball retrieval strategy.

7.10.5 Same Side Boundary Bug

A related troubleshooting issue emerged when the robot simultaneously detected the boundary with the front-left and back-left line sensors, or similarly on the right side. The team realised that this was not just a simple front or rear corner case, but a fundamentally different condition: the robot's entire side was aligned along the boundary line. This meant that the previously defined logic for front hits and back hits was no longer sufficient to describe the situation accurately. If treated as an ordinary corner hit, the robot could apply a correction that was too weak or even directionally inappropriate, causing it to continue scraping the line instead of escaping from it effectively.

To overcome this, the team reinterpreted the problem not as a normal edge event but as a side-contact condition that required its own dedicated recognition and response. The team therefore developed the idea of same-side hit detection, allowing the software to distinguish between a local corner event and a full side-alignment error. This improvement made the correction logic more physically representative of what was actually happening in the arena and reduced repeated failure caused by oversimplified sensor interpretation.

7.10.6 Opponent Detection Delay

A major software challenge involved distinguishing between an actual tennis ball and the opposing robot during sweep-based search. During turning sweeps, the side distance sensors could detect the opponent in a similar way to a ball, especially when the opponent passed through the field of view briefly. This caused the robot to lock onto the opponent as though it were a valid collection target, leading to false chasing behaviour and wasted time. The problem became more severe when the opponent sensor was positioned more centrally, because it could detect the opponent only for a short angular window during turning. Once the robot rotated slightly further, the opponent sensor would clear, but the side distance sensor might still continue to see the same object, allowing the program to relabel it as a ball.

Initially, we thought that by mounting opponent sensors to be aligned with the ball sensors, we could just program AND condition, to reduce the chances of opponent sensor absent when ball sensor triggering. However, the problem still persisted, and we revert to central positioning.

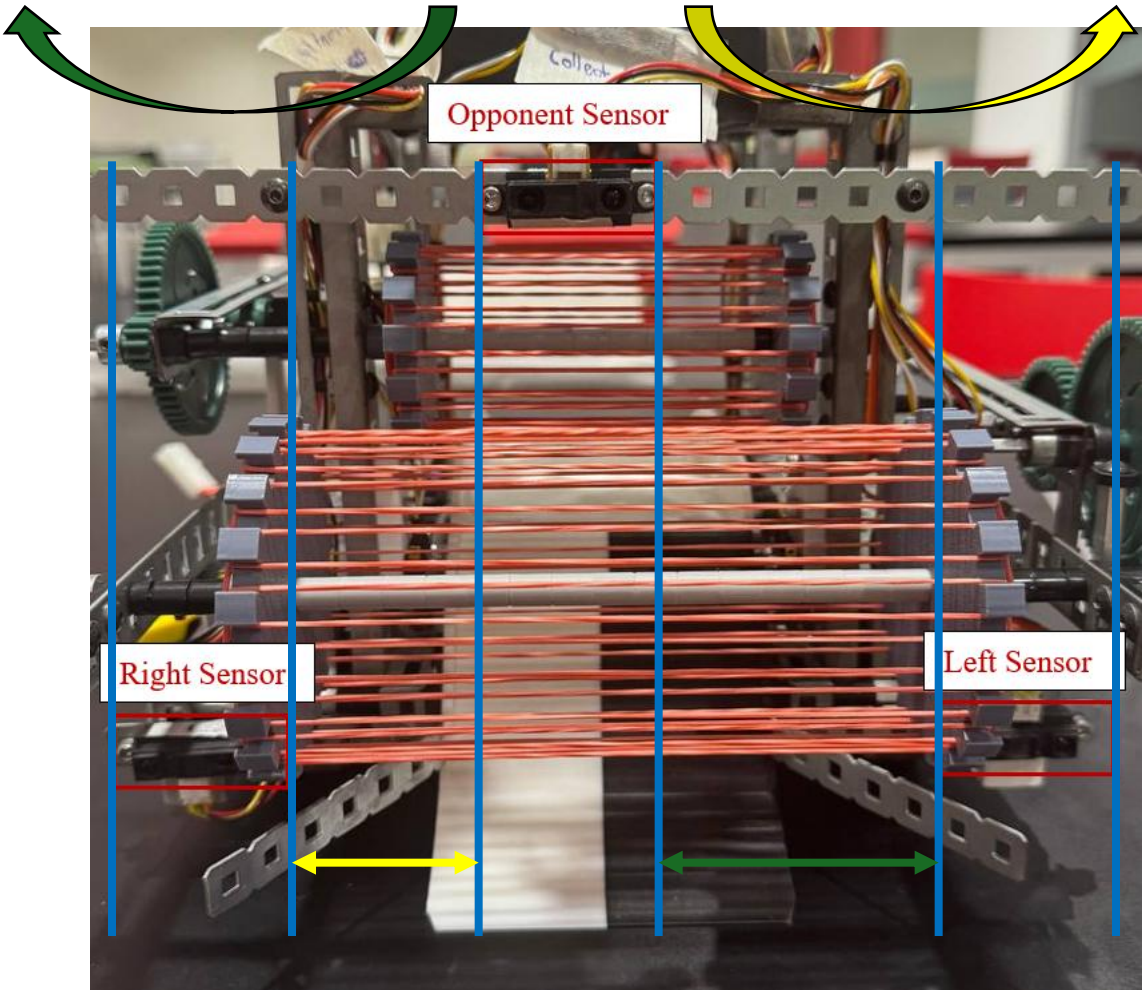


Figure 93: Illustration of Opponent Sensor & Ball Sensor Offset Detection

To overcome this, the team introduced the idea of temporary suppression or memory-based filtering in the opponent logic. Rather than treating opponent detection as valid only in the exact instant that the center sensor was active, the software was adjusted so that opponent detection created a short ignore window during which side-ball locking was disabled. Since the opponent sensor will always detect first before the ball sensor (recall we only use one ball sensor during sweep), this means we have to create an ignore or delay window when opponent sensor deactivates, allowing the side sensor lagging behind to bypass the opponent out of detection. This prevented the robot from immediately chasing an object that had only just been identified as the opponent. In forward-check mode, the team also implemented a more stable all-clear logic, where the robot stopped the drive but kept the collector active until the opponent sensor and side distance sensors all remained clear for a short time. These changes significantly improved target discrimination and demonstrated the importance of not relying solely on instantaneous sensor readings when interpreting moving obstacles.

7.10.7 Unreliable Home Delivery Logic

The team also faced significant issues with home-zone detection during the return phase. Originally, the logic relied heavily on yellow line detection at the rear of the robot to determine whether home had been reached. However, this proved unreliable in practice, because the robot could sometimes move too quickly and fail to register the line properly. At the same time, the back limit switch could not be used alone as a guaranteed home signal, since the back wall could flex and the switch could also be triggered by unintended collisions, including possible contact from the opponent. This meant that neither the line nor the switch could be treated as a perfectly reliable single-condition indicator of successful return.

To solve this, the team began treating home detection as a multi-condition logic problem rather than a one-sensor event. Instead of assuming that one reading alone was enough, the software was refined to consider combinations of factors such as rear line detection, heading alignment, switch activation, and timeout-based recovery behaviour. Additional breakout behaviour was also added so that the robot could recover if it failed to identify home after a prolonged return interval. This made the home-return routine more robust against missed detections and allowed the robot to continue operating despite imperfect real-world sensing. The insight gained from this issue was that end-of-task confirmation often requires sensor fusion and validation logic rather than dependence on a single threshold crossing.

7.10.8 Lane Change Not Alternating

A further debugging issue arose in the lane-change sequence used during the second pass. The robot repeatedly performed lane change to the right but did not consistently alternate to the left as intended. The team discovered that this was not simply a movement problem, but a logic problem involving the preservation and transition of the lane state variable. The software was intended to use a ‘state’ variable to determine whether the next lane shift should be to the right or to the left, but under some versions of the code the state was being reset or not maintained properly across cycles. As a result, even though the robot completed the right lane-change helper successfully, the next control decision still started from the same effective state and repeated the same lane direction.

To overcome this, the team examined the lane-change routine as a state-transition problem rather than only a motion problem. The fix involved ensuring that the lane state truly represented the next intended search corridor and was only changed when the lane-change event had been successfully completed. This improved the consistency of the lane alternation logic and allowed the second-pass search strategy to behave more systematically. The debugging process reinforced the importance of separating physical movement completion from logical state transition, especially in repeated autonomous search cycles.

7.10.9 Other Minor Bugs & Improvements

The programming team did countless hours and nights of refining the program by implementing minor fine-tuning lines of code to make the robot's behavior more reliable. For example, implementing stopDrive() between continuous searching in first pass or second pass sweep motion, preventing erratic behaviors or possible ignoring ball when moving too fast. In some cases, the sensor detected the ball, but the robot did not produce a clear or stable turning response before attempting to collect it. To improve this, the team introduced stopDrive() at specific transition points before initiating a turn, with the intention of ensuring that the robot came to a complete stop before changing direction. This improved the immediate turning response and made the first-pass targeting behaviour more stable and visually consistent.

However, after this modification was made, a new problem appeared in the recovery logic. The robot no longer performed the intended 360-degree recovery search consistently after a failed collection attempt. In other words, the added stopping behaviour solved one problem locally but changed the timing and state transition of the overall program enough to interfere with the subsequent miss-recovery routine. The team identified that the issue was not caused by the stopDrive() command alone, but the program's nested loops and break conditions depended heavily on motion timing. To overcome this, the team reviewed the flow of events more carefully and adjusted the logic so that stopping commands were only inserted where they improved motion stability without prematurely altering the state of the larger search-and-recovery sequence. This debugging process highlighted the importance of considering the interaction between local fixes and global control flow, especially in a long and highly conditional robotics program.

Other improvements include time delay between stopDrive() and stopCollect(), allowing the collector motor to run a bit more after stopping to allow the ball intake to continue and guarantee capture if stopped at the exact location of the ball. Time Delay is a reliable way to fine tune for searching and collecting or edge detection or opponent detection.

Tuning thresholds values are also regularly tested to be on the lookout for possible noise that could affect the robot's performance. While most teams try to force long distances for sensitive values, this is actually detrimental to their designs due to sensor ghosting, or accidental detection from arena's ramp or missing the ball. In fact, the less sensitive the sensors are by increasing the threshold value, the more precise our ball capture is.

8 FINAL ROBOT DESIGN OVERVIEW

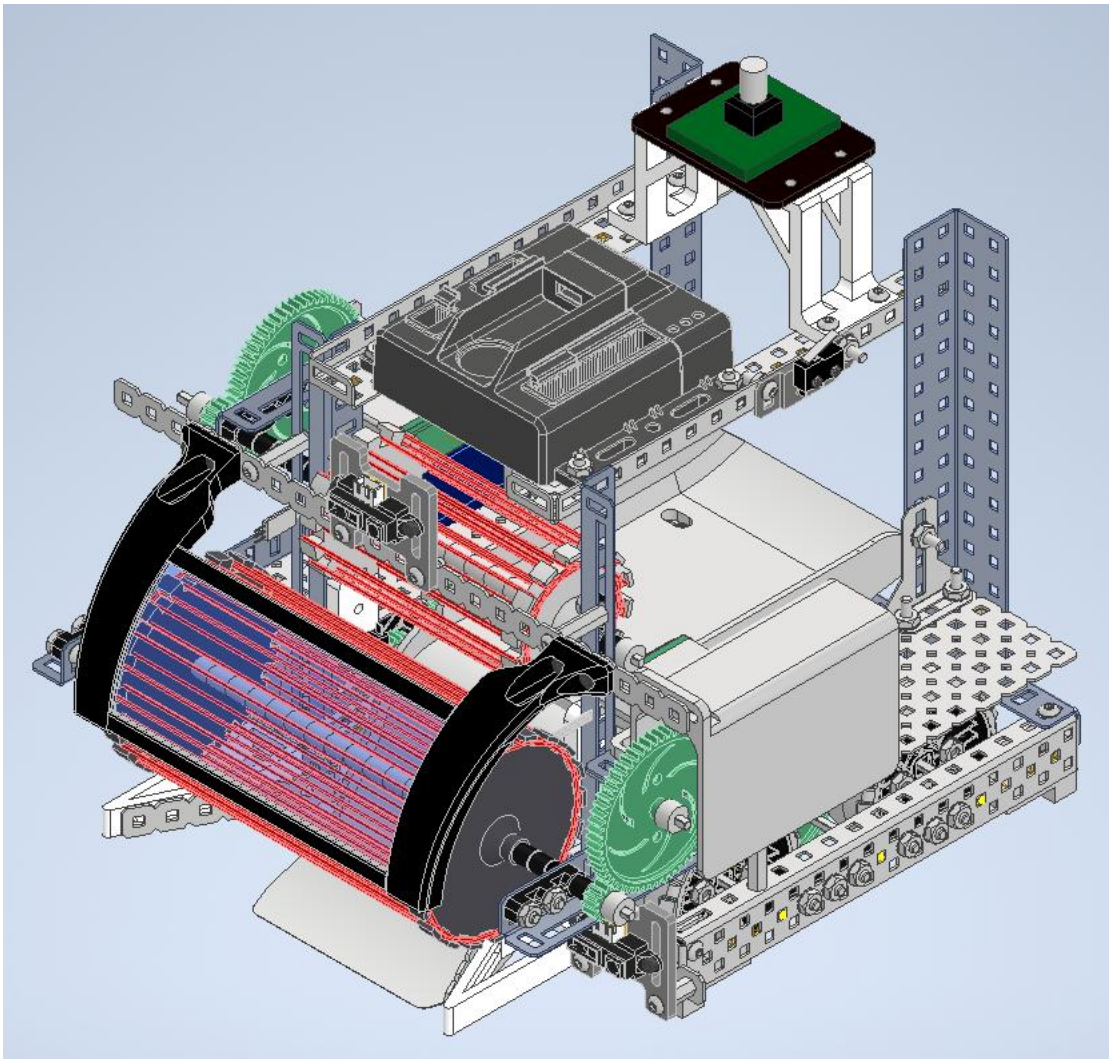


Figure 94: Isometric View of ROLLEX

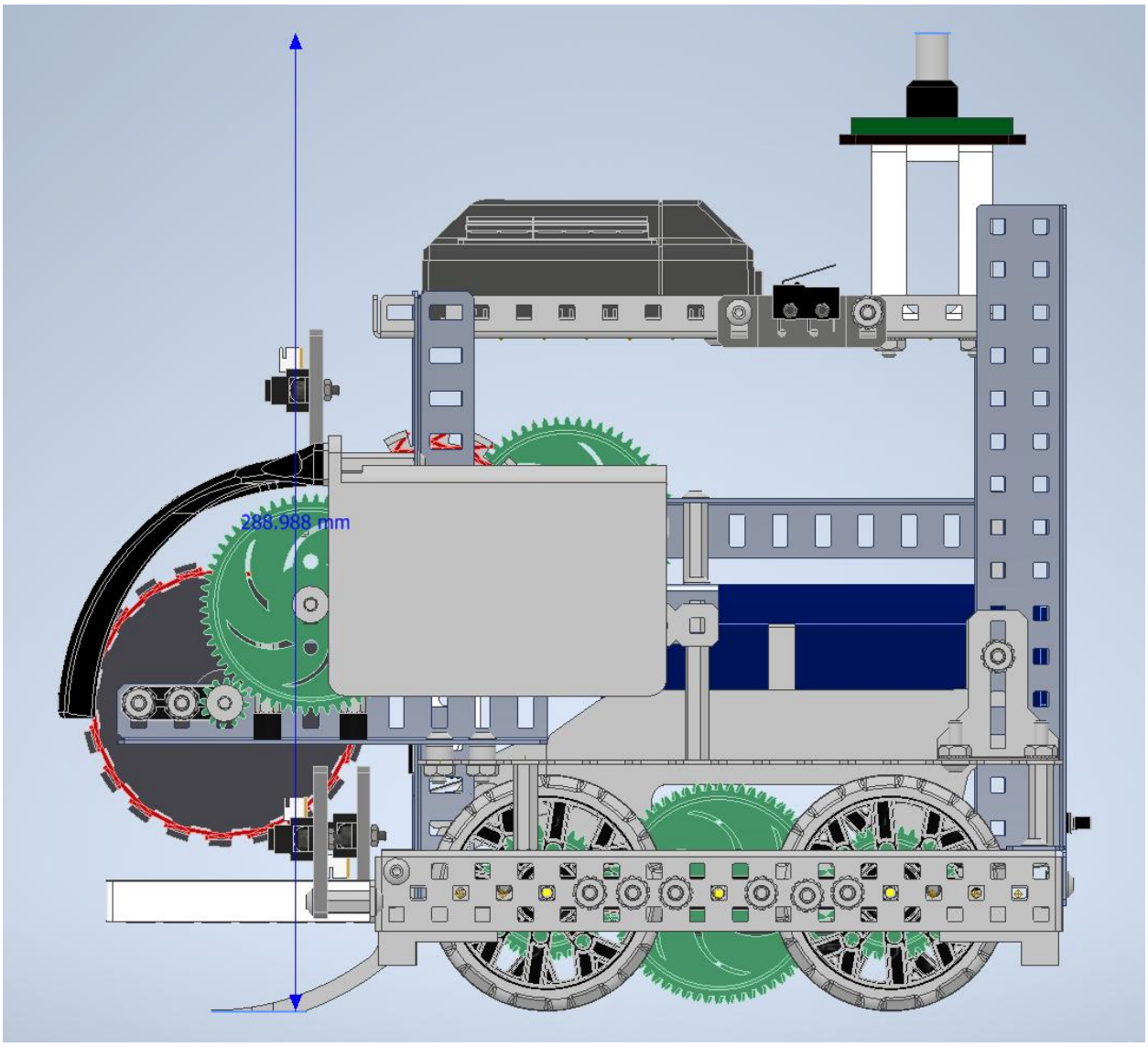


Figure 95: Right View of ROLLEX

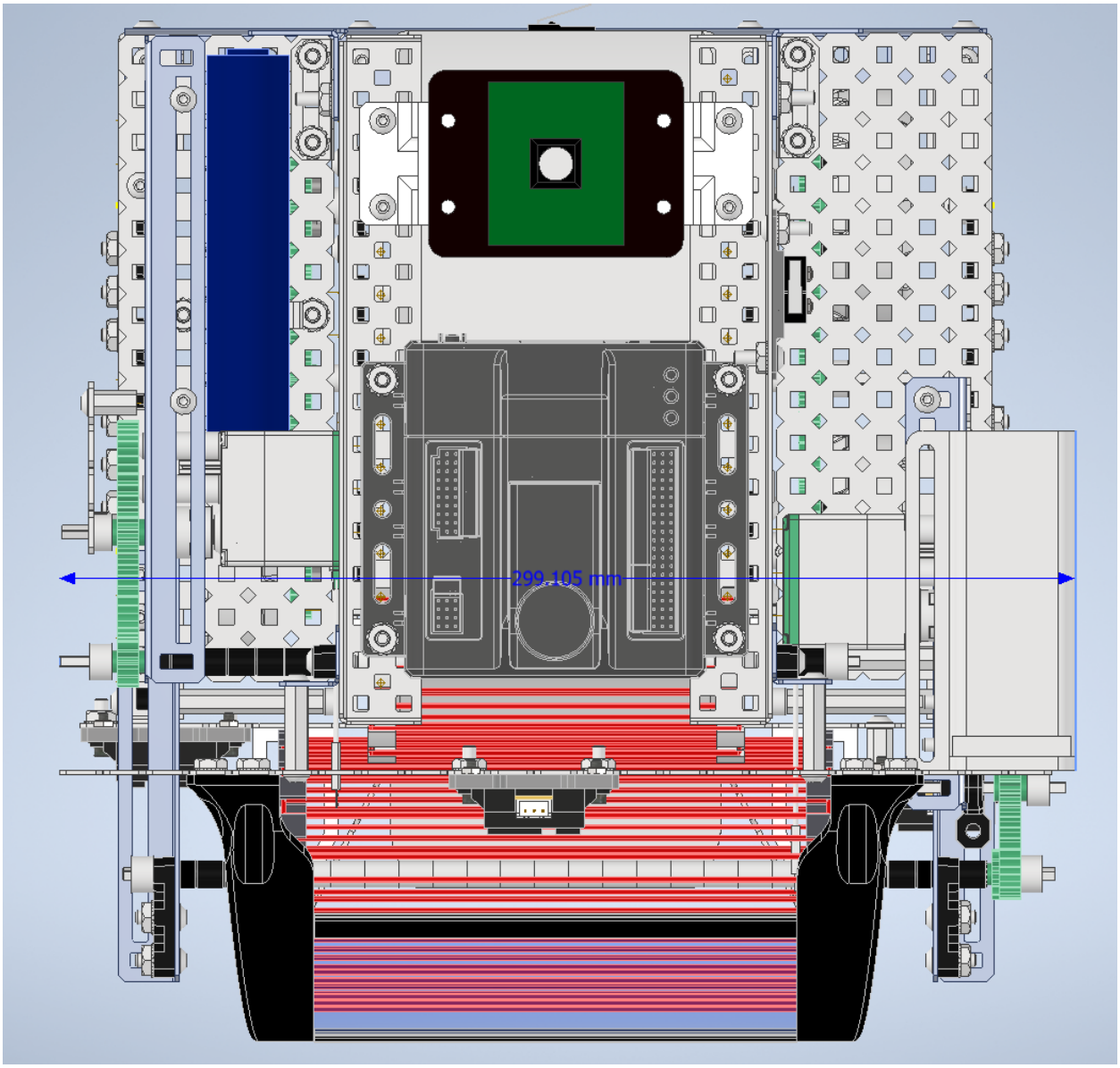


Figure 96: Top View of ROLLEX

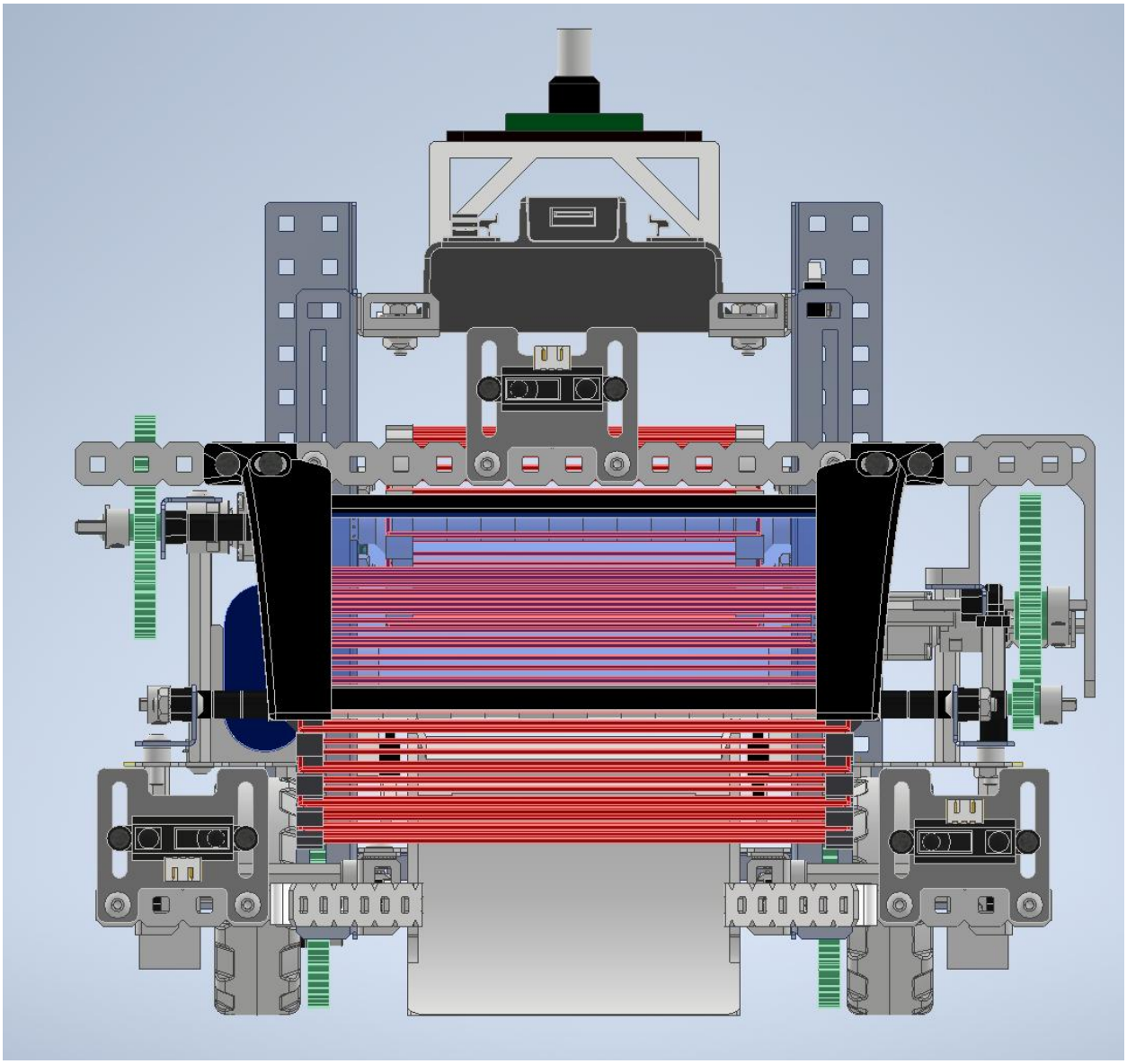


Figure 97: Front View of ROLLEX

9 COMPETITION

The competition for MA4012 Mechatronics Engineering Design occurs on week 13 of AY25/26 Semester 2 in N3.2-B1-03. **Figure 98** below shows the design of all 5 robots and their respective designs. The name in order from top to bottom are: Heng Max, Ball-e, Rollex (our design), Rolling Stone, Immigrants.

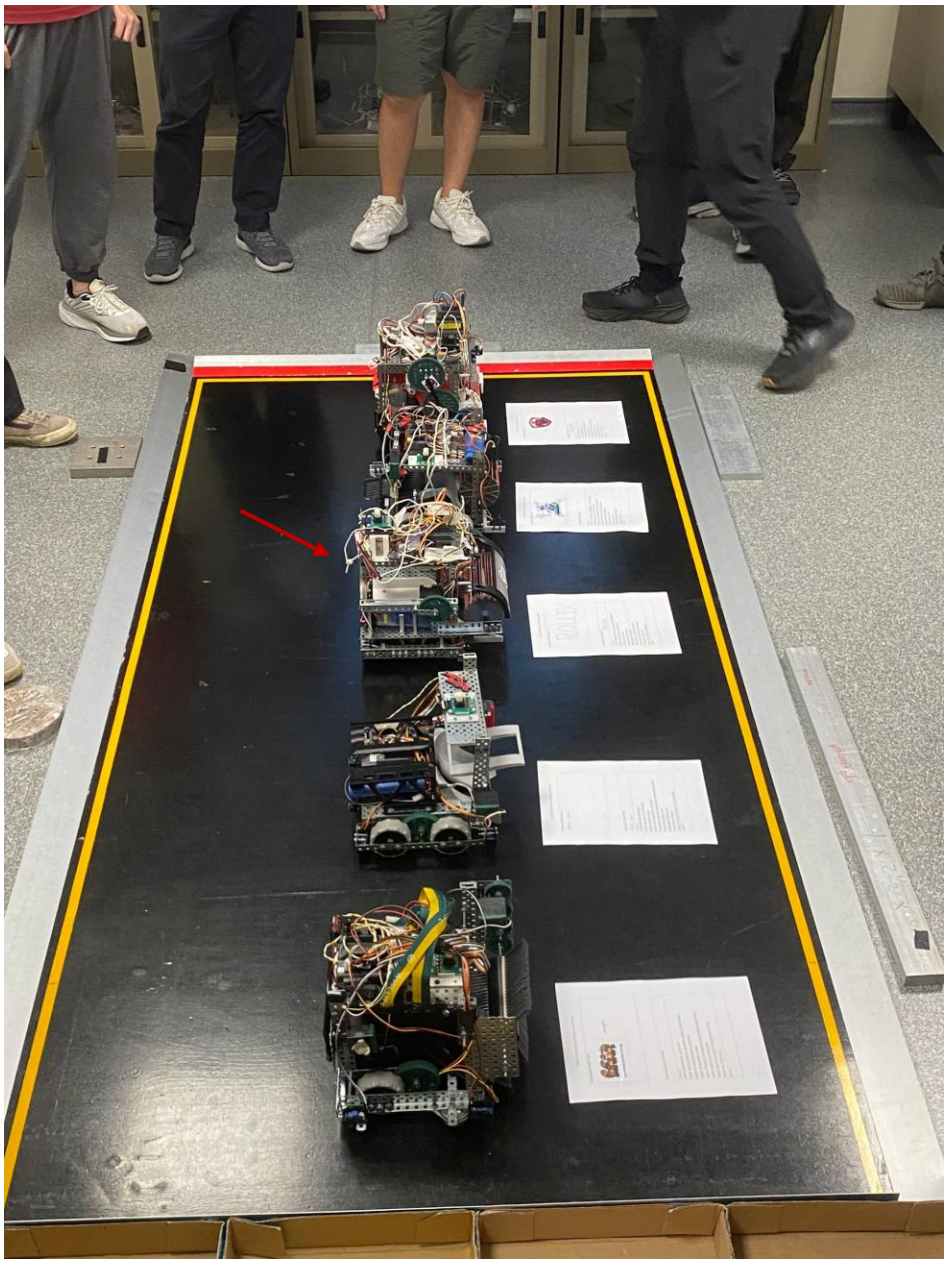


Figure 98: Picture of designs of others group's robot on the arena.

9.1 Competition Results

The competition consists of a round robin tournament style where each team competes against every other group. Table 18 below shows our results against each team, along with the finals. **Table 19** below shows the placement and the total amount of tennis balls each team deposited across all 4 rounds of round robin matches.

Round	**Against**	**Results**
1	Ball-e	2-1
2	Heng Max	2-1
3	Immigrants	2-0
4	Rolling Stone	2-0
Finals	Ball-e	2-0

Table 18: Round Robin competition results against other teams in MA4012.

Placement	**Team**	**Total balls deposited**
1	Rollex	10
2	Ball-e	4
3	Heng Max	2
4	Immigrants	1
5	Rolling Stone	0

Table 19: Placement and the total amount of tennis balls each team deposited.

9.2 Observation

9.2.1 Our Robot's Performance

Rollex demonstrated consistent and reliable performance throughout the competition. In all four rounds of the round robin and the finals, the robot successfully retrieved the first ball directly in front of it within the first 10 seconds of each match, reflecting the reliability of the line-tracking approach and the ball collection mechanism under competition conditions.

Performance on the second ball was generally successful across all rounds, though not without difficulty. In one round, the robot entered a stalemate with the opponent, requiring a manual reset before proceeding. Despite these interruptions, Rollex consistently recovered and completed retrieval of the second ball in every round.

The third ball, however, was not retrieved in any round of the competition. This was not a limitation of the robot's mechanical or software performance per se, but rather a consequence of match dynamics. By the time Rollex had secured the second ball, the opponent had either already deposited the third ball, was carrying it within their robot, or was physically blocking access to it. As the competition rules did not permit forceful displacement of an opposing robot, retrieval in these situations was not feasible within the match duration.

Overall, Rollex's consistent first and second ball retrieval across all five matches, resulting in a total of 8 balls deposited during round robin, 2 balls deposited during finals, and a first-place finish. Rollex demonstrated the robustness of the core collection and delivery pipeline under live competition conditions.

9.2.2 Observations of Competing Robots

Ball-e (2nd Place) Ball-e demonstrated occasional reliability on the first pass, successfully securing the first ball quickly in their opening match against Rollex. Their strategy involved navigating to the centre of the arena first, performing a 360° scan, and then proceeding to collect the ball. While this approach allowed for a broader initial search, the centre-first movement introduced a time delay that Rollex was able to exploit by driving directly to the ball. Notably, Ball-e's 360° scan was observed to trigger at inconsistent intervals across matches, suggesting that their scanning routine was not strictly conditional on ball search mode. A more robust implementation would confine the 360° scan exclusively to the ball-searching state. That said, Ball-e's ball collection and deposit mechanisms operated smoothly throughout the competition. Additionally, their search algorithm using 360° scan was copied from us by observing our test runs, which highlights not only how valuable our robot's capabilities are, but also how the reliability of failsafe function alone can outperform the other robots easily.

Heng Max (3rd Place) Heng Max encountered drifting issues at the start of their matches, with the robot veering to the right before detecting the boundary yellow line and triggering a breakout manoeuvre. Subsequent movement along the yellow line was slow and sluggish. After collecting the ball, the robot did not immediately return to deposit. Instead, it was hovering near the collection area before eventually navigating back. Like Ball-e, their ball collection and deposit mechanisms were smooth, but their search algorithm was poor, sluggish and random. The common observation across both 2nd and 3rd place teams is that while their end-effector hardware performed reliably, their navigation algorithms lacked the precision and robustness to make a direct, immediate approach to the ball, resulting in significantly longer time-to-first ball compared to Rollex.

Immigrants (4th Place) The match against Immigrants reached a standoff early on. Rather than proceeding toward the ball in front of them, Immigrants drove directly toward Rollex, raising the possibility that their algorithm excluded opponent-avoidance behaviour. The resulting standoff trapped Rollex between the boundary yellow line ahead and Immigrants behind, rendering the breakout routine ineffective. Following a manual reset, Rollex was able to smoothly collect and deliver the first ball within 10 seconds, proceeding to lane change and second pass. Meanwhile, Immigrants successfully drove straight and collected the ball but after collection, the robot repeatedly advanced to the boundary line and reversed in a loop, eventually becoming stuck in the corner between two boundary lines. Their ball collection mechanism appeared smooth, but their

deposit mechanism could not be assessed as the robot did not successfully complete a return run. It seems like their boundary edge detection was poor, lack of opponent detection or breakout, unreliable delivery, and sometimes successful detection and capture.

Rolling Stone (5th Place) Rolling Stone experienced a critical failure in their ball collection algorithm. Despite somewhat detecting the ball, the robot did not proceed to collect it and remained stationary for the duration of the match. As a result, no balls were deposited across any of their rounds.

9.2.3 Technical Issues or Unexpected Behaviours

Several technical limitations became apparent under live competition conditions that were difficult to anticipate during pre-competition testing.

Breakout and Collision Handling: The breakout sequence, triggered when the robot detected an obstruction ahead, was designed to reverse the robot to the home deposit area and restart the passing routine. While this functioned as intended in most scenarios, edge cases proved harder to handle. When blocked from the front by an opponent, the breakout sequence executed correctly. During our first match against Ball-E, our breakout function works as intended after 15 sec being jammed at home. However, in one instance during the match against Immigrants, the robot was pushed from behind past the boundary line, placing both robots in a stalemate outside the arena boundary. In this configuration, the boundary line could no longer trigger the breakout condition, leaving the robot unable to recover autonomously and requiring a manual reset.

Ball Detection at Arena Corners: When a ball had migrated to a corner of the arena, the robot's sensors were frequently unable to detect it at the required range, resulting in the ball being missed entirely. Corner positions constrained the robot's approach angle and reduced the effective detection window, making retrieval in these situations unreliable. It seems that the robot was somewhat lacking in broader search coverage due to left-biased search algorithm for third pass onwards.

Left-Biased Search Algorithm: A more systemic issue was observed during the match against Rolling Stone, stemming from the robot's fixed left-biased search logic. The robot is programmed to begin each pass from the left side of the arena, using boundary line detection on its left to determine when to transition to the middle. This logic functions correctly under normal conditions but broke down when the robot's starting position after a deposit was closer to the centre than expected.

In the affected match, after completing the first pass, the robot returned to a position near the centre rather than the left side. Without detecting the left boundary line, it defaulted to searching left of its current position — moving away from the ball on the right. Upon returning and depositing, the same condition repeated: positioned near the centre, the robot again searched left, collected the already-deposited ball's former position, and continued oscillating between the left and middle zones. The ball on the right side of the arena was never reached as a result. This revealed a dependency on consistent robot positioning after each deposit cycle that was not sufficiently robust to positional variation introduced by competition dynamics.

10 LESSONS LEARNT: POST COMPETITION ANALYSIS

10.1 Correlation Between Design Choices and Performance

The competition results validated the design decisions made during the concept evaluation stage. As shown previously in Table 13, Idea 1 received the highest weighted score of 450, driven largely by its ball collection capability (125/125), ball delivery capability (100/100), and pushing ability (75/75). These three criteria collectively proved to be the most decisive factors in the competition outcome, and Rollex reflects this directly with a first-place finish with 10 balls deposited, more than double the second-placed team

The roller-based collection mechanism with rubber bands was evaluated as the strongest concept due to its tolerance for positional misalignment and the flexible, passive guidance it provided. This translated directly into competition performance. Rollex consistently secured the first ball within 10 seconds across all five matches without a single failure. The wide capture zone and self-centring behaviour reduced the dependency on precise sensor alignment, which proved particularly valuable given that ball positions varied randomly each round.

The 4-wheel rear drive layout, which scored highest in pushing ability and resistance, also proved its worth under competitive conditions. In multiple stalemate situations, particularly against Immigrants, Rollex was able to hold its ground and recover, whereas opponents with lighter or less structurally protected drive bases were more vulnerable to displacement. The gear train configuration, while mechanically more complex, provided the traction and pushing force that gave Rollex an advantage in direct contact scenarios.

The dual-roller delivery system with limit switch sensors, evaluated as the most controlled and reliable delivery method, maintained a 100% delivery success rate throughout the competition. Every ball that was successfully collected was also successfully delivered, with no jams or misfires observed. This outcome justified the design decision to use a limit switch to confirm ball capture before initiating the return-home sequence, as it eliminated the possibility of returning home empty and wasting match time.

One area where the concept evaluation was less predictive was robot agility, which received a relatively low weighting of 10% and a score of 3/5 for Idea 1. In practice, the 4-wheel drive base's reduced turning agility did contribute to the stalemate situations observed in several rounds, particularly when trapped against the boundary. However, given that Rollex won all five matches, this trade-off between agility and structural robustness proved to be the correct one for the competition format.

10.2 Observed Limitations

While Rollex performed strongly throughout the competition, several limitations became apparent under live conditions that were not fully captured during pre-competition testing.

Position-Dependent Search Logic: The most significant limitation was the robot's reliance on a fixed left-biased search sequence that assumed a consistent starting position after each deposit cycle. During pre-competition testing, the robot reliably returned to approximately the same position after each delivery, making the left-boundary detection a dependable trigger for lane transitions. However, under competition conditions, opponent interactions and stalemate resets introduced positional variability that the algorithm was not designed to handle. As observed in the match against Rolling Stone, when the robot returned to a centre position rather than the left side, the lane transition logic failed to execute correctly, causing the robot to search repeatedly in already-cleared zones. This highlighted a fundamental limitation of open-loop positional assumptions in a dynamic competitive environment.

Breakout Sequence Coverage: The breakout sequence was designed around the assumption that the robot would always have at least one boundary line within sensor range to trigger recovery. This assumption held in the majority of situations, but failed in the specific case where the robot was pushed past the boundary line from behind by an opponent. With the boundary line now behind the robot rather than in front, the breakout condition could not be met autonomously. This edge case was difficult to anticipate during testing as it required a specific combination of opponent behaviour and robot positioning that did not arise in controlled test runs.

Ball Detection Range at Arena Corners: The sharp distance sensors were calibrated for reliable detection within a range of approximately 25 to 30 cm under open arena conditions. When balls migrated to corners, the constrained approach geometry reduced the effective detection window, as the robot could not position itself at an optimal angle relative to the ball. The arena walls limited the sweep arc available for scanning, and the sensor's field of view was insufficient to compensate for the angular offset. This was compounded by the fact that the search algorithm did not include a dedicated corner-search routine, meaning the robot would sweep past corner positions without registering a detection.

Back Limit Switch: The back limit switch is intended to be pressed against the collection boxes for delivery. However, there could be a chance where if opponent collide at us from behind, the robot might deliver the ball towards them. Fortunately, we studied the opponents design beforehand and their frame had hollow parts at the height of our limit switch. Even during the competition, colliding with us from behind did not deliver the ball into them. Nevertheless, the code could have bene further improved to prevent this.

10.3 Recommendations for Improvement

Based on the limitations identified during the competition, the following recommendations are proposed to improve the robustness and performance of the robot in future iterations.

Position-Aware Search Logic: The most impactful improvement would be to replace the fixed left-biased search sequence with a position-aware lane management system. Rather than relying on a single boundary line detection to determine the robot's current lane, the robot should track its relative position across the arena using a combination of compass heading, boundary line detections, and pass count. For example, a state variable could record which lane the robot last searched and use this information to determine the correct starting point for the next pass, regardless of where the robot physically lands after a deposit. This would make the search logic robust to positional variation introduced by opponent interactions and resets, eliminating the oscillation behaviour observed in the match against Rolling Stone.

Omnidirectional Breakout Recovery: The current breakout sequence is effective when the robot is obstructed from the front, but fails when pushed beyond the boundary from behind. A more robust recovery system should incorporate rear boundary detection as a dedicated breakout trigger. When both rear line sensors detect the boundary simultaneously under conditions where the robot is not in the home delivery state, the software should interpret this as an out-of-bounds condition and execute a forward escape manoeuvre rather than continuing to search for a front boundary line. Adding a timeout-based fallback that forces a forward movement after a fixed period of inactivity would further reduce the risk of the robot remaining stuck in an unrecoverable stalemate.

Corner-Targeted Search Routine: To improve ball retrieval in corner positions, a dedicated corner search subroutine should be implemented as a final sweep option when the standard search cycle completes without detection. This routine would direct the robot to approach each corner of the arena at a controlled angle, pause, and scan with both distance sensors before concluding that no ball is present in that zone. Mechanically, a wider sensor field of view or an angled sensor mount could also be considered to extend detection coverage into corner regions that the current horizontal sensor orientation cannot reliably reach.

Compass Redundancy or Software Filtering: To mitigate heading instability caused by electromagnetic interference during competition, a software-based filtering approach should be introduced. A moving average or majority-vote filter applied over several consecutive compass readings would reduce the impact of transient interference spikes on heading decisions, preventing erratic turning behaviour during homing. In the longer term, replacing the digital compass with an inertial measurement unit (IMU) that combines accelerometer and gyroscope data would provide significantly more robust heading estimation that is less susceptible to external magnetic disturbance.

Opponent State Awareness: The current software does not model the opponent's state or position beyond detecting its presence within the distance sensor's field of view. A more sophisticated approach would use the opponent sensor reading over time to infer whether the opponent is stationary or moving and adjust the robot's strategy accordingly. For example, if the opponent sensor registers a sustained stationary reading directly ahead, the robot could initiate a lateral

repositioning manoeuvre rather than waiting for a stalemate timeout. This would reduce the number of manual resets required during competition and allow the robot to recover more time for additional ball retrieval attempts.

Limit Switch: The back limit switch can be improved by not triggering for ball delivery unless yellow line is detected to authenticate the delivery, preventing any accidental collisions from losing our ball in possession.

CONCLUSION

In conclusion, the final robot design was developed as an integrated system in which the mechanical structure, collector mechanism, power and electrical arrangement, and software control strategy all worked together to achieve reliable ball collection and delivery. Throughout the design process, emphasis was placed not only on building a robot that could move and function, but on creating one that could operate consistently under practical arena conditions such as boundary encounters, sensing uncertainty, opponent interference, and repeated search cycles. The final oucome reflects a balance between speed, stability, control, and robustness, where each subsystem was designed with both performance and reliability in mind.

From a mechanical perspective, the vehicle and collector systems were designed to provide stable motion, effective ball intake, secure storage, and controlled dispensing. The drive base supported smooth navigation and turning, while the collector and storage system ensured that the ball could be retained after successful collection and preserved until delivery. The dispensing mechanism then completed the task by enabling controlled ball release at the home area. At the same time, the electrical and sensor layout was planned to support reliable communication between hardware and software, ensuring that sensor readings, motor actions, and controller decisions could function together in a coordinated manner.

The software played a particularly important role in translating the hardware design into intelligent robot behaviour. By structuring the program into modular functions for search, boundary correction, ball detection, opponent handling, lane changing, return-home navigation, and delivery, the robot was able to react to changing situations in a logical and systematic way. The use of staged search strategies, recovery behaviours, and sensor-based decision-making improved the robot's ability to continue operating even when conditions were imperfect. This demonstrated that successful robot performance depends not only on hardware quality, but also on well-planned control logic and effective integration between sensing and action.

Overall, this project highlighted the importance of iterative design, testing, and refinement in engineering practice. Many of the final design choices were not simply theoretical, but were shaped through repeated observation, troubleshooting, and practical improvement. The project therefore provided valuable insight into how autonomous systems are designed as complete working solutions rather than isolated components. Through this process, a deeper understanding was gained of how mechanical, electrical, and software subsystems must complement one another to produce a robot that is functional, efficient, and dependable in real operation.

GROUP REFLECTION

Initially, many of us were unsure of what to expect. Some of us felt anxious about the competitive nature of the module, the scale of the robotics challenge, and our own ability to contribute meaningfully while juggling other academic responsibilities. However, as the weeks progressed, the project gradually became one of the most meaningful and rewarding engineering experiences we had. It pushed us not only technically, but also personally, by forcing us to think more creatively, work more consistently, and respond constructively when things did not go according to plan. By the end of the semester, we had come to see MA4012 not simply as a module, but as a real engineering journey that revealed both our strengths and our weaknesses in design, communication, teamwork, and perseverance.

One of the strongest lessons that emerged from our collective reflection was the importance of iteration. Very few parts of the robot worked perfectly from the start. Whether it was the collector, the ramp, the roller, the drive system, the sensor arrangement, or the software logic, almost every major subsystem had to go through repeated cycles of testing, failure, redesign, and refinement. At times, this was frustrating and exhausting, especially when long hours of effort seemed to produce only small improvements. Yet, looking back, this was also one of the most valuable aspects of the project. It taught us that engineering is rarely a straight path, and that meaningful progress often comes from being willing to fail, learn from the failure, and keep improving patiently until the design becomes reliable. Rather than seeing setbacks as wasted effort, we began to understand them as necessary steps in arriving at a better solution.

As a team, we also came to appreciate how important communication, flexibility, and shared responsibility were. Although we began with different assigned roles, the project quickly showed us that no subsystem could be treated in isolation. Mechanical design affected sensing, sensing affected software decisions, and software performance depended on the physical response of the robot. Because of this, many of us found ourselves helping beyond our original roles, whether in mechanical design, coding, sensor testing, integration, or troubleshooting. This made the process more demanding, but also made it more meaningful, because it showed us how multiple systems constantly interacting with one another. This module made us appreciate the value of having teammates who were willing to step in where needed, share ideas openly, and keep working together even when the pressure increased.

The most important takeaway from our reflection was that perseverance mattered just as much as technical ability. There were many moments where progress felt slow, problems seemed repetitive, and the pressure of the final weeks became overwhelming. Some of us were balancing this project with Final Year Project work, other heavy modules, and personal fatigue, which made the experience even more intense. Yet this was also where much of our growth happened. We learned that engineering under real constraints requires more than just having a good idea. It requires discipline, consistency, patience, and the willingness to keep moving forward even when the outcome is still uncertain. In that sense, the final success of the robot did not come from one brilliant decision alone, but from the accumulation of many small improvements, careful observations, repeated troubleshooting, and the determination to keep going until the system worked.

Looking back, what stands out most is not only that we won, but how we got there. The result was satisfying, but what felt more meaningful was the process itself, the late nights, the repeated testing, the shared frustrations, the discussions, the redesigns, and the gradual realisation that our efforts were coming together into something reliable and competitive. As a team, we leave this project with a deeper appreciation of what engineering really involves in practice. It is not just about technical knowledge, but about integration, adaptability, collaboration, and resilience. More importantly, it reminded us that strong engineering outcomes are often built through steady collective effort rather than one big moment of success. In that sense, this project did not just give us a working robot, but also a much clearer understanding of how we aspire to grow as future engineers.

COLLATED PERSONAL REFLECTION

Name	Reflection
Tay Kai Jie, Worden	Throughout my journey in NTU Mechanical Engineering, MA4012 has been one of the most eye-opening and meaningful modules I have ever taken. It has been the most enjoyable experience I have had in school, and it reminded me why I chose engineering in the first place. This module did not just instil technical knowledge in me, but it reignited something deeper, my passion for creating, building, and solving problems through engineering. At the beginning, I was honestly anxious about this module. I thought it would be another difficult group project filled with disagreement, uneven contributions, and unnecessary stress. The thought of a competitive robotics project made me dread the semester ahead, because it felt intimidating and far beyond what I was prepared for. Nevertheless, as the weeks passed, MA4012 turned out to be completely different from what I feared. It exposed me to parts of engineering that I had never fully experienced before, especially the open-ended nature of design, the reality of hands-on building, and the discipline of turning ideas into something that actually works. Through this process, it revealed not only my technical strengths and weaknesses, but also parts of my character, my communication style, and the kind of engineer I am still becoming. What made MA4012 unique was how open and creative it was. In many other modules, engineering can feel very structured, where the goal is to solve a known problem with a known method, but this module challenged us to create, test, rethink, and refine. Before this, I never really considered myself a naturally creative or innovative person. I often doubted the quality of my ideas and had the tendency to jump too quickly into solutions without fully trusting the design process. However, this module taught me to slow down and think more carefully. Through functional analysis, sketching, idea generation, evaluation, and embodiment design, I learned that good design is not about getting the right answer immediately,

	but about refining your thinking until the design becomes clearer and stronger. I realised that there is no clear-cut outstanding piece, but to make what is original and uniquely yours outstanding through countless iterations and thought-provoking discussions. It also changed the way I look at products and systems around me, because I began to appreciate the thought, trade-offs, and intentionality behind good engineering design. One of the hardest parts of this project was the sheer weight of responsibility that came with it. The challenge was not only technical, although there were many real problems in both mechanical and software design, but also human. With everyone carrying other academic commitments and Final Year Projects, there were many moments where I felt alone, frustrated, and burdened. As the group leader, I initially felt as though I had simply been given a title that came with more responsibility than authority. At first, I tried to coordinate meetings, distribute responsibilities, and encourage everyone to contribute equally, because I genuinely wanted us to function well as a team. But over time, I realised that waiting for perfect balance or depending too heavily on everyone was not always enough to move the project forward. That realisation was difficult, but it also taught me something important about leadership. Leadership is not just about assigning tasks, it is about carrying responsibility, staying steady under pressure, and continuing to move the team forward even when things feel uncertain. What kept me going was the mindset of consistency and perseverance. The analogy I often thought about is how ants, though small, can carry grain by grain until they eventually build an entire underground city. In the same way, I learned to focus on small steps rather than being crushed by the scale of the whole project. I tried to remain open-minded, break down large problems into manageable tasks, and trust that steady effort would eventually amount to something meaningful. There were many sleepless nights, many moments of anxiety, and many times I questioned whether we were really making progress. But I learned to trust the process, to stay focused on our own growth, and also to observe and learn from other

	groups along the way. In the end, after 13 intense weeks, seeing our team emerge victorious made all those sacrifices felt worthwhile. It was not just a win in competition, but a victory built on grit, discipline, and quiet persistence. Repeated testing and troubleshooting were one of the most painful yet valuable parts of the journey. There were countless moments where we encountered setback after setback, and it felt as though long hours of work were producing very little progress. That kind of frustration is difficult to describe unless you have lived through it. But somewhere in that pain, I learned one of the deepest lessons of engineering. Failure is not something to despise, but something to learn from. This project taught me that progress often comes through repeated failure, reflection, and improvement. It taught me to stop seeing failure as proof that I am inadequate, and instead seeing it as part of the path towards becoming more capable. Although I felt discouraged, I learned to pick myself up, try again, and keep refining until something finally worked. If there is one quality that carried me through this module, it was grit. If I were asked whether I would do anything differently, honestly, I do not think I would. Not because everything was perfect, but because I know I gave this project my all. I worked on it wholeheartedly, with sincerity and commitment, and I have no regrets about the effort I put in. At the same time, that does not mean I have nothing left to improve on. On the contrary, this experience has made me more aware of how much further I want to grow. It encouraged me to continue sharpening my craft, deepen my knowledge, and to become a better engineer than I was before. MA4012 will always hold a special place in my heart for a personal reason as well. During my polytechnic days, I once worked on a similar autonomous robotics project where I was inexperienced, uncertain, and my robot was labelled as the worst-performing one in the cohort. That memory stayed with me for a long time. It made me doubt myself and I was constantly underestimated by my peers and superiors. At some

	point, I questioned whether I was truly capable in this field and should give up. But through years of perseverance, learning, and continuous effort, I have now lived long enough to see that story change. From being the student with the worst-performing robot to becoming the championship-winning team means more to me than I can ever put into words. It felt like redemption, but more importantly, it proves that failure does not have to define me. Failures are not final and setbacks are not the end of the story. If someone who once stood at the bottom can rise to the top with time, effort, and faithfulness to the process, then it is possible for you too. That is something I now believe not only for myself, but for others as well.
Eng Bing Xian	Looking back at this project, I realised that MA4012 was not just about building a robot, but about understanding how a full-scale engineering project flows in practice. From the beginning, during the allocation of the roles, I volunteered to design the collector mechanism as I am confident of my design knowledge and skills. However, I was unsure whether I could contribute well and if my efforts would transition to real development of the robot. With my core focus being the collector mechanism, I made it a point to ensure that it worked reliably and consistently. There were many moments where ideas that seemed correct in theory did not work as expected, and I had to go back, adjust, print and test again. Through this, it really showed me that engineering design is not about getting things right the first time, but about being willing to refine and the patience to sit down and make improvements until it works. One important lesson I took away is the essence of keeping things simple. At times, it was tempting to overthink or complicate the design like the 1st version of the Ball Ramp which had complex curves. But I realised that straightforward solutions often work best. Instead of adding more complexity, it was more effective to carefully think through how designs could be made simple and positioned to achieve the same result.

	Beyond designing, I also understood the need to do physical testing early. It is easy to convince yourself that a design will work based on logic, but only real testing reveals the actual behaviour. Many small details, such as positioning, alignment, and interaction between parts, only became clear through experimentation. Another aspect that stood out to me was teamwork. With the robot build evolving in the earlier weeks, the changes would affect my design, and vice versa. This made communication very important, updating each other and adapting accordingly. I also observed how much effort was required to keep the team moving forward, especially from the few of us taking on larger responsibilities. It made me appreciate that a successful project is not just about my personal effort, but about the deep commitment of other members of the team. This project also changed my perspective on what it means to be a mechatronics engineer. It is not about sticking strictly to one role, but about being flexible and willing to step in when needed. Even if my main focus was mechanical design, I had to understand how it interacts with sensors and the overall system, providing suggestions to ensure everything works together. In the end, what stood out to me the most was the importance of consistency and attention to detail. Making sure that the collection mechanism works properly and reliably, it contributed to the success of the entire system. Seeing the robot perform well during the competition gave me a sense of satisfaction, knowing that my contributions, though focused on one area, helped support the overall performance. Overall, MA4012 was a meaningful experience that taught me not just technical skills, but also patience, adaptability, and the importance of commitment to responsibilities and fulfilling them with quality. It reinforced the idea that progress comes from continuous effort, and that even small, well-executed contributions can make a significant difference in the final outcome.

Asher Chan	When starting this project, there were no strong expectations going in, but there was optimism about working well as a team, having collaborated with many teammates across previous projects. Looking back, this project turned out to be one of the most rewarding engineering experiences so far, both technically and personally. One of the most significant lessons was the importance of iterative testing and staying open to reconsidering design decisions. The drive system went through multiple configurations from a high-torque setup back to high-speed, then from an asymmetrical encoder-based assembly to a simpler symmetrical one. Each change driven by evaluation of what was actually working. The advice to remove the encoders was initially surprising, but it proved to be the right call and freed up critical space for the collector. This was a good reminder not to be too attached to any one design and to treat every test as useful data rather than just a validation exercise. My primary responsibility was the ball collection and delivery system, and this was where the most growth happened. Working closely with teammates on this, it became clear how much small decisions mattered. The curve of a ramp, the angle of a brush, the positioning of a second roller could make or break the reliability of the whole mechanism. The journey from a simple gated ramp concept to the final 2 roller collection and delivery system with a hinged flap limit switch involved several dead ends, including a brush solution that showed promise but never achieved full reliability. These setbacks were valuable as they pushed a more systematic approach to thinking through failure modes before committing to a design. There was also growth in communicating design intent clearly. Sketching out a rough ramp profile based on roller geometry constraints, or noticing that an uneven rubber band installation was causing roller wobble, were moments where close attention to the physical system paid off. Engineering intuition is built through this kind of hands-on observation. Winning the competition was a satisfying outcome, but what was more meaningful was that the team got there through genuine collaboration and systematic problem-solving. Every member's contribution came together in a way that made the robot consistently competitive across all rounds. This project

	shows that good engineering outcomes are as much a product of good teamwork as they are of good technical decisions. Overall this MA4012 project taught me that communication and dedication from group members are the most important factors that contribute to success of the project. Adapting to each other's roles when certain roles need more help and consistently meeting to work on the robot outside of class time helped us to develop ideas and eliminate those that don't work before we get stuck too deep into one of them. Although our robot did not function perfectly during the competition, we still did our best to adapt and overcome to any challenges that came along the way. The lessons that I have learnt here will be caried with me for all future engineering projects and this team will be the benchmark that all future teams will be compared to.
Lim Yue Feng	Reflecting on the MA4012 project, the greatest takeaway for me was the necessity of a holistic approach to mechatronics. While we were divided into mec1hanical and sensor sub-teams, I realized early on that these boundaries are fluid. To achieve a functional robot, I found myself diving into 'everything and anything', from refining the mechanical intake of the tennis balls to ensuring the sweep logic in the code accounted for physical sensor latency. The process taught me that engineering is rarely a straight line. We faced significant hurdles with the main roller's torque and the reliability of aging sensors, which required constant iteration and a willingness to pivot when initial designs failed. Beyond the technical challenges, managing the integration of these subsystems while keeping the team motivated under the final weeks of pressure was an invaluable lesson in leadership. Seeing our collective effort result in a 1st place finish was the perfect way to cap off our final semester, proving that robust engineering is as much about persistence and teamwork as it is about the initial design.
Nicky Chung Han Jun	Coming into the project, I was not confident in winning at all. I was not confident about the contributions that I would make into the team as I was not confident in my skills. However, as I started, I realized that a good solution does not immediately

	come from the start. Throughout this project, I learned that developing a working solution is not only about having a good idea, but also about being able to adapt when things do not go as planned. Many parts of the design process required patience, repeated testing, and careful adjustment before we could achieve a more reliable outcome. This showed me that engineering is often an iterative process rather than a straightforward one. I also learned the importance of working within constraints. In theory, certain ideas may seem feasible, but in practice they must still fit within the design limitations such as space, dimensions, component placement, and overall system integration. This made me realize that engineering decisions cannot be made by looking at one component alone, and that every design choice can affect the performance of the whole system. Furthermore, additional constraints such as sensor limitations due to old age can affect how we design and adapt our overall system as well. Therefore, many a time, repeating trials and different testing methods are considered and constantly done to optimize our robot in the best possible configuration. Another key takeaway from this project was the value of teamwork and communication. Since the robot was developed as a group, although on paper, I was stated to be on the sensor team, I was more involved in the mechanical side due to time constraints. The coding group started work on their base code using the sensors they brought home for trial and error, while the mechanical side was constantly figuring out the best arrangement and design parameters to allow the tennis ball to be caught and delivered smoothly. Since the robot was developed as a group, it was important to discuss ideas clearly, listen to different opinions, and make decisions together based on testing and practical considerations. It is also worth noting that everyone in the team was willing to contribute in any way, by meeting even on days out of class, to ensure progress on the robot. Through this, I learned that collaboration is important.

	The project also strengthened my problem-solving skills and taught me to be more resourceful. When certain ideas did not work as expected, we had to think of alternative ways to test and improve them instead. For example, in the design of the main roller, initially the roller is already spinning at 100% speed, but the ball could not power through the ramp into the collection point. Different methods were used such as throwing the ball at a faster speed to give it a higher momentum and simulate the robot traveling or lowering height the roller to give the ball more torque. This experience helped me become more open minded and willing to explore different approaches before arriving at a final solution Overall, the MA4012 project gave me a deeper appreciation of the engineering design process, and the effort required to turn an idea into a functional result. Beyond the technical knowledge gained, it also taught me the importance of adaptability, persistence, teamwork, and critical thinking, which I believe will be valuable in future engineering projects. Additionally, I am also proud of everyone for achieving 1st. Jia Yous for Finals, our last sem!!
Ooi Chee Ray	This project was a valuable experience that went beyond just designing and building a robot. One of the biggest challenges throughout the semester was managing the workload while juggling other commitments, especially my Final Year Project at the same time. There were periods where everything felt packed and stressed, and trying to keep up with multiple deadlines was not easy. Another important takeaway was the value of being part of a team where ideas could be shared openly and improved together. Different members brought different perspectives, and these discussions often helped us look at problems from new angles. Through feedback and gradual refinement, initial ideas were developed into better solutions over time. This showed me that collaboration is not only about dividing tasks, but also about learning from one another and growing ideas collectively.
Ng Jia Mei Shermaine	This project made me realise that I should always keep an open mind as in the first few weeks, everyone were throwing out

	amazing ideas and I realised that if we wanted to do well for this project, we should deeply consider all these ideas first and keep an open mind. Along the project, I tried with the best of my ability to help out anywhere (software, electrical hardware) whenever possible, and from doing so, I manage to gain so much knowledge, information and insights from each section. I'm grateful to have the opportunity to help out the software department especially during CNY week because I learnt about the calibration of sensors, and how heavily important it is to our project. I'm also very grateful to our leader for tasking me with designing and 3D-printing multiple components for our vehicle because it is part of my skillset, moreover 3D printing is just starting out to be a hobby of mine. Being able to challenge myself by designing components while trying to consider how they should be installed onto the vehicle was such a great opportunity for me to develop as an engineer. Not all designs that I've cranked out could be easily installed onto the vehicle, therefore its important to always visualise how you want to install a component that you're designing. This has been an amazing project with plenty of learning points that I had picked up on.
Chia Ansel	This project taught me that a successful robot is not built from one big idea alone, but through constant testing, iteration, and refinement. I learned that software and hardware cannot be treated as fully separate, since many coding decisions depended directly on sensor behaviour, placement, and the physical response of the robot. Many of the most effective improvements came from small but practical changes, such as adjusting sensor placement, refining the sweep logic, and allowing the sensors time to stabilise before the robot started. I also experienced how important it is to stay calm and focused under time pressure, especially during the final two weeks when the software had to be integrated quickly. Winning the competition made the experience especially meaningful, because it showed that the time and effort invested throughout the semester had paid off. Overall, this project strengthened my appreciation for adaptability, careful debugging, and the value of simple but robust engineering solutions.

Aravindhan S/O Subbu Pandian	This project taught me several valuable lessons on a personal level. One of the most important was the need to be adaptable. I started the project expecting to contribute mainly in the sensor-related scope, but as the work developed, my involvement shifted into other areas such as mechanical assembly, software logic, calibration support, and documentation. This taught me that in engineering projects, roles are not always fixed, and being useful to the team often means being willing to step into different tasks when needed. I learned that adaptability is not just about accepting change, but about continuing to contribute positively even when the project evolves in unexpected ways. Another important lesson was the value of taking initiative. Rather than limiting myself to my original assigned role, I looked for other ways to support the team, whether by helping with chassis and drivetrain assembly, assisting in the second round of sensor calibration, or drafting an initial program framework for the robot's autonomous behaviour. Even though not every contribution directly appeared in the final implementation, I learned that effort is still valuable when it strengthens understanding, supports team progress, and helps move the project forward. The project also taught me to appreciate the importance of systems thinking and teamwork. By being involved in different parts of the robot, I saw more clearly how mechanical, electrical, and software components are all interconnected. A design choice in one area could affect another, and this helped me understand that successful mechatronics design requires looking at the robot as a complete system rather than as separate parts. At the same time, I learned that good teamwork is built on communication, cooperation, and trust. Since the project involved many moving parts and changing responsibilities, it was important to stay aware of the team's progress and support one another where necessary. On a personal level, this experience also strengthened my confidence. Contributing across different areas showed me that I am capable of learning beyond my initial role and adapting to unfamiliar tasks. Winning the competition made these lessons even more meaningful, because it showed me that persistence, flexibility, and teamwork can lead to a successful outcome.

	Overall, this project was valuable not only because of the final result, but because it helped me grow in maturity, confidence, and understanding of what it means to work as part of a real engineering team.

PEER EVALUATION

Peer Review Form

Group Name:____Team Rollex_____ **Gp (no):**_3_

Peer Assessment by: TAY KAI JIE, WORDEN

Insert score of 1 to 10. **1.. Strongly Disagree 10… Strongly Agree**

Group Member Name:	**Punctual/ Committed**	**Proactive**	**Leadership**	**Groupplayer**	**Technical Knowledge**	**Communication**	**Teaching/ Coaching**		**Total**
ARAVINDHAN S/O SUBBU PANDIAN	10	10	10	10	10	10	10		70
CHIA ANSEL	10	10	10	10	10	10	10		70
NG JIA MEI SHERMAINE	10	10	10	10	10	10	10		70
ENG BING XIAN	10	10	10	10	10	10	10		70
ASHER CHAN YONGRUI	10	10	10	10	10	10	10		70
LIM YUE FENG	10	10	10	10	10	10	10		70
NICKY CHUNG HAN JUN	10	10	10	10	10	10	10		70
OOI CHEE RAY	10	10	10	10	10	10	10		70

Peer Review Form

Group Name:____Team Rollex_____ **Gp (no):**_3_

Peer Assessment by: ARAVINDHAN S/O SUBBU PANDIAN

Insert score of 1 to 10. **1.. Strongly Disagree 10… Strongly Agree**

Group Member Name:	**Punctual/ Committed**	**Proactive**	**Leadership**	**Groupplayer**	**Technical Knowledge**	**Communication**	**Teaching/ Coaching**		**Total**
TAY KAI JIE, WORDEN	10	10	10	10	10	10	10		70
CHIA ANSEL	10	10	10	10	10	10	10		70
NG JIA MEI SHERMAINE	10	10	10	10	10	10	10		70
ENG BING XIAN	10	10	10	10	10	10	10		70
ASHER CHAN YONGRUI	10	10	10	10	10	10	10		70
LIM YUE FENG	10	10	10	10	10	10	10		70
NICKY CHUNG HAN JUN	10	10	10	10	10	10	10		70
OOI CHEE RAY	10	10	10	10	10	10	10		70

Peer Review Form

Group Name:____Team Rollex_____ **Gp (no):**_3_

Peer Assessment by: CHIA ANSEL

Insert score of 1 to 10. **1.. Strongly Disagree 10… Strongly Agree**

Group Member Name:	**Punctual/ Committed**	**Proactive**	**Leadership**	**Groupplayer**	**Technical Knowledge**	**Communication**	**Teaching/ Coaching**		**Total**
TAY KAI JIE, WORDEN	10	10	10	10	10	10	10		70
ARAVINDHAN S/O SUBBU PANDIAN	10	10	10	10	10	10	10		70
NG JIA MEI SHERMAINE	10	10	10	10	10	10	10		70
ENG BING XIAN	10	10	10	10	10	10	10		70
ASHER CHAN YONGRUI	10	10	10	10	10	10	10		70
LIM YUE FENG	10	10	10	10	10	10	10		70
NICKY CHUNG HAN JUN	10	10	10	10	10	10	10		70
OOI CHEE RAY	10	10	10	10	10	10	10		70

Peer Review Form

Group Name:____Team Rollex_____ **Gp (no):**_3_

Peer Assessment by: NG JIA MEI SHERMAINE

Insert score of 1 to 10. **1.. Strongly Disagree 10… Strongly Agree**

Group Member Name:	**Punctual/ Committed**	**Proactive**	**Leadership**	**Groupplayer**	**Technical Knowledge**	**Communication**	**Teaching/ Coaching**		**Total**
TAY KAI JIE, WORDEN	10	10	10	10	10	10	10		70
ARAVINDHAN S/O SUBBU PANDIAN	10	10	10	10	10	10	10		70
CHIA ANSEL	10	10	10	10	10	10	10		70
ENG BING XIAN	10	10	10	10	10	10	10		70
ASHER CHAN YONGRUI	10	10	10	10	10	10	10		70
LIM YUE FENG	10	10	10	10	10	10	10		70
NICKY CHUNG HAN JUN	10	10	10	10	10	10	10		70
OOI CHEE RAY	10	10	10	10	10	10	10		70

Peer Review Form

Group Name:____Team Rollex_____ **Gp (no):**_3_

Peer Assessment by: ENG BING XIAN

Insert score of 1 to 10. **1.. Strongly Disagree 10… Strongly Agree**

Group Member Name:	**Punctual/ Committed**	**Proactive**	**Leadership**	**Groupplayer**	**Technical Knowledge**	**Communication**	**Teaching/ Coaching**		**Total**
TAY KAI JIE, WORDEN	10	10	10	10	10	10	10		70
ARAVINDHAN S/O SUBBU PANDIAN	10	10	10	10	10	10	10		70
CHIA ANSEL	10	10	10	10	10	10	10		70
NG JIA MEI SHERMAINE	10	10	10	10	10	10	10		70
ASHER CHAN YONGRUI	10	10	10	10	10	10	10		70
LIM YUE FENG	10	10	10	10	10	10	10		70
NICKY CHUNG HAN JUN	10	10	10	10	10	10	10		70
OOI CHEE RAY	10	10	10	10	10	10	10		70

Peer Review Form

Group Name:____Team Rollex_____ **Gp (no):**_3_

Peer Assessment by: ASHER CHAN YONGRUI

Insert score of 1 to 10. **1.. Strongly Disagree 10… Strongly Agree**

Group Member Name:	**Punctual/ Committed**	**Proactive**	**Leadership**	**Groupplayer**	**Technical Knowledge**	**Communication**	**Teaching/ Coaching**		**Total**
TAY KAI JIE, WORDEN	10	10	10	10	10	10	10		70
ARAVINDHAN S/O SUBBU PANDIAN	10	10	10	10	10	10	10		70
CHIA ANSEL	10	10	10	10	10	10	10		70
NG JIA MEI SHERMAINE	10	10	10	10	10	10	10		70
ENG BING XIAN	10	10	10	10	10	10	10		70
LIM YUE FENG	10	10	10	10	10	10	10		70
NICKY CHUNG HAN JUN	10	10	10	10	10	10	10		70
OOI CHEE RAY	10	10	10	10	10	10	10		70

Peer Review Form

Group Name:____Team Rollex_____ **Gp (no):**_3_

Peer Assessment by: LIM YUE FENG

Insert score of 1 to 10. **1.. Strongly Disagree 10… Strongly Agree**

Group Member Name:	**Punctual/ Committed**	**Proactive**	**Leadership**	**Groupplayer**	**Technical Knowledge**	**Communication**	**Teaching/ Coaching**		**Total**
TAY KAI JIE, WORDEN	10	10	10	10	10	10	10		70
ARAVINDHAN S/O SUBBU PANDIAN	10	10	10	10	10	10	10		70
CHIA ANSEL	10	10	10	10	10	10	10		70
NG JIA MEI SHERMAINE	10	10	10	10	10	10	10		70
ENG BING XIAN	10	10	10	10	10	10	10		70
ASHER CHAN YONGRUI	10	10	10	10	10	10	10		70
NICKY CHUNG HAN JUN	10	10	10	10	10	10	10		70
OOI CHEE RAY	10	10	10	10	10	10	10		70

Peer Review Form

Group Name:____Team Rollex_____ **Gp (no):**_3_

Peer Assessment by: NICKY CHUNG HAN JUN

Insert score of 1 to 10. **1.. Strongly Disagree 10… Strongly Agree**

Group Member Name:	**Punctual/ Committed**	**Proactive**	**Leadership**	**Groupplayer**	**Technical Knowledge**	**Communication**	**Teaching/ Coaching**		**Total**
TAY KAI JIE, WORDEN	10	10	10	10	10	10	10		70
ARAVINDHAN S/O SUBBU PANDIAN	10	10	10	10	10	10	10		70
CHIA ANSEL	10	10	10	10	10	10	10		70
NG JIA MEI SHERMAINE	10	10	10	10	10	10	10		70
ENG BING XIAN	10	10	10	10	10	10	10		70
ASHER CHAN YONGRUI	10	10	10	10	10	10	10		70
LIM YUE FENG	10	10	10	10	10	10	10		70
OOI CHEE RAY	10	10	10	10	10	10	10		70

Peer Review Form

Group Name:____Team Rollex_____ **Gp (no):**_3_

Peer Assessment by: OOI CHEE RAY

Insert score of 1 to 10. **1.. Strongly Disagree 10… Strongly Agree**

Group Member Name:	**Punctual/ Committed**	**Proactive**	**Leadership**	**Groupplayer**	**Technical Knowledge**	**Communication**	**Teaching/ Coaching**		**Total**
TAY KAI JIE, WORDEN	10	10	10	10	10	10	10		70
ARAVINDHAN S/O SUBBU PANDIAN	10	10	10	10	10	10	10		70
CHIA ANSEL	10	10	10	10	10	10	10		70
NG JIA MEI SHERMAINE	10	10	10	10	10	10	10		70
ENG BING XIAN	10	10	10	10	10	10	10		70
ASHER CHAN YONGRUI	10	10	10	10	10	10	10		70
LIM YUE FENG	10	10	10	10	10	10	10		70
NICKY CHUNG HAN JUN	10	10	10	10	10	10	10		70

APPENDIX A: SOFTWARE CODE

```
#pragma config(Sensor, in1,    Opponent,       sensorAnalog)
#pragma config(Sensor, in2,    FrontLeft_LineSensor, sensorReflection)
#pragma config(Sensor, in3,    FrontRight_LineSensor, sensorReflection)
#pragma config(Sensor, in4,    BackLeft_LineSensor, sensorReflection)
#pragma config(Sensor, in5,    BackRight_LineSensor, sensorReflection)
#pragma config(Sensor, in6,    LeftDist,       sensorAnalog)
#pragma config(Sensor, in7,    RightDist,      sensorAnalog)
#pragma config(Sensor, dgtl3,  startSwitch,    sensorDigitalIn)
#pragma config(Sensor, dgtl5,  rightRamp,      sensorDigitalIn)
#pragma config(Sensor, dgtl6,  leftRamp,       sensorDigitalIn)
#pragma config(Sensor, dgtl7,  BackSwitch,     sensorDigitalIn)
#pragma config(Sensor, dgtl8,  north,          sensorDigitalIn)
#pragma config(Sensor, dgtl9,  east,           sensorDigitalIn)
#pragma config(Sensor, dgtl10, south,          sensorDigitalIn)
#pragma config(Sensor, dgtl11, west,           sensorDigitalIn)
#pragma config(Motor,  port2,           collectMotor,  tmotorVex269_MC29, openLoop, reversed)
#pragma config(Motor,  port3,           rightMotor,    tmotorVex393_MC29, openLoop)
#pragma config(Motor,  port8,           leftMotor,     tmotorVex393_MC29, openLoop, reversed)
#pragma config(Motor,  port9,           dispenseMotor, tmotorVex269_MC29, openLoop, reversed)
//*!!Code automatically generated by 'ROBOTC' configuration wizard               !!*//

int state = 1;
//Robot Motor Functions:
void forward()
{
    motor[leftMotor]  = 65;
    motor[rightMotor] = 70;
}

void reverse()
{
    motor[leftMotor]  = -65;
    motor[rightMotor] = -70;
}

void turnLeft()
{
    motor[leftMotor]  = -60;   // Left wheel backward
    motor[rightMotor] =  60;   // Right wheel forward
}

void turnRight()
{
    motor[leftMotor]  = 60;   // Left wheel forward
    motor[rightMotor] = -60;   // Right wheel backward
}

void slowdown()
{
  motor[leftMotor]  = 60;
  motor[rightMotor] = 60;
}

void reverseSlow()
{
  motor[leftMotor]  = -60;
  motor[rightMotor] = -60;
}

void stopDrive()
{
    motor[leftMotor]  = 0;
    motor[rightMotor] = 0;
}

void collect()
{
    motor[collectMotor] = 100;
}

void stopCollect()
{
    motor[collectMotor] = 0;
}

void dispense()
{
  motor[dispenseMotor] = 90;
}

void stopDispense()
{
  motor[dispenseMotor] = 0;
}
//Robot's Sensor Values

bool ballCollected() //Detect if Ramp is closed
{
    return (SensorValue[leftRamp] == 0 || SensorValue[rightRamp] == 0);
}

bool opponentDetected() //Detect if Opponent is infront
{
    return (SensorValue[Opponent] > 800);   // TUNE THIS
}


```

```
bool leftDistDetected() //Detect if left sensor detects ball
{
    return (SensorValue[LeftDist] > 850);
}

bool rightDistDetected() //Detect if right sensor detects ball
{
    return (SensorValue[RightDist] > 800);
}


bool leftDistNotDetected() //Detect if left sensor no detection
{
    return (SensorValue[LeftDist] < 850);
}


bool rightDistNotDetected() //Detect if right sensor no detection
{
    return (SensorValue[RightDist] < 800);
}
int frontHitType()
{
    bool leftHit  = (SensorValue[FrontLeft_LineSensor] < 600);
    bool rightHit = (SensorValue[FrontRight_LineSensor] < 600);

    // already both hit now
    if(leftHit && rightHit)
    {
        return 3;
    }

    // left hit first: track for 100 ms to see if right also becomes hit
    if(leftHit && !rightHit)
    {
        for(int i = 0; i < 5; i++)   // 5 x 20 ms = 100 ms
        {
            if(SensorValue[FrontRight_LineSensor] < 600)
            {
                return 3;   // became full front line
            }
            wait1Msec(20);
        }
        return 1;   // stayed left-only
    }

    // right hit first: track for 100 ms to see if left also becomes hit
    if(!leftHit && rightHit)
    {
        for(int i = 0; i < 5; i++)   // 25 x 20 ms = 100 ms
        {
            if(SensorValue[FrontLeft_LineSensor] < 600)
            {
                return 3;   // became full front line
            }
            wait1Msec(20);
        }
        return 2;   // stayed right-only
    }

    return 0;   // no hit
}
int backHitType()
{
    bool leftHit  = (SensorValue[BackLeft_LineSensor] < 600);
    bool rightHit = (SensorValue[BackRight_LineSensor] < 600);

    // already both hit now
    if(leftHit && rightHit)
    {
        return 3;
    }

    // left hit first: track for 100 ms to see if right also becomes hit
    if(leftHit && !rightHit)
    {
        for(int i = 0; i < 5; i++)   // 5 x 20 ms = 100 ms
        {
            if(SensorValue[BackRight_LineSensor] < 600)
            {
                return 3;   // became full front line
            }
            wait1Msec(20);
        }
        return 1;   // stayed left-only
    }

    // right hit first: track for 100 ms to see if left also becomes hit
    if(!leftHit && rightHit)
    {
        for(int i = 0; i < 5; i++)   // 5 x 20 ms = 100 ms
        {
            if(SensorValue[BackLeft_LineSensor] < 600)
            {
                return 3;   // became full front line
            }
            wait1Msec(20);
```

```
        }
        return 2;   // stayed right-only
    }

    return 0;   // no hit
}

//Robot's Compass Direction:

bool isFacingNorth() //Facing Front
{
    return (SensorValue[north] == 0 &&
            SensorValue[east]  == 1 &&
            SensorValue[south] == 1 &&
            SensorValue[west]  == 1);
}

bool isFacingNorthWest() //Facing NorthWest
{
    return (SensorValue[north] == 0 &&
            SensorValue[east]  == 1 &&
            SensorValue[south] == 1 &&
            SensorValue[west]  == 0);
}

bool isFacingSouthEast() //Facing Right
{
    return (SensorValue[north] == 1 &&
            SensorValue[east]  == 0 &&
            SensorValue[south] == 0 &&
            SensorValue[west]  == 1);
}


bool isFacingSouth() //Facing Behind
{
    return (SensorValue[north] == 1 &&
            SensorValue[east]  == 1 &&
            SensorValue[south] == 0 &&
            SensorValue[west]  == 1);
}

//Robot's Complex Functions:
void pointNorth() //North
{
  while(true)
  {
    if (SensorValue[west] == 0)
    {
      turnRight();
      // wait1Msec(40);
      // stopDrive();
      // wait1Msec(20);
    }
    else if (SensorValue[east] == 0)
    {
      turnLeft();
      //wait1Msec(50);
      //stopDrive();
      //wait1Msec(20);
    }
    else if (isFacingNorth()) //face north
    {
      stopDrive();
      break;
    }
    else
    {
      turnRight();
      // wait1Msec(40);
      // stopDrive();
      // wait1Msec(20);
    }
  }
}
void alignNorth() // North
{
  while(true)
  {
    if (SensorValue[west] == 0) //turn right if facing left
    {
        turnRight();
        if (isFacingNorthWest()) //face north-west
        {
          stopDrive();
          break;
        }
    }
    else if (SensorValue[east] == 0) //turn left if facing right
    {
        turnLeft();
        if (isFacingNorthWest()) //face north-west
        {
          stopDrive();
          break;
        }
```

```
    }
    else //face other directions
    {
        turnRight();
        if (isFacingNorthWest()) //face north-west
        {
          stopDrive();
          break;
        }
    }
  }

  while (true)
  {
    if (isFacingNorth())
    {
      stopDrive();
      break;
    }
    else
    {
      turnRight();
    }
  }
}
bool frontLineDetected() //Detect if Robot reached front end
{
    return (frontHitType() == 3);
}

bool backLineDetected() //Detect if Robot reached front end
{
    return (backHitType() == 3);
}

bool backleftCornerHit()
{
    return (backHitType() == 1);
}

bool backrightCornerHit()
{
    return (backHitType() == 2);
}
bool frontleftCornerHit()
{
    return (frontHitType() == 1);
}
```

```

bool frontrightCornerHit()
{
    return (frontHitType() == 2);
}


void boundaryFrontCheck()
{
  while(frontleftCornerHit() || frontrightCornerHit())
    {
        bool hitFL  = frontleftCornerHit();
        bool hitFR = frontrightCornerHit();

        stopDrive();
        wait1Msec(20);
        reverseSlow(); //clearance of line
        wait1Msec(600);
        stopDrive();
        wait1Msec(20);

        if(hitFL)
        {
            turnRight();
            wait1Msec(40);
            slowdown();
            wait1Msec(700);
        }
        else if(hitFR)
        {
            turnLeft();
            wait1Msec(40);
            slowdown();
            wait1Msec(700);
        }

        stopDrive();
        wait1Msec(20);
    }
}
void boundaryBackCheck() //Boundary line
{
    while(backleftCornerHit() || backrightCornerHit())
    {
        bool hitBL  = backleftCornerHit();
        bool hitBR = backrightCornerHit();

        stopDrive();
```

```
        wait1Msec(20);
        slowdown(); //clearance of line
        wait1Msec(600);
        stopDrive();
        wait1Msec(20);

        if(hitBL)
        {
            turnLeft();
            wait1Msec(40);
            reverse();
            wait1Msec(700);
        }
        else if(hitBR)
        {
            turnRight();
            wait1Msec(40);
            reverse();
            wait1Msec(700);
        }

        stopDrive();
        wait1Msec(20);
    }
}

bool rightUntilNorth() //Left turn to check for ball
{
    clearTimer(T2);
    while(!isFacingNorth())
    {
      turnRight();

      if(frontLineDetected()) //Don't sweep if reached arena end
      {
          stopDrive();
          return false;
      }

      if(frontleftCornerHit() || frontrightCornerHit())//Stop this sweep if hit yellow line
      {
          stopDrive();
          return false;
      }

      if(opponentDetected())
        {
            clearTimer(T2);
        }

      if(time1[T2] < 500)
      {
          wait1Msec(20);
          continue;
      }

      if (leftDistDetected()) //If first signal detected
      {
        stopDrive();
        wait1Msec(20);

        while (true)
        {
          turnLeft(); //cotinue turn left until not detected

          if(leftDistNotDetected())
          {
            wait1Msec(180);
            stopDrive(); //ball center
            break;
          }
        } //end of while loop
        return true;
      }

    }
    stopDrive();
    return false;
}


bool leftUntilSouthWest() //Left turn to check for ball, use both sensors for sweep
{
    clearTimer(T2);
    while(!isFacingSouth())
    {
      turnLeft();

      if(frontLineDetected()) //Don't sweep if reached arena end
      {
          stopDrive();
          return false;
      }

      if(frontleftCornerHit() || frontrightCornerHit())//Stop this sweep if hit yellow line
      {
```

```
            stopDrive();
            return false;
        }

        if(opponentDetected())
          {
              clearTimer(T2);
          }

          if(time1[T2] < 500)
          {
              wait1Msec(20);
              continue;
          }

        if (rightDistDetected()) //If first signal detected
        {
          stopDrive();
          wait1Msec(20);
          while (true)
          {
            turnRight(); //cotinue turn right until not detected

            if(rightDistNotDetected())
            {
              wait1Msec(180);
              stopDrive(); //ball center
              break;
            }
          } //end of while loop
          return true;
        }

    }
    stopDrive();
    return false;
}

bool rightUntilSouthEast() //Right turn to check ball
{
    clearTimer(T2);
    while(!isFacingSouthEast())
    {
      turnRight();

      if(frontLineDetected()) //Don't sweep if reached arena end
      {
          stopDrive();
          return false;
      }

      if(frontleftCornerHit() || frontrightCornerHit())//Stop this sweep if hit yellow line
      {
          stopDrive();
          return false;
      }

      if(opponentDetected())
        {
            clearTimer(T2);
        }

        if(time1[T2] < 500)
        {
            wait1Msec(20);
            continue;
        }

      if (leftDistDetected()) //If first signal detected
      {
        stopDrive();
        wait1Msec(20);
        while (true)
        {
          turnLeft(); //cotinue turn left until not detected

          if(leftDistNotDetected())
          {
            wait1Msec(180);
            stopDrive(); //ball center
            break;
          }
        } //end of while loop
        return true;
      }

    }
     stopDrive();
     return false;
}

bool check360()
{
    clearTimer(T2);
    clearTimer(T1);
    turnLeft();
```

```
579
580         while(time1[T1] < 5000)
581         {
582           if(opponentDetected())
583             {
584                 clearTimer(T2);
585             }
586
587             if(time1[T2] < 500)
588             {
589                 wait1Msec(20);
590                 continue;
591             }
592
593           if (rightDistDetected()) //If first signal detected
594           {
595             stopDrive();
596             wait1Msec(20);
597             while (true)
598             {
599               turnRight(); //cotinue turn right until not detected
600               if(rightDistNotDetected())
601               {
602                 //wait1Msec(100);
603                 stopDrive(); //ball center
604                 break;
605               }
606             }
607             return true;
608           }
609         }
610         stopDrive();
611         return false;
612     }
613
614     bool checkForward() //Ball Detection as it moves forward every 500ms for sweep
615     {
616
617         for(int i = 0; i < 50; i++)   // 50 x 20 ms = 1000 ms total
618         {
619             if (ballCollected())
620             {
621                 stopDrive();
622                 stopCollect();
623                 return false;   // collected unseen, not "foundBall"
624             }
625              if (frontLineDetected()) //end of arena so stop!
626             {
627                 stopDrive();
628                 stopCollect();
629                 return false;
630             }
631
632             if (frontleftCornerHit() || frontrightCornerHit()) //Self-correct if hit side of arena!
633             {
634                 boundaryFrontCheck();
635                 continue;
636             }
637
638             if (opponentDetected())
639             {
640                 stopDrive();
641                 collect();   // keep roller on
642
643                 // wait until center opponent sensor clears
644                 while (opponentDetected())
645                 {
646                     wait1Msec(20);
647                 }
648
649                 // then require side sensors to be clear briefly too
650                 int clearCount = 0;
651                 while (clearCount < 10)   // 10 x 20 ms = 200 ms clear
652                 {
653                     if (!opponentDetected() &&
654                         leftDistNotDetected() &&
655                         rightDistNotDetected())
656                     {
657                         clearCount++;
658                     }
659                     else
660                     {
661                         clearCount = 0;
662                     }
663
664                     wait1Msec(20);
665                 }
666
667                 continue;
668             }
669
670
671             if (leftDistDetected()) //When left sensor detect
672             {
673                 while(leftDistDetected()) //Turn until not detected
674                 {
```

```
                turnLeft();
            }
            wait1Msec(50); //Center the Ball
            stopDrive();
            return true;
        }

        if (rightDistDetected()) //When right sensor detect
        {
            while(rightDistDetected()) //Readjust turn
            {
                turnRight();
            }
            wait1Msec(50); //Center the Ball
            stopDrive();
            return true;
        }
// When no sensors detect, move forward and on collector to repeat each small step
        forward();
        collect();
        wait1Msec(20);
    }
//After 500ms distance tuned, stop and return no ball detected OR ball collected but unseen
    stopDrive();
    stopCollect();
    return false;
}

bool sweepMotion() //Turns right, turns left, turns back to center, assume facing North
{ //If line detection, move on to next turn!
    if(rightUntilSouthEast()) return true;
    if(leftUntilSouthWest()) return true;
    if(rightUntilNorth()) return true;

    return false;
}


void changeLaneRight()
{

    while(!(SensorValue[north] == 0 && // make robot face 45 deg (north-east)
            SensorValue[east]  == 0 &&
            SensorValue[south] == 1 &&
            SensorValue[west]  == 1))
    {
        turnRight();
        wait1Msec(20);
    }

    motor[leftMotor] = 0; // stop before going forward, angled position
    motor[rightMotor] = 0;
    wait1Msec(100);

    for(int i = 0; i < 15; i++) //move forward by 10 counts
    {
        forward();
        wait1Msec(100);
        if(SensorValue[FrontRight_LineSensor] < 600) //Line detection
        {
            stopDrive(); //Stop
            wait1Msec(100);
            reverseSlow(); //reverse
            wait1Msec(500);
            pointNorth(); //point North
            wait1Msec(500);

            while (!((backLineDetected() || SensorValue[BackSwitch] == 0) && SensorValue[north] == 0))
            {
                reverse();
                wait1Msec(50);
            }
            wait1Msec(500);
            state = 2;
            return;
        }
    }
    stopDrive();
    pointNorth();
    wait1Msec(500);
}

void changeLaneLeft()
{

    while(!(SensorValue[north] == 0 && // make robot face 45 deg (north-west)
            SensorValue[east]  == 1 &&
            SensorValue[south] == 1 &&
            SensorValue[west]  == 0))
    {
        turnLeft();
        wait1Msec(20);
    }
    wait1Msec(250);
    stopDrive();
    wait1Msec(100);
```

```
771
772         for(int i = 0; i < 10; i++) //move forward by 10 counts diagonally
773         {
774             forward();
775             wait1Msec(100);
776             if(SensorValue[FrontLeft_LineSensor] < 600) //Line detection
777             {
778                 stopDrive(); //Stop
779                 wait1Msec(100);
780                 reverseSlow(); //reverse
781                 wait1Msec(500);
782                 pointNorth(); //point North
783                 wait1Msec(500);
784
785                 while (!((backLineDetected() || SensorValue[BackSwitch] == 0) && SensorValue[north] == 0))
786                 {
787                     reverse();
788                     wait1Msec(50);
789                 }
790                 wait1Msec(500);
791                 state = 1;
792                 return;
793             }
794         }
795         stopDrive();
796         pointNorth();
797         wait1Msec(500);
798     }
799
800     void goingHomeToDeliver()
801     {
802         pointNorth();
803         wait1Msec(200);
804         clearTimer(T3);
805         while (true)
806         {
807
808             reverse();
809
810              if ((backLineDetected() || SensorValue[BackSwitch] == 0) && SensorValue[north] == 0) //Reached Home
811             {
812                 stopDrive();
813                 clearTimer(T3);
814                 break;
815             }
816
817             // Step 3: if still not home after 15 sec, retry by moving forward then reverse again
818             if (!backLineDetected() && time1[T3] >= 15000) //Breakout Function
819             {
820                 stopDrive();
821                 wait1Msec(100);
822
823                 slowdown();
824                 wait1Msec(500);
825
826                 stopDrive();
827                 wait1Msec(100);
828
829                 clearTimer(T3);   // restart 10 s search window
830                 continue;
831             }
832
833             if (backleftCornerHit() || backrightCornerHit())
834             {
835               boundaryBackCheck();
836               continue;
837             }
838             // only mild correction if drift is obvious
839             if (SensorValue[west] == 0)
840             {
841                 motor[leftMotor] = 0;
842                 motor[rightMotor] = -70;
843             }
844             else if (SensorValue[east] == 0)
845             {
846                 motor[leftMotor] = -70;
847                 motor[rightMotor] = 0;
848             }
849
850             wait1Msec(20);
851         }
852     }
853
854     void ballDelivery()
855     {
856       while (true) //Ball Delivery
857       {
858         stopDrive();
859         dispense();
860         wait1Msec(1000);
861         stopDispense();
862         break;
863       }
864     }
865
866     task main()
```

```
{
  while(true)
  {
  if (SensorValue[startSwitch]==0)
  {
    bool FirstBall=false;
    slowdown();
    wait1Msec(500);

  while (true) //First Pass Search Algorithm: Moving Straight
  {
    forward(); //Move forward and collect if no ball detected or collected (Case 3)
    collect();
    //wait1Msec(500);

    if (ballCollected()) // Ball Inside then can break loop
    {
      stopDrive(); //Stop Robot
      wait1Msec(500); //Roller continue to ensure flap fully down
      stopCollect(); //Rest the Roller
      reverseSlow();
      wait1Msec(700); //reverse in whatever direction to align back to start position
      stopDrive();
      FirstBall = true;
      break; //Finish 1st pass, Ball is inside
    }

    else if (frontLineDetected()) //Line Detection => Miss Ball (Case 4)
      {
        stopDrive();
        wait1Msec(500);
        stopCollect(); //Stop first
        reverseSlow();
        wait1Msec(700);
        FirstBall = false;
        break; //Finish 1st pass, failed capture!
      }

    else if (frontleftCornerHit() || frontrightCornerHit()) //Only correct corner sensors
    {
      boundaryFrontCheck();
      continue;
    }

    // Ball spotted on the LEFT side (Case 1)
    else if (SensorValue[LeftDist] > 850)
    {
      stopDrive(); //Stop
      wait1Msec(100);
      while (true)
      {
        turnLeft();
        if (leftDistNotDetected()) //Turn until ball not detected
        {
          wait1Msec(100);
          stopDrive();
          break; //stop turn to center ball
        }
      }
      slowdown();
      collect(); // attempt to collect forward
      while (true)
      {
        if(ballCollected())
        {
            stopDrive();
            wait1Msec(500);
            stopCollect();
            reverseSlow(); //reverse back
            wait1Msec(700);
            FirstBall = true;
            break;
        }
        if (SensorValue[FrontLeft_LineSensor] < 600) //if line detected, will continue forward collection for abit and stop while collecting
        {
          wait1Msec(200);
          stopDrive();
          wait1Msec(500);
          stopCollect(); //Stop already
          reverseSlow(); //reverse back
          wait1Msec(700);
          if (ballCollected())
          {
            FirstBall = true;
          }
          else
          {
            FirstBall = false;
          }
          break; //Finish 1st pass check capture
        } //end of if case for edge

      } //end of while loop for forward collection
      break;
     }//end of case 1

```

```
963         // Ball spotted on the RIGHT side (Case 2)
964         else if (SensorValue[RightDist] > 800)
965         {
966           stopDrive();
967           wait1Msec(100);
968           while (true)
969           {
970             turnRight();
971             if (rightDistNotDetected()) //Turn until ball not detected
972             {
973               stopDrive();
974               break; //stop turning loop to align center
975             }
976           }
977           slowdown();
978           collect(); // attempt collection
979           while (true)
980           {
981             if(ballCollected())
982             {
983                 stopDrive();
984                 wait1Msec(500);
985                 stopCollect();
986                 reverseSlow(); //reverse back
987                 wait1Msec(700);
988                 FirstBall = true;
989                 break;
990             }
991             if (SensorValue[FrontRight_LineSensor] < 600) //if line detected, will continue forward collection for abit and stop while collecting
992             {
993               wait1Msec(200);
994               stopDrive();
995               wait1Msec(500);
996               stopCollect(); //Stop already
997               reverseSlow(); //reverse back
998               wait1Msec(700);
999               if (ballCollected())
1000              {
1001                FirstBall = true;
1002              }
1003              else
1004              {
1005                FirstBall = false;
1006              }
1007              break; //Finish 1st pass and check capture
1008            }
1009          }
1010          break;
1011        }
1012    }
1013    stopDrive();
1014    wait1Msec(20); //Stop first so that don't keep reversing for next loop.
1015
1016
1017    //Check Capture!
1018    while (true)
1019    {
1020      if (FirstBall)
1021      {
1022        break; //Move on to delivery
1023      }
1024      else //if ball not captured, retry
1025      {
1026        if (check360()) //Spin 360 to detect ball location
1027        {
1028            slowdown();
1029            collect();
1030
1031            bool gotBall = false;
1032            for(int i = 0; i < 75; i++)   // 1.5 s
1033            {
1034                if(ballCollected())
1035                {
1036                    gotBall = true;
1037                    break;
1038                }
1039
1040                if(frontLineDetected())
1041                {
1042                    stopDrive();
1043                    wait1Msec(500);
1044                    stopCollect();
1045
1046                    if(ballCollected())
1047                    {
1048                        gotBall = true;
1049                    }
1050                    else
1051                    {
1052                        gotBall = false;
1053                    }
1054                    break;
1055                }
1056
1057                if(frontleftCornerHit() || frontrightCornerHit())
1058                {
```

```
                    stopDrive();
                    wait1Msec(500);
                    stopCollect();

                    if(ballCollected())
                    {
                        gotBall = true;
                    }
                    else
                    {
                        gotBall = false;
                    }
                    break;
                }
                wait1Msec(20);
            }
            stopDrive();
            stopCollect();
            FirstBall = gotBall;
        }
        else //Ball not there anymore
        {
            pointNorth();
            break; //Give up and Home Empty-Hand
        }
      }
    }
    goingHomeToDeliver();

    if (FirstBall)
    {
      ballDelivery();
    }



    //Second Pass
    while(true)
    {
      while(true)
      {
        if (state == 1)
        {
            changeLaneRight();
            if (state==2)
            {
              changeLaneLeft(); //switch lane and move on
              break;
            }
            else
            {
              break; //if never yellow line, move on
            }
        }
        else if (state == 2)
        {
            changeLaneLeft();
            if (state==1)
            {
              changeLaneRight(); //switch lane and move on
              break;
            }
            else
            {
              break; //if never yellow line, move on
            }

        }
      }
      bool foundBall=false;

      for(int m = 0; m < 4; m++) //Loop Sweep Motion 3 times
      {
        pointNorth();   //Make robot face straight after lane change
        wait1Msec(100);

        if(checkForward()) //Move forward and check for ball detection too
        {
          foundBall = true; //Ball is detected ONLY IF left and right sensors detected during forward
          break;
        }
        else if (ballCollected()) //Ball not detected during forward BUT collected
        {
          break;
        }

        if(sweepMotion()) //Left,Right,North for ball detection
        {
          foundBall = true;
          break;
        }
      }
      //End of For Loop, Only stop sweeping if ball found or collected unseen

      while (true) //Ball Collection loop
```

```
    {
      if (foundBall) //Scenario 1: Ball Detected
      {
          slowdown(); //Forward slow
          collect(); //On Collector

          bool collected = false;

          for (int i = 0; i < 75; i++)   // 75 x 20 ms = 1500 ms of continuous tracking ball intake
          {
              if (ballCollected()) //Ball is inside
              {
                  stopDrive();
                  wait1Msec(100);
                  stopCollect();
                  collected = true;
                  break; //Stop robot, break loop
              }

              if(frontLineDetected())
              {
                  stopDrive();
                  wait1Msec(500);
                  stopCollect();
                  if (ballCollected()) //Ball is inside
                  {
                    collected = true;
                  }
                  else
                  {
                    collected = false;
                  }
                  break;
              }

              if(frontleftCornerHit() || frontrightCornerHit())
              {
                  stopDrive();
                  wait1Msec(500);
                  stopCollect();
                  if (ballCollected()) //Ball is inside
                  {
                    collected = true;
                  }
                  else
                  {
                    collected = false;
                  }
                  break;
              }

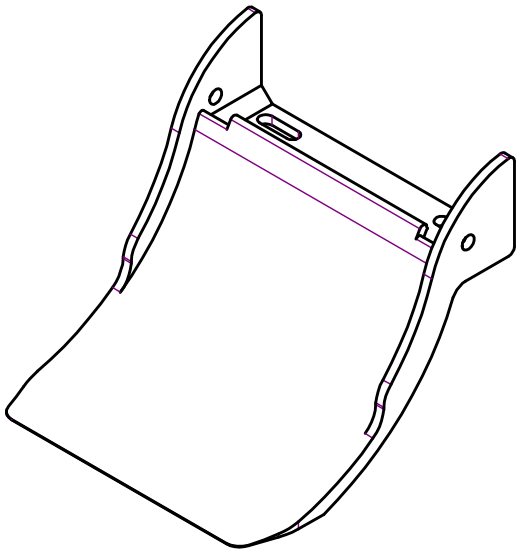
              wait1Msec(20); //Otherwise keep going until 1.5s over
          }

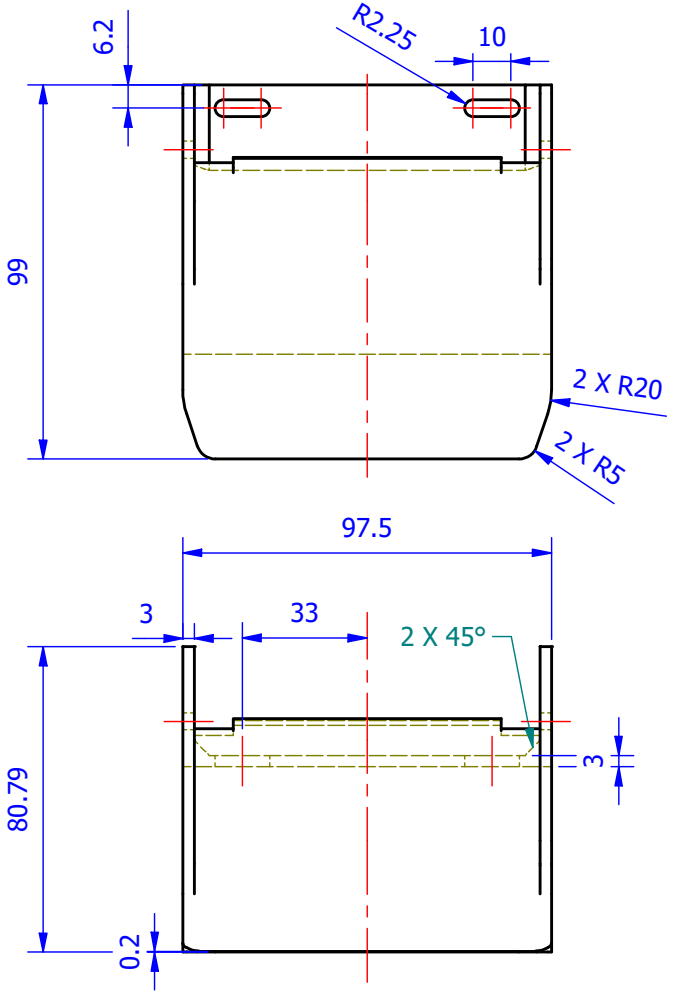
          if (collected)
          {
              break;   // go home
          }
          else //Ball Missed and Unsuccessful Collection
          {
              stopDrive();
              wait1Msec(500);
              stopCollect(); //Stop first
              reverseSlow();
              wait1Msec(700);

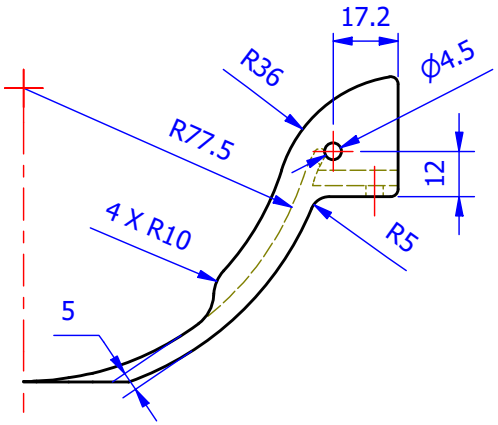
              if (check360()) //Spin 360 to detect ball location
              {
                  foundBall = true; //Found ball so try again to collect
              }
              else //Ball not there anymore
              {
                  pointNorth();
                  break; //Give up and Home Empty-Hand
              }
          }
      }

      //Scenario 2: Ball Undetected, Check if collected
      else if (!foundBall && ballCollected()) //Robot collects ball during forward without detecting it
      {
        stopDrive(); //Stop Robot
        stopCollect(); //Rest the Roller
        break; //Proceed to deliverHome
      }
      //Scenario 3: Ball Undetected, Ball Not collected and reached end
      else if (!foundBall && frontLineDetected()) //Robot has reached end of arena before end of 3 sweep
      {
        break;
      }
      //Scenario 4
      else //Robot has finished 3 sweeps, lane change
      {
        break;
      }
```

```
1251        }//Loop ends when ball is collected and ready to go Home!
1252
1253        goingHomeToDeliver();
1254
1255        if (ballCollected())
1256        {
1257          ballDelivery();
1258        }
1259        state = 2;
1260      }
1261    }
1262    }
1263    }
1264
```





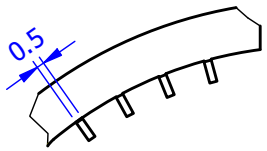


NOTES:

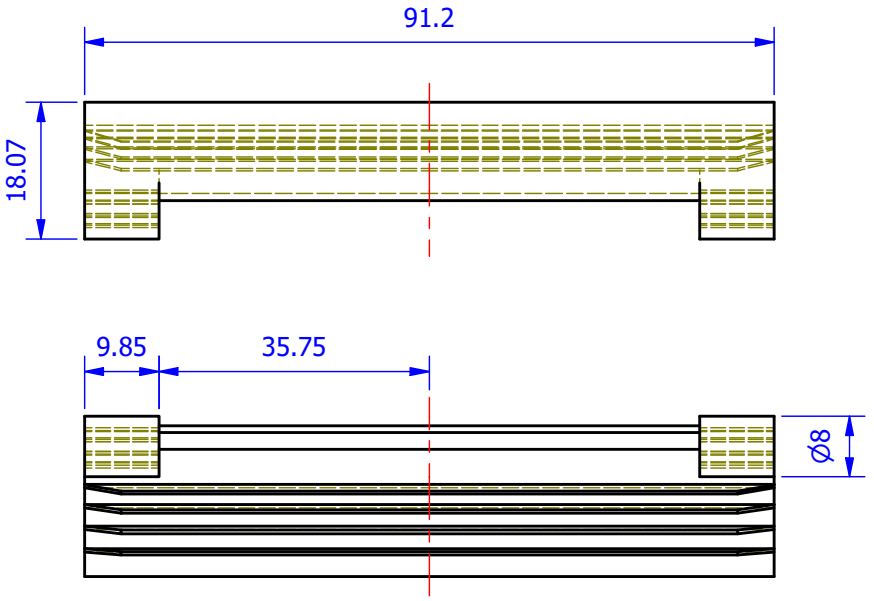
ALL FILLETS R2 UNLESS OTHERWISE SPECIFIED.

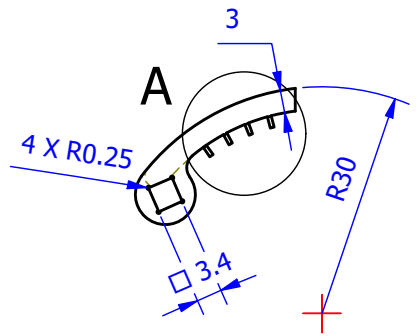
3D PRINTED PART.

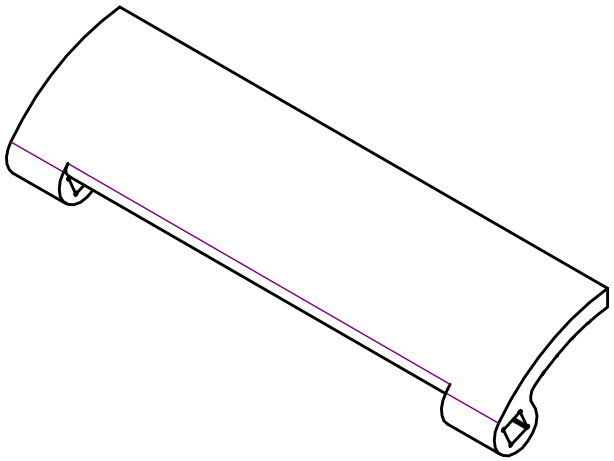
NANYANG TECHNOLOGICAL UNIVERSITY SCHOOL OF MECHANICAL AND AEROSPACE ENGINEERING					PROJECT TITLE MA4012	
DRAWN ENG BING XIAN	CHECKED SHERMAINE NG	DATE 14/04/2026	QTY 1	SCALE 1:2	PART NAME BALL RAMP	
LECTURER JOHN HENG	MATERIAL PLA		DRAWING NO. APPENDIX A1		CLASS/GROUP NO. GROUP 3	SIZE A4



A (2 : 1)



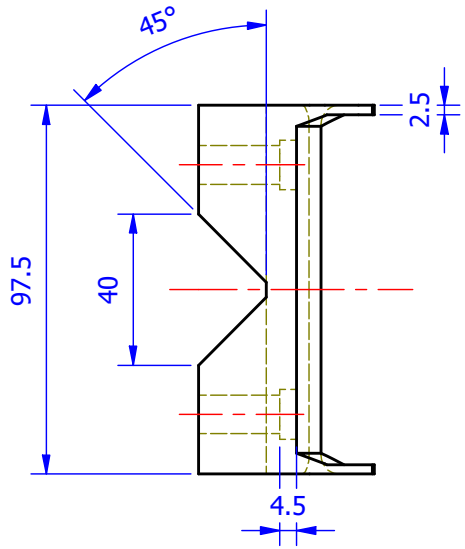


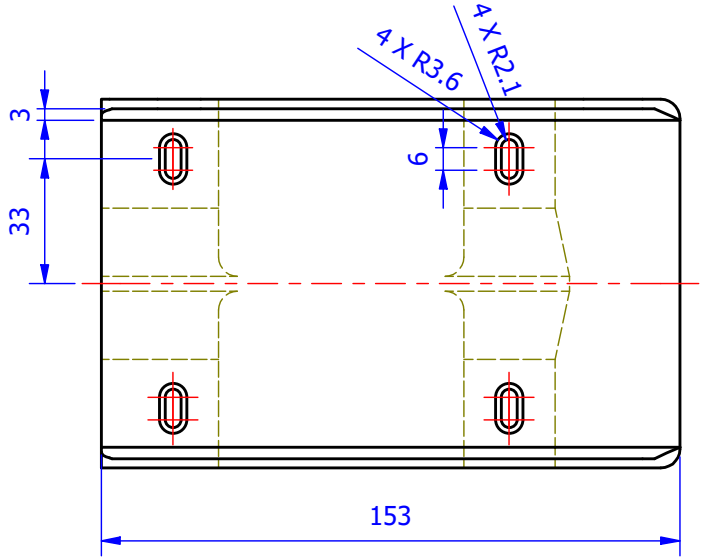


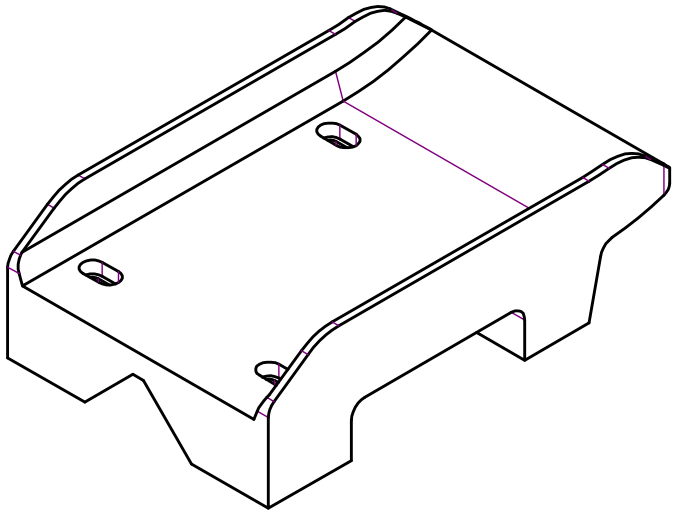
NOTES:

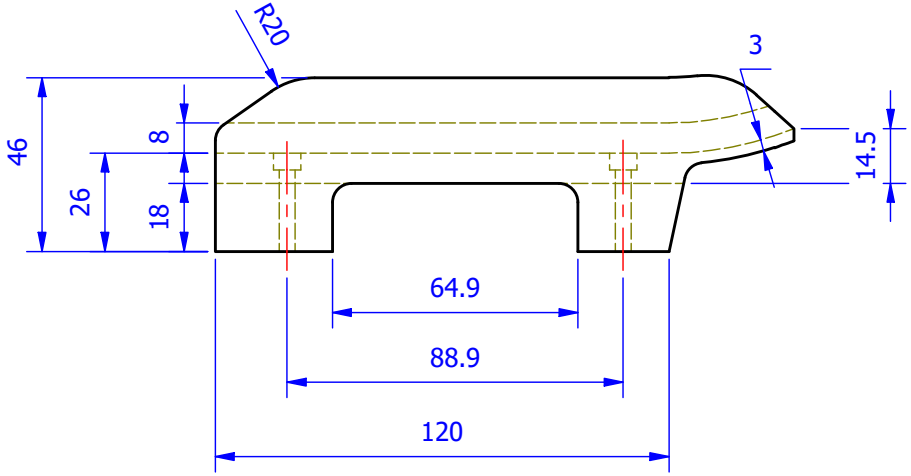
3D PRINTED PART.

NANYANG TECHNOLOGICAL UNIVERSITY SCHOOL OF MECHANICAL AND AEROSPACE ENGINEERING					PROJECT TITLE MA4012	
DRAWN ENG BING XIAN	CHECKED SHERMAINE NG	DATE 14/04/2026	QTY 1	SCALE 1:1	PART NAME TRIGGER FLAP	
LECTURER JOHN HENG	MATERIAL PLA		DRAWING NO. APPENDIX A2		CLASS/GROUP NO. GROUP 3	SIZE A4







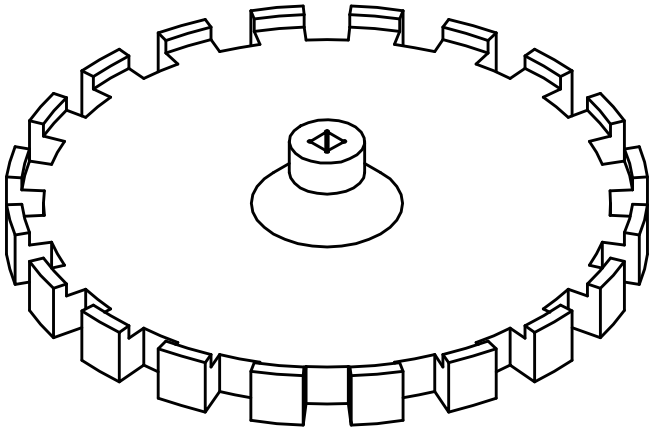


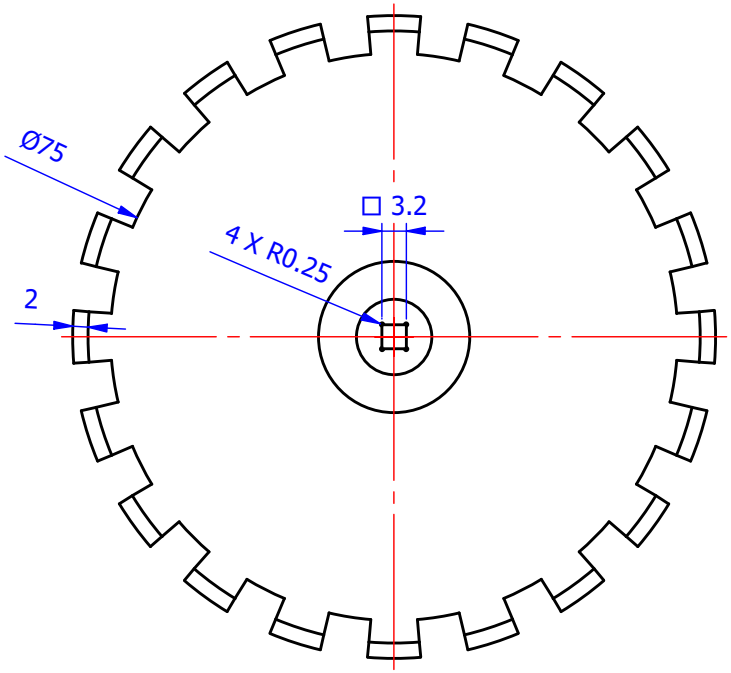
NOTES:

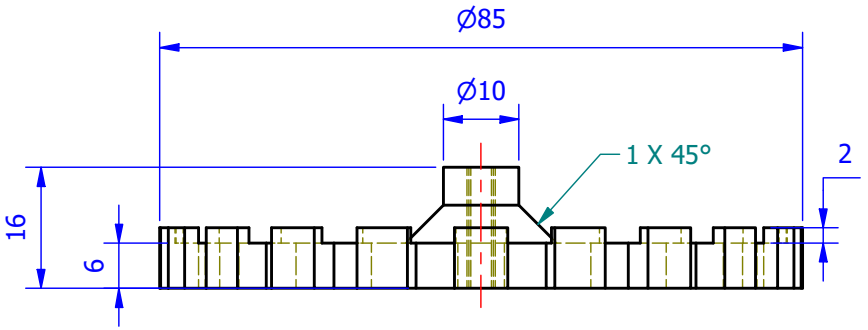
ALL FILLETS R5 UNLESS OTHERWISE SPECIFIED.

3D PRINTED PART.

NANYANG TECHNOLOGICAL UNIVERSITY SCHOOL OF MECHANICAL AND AEROSPACE ENGINEERING					PROJECT TITLE MA4012	
DRAWN ENG BING XIAN	CHECKED SHERMAINE NG	DATE 14/04/2026	QTY 1	SCALE 1:2	PART NAME DELIVERY RAMP	
LECTURER JOHN HENG	MATERIAL PLA		DRAWING NO. APPENDIX A3		CLASS/GROUP NO. GROUP 3	SIZE A4



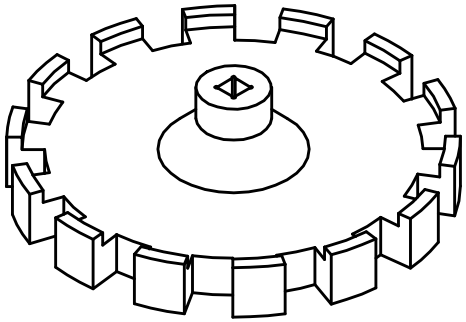


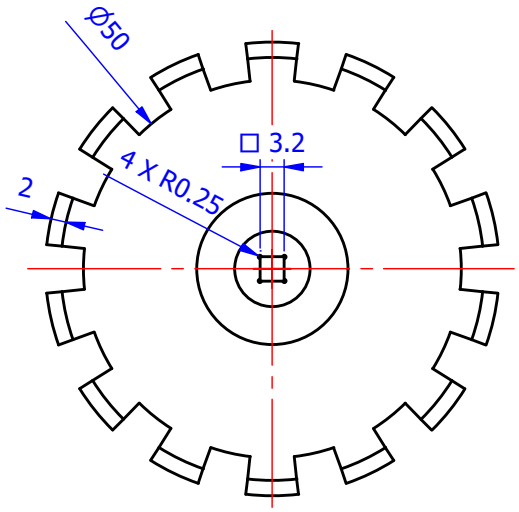


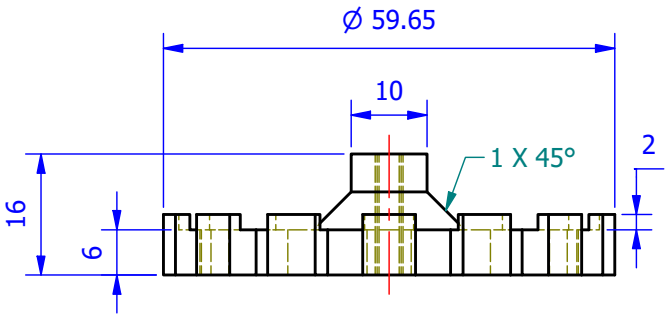
NOTES:

3D PRINTED PART.

NANYANG TECHNOLOGICAL UNIVERSITY SCHOOL OF MECHANICAL AND AEROSPACE ENGINEERING					PROJECT TITLE MA4012	
DRAWN ENG BING XIAN	CHECKED SHERMAINE NG	DATE 14/04/2026	QTY 2	SCALE 1:1	PART NAME MAIN ROLLER	
LECTURER JOHN HENG	MATERIAL PLA		DRAWING NO. APPENDIX A4		CLASS/GROUP NO. GROUP 3	SIZE A4



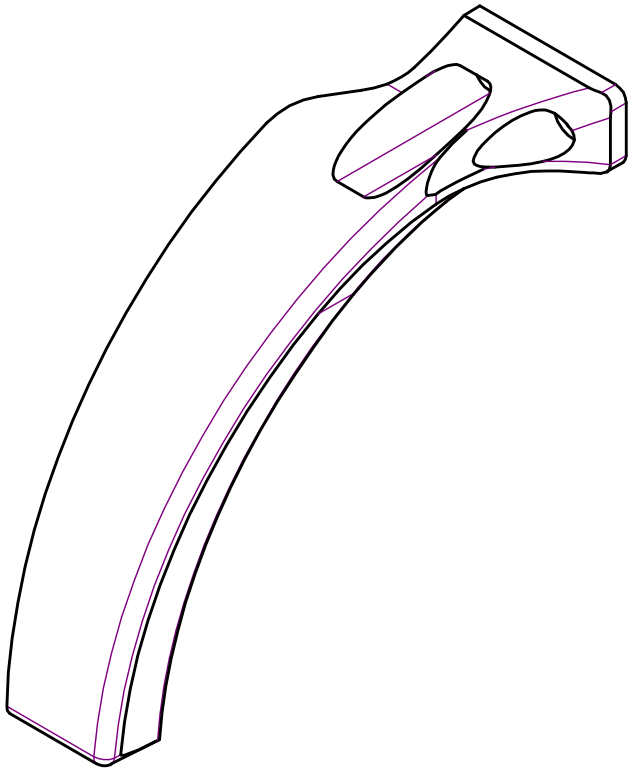


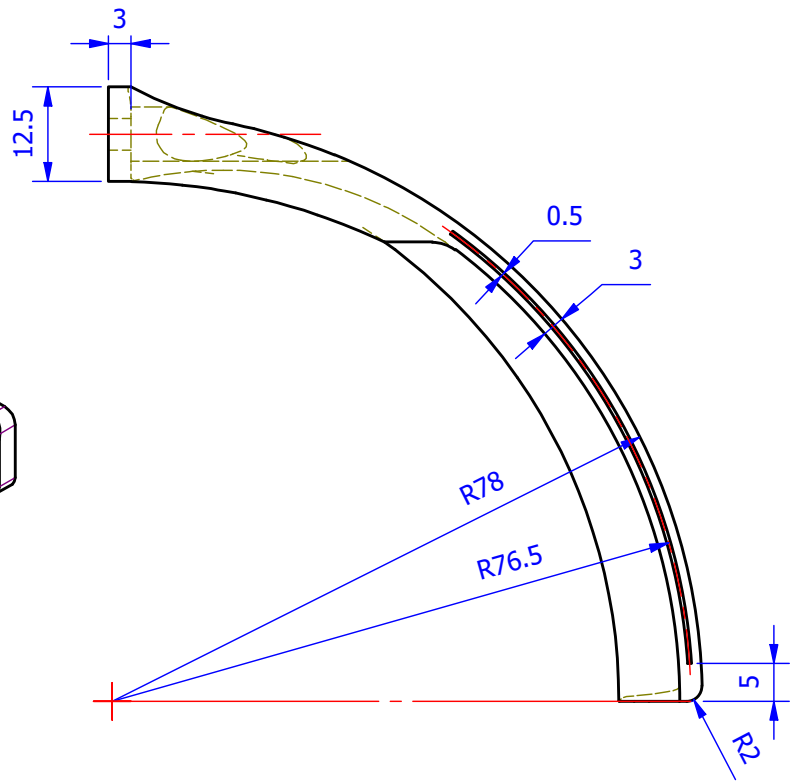


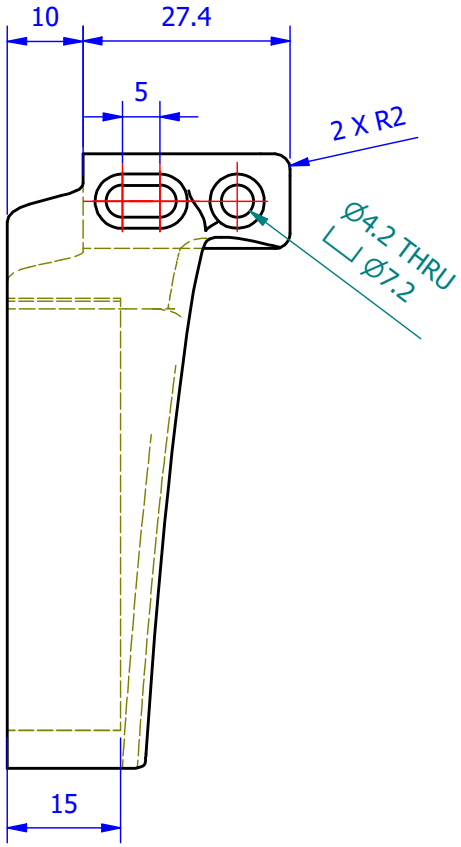
NOTES:

3D PRINTED PART.

NANYANG TECHNOLOGICAL UNIVERSITY SCHOOL OF MECHANICAL AND AEROSPACE ENGINEERING					PROJECT TITLE MA4012	
DRAWN ENG BING XIAN	CHECKED SHERMAINE NG	DATE 14/04/2026	QTY 2	SCALE 1:1	PART NAME DELIVERY ROLLER	
LECTURER JOHN HENG	MATERIAL PLA		DRAWING NO. APPENDIX A5		CLASS/GROUP NO. GROUP 3	SIZE A4



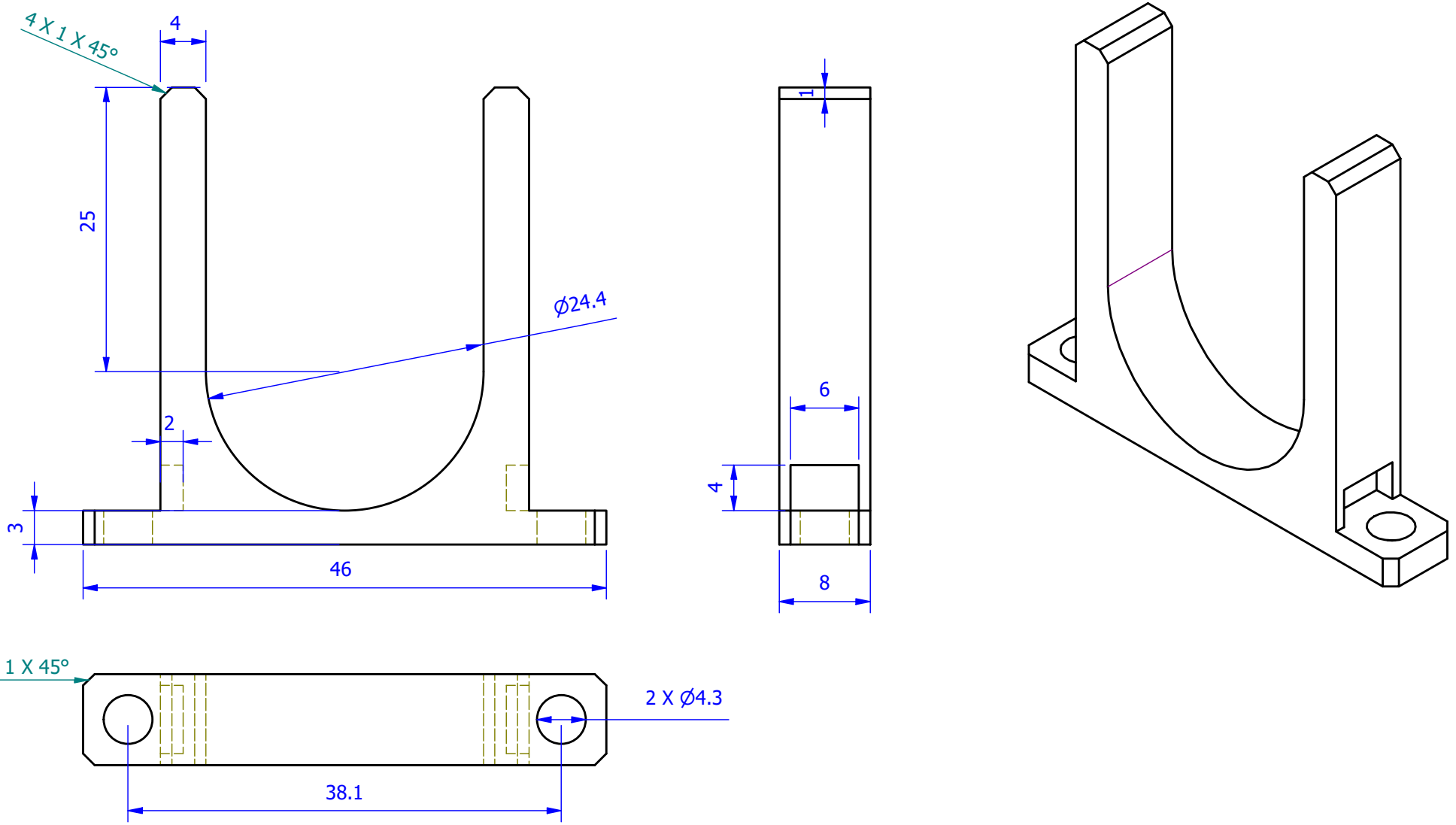




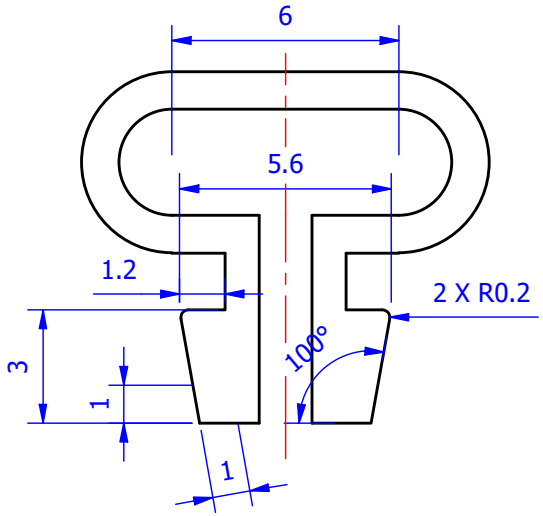
NOTES:

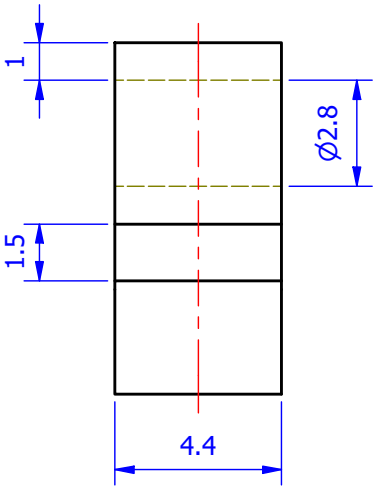
3D PRINTED PART

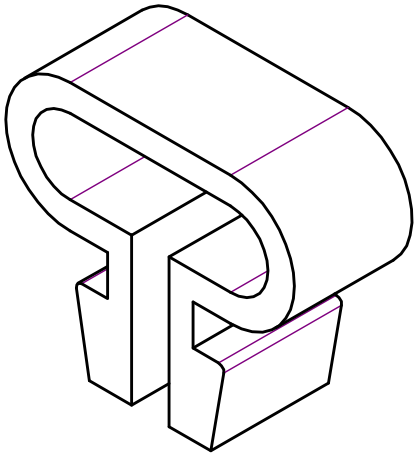
NANYANG TECHNOLOGICAL UNIVERSITY SCHOOL OF MECHANICAL AND AEROSPACE ENGINEERING					PROJECT TITLE MA4012	
DRAWN ENG BING XIAN	CHECKED SHERMAINE NG	DATE 14/04/2026	QTY 2	SCALE 1:1	PART NAME ROLLER GUARD	
LECTURER JOHN HENG	MATERIAL PLA		DRAWING NO. APPENDIX A6		CLASS/GROUP NO. GROUP 3	SIZE A4

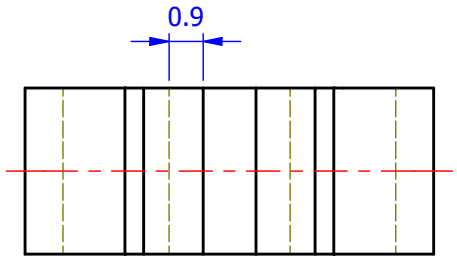


NANYANG TECHNOLOGICAL UNIVERSITY SCHOOL OF MECHANICAL AND AEROSPACE ENGINEERING					PROJECT TITLE MA4012	
DRAWN NG JIA MEI SHERMAINE	CHECKED ENG BING XIAN	DATE 16/04/2026	QTY 1	SCALE 2:1	PART NAME BATTERY HOLDER	
LECTURER JOHN HENG	MATERIAL PLA		DRAWING NO. APPENDIX A7		CLASS/GROUP NO. GROUP 3	SIZE A4

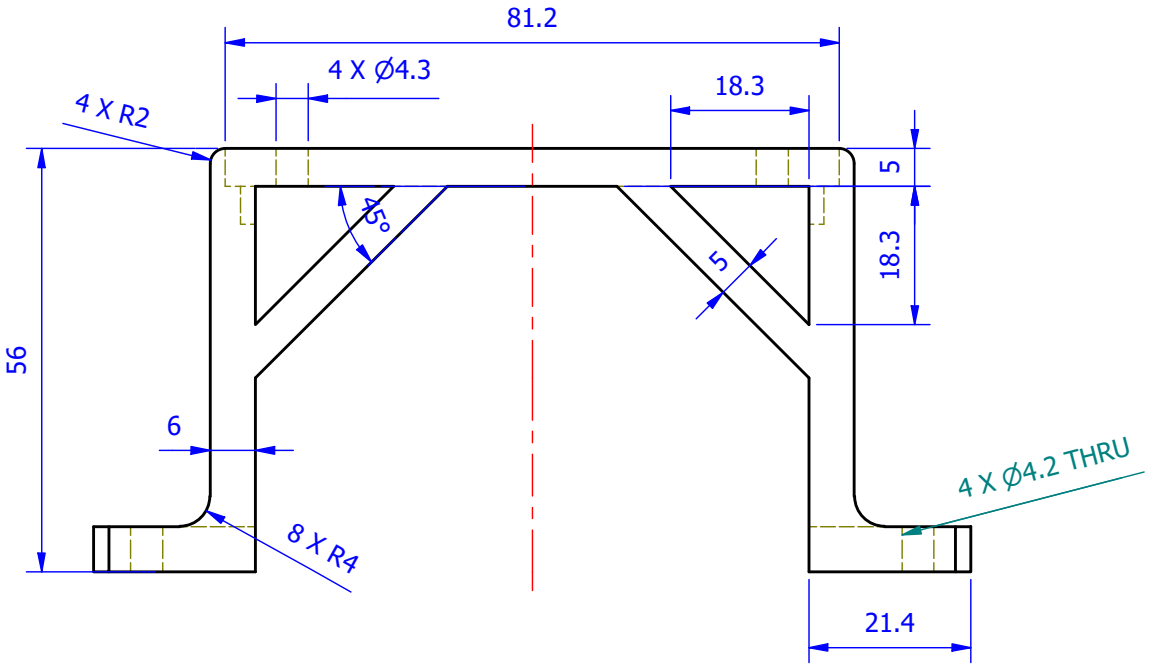


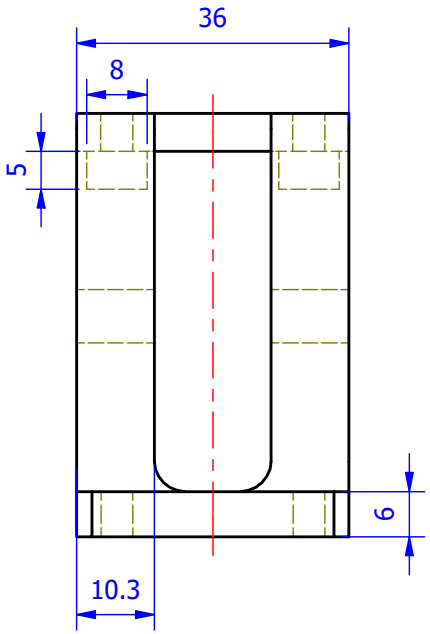


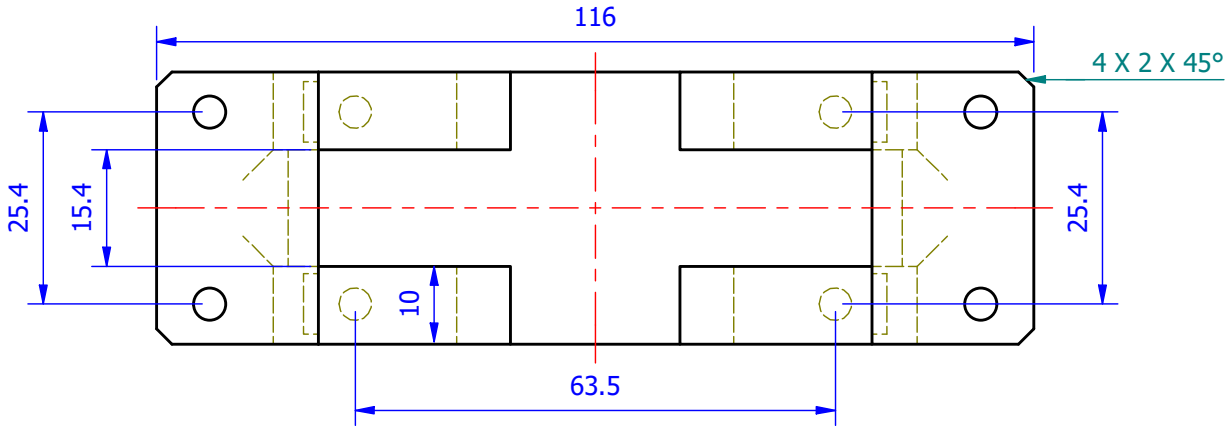


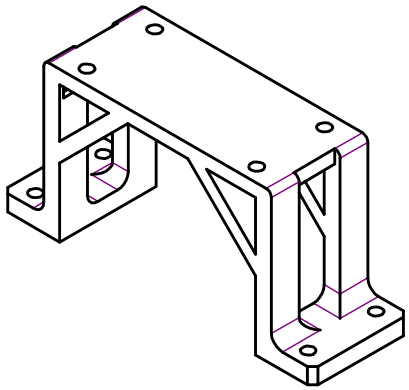


NANYANG TECHNOLOGICAL UNIVERSITY SCHOOL OF MECHANICAL AND AEROSPACE ENGINEERING					PROJECT TITLE MA4012	
DRAWN NG JIA MEI SHERMAINE	CHECKED ENG BING XIAN	DATE 16/04/2026	QTY	SCALE 5:1	PART NAME CABLE HOLDER	
LECTURER JOHN HENG	MATERIAL PLA		DRAWING NO. APPENDIX A8		CLASS/GROUP NO. GROUP 3	SIZE A4

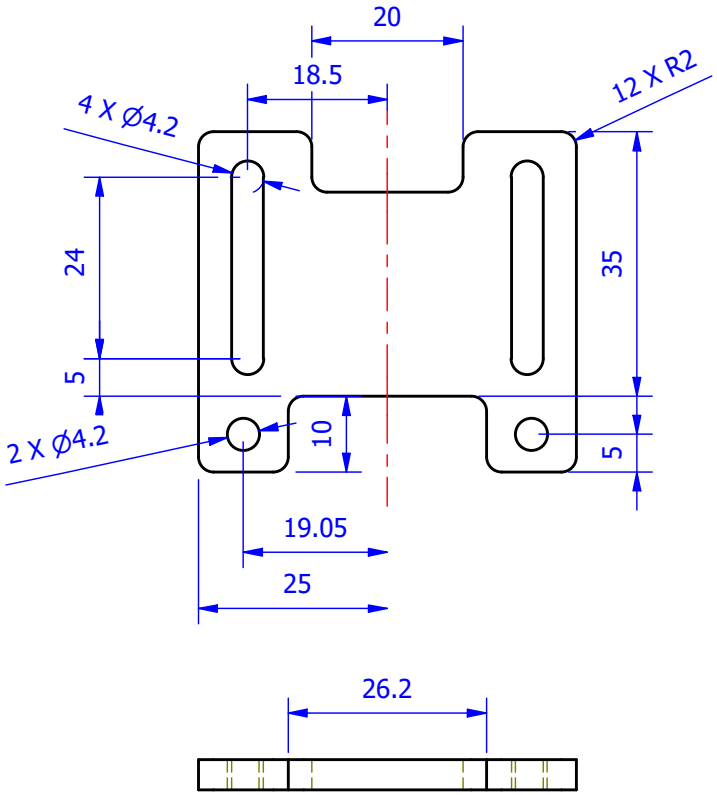


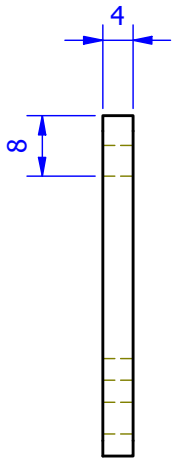


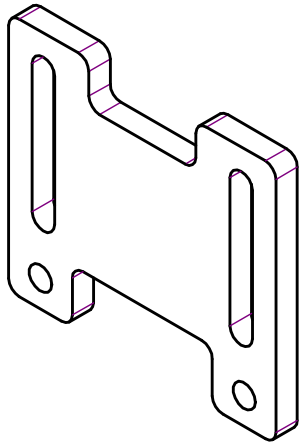




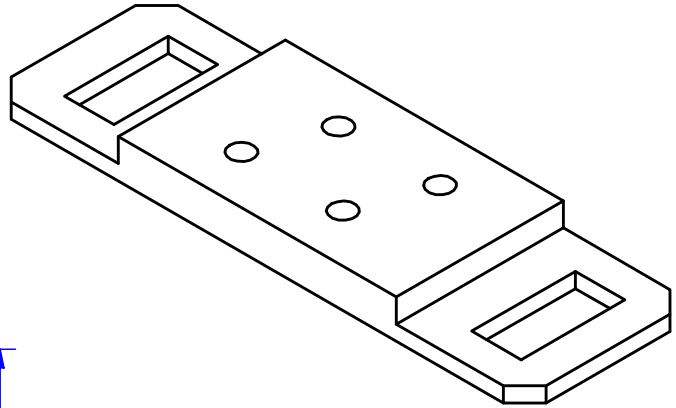
NANYANG TECHNOLOGICAL UNIVERSITY SCHOOL OF MECHANICAL AND AEROSPACE ENGINEERING					PROJECT TITLE MA4012	
DRAWN NG JIA MEI SHERMAINE	CHECKED ENG BING XIAN	DATE 16/04/2026	QTY 1	SCALE 1:1	PART NAME COMPASS HOLDER	
LECTURER JOHN HENG	MATERIAL PLA		DRAWING NO. APPENDIX A9		CLASS/GROUP NO. GROUP 3	SIZE A4

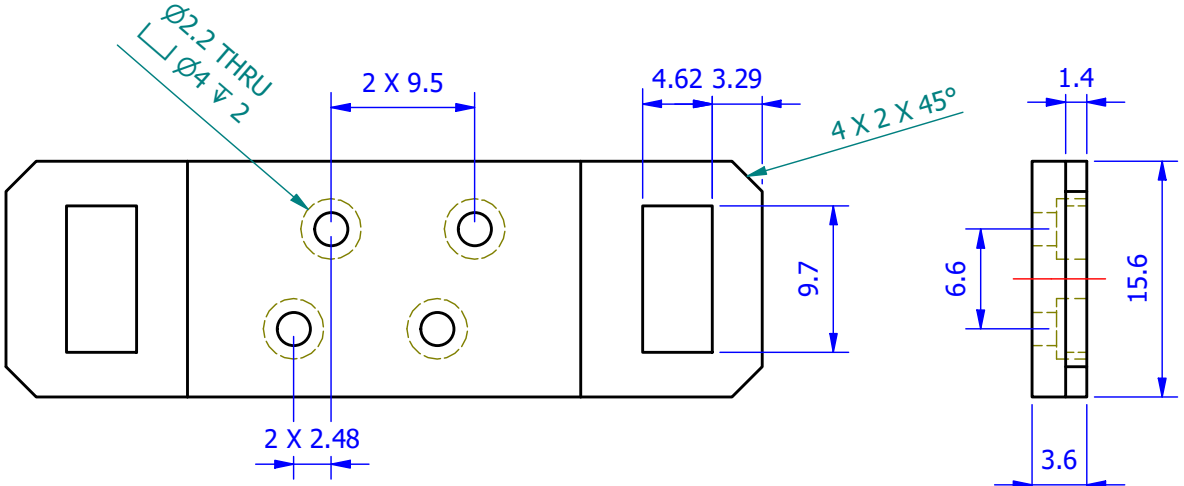


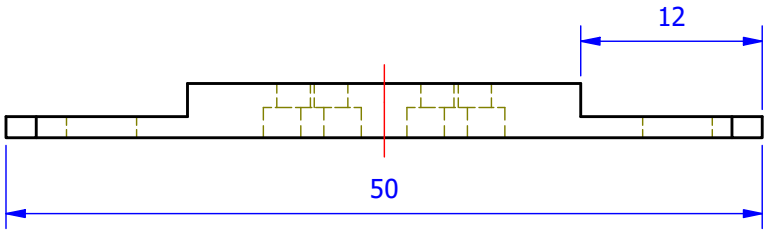




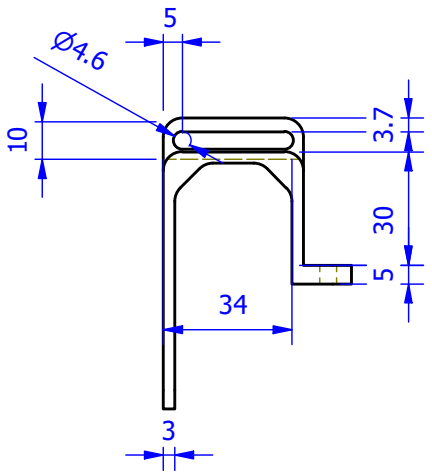
NANYANG TECHNOLOGICAL UNIVERSITY SCHOOL OF MECHANICAL AND AEROSPACE ENGINEERING					PROJECT TITLE MA4012	
DRAWN NG JIA MEI SHERMAINE	CHECKED ENG BING XIAN	DATE 16/04/2026	QTY 3	SCALE 1:1	PART NAME DISTANCE SENSOR HOLDER	
LECTURER JOHN HENG	MATERIAL PLA		DRAWING NO. APPENDIX A10		CLASS/GROUP NO. GROUP 3	SIZE A4

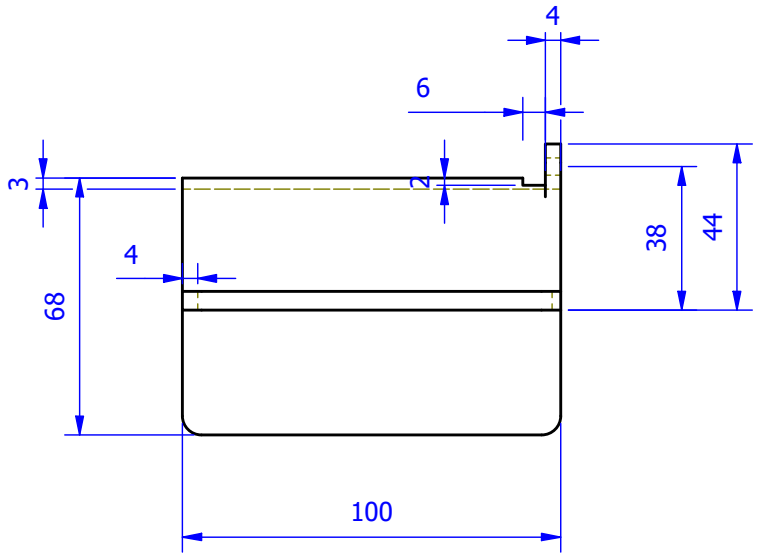


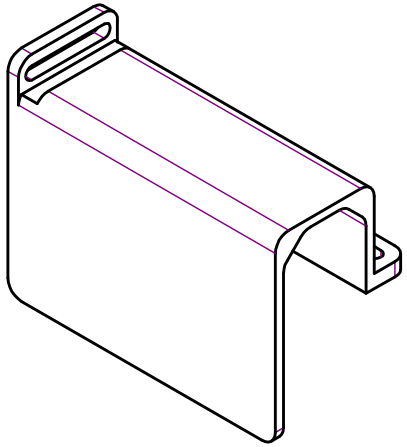


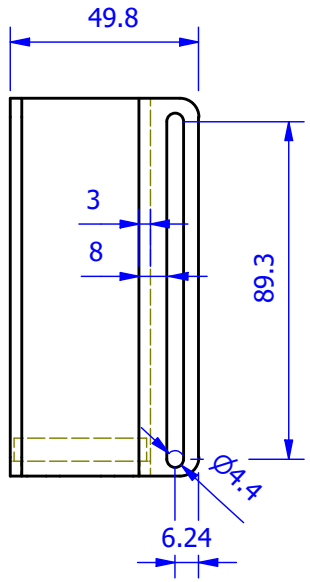


NANYANG TECHNOLOGICAL UNIVERSITY SCHOOL OF MECHANICAL AND AEROSPACE ENGINEERING					PROJECT TITLE MA4012	
DRAWN NG JIA MEI SHERMAINE	CHECKED ENG BING XIAN	DATE 16/04/2026	QTY 2	SCALE 2:1	PART NAME LIMIT SWITCH HOLDER	
LECTURER JOHN HENG	MATERIAL PLA		DRAWING NO. APPENDIX A11		CLASS/GROUP NO. GROUP 3	SIZE A4



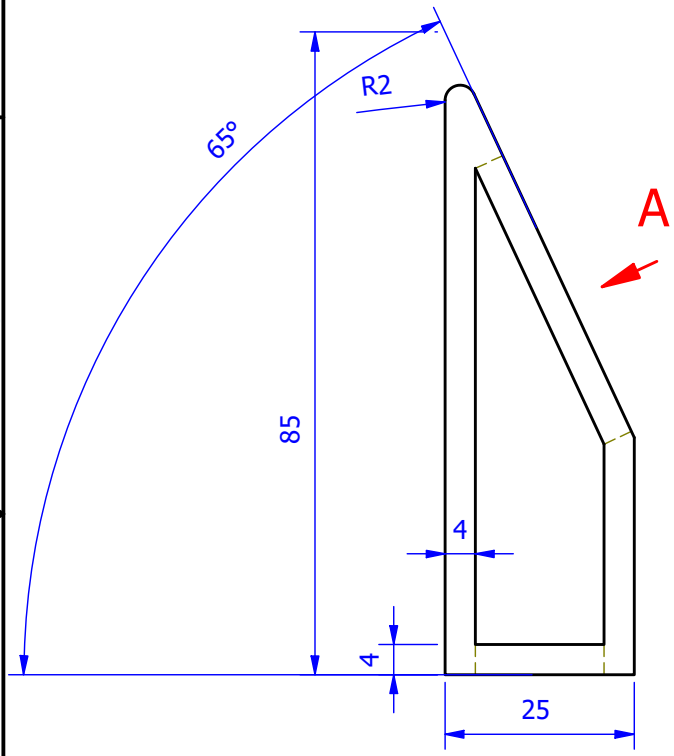


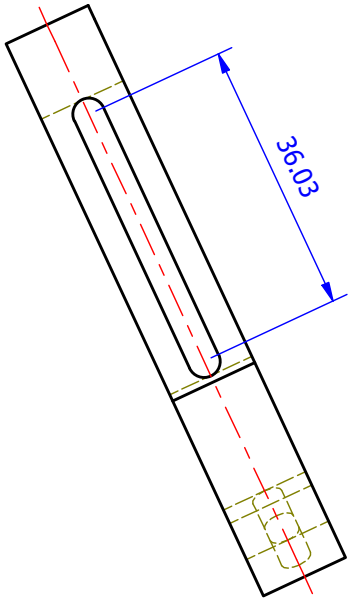


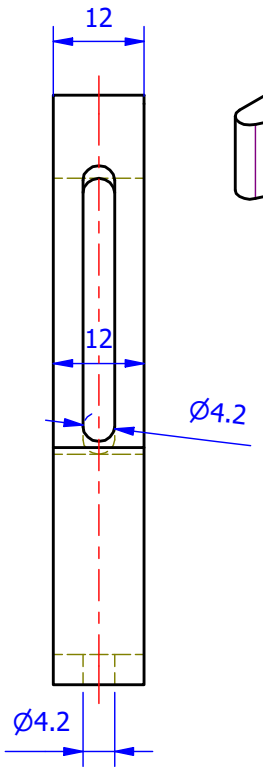


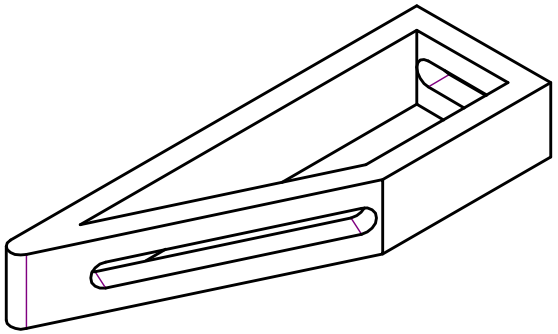
NANYANG TECHNOLOGICAL UNIVERSITY SCHOOL OF MECHANICAL AND AEROSPACE ENGINEERING					PROJECT TITLE MA4012	
DRAWN NG JIA MEI SHERMAINE	CHECKED ENG BING XIAN	DATE 16/04/2026	QTY 1	SCALE 1:2	PART NAME RIGHT GEAR PROTECTOR	
LECTURER JOHN HENG	MATERIAL PLA		DRAWING NO. APPENDIX A12		CLASS/GROUP NO. GROUP 3	SIZE A4

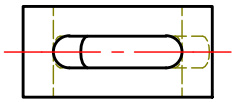
A (1 : 1)



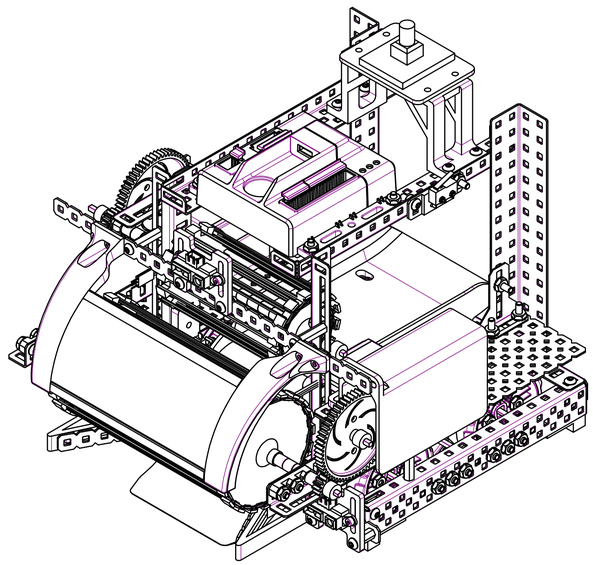


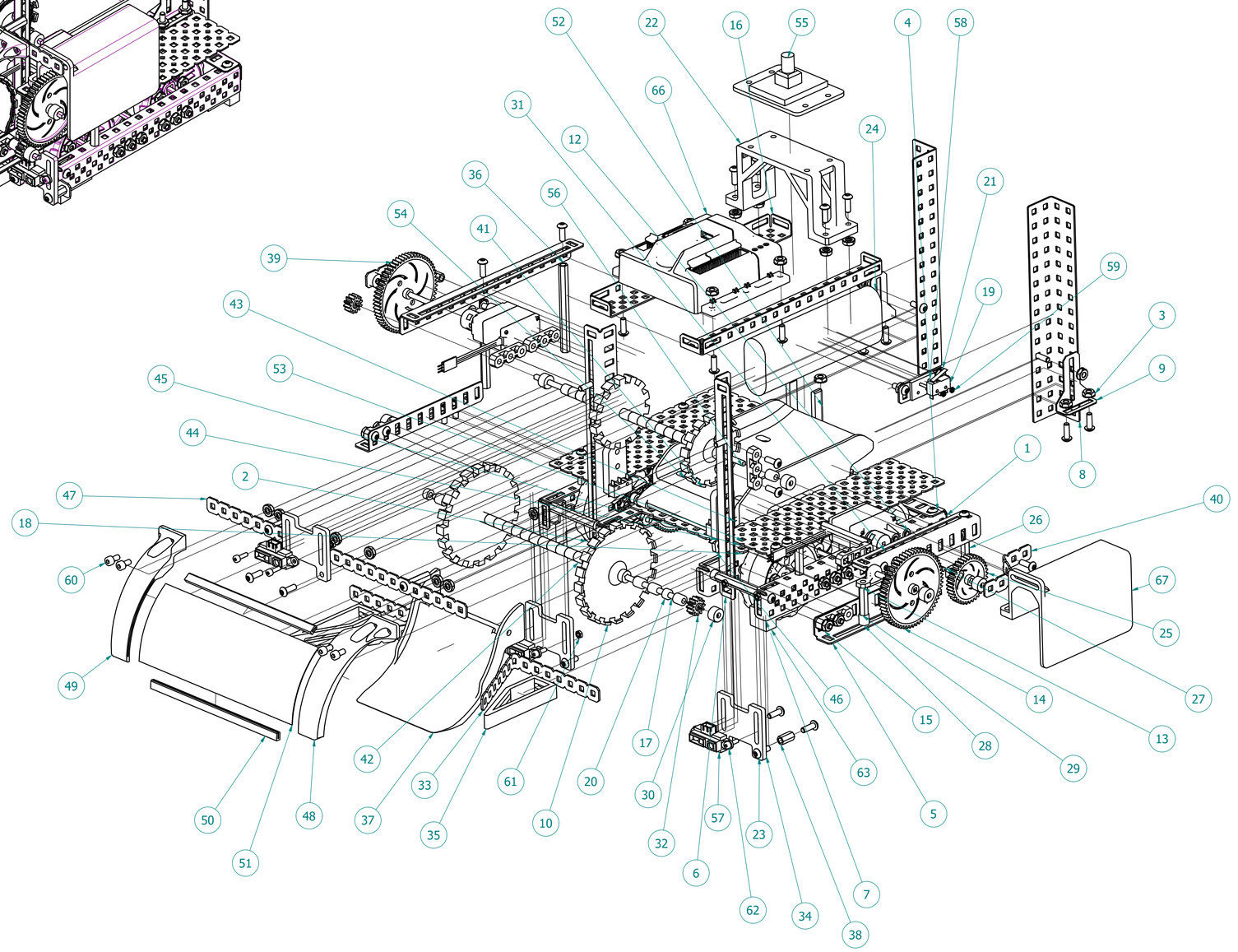






NANYANG TECHNOLOGICAL UNIVERSITY SCHOOL OF MECHANICAL AND AEROSPACE ENGINEERING					PROJECT TITLE MA4012	
DRAWN NG JIA MEI SHERMAINE	CHECKED ENG BING XIAN	DATE 16/04/2026	QTY 2	SCALE 1:1	PART NAME V Guide Reinforcement	
LECTURER JOHN HENG	MATERIAL PLA		DRAWING NO. APPENDIX A13		CLASS/GROUP NO. GROUP 3	SIZE A4

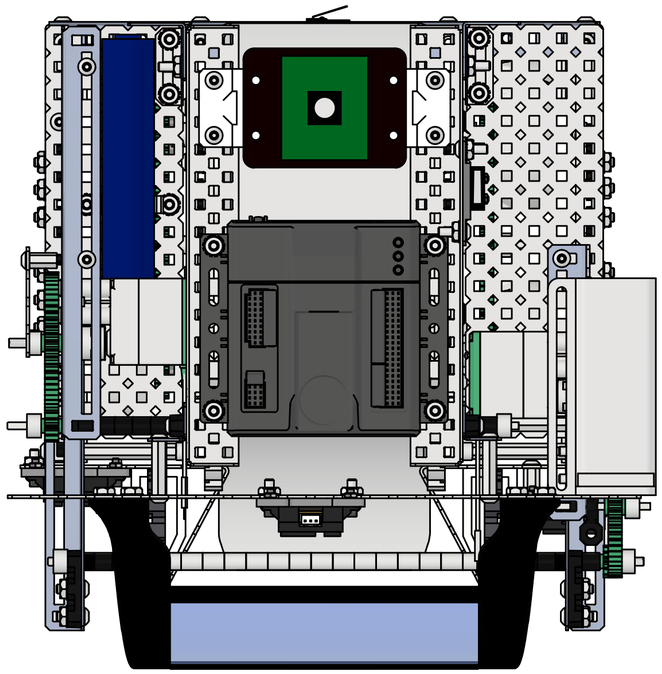


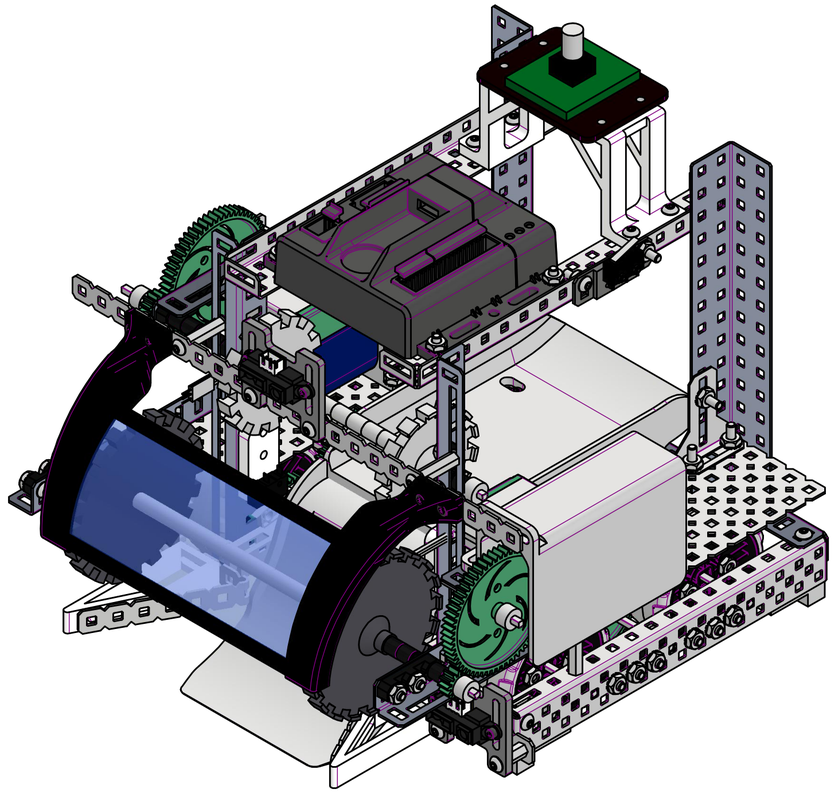


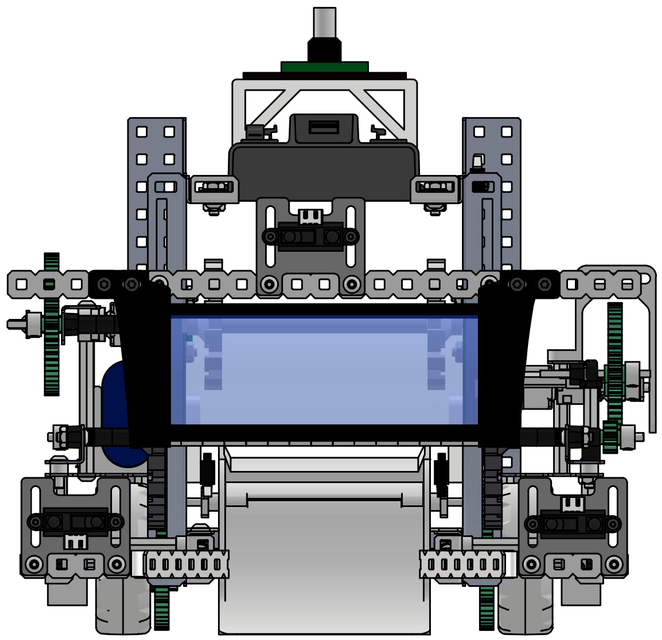
ITEM	PART NUMBER	QTY
67	Right gear protector	1
66	VEX Microcontroller	1
65	Rubber Band Short	14
64	Rubber Band	20
63	IR Line Tracking Module	4
62	Hexalobular socket head cap screw ISO 14579 - M3x10-8.8	6
61	ISO 4032 - M3	6
60	Hexalobular socket head cap screw ISO 14579 - M4x8-8.8	4
59	ISO 4032 - M2	4
58	Hexalobular socket pan head screw ISO 14583 - M2 x 10 - 4.8	4
57	IR Sharp sensor	3
56	Square Bar Roller	1
55	Compass	1
54	ISO 7380-1 - M4 x 30	4
53	Ramp Internal	1
52	Battery Holder	1
51	Plastic Protective Cover	1
50	Roller Guard Frame	2
49	Left Side Roller Guard Wideipt	1
48	Roller Guard Wide	1
47	Long Bar 1x23 (can be cut)	1
46	Lock Plate	2
45	Square Bar 135mm	1
44	Trigger Flap LS Holder	2
43	Ramp Roller v5 Trigger Flap	1
42	Collector Spacer	24
41	Ejecting Roller	2
40	Collector Bent Bar	1
39	Short Bar 1x4 (can be cut)	1
38	BEAM-0500	3
37	Ramp Roller v5	1
36	BEAM-3000	2
35	V Guide Reinforment 65 2.0	2
34	Distance sensor holder	3
33	V_guide	2
32	VEX-12-TOOTH-GEAR	2
31	Plastic Spacer 3mm	1
30	SHAFT-COLLAR	7
29	BEAM-1000	4
28	SPACER-THICK	13
27	SHAFT-2000	3
26	BEAM-2000	2
25	VEX-36-TOOTH-GEAR	1
24	VEX Battery Pack	1
23	SCREW-832-0500	57
22	Compass Holder	1
21	Limit switch holder 3	2
20	SPACER-THIN	9
19	Limit Switch	4
18	Long Bar External Ramp	3
17	SHAFT-12000	1
16	VL-RAIL-21-16 RevA	2
15	BEARING-FLAT	8
14	VEX-60-TOOTH-GEAR	2
13	BEARING-BLOCK	1
12	3-Wire Servo (276-2162)	2
11	Tennis Ball	1
10	Roller	2
9	ANGLE-GUSSET	2
8	Angle 2x2	2
7	Right main assembly	1
6	Slotted Angle 3	3
5	Slotted Angle 2	3
4	SCREW-832-0250	7
3	NUT-832-KEPS	50
2	Left main assembly	1
1	Slotted Angle	1

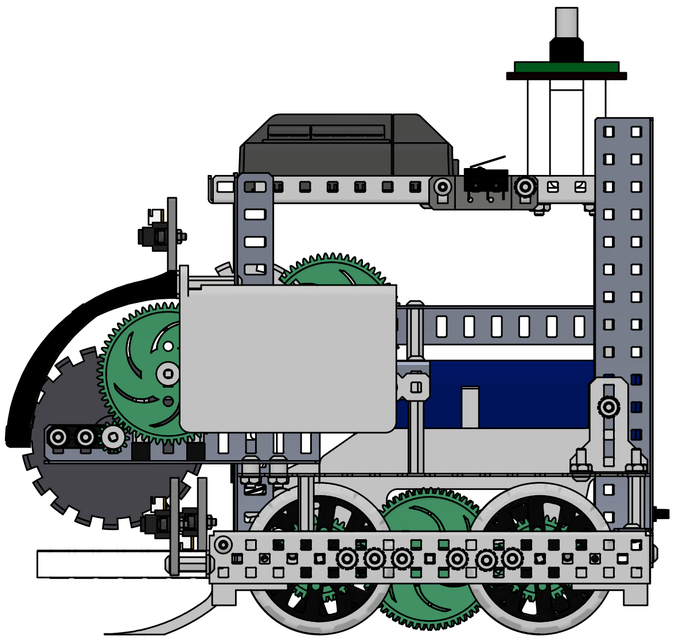
PARTS LIST

NANYANG TECHNOLOGICAL UNIVERSITY SCHOOL OF MECHANICAL AND AEROSPACE ENGINEERING				PROJECT TITLE MA4012	
DRAWN NG JIA MEI SHERMAINE	CHECKED ENG BING XIAN	DATE 16/04/2026	QTY	SCALE 1:2	PART NAME FULL ASSEMBLY
LECTURER JOHN HENG	MATERIAL		DRAWING NO. APPENDIX A14	CLASS/GROUP NO. GROUP 3	SIZE A1









NANYANG TECHNOLOGICAL UNIVERSITY SCHOOL OF MECHANICAL AND AEROSPACE ENGINEERING					PROJECT TITLE MA4012	
DRAWN NG JIA MEI SHERMAINE	CHECKED ENG BING XIAN	DATE 14/04/2026	QTY 1	SCALE 1:2	PART NAME FULL ASSEMBLY	
LECTURER JOHN HENG	MATERIAL		DRAWING NO. APPENDIX A15		CLASS/GROUP NO. GROUP 3	SIZE A2